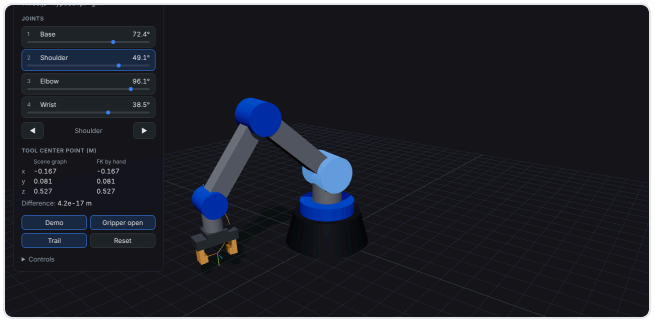
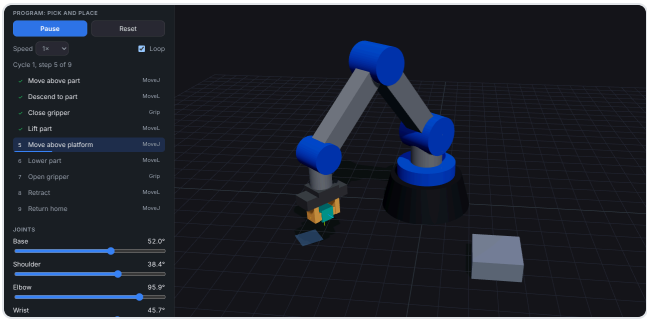
VENTION · PORTFOLIO WALKTHROUGH

Five Snacks, Line by Line

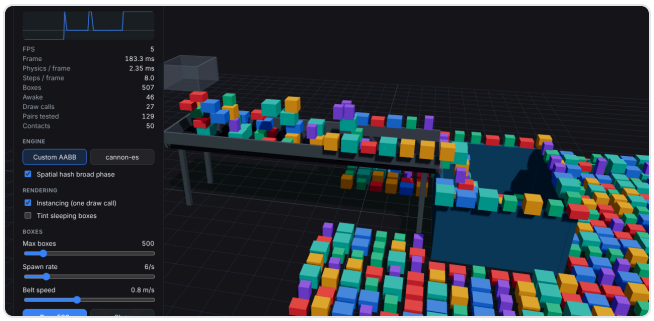
Robot kinematics, React, physics engines, legacy refactoring and databases in TypeScript: every file explained, with the concepts underneath.



1 · The Robot Arm



2 · The React Control Panel



3 · Conveyor + Physics

```
> npm run demo

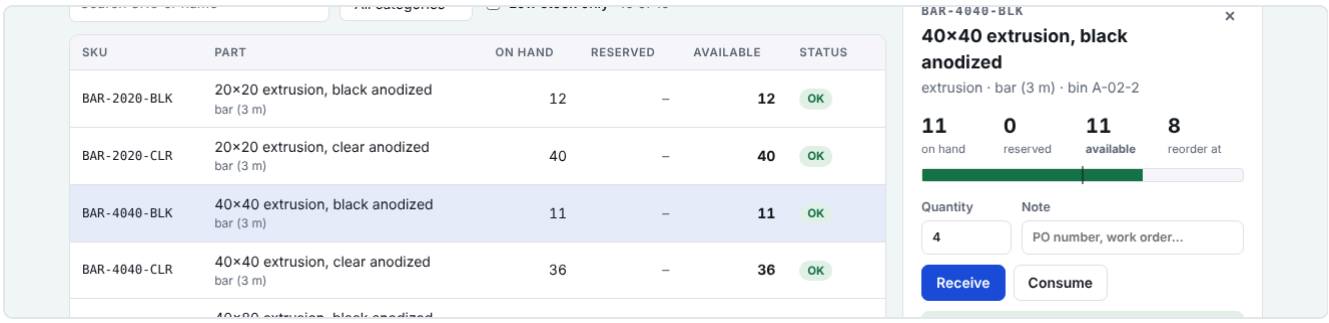
Quote Q-1762525200000

  40x40 extrusion 1200mm          4 ×    $35.10     $140.40
  40x80 extrusion 2400mm black    2 ×   $136.96     $273.92
  Plate PL-200                    4 ×    $32.00     $128.00
  Fasteners (pack of 50)          3 ×     $6.75      $20.25
  Linear actuator                 2 ×   $420.00     $840.00
  Motion controller               1 ×   $650.00     $650.00
  Assembly (8 h)                  1 ×   $280.00     $280.00

  Subtotal                                        $2,332.57
  Discount                                         -$205.26
  Shipping                                           $43.01
  Tax                                               $282.14
  Total                                           $2,452.46

  Ships in 17 working days
```

4 · The Legacy Refactor Kata



5 · Parts Inventory

Projects in `Portfolio/vention/` · Code quoted directly from the repository, with file and line numbers

Contents

How to read this book 6

Chapter 1 · The Robot Arm 8

1.1 Coordinate frames and transforms 9

1.2 Forward kinematics 10

1.3 Meshes, as the GPU sees them 11

1.4 glTF in one page 12

1.5 The frame loop and frame-rate independence 14

1.6 Conveyor belts: frames, kinematics and fixed steps 14

1.7 Rules as data 16

Line by line 17

- The arm as data 17
- Forward kinematics by hand 20
- The arm as a scene graph 24
- Loading and driving the gripper 31
- The stage 35
- The tool path trail 38
- The side panel 40
- Wiring it together 44
- Testing the kinematics 53
- Building the gripper model by hand 57
- Shapes in the scene: the data and the reach rule 68
- Inventory parts as shapes 72
- Drawing the shapes 76
- The Shapes panel 81
- Conveyor belts: how a belt carries things 93
- Drawing the belts 103
- The Conveyors tab, and running the line 109
- Splitters: rules that decide where items go 123
- Gates on the running line 129
- Drawing gates, and the Splitters tab 134

Interview questions 147

Exercises 150

Chapter 2 · The React Control Panel 152

2.1 Inverse kinematics 153

2.2 Joint space and Cartesian space 154

2.3 How React renders, and why the simulation lives outside it 154

2.4 useSyncExternalStore, and tearing 155
2.5 Effects, cleanup and StrictMode 155
2.6 Re-parenting with attach 156
Line by line 157
The robot program 157
Inverse kinematics 160
Running the program 164
The store between React and Three.js 172
The hook 179
Mounting the 3D view 179
The 3D side 181
Picking up the part 184
What changed in the stage 188
What changed in the gripper 190
The panel 191
Putting it together 199
Interview questions 200
Exercises 202

Chapter 3 · Conveyor + Physics 203

3.1 What a physics engine does in one step 204
3.2 Time: fixed steps and interpolation 204
3.3 Finding pairs: the broad phase 205
3.4 Contacts and impulses 206
3.5 Stacks, sleeping and islands 207
3.6 Performance in a browser 207
Line by line 208
The work cell as data 208
The fixed-timestep loop 211
Object pooling 213
One interface, two engines 215
The spatial hash 217
The custom engine 221
The cannon-es adapter 239
Spawning and recycling 245
Drawing thousands of boxes 249
The conveyor and bin 253
The frame loop 256
The frame-time graph 260
Interview questions 262
Exercises 264

Chapter 4 · The Legacy Refactor Kata 265

4.1 What makes code “legacy” 266

4.2 Characterization tests and the golden master 266

4.3 Refactoring in small, green steps 267

4.4 A pure core with side effects at the edge 267

4.5 Changing behaviour on purpose 268

4.6 Money and floating point 268

Line by line 269

The code as found 269

The plan, as commits 276

Generating the golden-master cases 277

The golden master 279

Spec tests: the rules, written down 283

Named numbers 286

Types 288

One rounding function 289

Pricing each item 290

Pricing rules 294

The result 296

Interview questions 300

Exercises 302

Chapter 5 · Parts Inventory 303

5.1 The shape of the system 304

5.2 Relational and document data models 304

5.3 Transactions, isolation and race conditions 305

5.4 Atomicity in MongoDB 306

5.5 Repositories and contract tests 307

5.6 Cut lists and bin packing 308

5.7 Choosing the database at deploy time 309

5.8 Setting up the database 309

5.9 Two apps, one database 312

Line by line 313

The data model and the one interface 313

Errors and shared helpers 318

From a machine design to stock 320

The reference implementation 327

The PostgreSQL schema 331

The PostgreSQL repository 337

The MongoDB repository 343

Choosing a database, and the service layer 351
The HTTP API 353
Contract tests: one suite, every database 360
Adding a part, end to end 365
Shapes for the robot arm's scene 373
The React page 382
Containers: Compose profiles and Kubernetes overlays 388
Conveyor belts in the database 394
Splitters: sorting rules as data, and editable parts 400
Interview questions 414
Exercises 417

Appendix A · Running the projects 418

Appendix B · Glossary 419

BEFORE YOU START

How to read this book

This book joins the separate walkthroughs into one and goes further. Each chapter covers one snack and has the same five parts: Each chapter covers one snack and has the same five parts:

1. **Concepts.** The ideas the code relies on, with diagrams, before any code: transforms and glTF, inverse kinematics and React's rendering model, physics engines, characterization testing.
2. **Line by line.** Every source file that matters, quoted from the repository in order, with each chunk explained. The code is sliced from the real files, so the line numbers match what you'll see in your editor. Snack 1, which previously covered only the gripper script, now covers the whole app.
3. **Going deeper.** Boxes inside the line-by-line parts that step back to what a production system, a full engine or a large team would do differently.
4. **Interview questions.** Questions this project invites, with model answers grounded in the code.
5. **Exercises.** Extensions to try, each with a hint.

Two reading paths work well. To prepare for a conversation about the projects, read each chapter's concepts and questions. To understand or extend the code, read the line-by-line parts with the project open next to the book.

The five snacks

	Project	Stack	What it shows
1	**The Robot Arm**	Three.js · TypeScript · glTF	A four-joint arm as a Three.js scene graph, forward kinematics written by hand and checked against it every frame, a gripper model built byte by byte as a glTF file, shapes and inventory parts placed on its floor from the database, and conveyor belts that carry them.
2	**The React Control Panel**	React · Three.js · TypeScript	A React panel drives a pick-and-place program through inverse kinematics, while the 3D view animates at full frame rate without re-rendering React every frame.
3	**Conveyor + Physics**	Three.js · cannon-es · TypeScript	A physics engine written from scratch, the same scene on cannon-es, and the optimisations that let thousands of boxes run at 60 fps in a browser.
4	**The Legacy Refactor Kata**	TypeScript · node:test	A messy 180-line quote calculator made small, typed and pure in 13 commits, with characterization tests proving exactly what changed and what didn't.

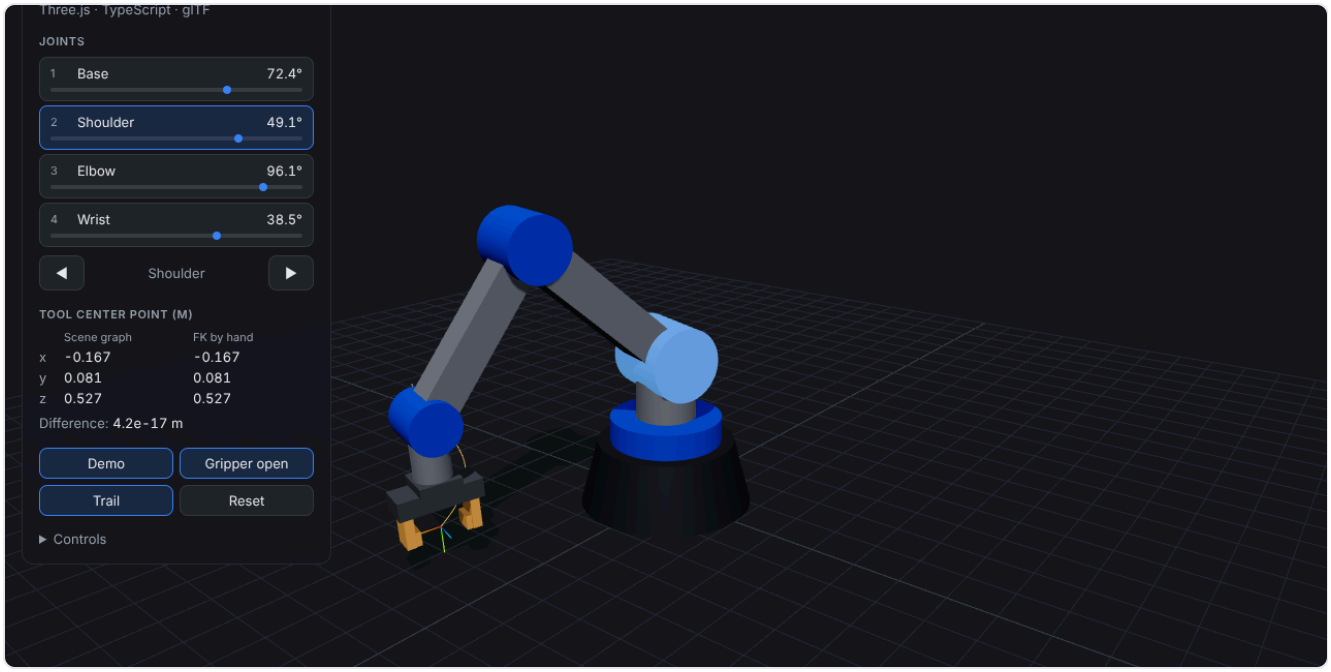
	Project	Stack	What it shows
5	**Parts Inventory**	Node.js · React · PostgreSQL or MongoDB · Docker · Kubernetes	A stock system for the parts the other snacks use: check a machine design against live stock and reserve it, all or nothing, on PostgreSQL or MongoDB behind one interface.

CHAPTER 1 · `vention/snack1-robot-arm`

The Robot Arm

A four-joint arm as a Three.js scene graph, forward kinematics written by hand and checked against it every frame, a gripper model built byte by byte as a glTF file, shapes and inventory parts placed on its floor from the database, and conveyor belts that carry them.

Three.js · TypeScript · glTF



Demo mode. The panel's TCP readout shows the scene graph's answer and the hand-written FK side by side; they differ by $4 \cdot 10^{-17}$ m.

The one idea to take away

The same question, “where is the tool?”, answered two independent ways, every frame:

`joint angles` → `Three.js scene graph` → `tcp.getWorldPosition()`

`joint angles` → `forwardKinematics() by hand` → `{ x, y, z }`

If the hierarchy were built wrong, the two would disagree immediately. The gripper is a file the script writes from scratch:

`boxes` → `vertices + normals + indices` →

`buffer → bufferView → accessor → mesh → node` → `.glb`

CONCEPTS · 1.1

Coordinate frames and transforms

Every object in a 3D scene has a **local transform**: a position, rotation and scale relative to its parent. Three.js packs the three into one 4×4 matrix (`object.matrix`) and computes the **world** matrix by multiplying down the tree: `matrixWorld = parent.matrixWorld × matrix`. Nothing more is needed to make a robot arm move like a robot arm: put each joint inside the previous one, and rotating a joint carries everything after it.

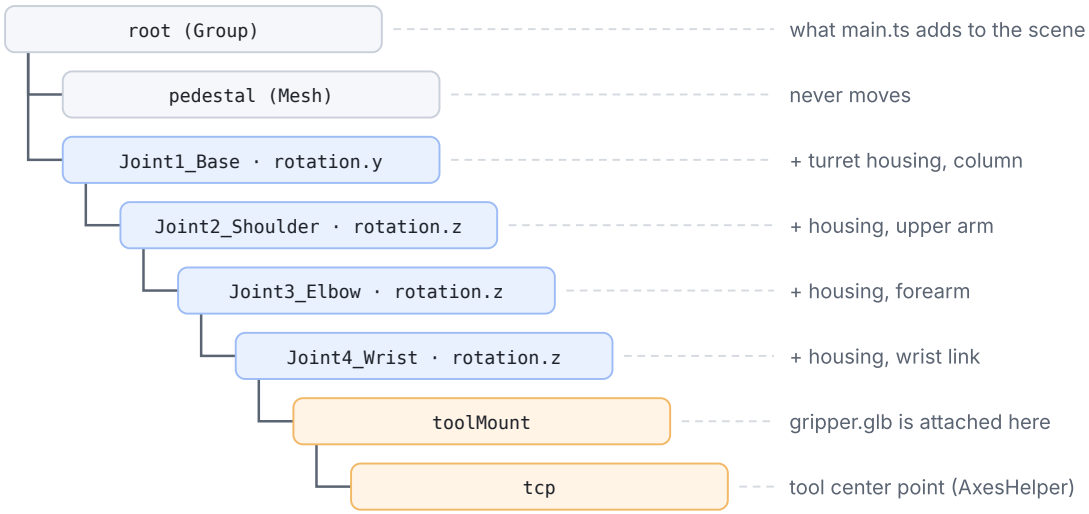


Figure 1.1. The arm's hierarchy. Blue nodes are empty joints that only rotate; the meshes hang off them. Rotating a joint moves everything below it in the tree.

The conventions are worth saying out loud, because most 3D bugs are convention bugs:

- **Right-handed, +Y up, meters.** Three.js, glTF and this project agree on all three, so no conversion happens anywhere.
- **Right-hand rule for rotation.** Point your right thumb along the axis; your fingers curl in the positive direction. A positive rotation about Z turns +Y toward −X, which is why a positive shoulder angle leans the arm toward −X.
- **Radians in the API, degrees for people.** The project stores degrees and converts once, in `RobotArm.setAngle`.

A transform matrix applies scale, then rotation, then translation (T·R·S). Read the arm's chain as alternating pairs: “translate up the link, then rotate about the joint”. That's exactly the list of matrices the test file multiplies together.

> Each joint here rotates about one axis only, so Euler-angle order never matters and **gimbal lock** can't happen. Free-floating orientations, like the tumbling boxes in Snack 3, use **quaternions** instead: four numbers that compose and interpolate smoothly without singular poses.

CONCEPTS · 1.2

Forward kinematics

Forward kinematics (FK) maps joint space (four angles) to task space (where the tool is). It always has exactly one answer and is cheap to compute, which makes it the foundation for everything else: drawing, checking clearances, and testing inverse kinematics in Snack 2.

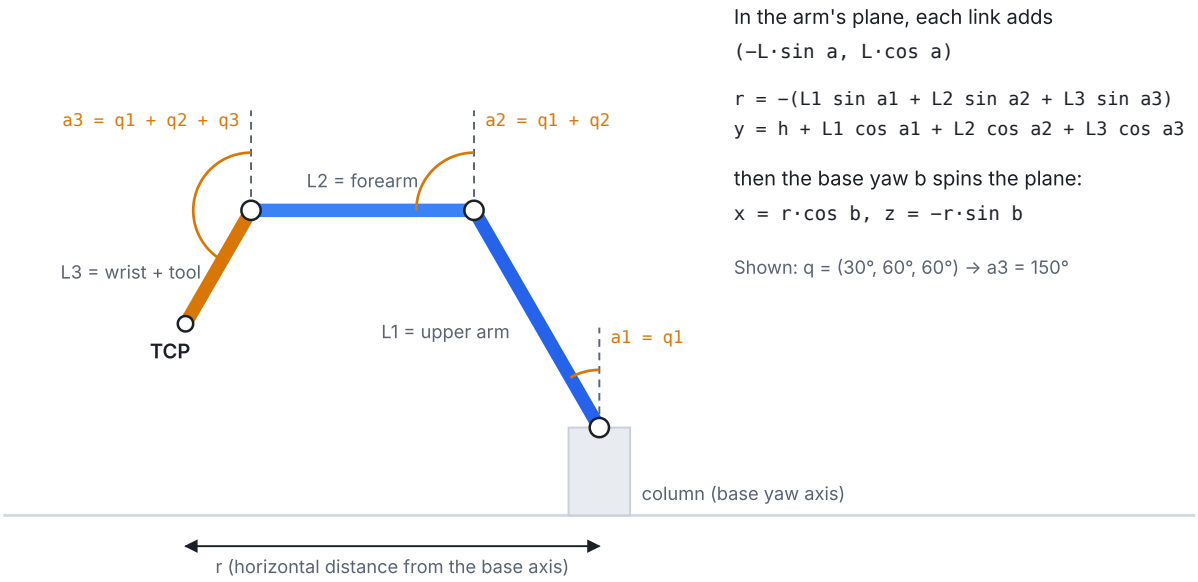


Figure 1.2. Side view of the arm's plane. Because the pitch joints are parallel, each link's angle from vertical is the sum of the joint angles before it, and the tool position is a sum of three link vectors.

The general method is a chain of 4×4 matrices, one per joint, usually described by **Denavit–Hartenberg** (DH) parameters: four numbers per joint (link length, twist, offset and angle). This arm doesn't need that machinery, because its pitch axes are parallel: everything after the base is a 2D problem in one plane. The test file still builds the general matrix chain, as an independent reference.

The **workspace** is the set of points the tool can reach. With every joint free it would be a sphere of radius 1.085 m around the shoulder; the joint limits carve pieces out of it. The demo-motion test checks one practical corner of it: the fingertips must stay above the floor.

CONCEPTS · 1.3

Meshes, as the GPU sees them

A mesh is a list of **vertices**, each with attributes (position, normal, maybe texture coordinates), plus an **index** list saying which three vertices form each triangle. The vertex shader moves every vertex by the model, view and projection matrices; the GPU fills in the triangles; the fragment shader colors each pixel using the normal interpolated across the triangle.

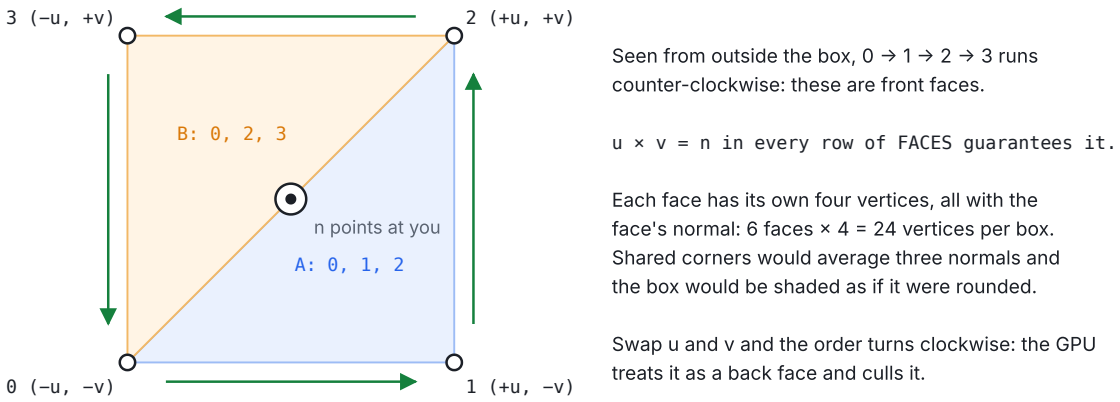


Figure 1.3. One face of a box, split into two triangles. The corner order decides which side is the front.

- **Winding** decides the front of a triangle: counter-clockwise as seen by the camera. Back faces are culled by default, which halves the work for closed shapes. Wrong winding looks like a model turned inside out.
- **Normals** drive lighting. Shared vertices give smooth shading (one averaged normal); duplicated vertices give hard edges. A box needs 24 vertices, not 8, to look like a box.
- **Indexed drawing** saves memory: 24 vertices and 36 indices instead of 36 full vertices, and the GPU can reuse vertices it has already transformed.

CONCEPTS · 1.4

glTF in one page

glTF 2.0 is the Khronos Group's open format for delivering 3D scenes, often called “the JPEG of 3D”. It holds a node hierarchy, meshes, PBR materials, textures, cameras, animations and skins. A `.gltf` is JSON plus separate binary and image files; a `.glb` packs everything into one binary file.

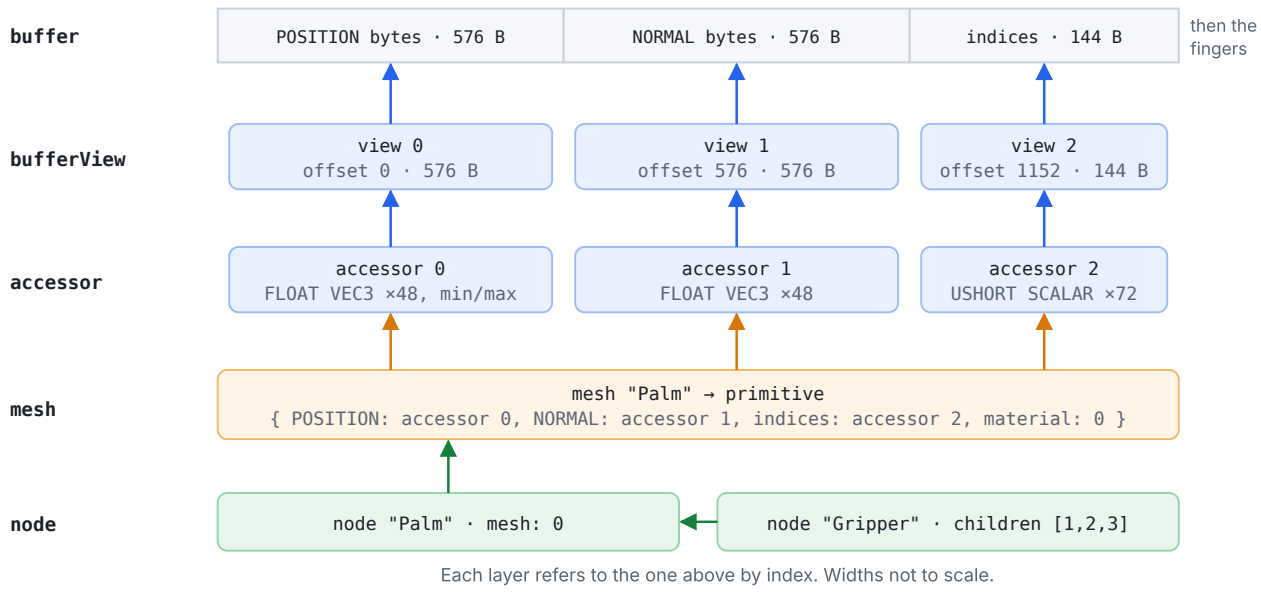


Figure 1.4. How the palm is stored. Raw bytes, then byte ranges, then typed views of those ranges, then a drawable mesh, then a node that places it.

Why three layers between bytes and meshes? A **bufferView** is a byte range that can be uploaded to the GPU as-is, and can **interleave** several attributes using a stride. An **accessor** says how to read part of a view (type, count, offset). So many meshes can share one blob, and a loader can hand byte ranges straight to WebGL with no parsing of individual numbers. That's what makes glTF fast to load.

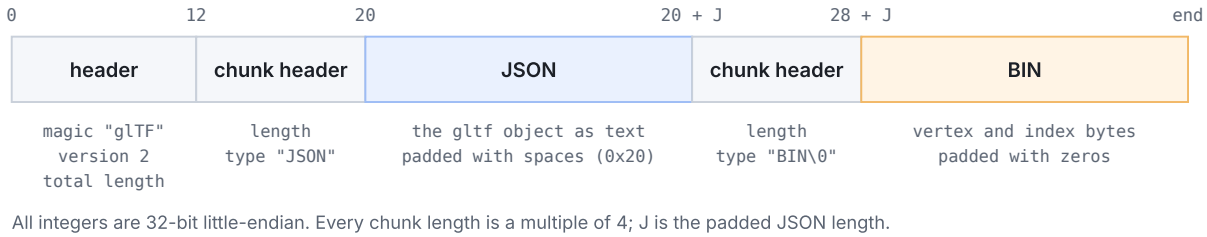


Figure 1.5. The .glb container: one file holding the JSON and the binary buffer, so the model loads with a single request.

Materials use the **metallic-roughness** PBR model: a base color, how metallic the surface is, and how rough. Three.js maps these directly to `MeshStandardMaterial`, so a model looks the same in any glTF viewer.

CONCEPTS · 1.5

The frame loop and frame-rate independence

`renderer.setAnimationLoop` (a wrapper around `requestAnimationFrame`) calls you once per display refresh: 60 times a second on most monitors, 120 or 144 on others. Anything that moves must therefore scale by the frame time `dt`, or it will run twice as fast on a 120 Hz screen.

Constant speeds are easy: `angle += speed × dt`. Smoothing is subtler. Moving a fixed fraction k of the remaining gap each frame leaves $(1 - k)^n$ of the gap after n frames, so the result depends on how many frames there were. Choosing **$k = 1 - e^{-\lambda \cdot dt}$** fixes it: after any sequence of frames adding up to time T, the remaining gap is $e^{-\lambda T}$, however the time was sliced. With $\lambda = 3$ (`DEMO_SMOOTHING`), 95% of the gap closes in one second.

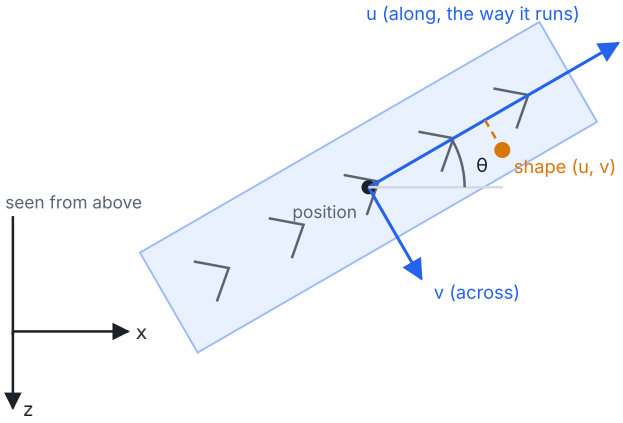
Two more frame-loop rules from this project: clamp `dt` (here to 50 ms), because a hidden tab stops calling you and the first frame back would otherwise see a huge step; and remember that Three.js updates world matrices lazily, during `render()`, so code that needs a world position between changes must ask for it (`getWorldPosition` does).

CONCEPTS · 1.6

Conveyor belts: frames, kinematics and fixed steps

A conveyor looks like a physics problem, but most of it is geometry. Three ideas carry the whole feature.

Local frames. Every belt can be turned to any angle, which makes “is this point on the belt?” awkward in world coordinates. Moved into the belt's own frame (subtract its position, rotate by minus its angle), every belt becomes the same axis-aligned rectangle. Robotics does this constantly: a part's position in the gripper's frame, a target in the camera's frame. The one rule is to use the same angle convention as the renderer, or what's drawn and what moves will disagree.



toBelt: world → belt

```
dx = x − position.x, dz = z − position.z
u = dx·cos θ − dz·sin θ
v = dx·sin θ + dz·cos θ
```

on the belt if

```
|u| ≤ length/2 and |v| ≤ width/2
```

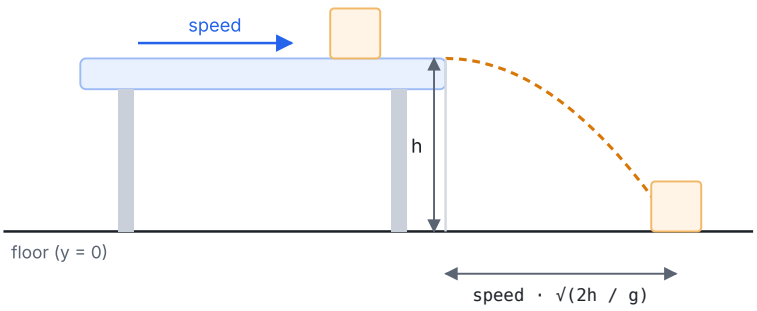
surface velocity

```
(vx, vz) = speed · (cos θ, −sin θ)
```

Same convention as Three.js's rotation.y, so the drawing and the motion always agree.

Figure 1.6. A belt's own frame. Turning the world point into (u, v) makes every belt an axis-aligned rectangle, whatever its angle.

Kinematic, not dynamic. A dynamic simulation computes forces (friction between belt and part, contact between parts) and gets motion from them. A kinematic one sets the motion directly: on a belt, move at the belt's speed. It can't show a box tipping over or two boxes jamming, but it's exact, cheap and testable, and it answers the layout question (where does each part end up?) that a line designer asks first. Off the end, gravity is the one force simulated, because where an item lands decides whether the next belt catches it.



Each step of dt, in the air:

```
vy ← vy − g·dt
y ← y + vy·dt
x, z ← x, z + (vx, vz)·dt
```

On landing:

y = the support under it
velocity = 0
next step, a belt there picks it up again

h = 0.3 m at 1 m/s:
0.25 s in the air, lands 0.25 m out

Figure 1.7. Off the end of a belt: the item keeps its forward speed and falls with gravity, a projectile, until something holds it up.

Fixed steps. The display runs at whatever rate the machine manages; the simulation shouldn't care. So each frame's time is cut into steps of at most 1/120 s. Without that, a slow frame would make a falling item drop further in one step than in several, landing in a different place, and a fast item could jump over a short belt entirely. Games call this decoupling the simulation rate from the frame rate.

CONCEPTS · 1.7

Rules as data

A splitter's behaviour could have been code: an `if` per case. Instead it's **data**, a list of field/operator/value conditions that a small interpreter (`route`) evaluates. That one choice is what makes everything else possible:

- **Stored and shared.** Rules live in the database like any other record, so every browser and every API replica sees the same ones.
- **Editable without a deploy.** A person changes sorting in the panel, a script changes it through the API, an operator changes it with SQL. None of them touches the app's code.
- **Explainable.** Because the interpreter is tiny, it can also report *which* rule fired, and preview decisions before anything is saved.
- **Safe by construction.** A condition can only name a known field and an operator that suits it. There's no way to put arbitrary code in a rule, which is why rules from the API can be trusted to run in every browser.

This is the pattern behind firewall rules, email filters, feature flags and the routing tables of real sortation systems. The usual trade-off applies: a rule language is limited by design, and when people start wanting “or”, grouping and arithmetic, it's time to decide how far to grow it.

Rules that read **live** data add one more property: the decision depends on the world when the item arrives, not when the rule was written. Change a part's category in the inventory and its items take a different exit, with no rule edited. That's the behaviour asked for here, and also the thing to be careful with: the same rule can sort the same part differently on two runs, so the panel shows the last decision and why.

CHAPTER 1

Line by line

20 parts, in the order data flows through the code.

PART 1.1 · `src/arm/jointSpecs.ts`

The arm as data

Every number that describes the arm lives in one file: link lengths, joint limits, the home pose and the demo motion. The arm, the panel and the tests all read from here.

Joint specs

src/arm/jointSpecs.ts · lines 1–12

```ts
import type { ArmDimensions } from "./kinematics";

export type Axis = "x" | "y" | "z";

/** One rotary joint. Angles are in degrees. */
export interface JointSpec {
  name: string;
  axis: Axis;
  min: number;
  max: number;
  home: number;
}
```

`Axis` is a **string literal union**: the only values TypeScript accepts are `"x"`, `"y"` and `"z"`. That pays off in RobotArm.ts, where `rotation[spec.axis]` is type-checked: a typo like `"w"` is a compile error, not a silent no-op.

A `JointSpec` is pure data, in degrees, because degrees are what people read and type. Conversion to radians happens at exactly one place, where the angle is handed to Three.js.

Link lengths

src/arm/jointSpecs.ts · lines 14–22

```
/** Link lengths in meters (glTF and Three.js both treat 1 unit as 1 m). */
export const ARM: ArmDimensions = {
  baseHeight: 0.15,
  shoulderHeight: 0.2,
  upperArm: 0.45,
  forearm: 0.4,
  wrist: 0.12,
  tool: 0.115,
};
```

Meters, because glTF and Three.js both treat one unit as one meter, and keeping real units means numbers like “45 cm upper arm” can be checked against a spec sheet. The arm stretches to 1.085 m from the shoulder (0.45 + 0.4 + 0.12 + 0.115), roughly the size of a small collaborative robot.

`tool` is the distance from the wrist flange to the **tool center point** (TCP), the point between the fingertips that everything else in the project aims.

Joints and the home pose

src/arm/jointSpecs.ts · lines 24–33

```
/**
 * The arm is built from this list, so adding or re-limiting a joint is a data
 * change. Home pose: gripper pointing straight down (30 + 90 + 60 = 180 degrees).
 */
export const JOINTS: readonly JointSpec[] = [
  { name: "Base", axis: "y", min: -180, max: 180, home: 45 },
  { name: "Shoulder", axis: "z", min: -100, max: 100, home: 30 },
  { name: "Elbow", axis: "z", min: -140, max: 140, home: 90 },
  { name: "Wrist", axis: "z", min: -120, max: 120, home: 60 },
];
```

One yaw joint about Y (the base turns the whole arm), then three pitch joints about Z. The array is the **single source of truth**: RobotArm.ts builds one joint per entry, the panel builds one row per entry, and the keyboard handler accepts keys 1 to `JOINTS.length`.

The home angles are chosen so the three pitch angles add up to 180° (30 + 90 + 60). Pitch angles accumulate along a chain of parallel joints, so a total of 180° means the last link points straight down: the gripper is ready to pick something off the floor. A test checks exactly this.

`readonly JointSpec[]` stops any code from pushing to or reassigning entries of the array. It's a compile-time promise only, which is all that's needed.

The demo motion

src/arm/jointSpecs.ts · lines 35–56

```
/**
 * Demo motion: each joint sways around its home angle at its own frequency, so
 * the combined path never quite repeats. Amplitudes are chosen so the gripper
 * stays above the floor (checked in tests/kinematics.test.ts).
 */
const DEMO_MOTION = [
  { amplitude: 70, frequency: 0.07 },
  { amplitude: 20, frequency: 0.11 },
  { amplitude: 18, frequency: 0.13 },
  { amplitude: 30, frequency: 0.17 },
] as const;

/** Target angle in degrees for joint `index` at `time` seconds into the demo. */
export function demoTarget(index: number, time: number): number {
  const { amplitude, frequency } = DEMO_MOTION[index];
  return JOINTS[index].home + amplitude * Math.sin(2 * Math.PI * frequency * time + index);
}

/** Whether the gripper should be open at `time` seconds into the demo. */
export function demoGripperOpen(time: number): boolean {
  return Math.sin(2 * Math.PI * 0.15 * time) > 0;
}
```

Each joint sways around its home angle as a sine wave: `home + amplitude · sin(2π · frequency · t + index)`. The frequencies (0.07, 0.11, 0.13, 0.17 Hz) share no simple ratio, so the combined motion takes a very long time to repeat, and the `+ index` phase offset stops all joints from starting in step.

`as const` freezes the table's types to its literal values and makes it read-only. Amplitudes were tuned down after the floor-clearance test failed with the first values: the gripper dipped through the floor at certain moments. That test, and not eyeballing, is what decided them.

`demoGripperOpen` is a slow square wave: open for half of every 6.7 s cycle (0.15 Hz), closed for the other half.

PART 1.2 · `src/arm/kinematics.ts`

Forward kinematics by hand

Given the four joint angles, where is the tool? Three.js already knows, through its scene graph. This file works it out independently, with no Three.js at all, so the two answers can check each other.

Why the problem is small

src/arm/kinematics.ts · lines 1–11

```
/**
 * Forward kinematics for the arm, written by hand with no Three.js dependency.
 *
 * The arm is a base that yaws about Y, followed by three joints that all pitch
 * about their local Z axis. Because the pitch joints are parallel, everything
 * after the base yaw lies in one vertical plane, so FK reduces to summing link
 * vectors in that plane, then rotating the plane about Y by the base angle.
 *
 * main.ts compares this against Three.js's own scene-graph result every frame,
 * which is a good sanity check that the hierarchy is built the way we think.
 */
```

The comment states the insight that makes the code short. The three pitch joints are **parallel** (all rotate about their own Z axis), so every link after the base lies in one vertical plane. Forward kinematics (FK) becomes a 2D problem in that plane, followed by one rotation of the whole plane about the vertical axis by the base angle.

A general 6-axis arm needs a full chain of 4×4 matrices. That's what the test file builds as an independent reference.

Data shapes

src/arm/kinematics.ts · lines 13–32

```
export interface ArmDimensions {
  /** Floor to the base (yaw) joint. */
  baseHeight: number;
  /** Base joint to the shoulder joint. */
  shoulderHeight: number;
  /** Shoulder joint to elbow joint. */
  upperArm: number;
  /** Elbow joint to wrist joint. */
  forearm: number;
  /** Wrist joint to the tool flange. */
  wrist: number;
  /** Tool flange to the tool center point (between the gripper fingertips). */
  tool: number;
}

export interface Vec3 {
  x: number;
  y: number;
  z: number;
}
```

`ArmDimensions` documents each length with a doc comment, so editors show “Floor to the base (yaw) joint” on hover. `Vec3` is a plain structural type: Three.js's `Vector3` satisfies it, but so does `{ x: 1, y: 2, z: 3 }`, which keeps this module testable in plain Node with no WebGL.

The forward kinematics function

src/arm/kinematics.ts · lines 34–45

```
/**
 * World position of the tool center point.
 * @param q joint angles in radians: [base yaw, shoulder, elbow, wrist]
 */
export function forwardKinematics(d: ArmDimensions, q: readonly number[]): Vec3 {
  const [base = 0, shoulder = 0, elbow = 0, wrist = 0] = q;

  // Each pitch joint adds to the angle of every link after it.
  const a1 = shoulder;
  const a2 = shoulder + elbow;
  const a3 = shoulder + elbow + wrist;
  const distal = d.wrist + d.tool;
```

Destructuring with defaults (`base = 0`) means a short array still works. Then the key step: the **absolute** angle of each link from vertical is the sum of every pitch joint before it. Tilt the shoulder 30° and the elbow 90°, and the forearm is 120° from vertical. `distal` treats the wrist link and tool as one straight segment, since nothing bends between them.

Summing link vectors

src/arm/kinematics.ts · lines 47–54

```
// Rotating (0, L, 0) about Z by a gives (-L sin a, L cos a, 0).
const r = -(d.upperArm * Math.sin(a1) + d.forearm * Math.sin(a2) + distal * Math.sin(a3));
const y =
  d.baseHeight +
  d.shoulderHeight +
  d.upperArm * Math.cos(a1) +
  d.forearm * Math.cos(a2) +
  distal * Math.cos(a3);
```

In the arm's plane, a link of length L at angle a from vertical contributes (−L sin a, L cos a). The minus sign is the arm's convention: a positive rotation about Z tips +Y toward −X, so the arm leans toward −X. `r` is the signed horizontal distance from the base axis to the TCP and `y` is its height.

Height starts from the shoulder joint (`baseHeight + shoulderHeight`), since the column below it never tilts.

Turning the plane

src/arm/kinematics.ts · lines 56–58

```
  // Rotating (r, y, 0) about Y by `base` gives (r cos b, y, -r sin b).
  return { x: r * Math.cos(base), y, z: -r * Math.sin(base) };
}
```

Rotating a point (r, y, 0) about the vertical axis by angle b gives (r cos b, y, −r sin b). Height doesn't change, since yaw only spins the plane. The sign on z is again the right-hand rule: positive rotation about +Y turns +X toward −Z.

`main.ts` calls this every frame and the panel shows its answer next to Three.js's, with the difference: around 10^{-17} m, which is floating-point noise. If the hierarchy were built wrong, that number would jump to centimeters immediately.

Small helpers

src/arm/kinematics.ts · lines 60–69

```
/** Distance from the shoulder joint to the TCP with the arm fully stretched. */
export function maxReach(d: ArmDimensions): number {
  return d.upperArm + d.forearm + d.wrist + d.tool;
}

export function clamp(value: number, min: number, max: number): number {
  return Math.min(max, Math.max(min, value));
}

export const DEG = Math.PI / 180;
```

`maxReach` is the stretched-out length from the shoulder, used by the tests as an upper bound. `clamp` keeps a value within limits; `Math.min(max, Math.max(min, value))` reads inside out: first raise to at least `min`, then lower to at most `max`. `DEG` converts degrees to radians when multiplied (and radians to degrees when divided).

GOING DEEPER

The Jacobian, and singularities

The next step after FK is velocity. The **Jacobian** J is the matrix of partial derivatives ∂(tool position)/∂(joint angle): 3 rows by 4 columns here. Multiply it by joint speeds and you get the tool's speed. Real controllers invert it to execute straight-line moves (joint speeds = J^{-1} × desired tool speed), and numerical IK solvers are built on it.

Where J loses rank, the arm is at a **singularity**: some tool direction can't be produced at any joint speed. For this arm the obvious one is full stretch, where the tool can't move further outward. Near a singularity, a straight-line move demands enormous joint speeds, which is why industrial controllers slow down or refuse near them.

PART 1.3 · `src/arm/RobotArm.ts`

The arm as a scene graph

The arm is a chain of nested Three.js objects. Rotating one joint moves everything below it, so there's no kinematics code here at all: the hierarchy is the kinematics.

The hierarchy

src/arm/RobotArm.ts · lines 8–23

```
/**
 * The arm as a Three.js scene-graph hierarchy:
 *
 *   root
 *   ├── pedestal
 *   └── Joint1 (yaw, Y)        ← turret, column
 *       └── Joint2 (pitch, Z)  ← shoulder housing, upper arm
 *           └── Joint3 (pitch, Z)  ← elbow housing, forearm
 *               └── Joint4 (pitch, Z)  ← wrist housing, wrist link
 *                   └── toolMount  ← gripper (glTF) attaches here
 *                       └── tcp    ← tool center point
 *
 * Each joint is an empty Object3D placed at the end of its parent link. Rotating
 * a joint moves everything below it, exactly like parenting Transforms in Unity.
 * Link meshes are offset by half their length so they extend from the joint.
 */
```

Each joint is an empty `Object3D`, a transform with no geometry, placed at the end of its parent link. The visible meshes hang off the joints. This separation is the main design choice: rotation lives on the joint, shape lives on the mesh, and a mesh can be offset or swapped without touching the motion.

Unity developers will recognise it as parenting empty GameObjects. Three.js computes each object's world transform as its parent's world transform times its own local one, all the way up the tree.

Fields

src/arm/RobotArm.ts · lines 24–34

```
export class RobotArm {
  readonly root = new THREE.Group();
  readonly joints: THREE.Object3D[] = [];
  /** Clickable joint housings; userData.jointIndex says which joint each belongs to. */
  readonly housings: THREE.Mesh<THREE.BufferGeometry, THREE.MeshStandardMaterial>[] = [];
  readonly toolMount = new THREE.Object3D();
  readonly tcp = new THREE.Object3D();

  private readonly dims: ArmDimensions;
  private readonly specs: readonly JointSpec[];
  private readonly angles: number[]; // degrees
```

`root` is what main.ts adds to the scene. `joints` is indexed the same way as `JOINTS`, so `joints[2]` is the elbow. `housings` are the blue cylinders the user can click. `toolMount` is where the gripper attaches, and `tcp` marks the tool center point.

`angles` stores degrees as the source of truth. Reading an angle back from `rotation.z` would work, but would give radians and float round-trip noise; keeping the commanded value avoids both.

Constructor

src/arm/RobotArm.ts · lines 36–42

```
constructor(dims: ArmDimensions, specs: readonly JointSpec[]) {
  this.dims = dims;
  this.specs = specs;
  this.angles = specs.map((s) => s.home);
  this.build();
  specs.forEach((s, i) => this.setAngle(i, s.home));
}
```

Build the hierarchy once, then apply the home pose through `setAngle`, so the same clamping and conversion run as for any later change. There is only one code path that rotates a joint.

Setting an angle

src/arm/RobotArm.ts · lines 44–51

```
/** Sets joint `index` in degrees, clamped to its limits. Returns the applied angle. */
setAngle(index: number, degrees: number): number {
  const spec = this.specs[index];
  const value = clamp(degrees, spec.min, spec.max);
  this.angles[index] = value;
  this.joints[index].rotation[spec.axis] = value * DEG;
  return value;
}
```

Clamp to the joint's limits, remember the value, then set the rotation on the joint's axis in radians. `rotation[spec.axis]` is computed property access: for the base it's `rotation.y`, for the others `rotation.z`. Returning the applied angle lets a caller see that a request was clamped.

Setting `rotation` doesn't recompute any matrices. Three.js does that lazily, when the scene is rendered or when someone asks for a world position.

Reading angles and highlighting

src/arm/RobotArm.ts · lines 53–70

```
getAngle(index: number): number {
  return this.angles[index];
}

get anglesDeg(): readonly number[] {
  return this.angles;
}

anglesRad(): number[] {
  return this.angles.map((a) => a * DEG);
}

/** Makes the selected joint's housing glow; pass null to clear. */
highlight(index: number | null): void {
  this.housings.forEach((h, i) => {
    h.material.emissive.setHex(i === index ? SELECTED_GLOW : 0x000000);
  });
}
```

`anglesDeg` returns the internal array typed as `readonly`, so the panel can read it every frame without a copy. `anglesRad` makes a new array, which is fine because it's only used for the FK comparison.

Highlighting sets the material's **emissive** color, light the surface gives off regardless of lighting, so the selected housing glows. That only works because each housing has its own material (see `addHousing`).

Building the chain

src/arm/RobotArm.ts · lines 72–110

```
private build(): void {
  const d = this.dims;
  const linkMat = new THREE.MeshStandardMaterial({ color: 0xc9ced6, metalness: 0.7, roughness:
0.35 });
  const baseMat = new THREE.MeshStandardMaterial({ color: 0x2a2e35, metalness: 0.4, roughness:
0.6 });

  const pedestal = solid(new THREE.CylinderGeometry(0.16, 0.2, d.baseHeight, 48), baseMat);
  pedestal.position.y = d.baseHeight / 2;
```

```
  this.root.add(pedestal);

  // Joint 1: base yaw
  const j1 = this.addJoint(this.root, d.baseHeight);
  const turret = this.addHousing(j1, new THREE.CylinderGeometry(0.13, 0.13, 0.06, 48));
  turret.position.y = 0.03;
  j1.add(along(solid(new THREE.CylinderGeometry(0.07, 0.07, d.shoulderHeight, 32), linkMat),
d.shoulderHeight));

  // Joint 2: shoulder pitch
  const j2 = this.addJoint(j1, d.shoulderHeight);
  this.addHousing(j2, zCylinder(0.085, 0.17));
  j2.add(along(solid(new THREE.BoxGeometry(0.09, d.upperArm, 0.09), linkMat), d.upperArm));

  // Joint 3: elbow pitch
  const j3 = this.addJoint(j2, d.upperArm);
  this.addHousing(j3, zCylinder(0.07, 0.15));
  j3.add(along(solid(new THREE.BoxGeometry(0.075, d.forearm, 0.075), linkMat), d.forearm));

  // Joint 4: wrist pitch
  const j4 = this.addJoint(j3, d.forearm);
  this.addHousing(j4, zCylinder(0.055, 0.12));
  j4.add(along(solid(new THREE.CylinderGeometry(0.04, 0.04, d.wrist, 32), linkMat), d.wrist));

  this.toolMount.name = "ToolMount";
  this.toolMount.position.y = d.wrist;
  j4.add(this.toolMount);

  this.tcp.name = "TCP";
  this.tcp.position.y = d.tool;
  this.tcp.add(new THREE.AxesHelper(0.06));
  this.toolMount.add(this.tcp);
}
```

Read it as the arm, bottom to top. The pedestal is a tapered cylinder centered at half its height so it sits on the floor. Each joint is created at the end of the previous link (`addJoint(parent, offsetY)`), gets a housing, and then gets its link mesh, which `along()` shifts up by half its length so it runs from the joint to the next one.

Every link points along its joint's local +Y. That's why FK can use (0, L, 0) for every link: all the geometry is “straight up” in local space, and only the rotations differ.

`toolMount` sits at the end of the wrist link and `tcp` a further `d.tool` beyond it. An `AxesHelper` on the TCP draws small red, green and blue axis lines, which is how you can see the tool's orientation in the screenshot.

Adding joints and housings

src/arm/RobotArm.ts · lines 112–135

```
private addJoint(parent: THREE.Object3D, offsetY: number): THREE.Object3D {
  const index = this.joints.length;
  const joint = new THREE.Object3D();
  joint.name = `Joint${index + 1}_${this.specs[index].name}`;
  joint.position.y = offsetY;
  parent.add(joint);
  this.joints.push(joint);
  return joint;
}

private addHousing(joint: THREE.Object3D, geometry: THREE.BufferGeometry) {
  // Each housing gets its own material so it can be highlighted on its own.
  const material = new THREE.MeshStandardMaterial({
    color: HOUSING_COLOR,
    metalness: 0.3,
    roughness: 0.45,
    emissiveIntensity: 0.6,
  });
  const housing = solid(geometry, material);
  housing.userData.jointIndex = this.housings.length;
  joint.add(housing);
  this.housings.push(housing);
  return housing;
}
```

`addJoint` names each joint (`Joint2_Shoulder`), which makes the hierarchy readable in Three.js's devtools, and pushes it so `joints[i]` lines up with `specs[i]`.

`addHousing` stores the joint index in `userData`, Three.js's free-form slot for app data on any object. When a click ray hits a housing, main.ts reads `userData.jointIndex` to know which joint was clicked. Each housing gets its **own** material, since a shared material would make all of them glow at once.

Mesh helpers

src/arm/RobotArm.ts · lines 138–154

```
function solid<M extends THREE.Material>(geometry: THREE.BufferGeometry, material: M) {
  const mesh = new THREE.Mesh(geometry, material);
  mesh.castShadow = true;
  mesh.receiveShadow = true;
  return mesh;
}

/** Shifts a link mesh so it runs from its joint (y = 0) to y = length. */
function along<T extends THREE.Object3D>(mesh: T, length: number): T {
  mesh.position.y = length / 2;
  return mesh;
}

/** Cylinder whose axis is Z, matching a pitch joint's rotation axis. */
function zCylinder(radius: number, length: number): THREE.BufferGeometry {
  return new THREE.CylinderGeometry(radius, radius, length, 40).rotateX(Math.PI / 2);
}
```

`solid` turns on shadows for every part. `along` shifts a mesh's center up by half its length. `zCylinder` exists because Three.js cylinders stand along Y, but a pitch joint's housing should lie along its rotation axis, Z: rotating the geometry 90° about X lays it down. `rotateX` on a geometry changes the vertices once, so the mesh itself keeps an identity rotation.

The generic `<M extends THREE.Material>` keeps the precise material type, so `housings` can be typed as meshes with a `MeshStandardMaterial`, and `material.emissive` type-checks.

GOING DEEPER

When world matrices are actually computed

Setting `rotation.z` only stores a number. During `renderer.render()`, Three.js calls `scene.updateMatrixWorld()`, which walks the tree: each object rebuilds `matrix` from position, quaternion and scale (if `matrixAutoUpdate` is on, the default) and multiplies by its parent's world matrix. For a 20-node arm that's nothing; for 50,000 static objects it adds up, which is why static scenery often sets `matrixAutoUpdate = false` and updates by hand.

`getWorldPosition` calls `updateWorldMatrix(true, false)`: update the parents, then itself, but not the children. That's why main.ts can read the TCP right after changing angles and get the right answer.

PART 1.4 · `src/arm/Gripper.ts`

Loading and driving the gripper

The gripper is a glTF file (built by the script at the end of this chapter). This class loads it, finds the fingers by name, and slides them.

Constants and fields

src/arm/Gripper.ts · lines 1–25

```
import * as THREE from "three";
import { GLTFLoader } from "three/addons/loaders/GLTFLoader.js";

const CLOSED_X = 0.025;
const OPEN_X = 0.07;
const SPEED = 2.5; // full open/close strokes per second

/**
 * Parallel two-finger gripper loaded from glTF. The model's finger nodes are
 * found by name and slid along X, so the asset stays a plain static file with
 * no animation clips baked in.
 */
export class Gripper {
  readonly object: THREE.Object3D;
  private readonly left: THREE.Object3D;
  private readonly right: THREE.Object3D;
  private openness = 1; // 0 = closed, 1 = open
  private target = 1;

  private constructor(object: THREE.Object3D, left: THREE.Object3D, right: THREE.Object3D) {
    this.object = object;
    this.left = left;
    this.right = right;
    this.apply();
  }
```

The fingers sit 2.5 cm from the center when closed and 7 cm when open. `openness` is the animated value from 0 to 1, `target` is where it's heading. The constructor is **private**, so the only ways to get a `Gripper` are the two static factories below, each of which guarantees both finger nodes exist.

Loading the model

src/arm/Gripper.ts · lines 27–41

```
static async load(url: string): Promise<Gripper> {
  const gltf = await new GLTFLoader().loadAsync(url);
  const root = gltf.scene;
  const left = root.getObjectByName("FingerLeft");
  const right = root.getObjectByName("FingerRight");
  if (!left || !right) throw new Error(`${url} has no FingerLeft/FingerRight nodes`);

  root.traverse((child) => {
    if (child instanceof THREE.Mesh) {
      child.castShadow = true;
      child.receiveShadow = true;
    }
  });
  return new Gripper(root, left, right);
}
```

`loadAsync` fetches and parses the `.glb` and returns a promise. `gltf.scene` is an ordinary Three.js `Group` built from the file's nodes. `getObjectByName` searches the tree for the two node names the build script wrote; if they're missing, loading fails loudly instead of producing a gripper that silently doesn't move.

`traverse` visits every descendant. glTF has no notion of shadows, so they're switched on here for each mesh.

A fallback

src/arm/Gripper.ts · lines 43–61

```
/** Simple boxes, used if the glTF file can't be loaded. */
static placeholder(): Gripper {
  const material = new THREE.MeshStandardMaterial({ color: 0x888888 });
  const root = new THREE.Group();
  const palm = new THREE.Mesh(new THREE.BoxGeometry(0.18, 0.06, 0.07), material);
  palm.position.y = 0.03;
  const finger = () => {
    const pivot = new THREE.Object3D();
    pivot.position.y = 0.06;
    const mesh = new THREE.Mesh(new THREE.BoxGeometry(0.02, 0.07, 0.05), material);
    mesh.position.y = 0.035;
    pivot.add(mesh);
    return pivot;
  };
  const left = finger();
  const right = finger();
  root.add(palm, left, right);
  return new Gripper(root, left, right);
}
```

If the file can't be loaded (wrong path, blocked request), main.ts uses this instead: grey boxes with the same structure, a pivot 6 cm up for each finger. Because the class only ever moves the two pivots, everything else works unchanged. A missing asset degrades the look, not the app.

Opening, closing and animating

src/arm/Gripper.ts · lines 63–86

```
  get isOpen(): boolean {
    return this.target === 1;
  }

  setOpen(open: boolean): void {
    this.target = open ? 1 : 0;
  }

  update(dt: number): void {
    if (this.openness === this.target) return;
    const step = SPEED * dt;
    this.openness =
      this.openness < this.target
        ? Math.min(this.target, this.openness + step)
        : Math.max(this.target, this.openness - step);
    this.apply();
  }

  private apply(): void {
    const x = CLOSED_X + (OPEN_X - CLOSED_X) * this.openness;
    this.left.position.x = -x;
    this.right.position.x = x;
  }
}
```

`setOpen` only changes the target, so callers never deal with animation. `update(dt)` moves `openness` toward the target at 2.5 strokes per second, using `Math.min` / `Math.max` so it lands exactly on the target instead of overshooting. Returning early when there's nothing to do keeps the per-frame cost near zero.

`apply` maps openness to a distance and moves the fingers symmetrically: left to −x, right to +x. The fingers are animated in code rather than with glTF animation clips, which keeps the asset a plain static file.

PART 1.5 · `src/scene/stage.ts`

The stage

Everything except the arm: renderer, camera, orbit controls, lights and floor.

Renderer

src/scene/stage.ts · lines 5–13

```
export function createStage(canvas: HTMLCanvasElement) {
  const renderer = new THREE.WebGLRenderer({ canvas, antialias: true });
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
  renderer.toneMapping = THREE.ACESFilmicToneMapping;
  renderer.shadowMap.enabled = true;
  renderer.shadowMap.type = THREE.PCFSoftShadowMap;

  const scene = new THREE.Scene();
  scene.background = new THREE.Color(0x15181d);
```

`setPixelRatio` renders at the screen's real resolution on high-DPI displays, capped at 2× because 3× costs 2.25 times the pixels for a barely visible gain. **ACES filmic tone mapping** compresses bright highlights the way film does, instead of clipping them to white. `PCFSoftShadowMap` blurs shadow edges.

Camera and orbit controls

src/scene/stage.ts · lines 15–24

```
const camera = new THREE.PerspectiveCamera(45, 1, 0.01, 50);
camera.position.set(1.3, 1.0, 1.6);

const controls = new OrbitControls(camera, canvas);
controls.target.set(0, 0.4, 0);
controls.enableDamping = true;
controls.minDistance = 0.5;
controls.maxDistance = 5;
controls.maxPolarAngle = Math.PI * 0.49; // stay above the floor
controls.update();
```

A 45° field of view, with near and far clip planes at 1 cm and 50 m. Keeping the near plane as far out as possible matters: depth precision is spread between near and far, and a tiny near plane causes flickering surfaces.

`OrbitControls` lets the user drag to orbit around `target` (40 cm up, roughly the arm's middle). Damping gives the motion inertia, which requires `controls.update()` every frame. `maxPolarAngle` just under 90° stops the camera going below the floor.

Lights and shadows

src/scene/stage.ts · lines 26–35

```
scene.add(new THREE.HemisphereLight(0xdfe8ff, 0x2a2622, 1.2));

const sun = new THREE.DirectionalLight(0xffffff, 2.2);
sun.position.set(2, 4, 2.5);
sun.castShadow = true;
sun.shadow.mapSize.set(2048, 2048);
Object.assign(sun.shadow.camera, { left: -1.5, right: 1.5, top: 1.5, bottom: -1.5, near: 0.5,
far: 10 });
sun.shadow.camera.updateProjectionMatrix();
sun.shadow.normalBias = 0.02;
scene.add(sun);
```

A **hemisphere light** gives cool light from above and warm bounce from below, a cheap stand-in for sky and ground. The directional “sun” casts the shadows. Its shadow camera is an orthographic box (±1.5 m) that only needs to cover the arm's reach: the smaller the box, the sharper the 2048×2048 shadow map. `normalBias` nudges the lookup along the surface normal to avoid “shadow acne”, stripes caused by a surface shadowing itself.

Floor, grid and resizing

src/scene/stage.ts · lines 37–60

```
  // An invisible floor that only shows shadows, plus a grid at 10 cm spacing.
  const floor = new THREE.Mesh(
    new THREE.PlaneGeometry(10, 10).rotateX(-Math.PI / 2),
    new THREE.ShadowMaterial({ opacity: 0.35 }),
  );
  floor.receiveShadow = true;
  scene.add(floor);

  const grid = new THREE.GridHelper(4, 40, 0x3a414d, 0x262b33);
  grid.position.y = 0.001; // avoid z-fighting with the shadow plane
  scene.add(grid);

  function resize() {
    const width = canvas.clientWidth;
    const height = canvas.clientHeight;
    renderer.setSize(width, height, false);
    camera.aspect = width / height;
    camera.updateProjectionMatrix();
  }
  window.addEventListener("resize", resize);
  resize();

  return { renderer, scene, camera, controls };
}
```

The floor is a `ShadowMaterial` plane, invisible except where shadows fall on it, with a grid drawn 1 mm above to avoid **z-fighting** (two surfaces at the same depth flickering as they win the depth test on alternate pixels).

`resize` reads the canvas's CSS size and sets the drawing buffer to match. Passing `false` stops Three.js from overwriting the canvas's CSS size. The camera's aspect ratio must match, followed by `updateProjectionMatrix`, or the scene stretches.

PART 1.6 · `src/scene/Trail.ts`

The tool path trail

A line tracing where the TCP has been. Small, but a good example of writing per-frame code that never allocates.

A fixed buffer

src/scene/Trail.ts · lines 3–32

```
/**
 * A line tracing the recent path of a point. The vertex buffer is allocated once
 * and reused, so adding points never allocates or creates new GPU buffers.
 */
export class Trail {
  readonly line: THREE.Line;
  private readonly positions: Float32Array;
  private readonly attribute: THREE.BufferAttribute;
  private readonly capacity: number;
  private count = 0;
  private readonly last = new THREE.Vector3(Infinity, Infinity, Infinity);

  constructor(capacity = 800) {
    this.capacity = capacity;
    this.positions = new Float32Array(capacity * 3);
    this.attribute = new THREE.BufferAttribute(this.positions, 3);
    this.attribute.setUsage(THREE.DynamicDrawUsage);

    const geometry = new THREE.BufferGeometry();
    geometry.setAttribute("position", this.attribute);
    geometry.setDrawRange(0, 0);

    this.line = new THREE.Line(
      geometry,
      new THREE.LineBasicMaterial({ color: 0xffa94d, transparent: true, opacity: 0.85 }),
    );
    // The bounding sphere would be computed once from the empty buffer and then
    // cull the line wrongly, so skip frustum culling for this one object.
    this.line.frustumCulled = false;
  }
```

The positions live in one `Float32Array` sized for 800 points, allocated once. `DynamicDrawUsage` tells WebGL the data will change often. `setDrawRange(0, 0)` means draw nothing yet: the buffer is full size, but only the first `count` points are used.

Frustum culling skips objects outside the camera's view using a bounding sphere. Three.js computes that sphere once, here from an empty buffer, so the trail would be culled as soon as it grew. Turning culling off for this one object is simpler than recomputing the sphere.

Adding a point

src/scene/Trail.ts · lines 34–47

```
push(point: THREE.Vector3): void {
  if (this.last.distanceToSquared(point) < 1e-6) return; // moved less than 1 mm
  this.last.copy(point);

  if (this.count === this.capacity) {
    this.positions.copyWithin(0, 3); // drop the oldest point
    this.count--;
  }
  point.toArray(this.positions, this.count * 3);
  this.count++;

  this.attribute.needsUpdate = true;
  this.line.geometry.setDrawRange(0, this.count);
}
```

Points closer than 1 mm to the last one are skipped (comparing squared distance avoids a square root). When the buffer is full, `copyWithin(0, 3)` shifts every value back by one point, dropping the oldest. That's 2,400 floats moved, a few microseconds, and it keeps the line in order for drawing. A ring buffer would avoid the copy but would need two draw ranges.

`point.toArray(positions, offset)` writes x, y, z straight into the buffer. `needsUpdate` tells Three.js to upload it to the GPU before the next draw.

Clearing and visibility

src/scene/Trail.ts · lines 49–58

```
  clear(): void {
    this.count = 0;
    this.last.set(Infinity, Infinity, Infinity);
    this.line.geometry.setDrawRange(0, 0);
  }

  set visible(on: boolean) {
    this.line.visible = on;
  }
}
```

Clearing doesn't zero the buffer, it just sets the count to 0: old values are overwritten as new points arrive. A setter (`set visible`) lets main.ts write `trail.visible = on` like a plain property.

GOING DEEPER

Why per-frame allocation matters

JavaScript frees memory with a garbage collector. Short-lived objects are cheap to collect, but not free: allocate a few `Vector3`s per object per frame and, at 60 fps, the collector runs often enough to show up as occasional dropped frames. The habits in this project avoid that: reuse scratch objects (`tcpWorld` in main.ts), preallocate typed arrays (the trail), and keep closures out of hot loops. Snack 3 takes the same idea to its conclusion with structure-of-arrays physics.

PART 1.7 · `src/ui/panel.ts`

The side panel

Plain DOM, no framework. The panel only displays state and reports user intent through handlers; it never touches the arm.

Handlers and helpers

src/ui/panel.ts · lines 4–32

```
export type JogDirection = -1 | 0 | 1;
```

```
export interface PanelHandlers {
  onSelect(index: number): void;
  onJog(direction: JogDirection): void;
  onToggleDemo(): void;
  onToggleGripper(): void;
  onToggleTrail(): void;
  onReset(): void;
}

function byId<T extends HTMLElement = HTMLElement>(id: string): T {
  const el = document.getElementById(id);
  if (!el) throw new Error(`Missing #${id} in index.html`);
  return el as T;
}

/** Writes only when the text changed, to avoid needless DOM work every frame. */
function setText(el: HTMLElement, text: string) {
  if (el.textContent !== text) el.textContent = text;
}

function formatDeg(value: number) {
  return `${(Math.abs(value) < 0.05 ? 0 : value).toFixed(1)}°`;
}

function formatMeters(value: number) {
  return (Math.abs(value) < 0.0005 ? 0 : value).toFixed(3);
}
```

`PanelHandlers` is the panel's whole contract with the app: six callbacks. main.ts decides what they do. `byId` fails loudly if index.html is missing an element, instead of a confusing `null` error later.

`setText` only writes when the text changed. `update` runs every frame, and writing the same text 60 times a second would still make the browser do layout work. `formatDeg` and `formatMeters` snap tiny values to 0, so the display never shows “−0.0°”.

Building the joint list

src/ui/panel.ts · lines 35–55

```
export function createPanel(specs: readonly JointSpec[], handlers: PanelHandlers) {
  const list = byId("joints");
  const rows = specs.map((spec, i) => {
    const button = document.createElement("button");
    button.type = "button";
    button.className = "joint";
    button.innerHTML = `
      <span class="key">${i + 1}</span>
      <span class="name">${spec.name}</span>
      <span class="value"></span>
      <span class="bar" title="${spec.min}° to ${spec.max}°"><span class="marker"></span>
</span>`;
    button.addEventListener("click", () => handlers.onSelect(i));
    const item = document.createElement("li");
    item.append(button);
    list.append(item);
    return {
      button,
      value: button.querySelector<HTMLElement>(".value")!,
      marker: button.querySelector<HTMLElement>(".marker")!,
    };
  });
```

One button per joint spec, with its key number, name, live value and a bar whose `title` shows the limits on hover. Each row keeps references to the elements it will update, so the per-frame code never queries the DOM.

Press-and-hold jog buttons

src/ui/panel.ts · lines 57–67

```
for (const button of document.querySelectorAll<HTMLButtonElement>("[data-jog]")) {
  const direction = Number(button.dataset.jog) as JogDirection;
  button.addEventListener("pointerdown", (e) => {
    button.setPointerCapture(e.pointerId);
    handlers.onJog(direction);
  });
  const stop = () => handlers.onJog(0);
  button.addEventListener("pointerup", stop);
  button.addEventListener("pointercancel", stop);
  button.addEventListener("lostpointercapture", stop);
}
```

Jogging must last exactly as long as the button is held. `setPointerCapture` keeps sending pointer events to the button even if the pointer slides off it, so the release is never missed. Stopping on `pointerup`, `pointercancel` and `lostpointercapture` covers every way a press can end, including touch interruptions.

Updating every frame

src/ui/panel.ts · lines 83–98

```
return {
  update(anglesDeg: readonly number[], scenePos: Vec3, fkPos: Vec3) {
    rows.forEach((row, i) => {
      const spec = specs[i];
      setText(row.value, formatDeg(anglesDeg[i]));
      row.marker.style.left = `${((anglesDeg[i] - spec.min) / (spec.max - spec.min)) * 100}%`;
    });
    const s = [scenePos.x, scenePos.y, scenePos.z];
    const f = [fkPos.x, fkPos.y, fkPos.z];
    for (let k = 0; k < 3; k++) {
      setText(scene[k], formatMeters(s[k]));
      setText(fk[k], formatMeters(f[k]));
    }
    const error = Math.hypot(s[0] - f[0], s[1] - f[1], s[2] - f[2]);
    setText(diff, `${error.toExponential(1)} m`);
  },
```

Sets each joint's value and moves its marker along the bar as a percentage of the joint's range. Then the TCP readout: both positions and the distance between them, in exponent notation, since it's always around 10^{-17}.

Toggles

src/ui/panel.ts · lines 100–122

```
  setSelected(index: number) {
    rows.forEach((row, i) => row.button.classList.toggle("selected", i === index));
    setText(selectedName, specs[index].name);
  },

  setDemo(on: boolean) {
    demo.setAttribute("aria-pressed", String(on));
  },

  setGripperOpen(open: boolean) {
    gripper.setAttribute("aria-pressed", String(open));
    setText(gripper, open ? "Gripper open" : "Gripper closed");
  },

  setTrail(on: boolean) {
    trail.setAttribute("aria-pressed", String(on));
  },

  showStatus(message: string) {
    status.textContent = message;
    status.hidden = false;
  },
};
```

Toggle buttons use `aria-pressed`, which screen readers announce as “pressed” or “not pressed”, and the CSS styles the same attribute. One attribute serves accessibility and appearance, so they can't disagree.

PART 1.8 · `src/main.ts`

Wiring it together

The entry point: builds the scene, connects input, and runs the frame loop. All app state lives here.

Setup and state

src/main.ts · lines 13–65

```
const JOG_SPEED = 60; // degrees per second while a key or jog button is held
const DEMO_SMOOTHING = 3; // higher = joints follow the demo path more tightly

const canvas = document.getElementById("scene") as HTMLCanvasElement;
const { renderer, scene, camera, controls } = createStage(canvas);

const arm = new RobotArm(ARM, JOINTS);
scene.add(arm.root);

const trail = new Trail();
scene.add(trail.line);

let gripper: Gripper | null = null;

// Shapes on the floor, stored in the inventory database (Snack 5's API).
const shapes = createShapes({ root: document.getElementById("shapes-panel")!, scene, camera,
canvas, arm: ARM });

// Conveyor belts, also stored there. Shapes on a running belt ride along.
const belts = createConveyors({
  root: document.getElementById("conveyors-panel")!,
  runButton: document.getElementById("btn-run") as HTMLButtonElement,
  scene, camera, canvas, shapes,
});

// Splitters: gates on the belts that send each item left, right or straight on, by your rules.
const splitters = createSplitters({ root: document.getElementById("splitters-panel")!, scene,
camera, canvas, shapes, conveyors: belts });

// The right-hand panel's tabs.
const tabs = {
  shapes: document.getElementById("tab-shapes")!,
  conveyors: document.getElementById("tab-conveyors")!,
  splitters: document.getElementById("tab-splitters")!,
};
function showTab(name: keyof typeof tabs) {
  for (const [key, tab] of Object.entries(tabs)) {
    const on = key === name;
```

```
    tab.setAttribute("aria-selected", String(on));
    document.getElementById(tab.getAttribute("aria-controls")!)!.hidden = !on;
  }
}
tabs.shapes.addEventListener("click", () => showTab("shapes"));
tabs.conveyors.addEventListener("click", () => showTab("conveyors"));
tabs.splitters.addEventListener("click", () => showTab("splitters"));
belts.onShowTab(() => showTab("conveyors"));
splitters.onShowTab(() => showTab("splitters"));

const state = {
  selected: 1,
  demo: true,
  demoTime: 0,
  trail: true,
  jog: 0 as JogDirection,
};
```

`JOG_SPEED` is 60°/s while a key is held. The arm and the trail are added to the scene immediately; the gripper is `null` until its file loads, which is why later code says `gripper?.update(dt)`.

The Shapes panel and the Conveyors tab are created here too, with the scene, camera and canvas they need, and the two tabs that switch between them (all covered in the last parts of this chapter). Everything about shapes and belts lives in their own modules; main.ts only hands them the pointer, the keyboard and the frame loop.

`state` is one plain object for everything the UI can change. `0 as JogDirection` tells TypeScript the field holds the type −1 | 0 | 1, not just the literal 0.

Connecting the panel

src/main.ts · lines 67–77

```
const panel = createPanel(JOINTS, {
  onSelect: select,
  onJog: (direction) => {
    state.jog = direction;
    if (direction !== 0) setDemo(false);
  },
  onToggleDemo: () => setDemo(!state.demo),
  onToggleGripper: toggleGripper,
  onToggleTrail: () => setTrail(!state.trail),
  onReset: reset,
});
```

Each handler is one line. Jogging from the panel stops the demo, because the demo would otherwise overwrite the angle on the very next frame. `select` and `reset` are function declarations further down, which JavaScript hoists, so they can be referenced here.

Loading the gripper

src/main.ts · lines 79–91

```
// ---- Gripper (glTF) ----

Gripper.load(`${import.meta.env.BASE_URL}models/gripper.glb`)
  .catch((err: unknown) => {
    console.error(err);
    panel.showStatus("Couldn't load models/gripper.glb, so a placeholder gripper is shown.");
    return Gripper.placeholder();
  })
  .then((loaded) => {
    gripper = loaded;
    arm.toolMount.add(loaded.object);
    panel.setGripperOpen(loaded.isOpen);
  });
```

A promise chain: try the real model, and on any failure log it, tell the user, and fall back to the placeholder. Either way `.then` receives a working `Gripper` and mounts it on the arm's tool mount, so it moves with the wrist.

`import.meta.env.BASE_URL` is Vite's base path, so the URL is right whether the app runs at the site root or in a subfolder.

Actions

src/main.ts · lines 95–129

```
function select(index: number) {
  state.selected = (index + JOINTS.length) % JOINTS.length;
  arm.highlight(state.selected);
  panel.setSelected(state.selected);
}

function setDemo(on: boolean) {
  state.demo = on;
  panel.setDemo(on);
}

function toggleGripper() {
  setDemo(false); // otherwise the demo would immediately override it
  setGripperOpen(!(gripper?.isOpen ?? true));
}

function setGripperOpen(open: boolean) {
  if (!gripper || gripper.isOpen === open) return;
  gripper.setOpen(open);
  panel.setGripperOpen(open);
}

function setTrail(on: boolean) {
  state.trail = on;
  trail.visible = on;
  if (!on) trail.clear();
  panel.setTrail(on);
}

function reset() {
  setDemo(false);
  JOINTS.forEach((spec, i) => arm.setAngle(i, spec.home));
  setGripperOpen(true);
  trail.clear();
}
```

Every user action goes through one of these functions, whether it came from a key, a button or a click on the arm. Each updates `state`, the 3D scene and the panel together, so they can't drift apart.

`(index + JOINTS.length) % JOINTS.length` wraps around in both directions: pressing up on the first joint selects the last. Adding the length first matters, because in JavaScript `-1 % 4` is −1, not 3.

Keyboard

src/main.ts · lines 133–169

```
const held = new Set<string>();

window.addEventListener("keydown", (e) => {
  // Typing a shape's name or size must not move the arm (R would reset it, G open the
gripper…).
  const t = e.target;
  if (t instanceof HTMLInputElement || t instanceof HTMLSelectElement || t instanceof
HTMLTextAreaElement) return;
  if (splitters.handleKey(e) || belts.handleKey(e) || shapes.handleKey(e)) {
    e.preventDefault();
    return;
  }
  // Let a focused button handle its own Space/Enter instead of toggling twice.
  if (e.target instanceof HTMLButtonElement && (e.key === " " || e.key === "Enter")) return;
  const key = e.key.length === 1 ? e.key.toLowerCase() : e.key;

  if (/^[1-9]$/.test(key) && Number(key) <= JOINTS.length) select(Number(key) - 1);
  else if (key === "ArrowUp") select(state.selected - 1);
  else if (key === "ArrowDown") select(state.selected + 1);
  else if (key === "ArrowLeft" || key === "ArrowRight" || key === "a" || key === "d") {
    held.add(key);
    setDemo(false);
  } else if (key === "g" && !e.repeat) toggleGripper();
  else if (key === " " && !e.repeat) setDemo(!state.demo);
  else if (key === "t" && !e.repeat) setTrail(!state.trail);
  else if (key === "r" && !e.repeat) reset();
  else return;

  e.preventDefault();
});

window.addEventListener("keyup", (e) => held.delete(e.key.length === 1 ? e.key.toLowerCase() :
```

```
e.key));
window.addEventListener("blur", () => held.clear());

function keyboardDirection(): number {
  const negative = held.has("ArrowLeft") || held.has("a") ? 1 : 0;
  const positive = held.has("ArrowRight") || held.has("d") ? 1 : 0;
  return positive - negative;
}
```

Held keys go into a `Set`, so jogging continues while the key is down and stops on keyup, with no dependence on the operating system's key-repeat rate. Toggle keys check `!e.repeat`, so holding G doesn't flicker the gripper. `preventDefault` stops arrows and space from scrolling the page, but only for keys the app used.

The first lines are new with the Shapes panel. A key typed into a text field must stay in that field: typing “r” in a shape's name would otherwise reset the arm. Then the shapes module gets the first look at every key (Esc cancels placing, Delete removes the selected shape), and only keys it doesn't use reach the arm.

On `blur` (the window loses focus) all held keys are cleared. Otherwise, releasing a key while another window has focus would leave the joint jogging forever.

Clicking a joint

src/main.ts · lines 173–205

```
const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();
let pressX = 0;
let pressY = 0;

function housingAt(e: PointerEvent) {
  const rect = canvas.getBoundingClientRect();
  pointer.set(((e.clientX - rect.left) / rect.width) * 2 - 1, -((e.clientY - rect.top) /
rect.height) * 2 + 1);
  raycaster.setFromCamera(pointer, camera);
  return raycaster.intersectObjects(arm.housings, false)[0]?.object;
}

canvas.addEventListener("pointerdown", (e) => {
  pressX = e.clientX;
  pressY = e.clientY;
});
canvas.addEventListener("pointerup", (e) => {
```

```
  if (Math.hypot(e.clientX - pressX, e.clientY - pressY) > 4) return; // that was an orbit drag
  // Whoever is placing something gets the click; then shapes (they sit on top of belts), then
  // splitter gates (they stand over belts), then belts.
  if (splitters.isActive() && splitters.handleClick(e)) return;
  if (belts.isActive() && belts.handleClick(e)) return;
  if (shapes.handleClick(e)) return; // placing, moving or selecting a shape
  if (splitters.handlePick(e)) return;
  if (belts.handlePick(e)) return;
  const hit = housingAt(e);
  if (hit) select(hit.userData.jointIndex as number);
});
canvas.addEventListener("pointermove", (e) => {
  if (e.buttons !== 0) return;
  if (splitters.isActive() ? splitters.handleHover(e) : belts.isActive() ? belts.handleHover(e)
: shapes.handleHover(e)) return;
  canvas.style.cursor = housingAt(e) || splitters.hoversGate(e) || belts.hoversBelt(e) ?
"pointer" : "";
});
```

To find what's under the mouse, convert the pointer position to **normalized device coordinates** (−1 to 1 on both axes, with +y up, hence the minus sign), then cast a ray from the camera through that point. `intersectObjects` returns hits sorted by distance; the first is the nearest housing.

Orbit controls also use the mouse, so a click is only a click if the pointer moved less than 4 pixels between press and release. Anything more was a drag to orbit the camera.

A genuine click goes to the shapes module first: while placing or moving a shape, a click on the floor is its drop point, and a click on a shape selects it. Only when the shapes module doesn't want the click does it select a joint. Hover works the same way, so the cursor is a crosshair while placing and a pointer over a shape.

The frame loop

`src/main.ts · lines 209–236`

```
const tcpWorld = new THREE.Vector3();
let lastTime = performance.now();

select(state.selected);
panel.setDemo(state.demo);
panel.setTrail(state.trail);

renderer.setAnimationLoop((time) => {
  const dt = Math.min((time - lastTime) / 1000, 0.05); // clamp after a hidden tab resumes
```

```
  lastTime = time;

  if (state.demo) {
    state.demoTime += dt;
    // Frame-rate independent easing toward the demo path, so toggling the demo
    // on from any pose glides there instead of jumping.
    const k = 1 - Math.exp(-DEMO_SMOOTHING * dt);
    JOINTS.forEach((_, i) => {
      const current = arm.getAngle(i);
      arm.setAngle(i, current + (demoTarget(i, state.demoTime) - current) * k);
    });
    setGripperOpen(demoGripperOpen(state.demoTime));
  } else {
    const direction = keyboardDirection() + state.jog;
    if (direction !== 0) {
      const i = state.selected;
      arm.setAngle(i, arm.getAngle(i) + Math.sign(direction) * JOG_SPEED * dt);
    }
  }
```

`setAnimationLoop` calls the function once per display refresh. `dt` is the frame time in seconds, capped at 50 ms so a tab that was hidden for a minute doesn't make the arm jump.

In demo mode, each joint eases toward its demo target with `k = 1 − e^(−3·dt)`. That's **frame-rate-independent smoothing**: a plain “move 5% of the way each frame” would converge twice as fast at 120 fps as at 60. With the exponential form, the result after one second is the same however it's sliced into frames. Turning the demo on from any pose glides instead of jumping.

Otherwise, the selected joint moves at `JOG_SPEED` in the direction of whatever is held. Multiplying by `dt` makes the speed the same in degrees per second at any frame rate.

Measuring and drawing

src/main.ts · lines 238–250

```
  gripper?.update(dt);
  belts.update(dt); // moves shapes riding the belts while running
  splitters.update(dt); // swings the gates' flaps

  // getWorldPosition walks up the hierarchy and refreshes world matrices first,
  // so it's correct even though the joints changed since the last render.
  arm.tcp.getWorldPosition(tcpWorld);
  if (state.trail) trail.push(tcpWorld);
  panel.update(arm.anglesDeg, tcpWorld, forwardKinematics(ARM, arm.anglesRad()));

  controls.update();
  renderer.render(scene, camera);
});
```

`getWorldPosition` updates the world matrices up the chain and returns the TCP's position: the scene graph's answer. `forwardKinematics` gives the hand-written answer from the same angles. Both go to the panel side by side, so every frame is a live test that the hierarchy and the math agree.

GOING DEEPER

Picking at scale

`Raycaster.intersectObjects` tests the ray against every triangle of the objects it's given, after a quick bounding-sphere check. Four housings is trivial. A factory layout with thousands of parts needs either a **bounding volume hierarchy** (the three-mesh-bvh library makes raycasts logarithmic) or **GPU picking**: render each object's ID as a color into an off-screen buffer and read back the one pixel under the mouse.

PART 1.9 · `tests/kinematics.test.ts`

Testing the kinematics

Seven tests, run with Node's built-in test runner. The interesting one doesn't check hand-picked answers: it checks the function against a completely different way of computing the same thing.

Setup

tests/kinematics.test.ts · lines 1–12

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { DEG, forwardKinematics, maxReach, type Vec3 } from "../src/arm/kinematics";
import { ARM, JOINTS, demoTarget } from "../src/arm/jointSpecs";

const H = ARM.baseHeight + ARM.shoulderHeight; // shoulder joint height
const REACH = maxReach(ARM);

function assertClose(actual: Vec3, expected: Vec3, tolerance = 1e-12) {
  const err = Math.hypot(actual.x - expected.x, actual.y - expected.y, actual.z - expected.z);
  assert.ok(err < tolerance, `expected ${JSON.stringify(expected)}, got
${JSON.stringify(actual)} (error ${err})`);
}
```

`H` is the shoulder height and `REACH` the stretched length. `assertClose` compares positions by distance with a tolerance of 10^{-12}, since floating-point results are rarely exactly equal, and puts both values in the failure message.

An independent reference

tests/kinematics.test.ts · lines 16–44

```
type Mat4 = number[]; // row-major

function multiply(a: Mat4, b: Mat4): Mat4 {
  const out = new Array<number>(16).fill(0);
  for (let r = 0; r < 4; r++)
    for (let c = 0; c < 4; c++)
      for (let k = 0; k < 4; k++) out[r * 4 + c] += a[r * 4 + k] * b[k * 4 + c];
  return out;
}
const translateY = (y: number): Mat4 => [1, 0, 0, 0, 0, 1, 0, y, 0, 0, 1, 0, 0, 0, 0, 1];
const rotateY = (t: number): Mat4 => {
  const c = Math.cos(t), s = Math.sin(t);
  return [c, 0, s, 0, 0, 1, 0, 0, -s, 0, c, 0, 0, 0, 0, 1];
};
const rotateZ = (t: number): Mat4 => {
  const c = Math.cos(t), s = Math.sin(t);
  return [c, -s, 0, 0, s, c, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1];
```

```
};

function matrixChainFK(q: number[]): Vec3 {
  const chain = [
    translateY(ARM.baseHeight), rotateY(q[0]),
    translateY(ARM.shoulderHeight), rotateZ(q[1]),
    translateY(ARM.upperArm), rotateZ(q[2]),
    translateY(ARM.forearm), rotateZ(q[3]),
    translateY(ARM.wrist + ARM.tool),
  ].reduce(multiply);
  return { x: chain[3], y: chain[7], z: chain[11] }; // translation column
}
```

The same arm as a chain of 4×4 matrices: translate up a link, rotate about the joint's axis, translate, rotate… and multiply them all together (`reduce(multiply)`). The position of the tool is the translation column of the result. This is exactly what a scene graph does internally, written out with no shortcuts and no shared code with kinematics.ts.

Known poses

tests/kinematics.test.ts · lines 48–66

```
test("zero pose points straight up", () => {
  assertClose(forwardKinematics(ARM, [0, 0, 0, 0]), { x: 0, y: H + REACH, z: 0 });
});

test("shoulder +90 degrees lays the arm along -X", () => {
  assertClose(forwardKinematics(ARM, [0, 90 * DEG, 0, 0]), { x: -REACH, y: H, z: 0 });
});

test("base +90 degrees swings that pose to +Z", () => {
  assertClose(forwardKinematics(ARM, [90 * DEG, 90 * DEG, 0, 0]), { x: 0, y: H, z: REACH });
});

test("home pose has the gripper pointing straight down", () => {
  const q = JOINTS.map((j) => j.home * DEG);
  const tcp = forwardKinematics(ARM, q);
  const flange = forwardKinematics({ ...ARM, wrist: 0, tool: 0 }, q); // the wrist joint
  assert.ok(Math.abs(tcp.x - flange.x) < 1e-12 && Math.abs(tcp.z - flange.z) < 1e-12);
  assert.ok(tcp.y < flange.y);
});
```

Poses you can check in your head: everything at 0 points straight up; shoulder at +90° lays the arm flat along −X; base +90° swings that to +Z. These pin down the sign conventions, which are the most common source of FK bugs.

The home-pose test computes the wrist joint's position by running FK with zero-length wrist and tool, then checks the TCP is directly below it: the gripper points straight down.

Random poses, and limits

tests/kinematics.test.ts · lines 68–96

```
test("matches a 4x4 matrix chain for 1,000 random poses", () => {
  let seed = 42; // deterministic pseudo-random numbers
  const random = () => ((seed = (seed * 1664525 + 1013904223) % 2 ** 32) / 2 ** 32);
  for (let i = 0; i < 1000; i++) {
    const q = JOINTS.map((j) => (j.min + random() * (j.max - j.min)) * DEG);
    assertClose(forwardKinematics(ARM, q), matrixChainFK(q));
  }
});

test("never reaches farther than the stretched arm", () => {
  for (let i = 0; i < 500; i++) {
    const q = [i * 0.7, Math.sin(i), Math.cos(i * 1.3), Math.sin(i * 0.4) * 2];
    const p = forwardKinematics(ARM, q);
    assert.ok(Math.hypot(p.x, p.y - H, p.z) <= REACH + 1e-12);
  }
});

test("demo motion stays within joint limits and keeps the gripper above the floor", () => {
  const FINGERTIP_BELOW_TCP = 0.015; // fingertips extend a little past the TCP
  let lowest = Infinity;
  for (let t = 0; t < 600; t += 0.05) {
    const anglesDeg = JOINTS.map((_, i) => demoTarget(i, t));
    anglesDeg.forEach((a, i) => {
      assert.ok(a >= JOINTS[i].min && a <= JOINTS[i].max, `${JOINTS[i].name} = ${a} at
t=${t}`);
    });
    lowest = Math.min(lowest, forwardKinematics(ARM, anglesDeg.map((a) => a * DEG)).y);
  }
  assert.ok(lowest - FINGERTIP_BELOW_TCP > 0.02, `gripper dips to ${lowest.toFixed(3)} m`);
});
```

1,000 random poses within the joint limits, each compared with the matrix chain. A small inline random generator with a fixed seed makes the “random” poses identical on every run, so a failure can always be reproduced.

The last test samples ten minutes of the demo motion every 50 ms and checks every angle is within its limits and the fingertips stay at least 2 cm above the floor. That's the test that forced the demo amplitudes down.

PART 1.10 · `scripts/build-gripper.ts`

Building the gripper model by hand

This script writes `public/models/gripper.glb`, a binary glTF 2.0 file, with no 3D software and no libraries. Doing it by hand shows exactly what's inside a glTF file.

The header comment

scripts/build-gripper.ts · lines 1–14

```
/**
 * Writes public/models/gripper.glb: a two-finger parallel gripper as a binary glTF 2.0 file.
 *
 * No dependencies: geometry, buffer layout and the GLB container are built by hand, which
 * keeps the asset reproducible from source. Run with `npm run build:gripper`.
 *
 * Node hierarchy (names matter, Gripper.ts looks them up):
 *   Gripper
 *   ├── Palm
 *   ├── FingerLeft   (origin at the finger root, slides along X)
 *   └── FingerRight
 *
 * Units are meters, +Y points away from the wrist flange, matching the glTF convention.
 */
```

Documentation only. It says what the script produces, that it has no dependencies, how to run it, and the node hierarchy the app expects. The names matter because Gripper.ts looks up `FingerLeft` and `FingerRight` by name.

Imports

scripts/build-gripper.ts · lines 15–17

```
import { mkdirSync, writeFileSync } from "node:fs";
import { dirname } from "node:path";
import { fileURLToPath } from "node:url";
```

- `mkdirSync`: creates a folder, used to make sure `public/models/` exists.
- `writeFileSync`: writes the finished bytes to disk.
- `dirname`: gets the folder part of a path (`.../public/models/gripper.glb` becomes `.../public/models`).
- `fileURLToPath`: turns a `file:///...` URL into a normal path. ES modules know their own location as a URL, not a path.

The `node:` prefix means “Node's built-in module”, not something from npm.

Types

scripts/build-gripper.ts · lines 19–24

```
type V3 = [number, number, number];

interface Box {
  center: V3;
  size: V3;
}
```

A 3D vector as a fixed-length tuple, and a box described by its center and its full width, height and depth. The whole gripper is built from boxes.

The face table

scripts/build-gripper.ts · lines 26–36

```
// For each face: outward normal n, and in-plane axes u, v with u × v = n,
// so the corners below wind counter-clockwise when seen from outside (glTF front faces).
const FACES: { n: V3; u: V3; v: V3 }[] = [
  { n: [1, 0, 0], u: [0, 1, 0], v: [0, 0, 1] },
  { n: [-1, 0, 0], u: [0, 0, 1], v: [0, 1, 0] },
  { n: [0, 1, 0], u: [0, 0, 1], v: [1, 0, 0] },
  { n: [0, -1, 0], u: [1, 0, 0], v: [0, 0, 1] },
  { n: [0, 0, 1], u: [1, 0, 0], v: [0, 1, 0] },
  { n: [0, 0, -1], u: [0, 1, 0], v: [1, 0, 0] },
];
const CORNERS: [number, number][] = [[-1, -1], [1, -1], [1, 1], [-1, 1]];
```

One row per face of a box. `n` is the **normal**, the direction the face points; `u` and `v` are two directions lying flat on the face, used to step from the face's center to its corners.

The comment says `u × v = n`. That's the important part: it guarantees the corners come out **counter-clockwise when viewed from outside**, which is how glTF and WebGL decide which side of a triangle is the front. Get it backwards and the faces are invisible from outside, because back faces are culled.

`CORNERS` lists the four corners as (u, v) steps: bottom-left, bottom-right, top-right, top-left. Going round in this order is the counter-clockwise loop.

Turning boxes into vertex data

scripts/build-gripper.ts · lines 38–56

```
/** Boxes merged into one flat-shaded mesh: 4 vertices and 2 triangles per face. */
function boxesToMesh(boxes: Box[]) {
  const positions: number[] = [];
  const normals: number[] = [];
  const indices: number[] = [];
  for (const { center, size } of boxes) {
    for (const { n, u, v } of FACES) {
      const first = positions.length / 3;
      for (const [su, sv] of CORNERS) {
        for (let k = 0; k < 3; k++) {
          positions.push(center[k] + ((n[k] + su * u[k] + sv * v[k]) * size[k]) / 2);
        }
        normals.push(...n);
      }
      indices.push(first, first + 1, first + 2, first, first + 2, first + 3);
    }
  }
  return { positions, normals, indices };
}
```

Three flat arrays, which is what the GPU wants: `positions` (x, y, z per vertex), `normals` (one direction per vertex, for lighting) and `indices` (three vertex numbers per triangle). For every box and every face, `first` remembers the number of the first vertex the face adds (three numbers per vertex, hence ÷ 3).

The corner formula reads: **start at the box center, step out to the face (`n`), step sideways to the corner (± `u`, ± `v`), and scale each step by half the box size on that axis.** It works because `n`, `u` and `v` each point along a different axis, so on any one axis only one of them is nonzero.

Every corner of a face gets that face's normal. That's why each face has its own four vertices instead of sharing corners with its neighbours: shared corners would average the normals and the box would look rounded instead of crisp.

Finally the 4-corner face is split into two triangles, 0-1-2 and 0-2-3, both keeping the counter-clockwise order. One box is 24 vertices and 12 triangles.

The gripper's shape

scripts/build-gripper.ts · lines 58–71

```
// ---- Geometry ----

const palm: Box[] = [
  { center: [0, 0.01, 0], size: [0.09, 0.02, 0.09] }, // mounting flange
  { center: [0, 0.04, 0], size: [0.18, 0.04, 0.07] }, // body, top face at y = 0.06
];

/** side = +1 for the left finger (pad faces +X, toward the center), -1 for the right. */
function finger(side: 1 | -1): Box[] {
  return [
    { center: [0, 0.035, 0], size: [0.02, 0.07, 0.05] }, // finger
    { center: [side * 0.016, 0.055, 0], size: [0.012, 0.03, 0.044] }, // grip pad
  ];
}
```

The palm is two boxes, in meters: a 9 × 2 × 9 cm mounting flange from y = 0 to 2 cm, sitting flush on the wrist, and an 18 cm wide body from 2 to 6 cm.

A finger is a 2 × 7 × 5 cm bar plus a grip pad near its tip, on the inner side. `side` flips which side that is: `+1` puts the pad at +X for the left finger, `−1` at −X for the right. The finger's y runs from 0 to 7 cm because its origin is its root, where it attaches to the palm.

glTF constants and storage

scripts/build-gripper.ts · lines 73–84

```
// ---- glTF JSON + binary buffer ----

const ARRAY_BUFFER = 34962;
const ELEMENT_ARRAY_BUFFER = 34963;
const FLOAT = 5126;
const UNSIGNED_SHORT = 5123;

const binChunks: Buffer[] = [];
let binLength = 0;
const bufferViews: object[] = [];
const accessors: object[] = [];
const meshes: object[] = [];
```

WebGL's numeric codes, which glTF reuses: `ARRAY_BUFFER` marks vertex data, `ELEMENT_ARRAY_BUFFER` triangle indices, `FLOAT` a 32-bit float and `UNSIGNED_SHORT` a 16-bit whole number (0 to 65,535, plenty for 48 vertices).

A glTF file has two parts: a **JSON** description and one **binary blob** of raw numbers. `binChunks` collects pieces of the blob and `binLength` tracks its size; the other three arrays are the JSON sections that point into it.

Appending to the blob

scripts/build-gripper.ts · lines 86–96

```
function addBufferView(data: Buffer, target: number): number {
  const padding = (4 - (binLength % 4)) % 4; // keep every view 4-byte aligned
  if (padding) {
    binChunks.push(Buffer.alloc(padding));
    binLength += padding;
  }
  bufferViews.push({ buffer: 0, byteOffset: binLength, byteLength: data.length, target });
  binChunks.push(data);
  binLength += data.length;
  return bufferViews.length - 1;
}
```

glTF requires each block of float data to start at a byte offset divisible by 4. `(4 - (binLength % 4)) % 4` is how many filler bytes get there: if the blob is 10 bytes long, 10 % 4 = 2, so 2 bytes of padding. The outer `% 4` turns “4 needed” into 0 when already aligned.

It matters because index data (2 bytes each) can leave the blob at an odd length. Then a **bufferView** is recorded: “in buffer 0, starting at this byte, this many bytes, used for vertices or indices”. The function returns the view's index, because glTF refers to everything by its position in an array.

Accessors

scripts/build-gripper.ts · lines 98–101

```
function addAccessor(accessor: object): number {
  accessors.push(accessor);
  return accessors.length - 1;
}
```

An **accessor** says how to read a bufferView: “48 items, each 3 floats (VEC3)”. Three layers, then: the buffer holds raw bytes, a bufferView marks a byte range within it, and an accessor says how to interpret that range as numbers.

One complete mesh

scripts/build-gripper.ts · lines 103–113

```
function addMesh(name: string, boxes: Box[], material: number): number {
  const { positions, normals, indices } = boxesToMesh(boxes);
  const pos = new Float32Array(positions);
  const count = pos.length / 3;

  const min: V3 = [Infinity, Infinity, Infinity];
  const max: V3 = [-Infinity, -Infinity, -Infinity];
  for (let i = 0; i < pos.length; i++) {
    min[i % 3] = Math.min(min[i % 3], pos[i]);
    max[i % 3] = Math.max(max[i % 3], pos[i]);
  }
```

Build the vertex data and convert the positions to a `Float32Array`, the exact byte format glTF stores. Then compute the smallest and largest x, y and z: glTF **requires** these bounds for positions, and loaders use them for culling. `i % 3` picks the axis as the loop walks the flat list.

The bounds are read from the `Float32Array`, not the original numbers, on purpose: converting to 32 bits rounds slightly, and the bounds must match what's actually stored.

scripts/build-gripper.ts · lines 115–130

```
  const position = addAccessor({
    bufferView: addBufferView(Buffer.from(pos.buffer), ARRAY_BUFFER),
    componentType: FLOAT, count, type: "VEC3", min, max,
  });
  const normal = addAccessor({
    bufferView: addBufferView(Buffer.from(new Float32Array(normals).buffer), ARRAY_BUFFER),
    componentType: FLOAT, count, type: "VEC3",
  });
  const index = addAccessor({
    bufferView: addBufferView(Buffer.from(new Uint16Array(indices).buffer),
ELEMENT_ARRAY_BUFFER),
    componentType: UNSIGNED_SHORT, count: indices.length, type: "SCALAR",
  });

  meshes.push({ name, primitives: [{ attributes: { POSITION: position, NORMAL: normal },
indices: index, material }] });
  return meshes.length - 1;
}
```

Each accessor call also writes its bytes to the blob. `pos.buffer` is the raw memory behind the typed array and `Buffer.from` wraps it without copying. Normals don't need bounds; indices are 16-bit, one number each (`SCALAR`).

The mesh entry ties it together. A **primitive** is one drawable piece: “positions from accessor N, normals from M, triangles from the index accessor, with this material”. Triangles are the default mode, so it isn't stated.

Building the three meshes

scripts/build-gripper.ts · lines 132–136

```
const ANODIZED = 0;
const ACCENT = 1;
const palmMesh = addMesh("Palm", palm, ANODIZED);
const leftMesh = addMesh("FingerLeft", finger(1), ACCENT);
const rightMesh = addMesh("FingerRight", finger(-1), ACCENT);
```

`ANODIZED` and `ACCENT` name the two material slots. The dark palm and two orange, mirror-image fingers are built, keeping each mesh's index.

The glTF JSON

scripts/build-gripper.ts · lines 138–148

```
const OPEN_X = 0.07;
const gltf = {
  asset: { version: "2.0", generator: "robot-arm/scripts/build-gripper.ts" },
  scene: 0,
  scenes: [{ name: "Gripper", nodes: [0] }],
  nodes: [
    { name: "Gripper", children: [1, 2, 3] },
    { name: "Palm", mesh: palmMesh },
    { name: "FingerLeft", mesh: leftMesh, translation: [-OPEN_X, 0.06, 0] },
    { name: "FingerRight", mesh: rightMesh, translation: [OPEN_X, 0.06, 0] },
  ],
```

`asset.version` is required; `generator` records which tool made the file. One scene, whose top node is node 0.

The **nodes** form the hierarchy, like Object3Ds in Three.js or GameObjects in Unity. “Gripper” is an empty parent with three children. Each finger is placed 7 cm to the side and 6 cm up (the top of the palm), so the model starts open. These are the names Gripper.ts searches for, and moving a finger means changing its node's position.

scripts/build-gripper.ts · lines 149–163

```
  meshes,
  materials: [
    {
      name: "Anodized",
      pbrMetallicRoughness: { baseColorFactor: [0.16, 0.17, 0.19, 1], metallicFactor: 0.6,
roughnessFactor: 0.45 },
    },
    {
      name: "Accent",
      pbrMetallicRoughness: { baseColorFactor: [0.95, 0.42, 0.08, 1], metallicFactor: 0.1,
roughnessFactor: 0.55 },
    },
  ],
  accessors,
  bufferViews,
  buffers: [{ byteLength: binLength }],
};
```

Two **PBR** (physically based) materials: `baseColorFactor` is red, green, blue and alpha from 0 to 1, `metallicFactor` runs from plastic (0) to metal (1), `roughnessFactor` from mirror (0) to matte (1). Three.js's `GLTFLoader` turns them into `MeshStandardMaterial`, the same material the arm uses. The single buffer's size is declared last; in a .glb that buffer is the binary chunk after the JSON.

Packing it into a .glb

scripts/build-gripper.ts · lines 165–178

```
// ---- GLB container: 12-byte header, JSON chunk (space padded), BIN chunk (zero padded) ----

function pad(data: Buffer, fill: number): Buffer {
  const padding = (4 - (data.length % 4)) % 4;
  return padding ? Buffer.concat([data, Buffer.alloc(padding, fill)]) : data;
}

const json = pad(Buffer.from(JSON.stringify(gltf)), 0x20);
const bin = pad(Buffer.concat(binChunks), 0x00);

const header = Buffer.alloc(12);
header.writeUInt32LE(0x46546c67, 0); // "glTF"
header.writeUInt32LE(2, 4);
header.writeUInt32LE(12 + 8 + json.length + 8 + bin.length, 8);
```

A `.glb` is one binary file: a 12-byte header, a JSON chunk, a binary chunk. Each chunk's length must be a multiple of 4. The JSON is padded with spaces (`0x20`) so it still parses; the binary with zeros.

The header is three 32-bit numbers in **little-endian** order (lowest byte first, which glTF requires): the magic number `0x46546c67`, which reads `g l T F` as bytes and lets a loader recognise the file; the container version, 2; and the total size, 12 for the header plus 8 + JSON and 8 + binary for the two chunks.

scripts/build-gripper.ts · lines 180–193

```
function chunkHeader(length: number, type: number): Buffer {
  const h = Buffer.alloc(8);
  h.writeUInt32LE(length, 0);
  h.writeUInt32LE(type, 4);
  return h;
}

const glb = Buffer.concat([
  header,
  chunkHeader(json.length, 0x4e4f534a), // "JSON"
  json,
  chunkHeader(bin.length, 0x004e4942), // "BIN\0"
  bin,
]);
```

Every chunk starts with 8 bytes: its length, then a type code. Like the magic number, the codes spell `JSON` and `BIN` when read as bytes. Then the five pieces are concatenated in order.

Writing the file

scripts/build-gripper.ts · lines 195–198

```
const out = fileURLToPath(new URL("../public/models/gripper.glb", import.meta.url));
mkdirSync(dirname(out), { recursive: true });
writeFileSync(out, glb);
console.log(`Wrote ${out} (${glb.length} bytes)`);
```

`import.meta.url` is this script's own location, so resolving `../public/models/gripper.glb` against it puts the output in the right place **whichever folder you run the command from**. `recursive: true` makes any missing folders and doesn't complain if they exist. The finished file is 6,376 bytes.

GOING DEEPER

What a production asset pipeline adds

Real models come from CAD or Blender, not hand-written boxes. A typical web pipeline then runs tools like gltf-transform or gltfpack to merge duplicate data, **quantize** positions and normals to 16 or 8 bits, compress geometry with Draco or meshopt, convert textures to KTX2/Basis (compressed on the GPU too), and checks the result with the Khronos glTF Validator.

For a browser-based design tool, CAD formats such as STEP describe exact surfaces, not triangles, so parts are **tessellated** into meshes first, at a level of detail chosen per part. Writing a glTF by hand, as here, is the best way to understand what all of those tools are producing.

PART 1.11 · `src/shapes/types.ts`

Shapes in the scene: the data and the reach rule

The arm's floor can hold shapes: boxes, cylinders, spheres and cones, or inventory parts drawn as one of those. They're stored in the inventory database (Chapter 5), so this app is now a client of that API.

The shape type

src/shapes/types.ts · lines 1–30

```
/** Shapes as the inventory API stores them (snack5-inventory/api/src/domain/shapes.ts). */

export type ShapeKind = "box" | "cylinder" | "sphere" | "cone";
export const SHAPE_KINDS: readonly ShapeKind[] = ["box", "cylinder", "sphere", "cone"];

/** Bounding box in meters: width, height, depth. Round shapes have depth = width. */
export interface Size {
  x: number;
  y: number;
  z: number;
}

export interface Shape {
  id: string;
  name: string;
```

```
  kind: ShapeKind;
  size: Size;
  color: string;
  /** Floor position of the center, in meters. */
  position: { x: number; z: number };
  /** Degrees about the vertical axis. */
  rotationY: number;
  /** The inventory part this shape stands for, or null for a plain shape. */
  sku: string | null;
  createdAt: string;
  updatedAt: string;
}

export type NewShape = Omit<Shape, "id" | "createdAt" | "updatedAt">;
export type ShapePatch = Partial<Pick<Shape, "name" | "color" | "position" | "rotationY">>;
```

This mirrors the API's type field for field, including the rule that `size` is always a bounding box: a cylinder's depth equals its width, a sphere's three sizes are its diameter. One meaning for `size` lets the drawing code and the reach code treat every kind the same way.

Only x and z are stored for the position. Every shape rests on the floor, so its height decides where its center is; storing y as well would allow shapes floating in the air or sunk into the floor.

Can the gripper get there?

src/shapes/reach.ts · lines 1–50

```
/**
 * Can the arm reach a shape? Pure math on the arm's dimensions, no Three.js, so it's
 * unit-tested like the kinematics.
 *
 * The gripper picks from above, so the question is whether the tool center point can get
 * to the middle of the shape's top face. The arm is a chain hinged at the shoulder, so the
 * points it can reach lie within a sphere of radius maxReach around the shoulder joint.
 * A small margin keeps "reachable" from meaning "only with the arm locked straight".
 */
import { maxReach, type ArmDimensions } from "../arm/kinematics";
import type { Shape } from "./types";

export type Reach = "reachable" | "too-far" | "too-close" | "overlaps-base";

/** Radius of the pedestal at the floor (RobotArm.ts builds it 0.2 m at the bottom). */
export const BASE_RADIUS = 0.2;
```

```
/** Closer than this, the arm would have to fold back on its own column. */
export const MIN_REACH_RADIUS = 0.25;
export const REACH_MARGIN = 0.03;

/** Radius of the circle a shape's footprint fits in, whatever its rotation. */
export function footprintRadius(shape: Pick<Shape, "kind" | "size">): number {
  return shape.kind === "box" ? Math.hypot(shape.size.x, shape.size.z) / 2 : shape.size.x / 2;
}

/** baseY is what the shape stands on: 0 on the floor, a belt's height on a conveyor. */
export function reachOf(shape: Pick<Shape, "kind" | "size" | "position">, arm: ArmDimensions,
baseY = 0): Reach {
  const { x, z } = shape.position;
  const radius = Math.hypot(x, z); // horizontal distance from the base axis
  if (radius - footprintRadius(shape) < BASE_RADIUS) return "overlaps-base";
  if (radius < MIN_REACH_RADIUS) return "too-close";
  const shoulderY = arm.baseHeight + arm.shoulderHeight;
  const topY = baseY + shape.size.y;
  const distance = Math.hypot(radius, topY - shoulderY);
  return distance <= maxReach(arm) - REACH_MARGIN ? "reachable" : "too-far";
}

/** How far out, along the floor, the arm can reach a point at height y. */
export function reachRadiusAt(y: number, arm: ArmDimensions): number {
  const r = maxReach(arm) - REACH_MARGIN;
  const dy = y - (arm.baseHeight + arm.shoulderHeight);
  return Math.sqrt(Math.max(0, r * r - dy * dy));
}

export const REACH_LABEL: Record<Reach, string> = {
  reachable: "in reach",
  "too-far": "out of reach",
  "too-close": "too close",
  "overlaps-base": "hits the base",
};
```

The gripper picks from above, so the target is the middle of the shape's top face. Every point the TCP can reach lies inside a sphere of radius `maxReach` around the shoulder, because the chain is hinged there: that's the whole test, `Math.hypot(radius, topY - shoulderY) <= maxReach - margin`. The 3 cm margin means “in reach” doesn't require the arm locked dead straight.

Two checks come first. A shape whose footprint overlaps the pedestal's 0.2 m radius would collide with the base, and a point closer than 0.25 m would need the arm to fold back on its own column. A box's footprint radius is half its diagonal, so the check holds at any rotation.

`reachRadiusAt(y)` turns the sphere into a circle at one height: Pythagoras on the sphere's radius and the height difference. At floor level it's the blue ring drawn on the floor.

Testing the reach rule

tests/reach.test.ts · lines 1–36

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { ARM } from "../src/arm/jointSpecs";
import { maxReach } from "../src/arm/kinematics";
import { BASE_RADIUS, footprintRadius, reachOf, reachRadiusAt, REACH_MARGIN } from
"../src/shapes/reach";

const cube = (x: number, z: number, cm = 20) => ({ kind: "box" as const, size: { x: cm / 100,
y: cm / 100, z: cm / 100 }, position: { x, z } });

test("a small box in front of the arm is reachable", () => {
  assert.equal(reachOf(cube(0.6, 0), ARM), "reachable");
  assert.equal(reachOf(cube(0, -0.6), ARM), "reachable"); // the base turns, so direction
doesn't matter
});

test("far away is out of reach, and the limit matches the reach sphere", () => {
  assert.equal(reachOf(cube(1.5, 0), ARM), "too-far");
  // A thin 1 cm plate's top is at y = 0.01: the limit is the reach sphere's radius at that
height.
  const r = reachRadiusAt(0.01, ARM);
  const plate = (x: number) => ({ kind: "box" as const, size: { x: 0.05, y: 0.01, z: 0.05 },
position: { x, z: 0 } });
  assert.equal(reachOf(plate(r - 0.001), ARM), "reachable");
  assert.equal(reachOf(plate(r + 0.001), ARM), "too-far");
});

test("tall shapes reach higher, so they can be reached further out", () => {
  const tall = { kind: "cylinder" as const, size: { x: 0.1, y: 0.35, z: 0.1 }, position: { x:
1.04, z: 0 } };
  const flat = { ...tall, size: { x: 0.1, y: 0.02, z: 0.1 } };
  assert.equal(reachOf(tall, ARM), "reachable"); // top level with the shoulder: full reach
```

```
sideways
  assert.equal(reachOf(flat, ARM), "too-far");
  assert.ok(Math.abs(reachRadiusAt(ARM.baseHeight + ARM.shoulderHeight, ARM) - (maxReach(ARM) -
REACH_MARGIN)) < 1e-12);
});

test("shapes overlapping the pedestal, or tucked against it, are flagged", () => {
  assert.equal(reachOf(cube(0.25, 0), ARM), "overlaps-base"); // 20 cm box reaches within 0.2 m
of the axis
  assert.equal(reachOf({ kind: "sphere", size: { x: 0.04, y: 0.04, z: 0.04 }, position: { x:
0.23, z: 0 } }, ARM), "too-close");
  assert.ok(footprintRadius(cube(0, 0)) > 0.1); // a box's footprint circle covers its corners
  assert.equal(BASE_RADIUS, 0.2);
});
```

The boundary test is the interesting one: it computes the exact limit for a 1 cm plate from `reachRadiusAt`, then checks a plate 1 mm inside is reachable and 1 mm outside is not. The tall-shapes test captures something a person might not expect: a shape whose top is level with the shoulder can be reached furthest out, because the arm reaches sideways there at full length.

PART 1.12 · **src/shapes/fromPart.ts**

Inventory parts as shapes

The inventory knows SKUs, names and stock, but not dimensions. This module gives each part a plausible shape from what its SKU and category say.

Reading sizes out of SKUs

src/shapes/fromPart.ts · lines 1–33

```
/**
 * Turns an inventory part into a shape for the scene. The inventory doesn't store
 * dimensions, so they're read from the SKU where it carries them (BAR-4040 is a 40 x 40 mm
 * profile, PL-200 a 200 mm plate, LA-300 a 300 mm stroke) and otherwise come from a
 * typical size for the category. Pure, so it's unit-tested.
 */
import type { NewShape, ShapeKind, Size } from "./types";

/** The fields of an inventory part this needs (snack5-inventory/api/src/domain/types.ts). */
export interface InventoryPart {
  sku: string;
```

```
  name: string;
  category: string;
  unit: string;
  onHand: number;
  available: number;
}

export interface PartLook {
  kind: ShapeKind;
  size: Size;
  color: string;
}

/** A cut piece, not a whole 3 m bar, so it fits on the arm's floor. */
export const EXTRUSION_PIECE_M = 0.5;

const mm = (n: number) => n / 1000;

/** The first run of digits in a SKU, e.g. 4040 in BAR-4040-CLR. */
function digits(sku: string): string | null {
  return /(\d+)/.exec(sku)?.[1] ?? null;
}
```

`InventoryPart` lists only the fields this module uses, a deliberately narrow type. The full part has more, and any object with at least these fields fits (TypeScript's structural typing), so the API's response can be passed straight in.

src/shapes/fromPart.ts · lines 35–66

```
export function lookFor(part: Pick<InventoryPart, "sku" | "category">): PartLook {
  const d = digits(part.sku);
  switch (part.category) {
    case "extrusion": {
      // BAR-2020 / BAR-4040 / BAR-4080: width and height of the profile in mm.
      const w = d && d.length === 4 ? Number(d.slice(0, 2)) : 40;
      const h = d && d.length === 4 ? Number(d.slice(2)) : 40;
      const black = /-BLK\b/.test(part.sku);
      // Lying on the floor along x, taller side up.
      return { kind: "box", size: { x: EXTRUSION_PIECE_M, y: mm(Math.max(w, h)), z:
mm(Math.min(w, h)) }, color: black ? "#2b2f36" : "#c3c9d1" };
    }
    case "plate": {
```

```
      if (/^BR-/.test(part.sku)) return { kind: "box", size: { x: 0.04, y: 0.04, z: 0.04 },
color: "#9aa3af" }; // corner bracket
      const side = d ? Math.min(Math.max(Number(d), 50), 1000) : 150;
      return { kind: "box", size: { x: mm(side), y: 0.01, z: mm(side) }, color: "#9aa3af" };
    }
    case "fastener":
      return { kind: "box", size: { x: 0.08, y: 0.04, z: 0.06 }, color: "#b7791f" }; // a small
box of bolts
    case "motor":
      return { kind: "cylinder", size: { x: 0.09, y: 0.14, z: 0.09 }, color: "#1f2937" };
    case "actuator": {
      const stroke = d ? Math.min(Number(d), 1500) : 300;
      return { kind: "box", size: { x: mm(stroke) + 0.1, y: 0.06, z: 0.06 }, color: "#3b82f6"
};
    }
    case "controller":
      return { kind: "box", size: { x: 0.2, y: 0.08, z: 0.15 }, color: "#334155" };
    case "gripper":
      return { kind: "box", size: { x: 0.12, y: 0.08, z: 0.06 }, color: "#f59e0b" };
    default:
      return { kind: "box", size: { x: 0.1, y: 0.1, z: 0.1 }, color: "#94a3b8" };
  }
}
```

Vention-style SKUs carry dimensions: `BAR-4080-BLK` is a 40 × 80 mm black profile, `PL-300` a 300 mm plate, `LA-300` a 300 mm stroke. `digits` takes the first run of numbers and each category interprets it. Bars are drawn as a 50 cm cut piece, not a full 3 m bar that would dwarf the arm, lying on their narrow side with the tall side up.

Exceptions are written down rather than guessed: `BR-90` is a 90° bracket, not a 90 mm one, so brackets get a fixed 4 cm block. Anything unknown gets a neutral 10 cm box, so a new category never breaks the scene.

Linking a shape to its part

src/shapes/fromPart.ts · lines 68–78

```
/** A new shape for `part` at (x, z), named after it and linked to it by SKU. */
export function shapeForPart(part: InventoryPart, x: number, z: number, look: PartLook =
lookFor(part), rotationY = 0): NewShape {
  return { name: part.name.slice(0, 40), ...look, position: { x, z }, rotationY, sku: part.sku
};
}

/** How many shapes in the scene stand for each SKU. */
export function placedBySku(shapes: readonly { sku: string | null }[]): Map<string, number> {
  const counts = new Map<string, number>();
  for (const s of shapes) if (s.sku) counts.set(s.sku, (counts.get(s.sku) ?? 0) + 1);
  return counts;
}
```

A part's shape carries its SKU. That link is what lets the panel say “2 in scene, 1 on hand”: `placedBySku` counts shapes per SKU, and the panel compares with stock. Placing a part doesn't change stock: the scene is a layout, and holding stock is what reservations are for (Chapter 5).

tests/fromPart.test.ts · lines 1–31

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { EXTRUSION_PIECE_M, lookFor, placedBySku, shapeForPart } from "../src/shapes/fromPart";

test("extrusions: the profile comes from the SKU, black or clear from the suffix", () => {
  const black = lookFor({ sku: "BAR-4080-BLK", category: "extrusion" });
  assert.deepEqual(black.size, { x: EXTRUSION_PIECE_M, y: 0.08, z: 0.04 });
  assert.equal(black.color, "#2b2f36");
  assert.deepEqual(lookFor({ sku: "BAR-2020-CLR", category: "extrusion" }).size, { x: 0.5, y:
0.02, z: 0.02 });
});

test("plates, brackets and actuators take their size from the SKU", () => {
  assert.deepEqual(lookFor({ sku: "PL-300", category: "plate" }).size, { x: 0.3, y: 0.01, z:
0.3 });
  assert.deepEqual(lookFor({ sku: "BR-90", category: "plate" }).size, { x: 0.04, y: 0.04, z:
0.04 }); // 90 is the angle, not a size
  const la = lookFor({ sku: "LA-300", category: "actuator" });
  assert.ok(Math.abs(la.size.x - 0.4) < 1e-12); // 300 mm stroke + 100 mm body
```

```
});

test("every category gets a shape, and unknown ones a neutral box", () => {
  for (const category of ["fastener", "motor", "controller", "gripper", "something-new"]) {
    const look = lookFor({ sku: "X-1", category });
    assert.ok(look.size.x > 0 && look.size.y > 0 && look.size.z > 0, category);
  }
  assert.equal(lookFor({ sku: "SRV-400", category: "motor" }).kind, "cylinder");
});

test("a part's shape is linked by SKU and named after the part", () => {
  const s = shapeForPart({ sku: "MC-1", name: "Motion controller", category: "controller",
unit: "piece", onHand: 4, available: 4 }, 0.5, -0.3);
  assert.deepEqual([s.sku, s.name, s.position], ["MC-1", "Motion controller", { x: 0.5, z: -0.3
}]);
  assert.deepEqual(placedBySku([{ sku: "MC-1" }, { sku: null }, { sku: "MC-1" }, { sku: "PL-
200" }]), new Map([["MC-1", 2], ["PL-200", 1]]));
});
```

PART 1.13 · `src/shapes/ShapeLayer.ts`

Drawing the shapes

One Three.js group holds the stored shapes, a see-through preview for placing, and the reach ring.

The layer

src/shapes/ShapeLayer.ts · lines 1–46

```
import * as THREE from "three";
import { DEG } from "../arm/kinematics";
import type { Shape, ShapeKind, Size } from "./types";

/**
 * Draws the stored shapes in the scene, plus a see-through "ghost" that follows the mouse
 * while placing one, and a faint ring on the floor showing how far the arm reaches.
 *
 * sync() is the only way shapes change: give it the list from the API and it adds, updates
 * and removes meshes to match, keyed by shape id, so polling the API is enough to show
 * changes made from another browser.
 */
```

```
export class ShapeLayer {
  readonly root = new THREE.Group();
  private readonly meshes = new Map<string, THREE.Mesh<THREE.BufferGeometry,
THREE.MeshStandardMaterial>>();
  private readonly ghost: THREE.Mesh<THREE.BufferGeometry, THREE.MeshStandardMaterial>;
  private ghostKey = "";
  private selectedId: string | null = null;

  /**
   * What a shape at (x, z) stands on: the floor (0) unless a conveyor is there. Set by the
   * conveyor controller, so this layer doesn't need to know about belts.
   */
  supportAt: (x: number, z: number) => number = () => 0;

  constructor(reachRadius: number) {
    this.root.name = "Shapes";

    this.ghost = new THREE.Mesh(
      new THREE.BoxGeometry(1, 1, 1),
      new THREE.MeshStandardMaterial({ transparent: true, opacity: 0.45, depthWrite: false }),
    );
    this.ghost.visible = false;
    this.root.add(this.ghost);

    // Floor-level reach: a thin ring, drawn just above the grid to avoid z-fighting.
    const ring = new THREE.Mesh(
      new THREE.RingGeometry(reachRadius - 0.006, reachRadius + 0.006, 128),
      new THREE.MeshBasicMaterial({ color: 0x3b82f6, transparent: true, opacity: 0.5 }),
    );
    ring.rotation.x = -Math.PI / 2;
    ring.position.y = 0.002;
    ring.name = "ReachRing";
    this.root.add(ring);
  }
```

The ghost is a normal mesh with a transparent material. `depthWrite: false` stops it hiding things behind it in the depth buffer, so the floor grid and other shapes stay visible through it, which is what makes it read as a preview.

The reach ring sits 2 mm above the floor, like the grid in stage.ts, to avoid z-fighting with the floor.

Syncing with the database

src/shapes/ShapeLayer.ts · lines 47–90

```
/** Makes the scene match `shapes` exactly. */
sync(shapes: readonly Shape[]): void {
  const keep = new Set(shapes.map((s) => s.id));
  for (const [id, mesh] of this.meshes) {
    if (!keep.has(id)) {
      this.root.remove(mesh);
      mesh.geometry.dispose();
      mesh.material.dispose();
      this.meshes.delete(id);
    }
  }
  for (const s of shapes) {
    let mesh = this.meshes.get(s.id);
    const key = geometryKey(s.kind, s.size);
    if (!mesh) {
      mesh = new THREE.Mesh(geometryFor(s.kind, s.size), new THREE.MeshStandardMaterial({
roughness: 0.55, metalness: 0.05 }));
      mesh.castShadow = true;
      mesh.receiveShadow = true;
      mesh.userData.shapeId = s.id;
      mesh.userData.geometryKey = key;
      this.meshes.set(s.id, mesh);
      this.root.add(mesh);
    } else if (mesh.userData.geometryKey !== key) {
      mesh.geometry.dispose();
      mesh.geometry = geometryFor(s.kind, s.size);
      mesh.userData.geometryKey = key;
    }
    mesh.name = s.name;
    mesh.material.color.set(s.color);
    mesh.userData.halfHeight = s.size.y / 2;
    place(mesh, s.size, s.position.x, s.position.z, s.rotationY, this.supportAt(s.position.x,
s.position.z));
  }
  this.select(this.selectedId && keep.has(this.selectedId) ? this.selectedId : null);
}

/**
```

```
 * Moves one shape's mesh without touching the database: used every frame while the
 * belts run. y is the height of the shape's base.
 */
pose(id: string, x: number, y: number, z: number): void {
  const mesh = this.meshes.get(id);
  if (mesh) mesh.position.set(x, y + (mesh.userData.halfHeight as number), z);
}
```

`sync` makes the scene match a list from the API exactly: remove meshes whose id is gone (disposing their geometry and material, since GPU memory isn't garbage-collected), add new ones, update the rest. This is the same idea as React's reconciliation, by hand: the API's list is the state, and the scene is redrawn to match it.

A geometry is rebuilt only when kind or size changed, tracked by a `geometryKey` string in `userData`. Moving or recoloring a shape, the common case, touches only its transform and material.

Selecting, picking and the ghost

src/shapes/ShapeLayer.ts · lines 91–119

```
/** Highlights one shape (or none) by making it glow slightly. */
select(id: string | null): void {
  this.selectedId = id;
  for (const [shapeId, mesh] of this.meshes) {
    mesh.material.emissive.set(shapeId === id ? 0x3b3b3b : 0x000000);
  }
}

/** The id of the nearest shape under the ray, if any. */
pick(raycaster: THREE.Raycaster): string | null {
  const hit = raycaster.intersectObjects([...this.meshes.values()], false)[0];
  return (hit?.object.userData.shapeId as string | undefined) ?? null;
}

showGhost(kind: ShapeKind, size: Size, color: string, x: number, z: number, rotationY: number):
void {
  const key = geometryKey(kind, size);
  if (key !== this.ghostKey) {
    this.ghost.geometry.dispose();
    this.ghost.geometry = geometryFor(kind, size);
    this.ghostKey = key;
  }
  this.ghost.material.color.set(color);
```

```
  place(this.ghost, size, x, z, rotationY, this.supportAt(x, z));
  this.ghost.visible = true;
}

hideGhost(): void {
  this.ghost.visible = false;
}
```

Selection reuses the emissive glow the joint housings use. Picking raycasts only against shape meshes and reads the id from `userData`, as joint picking reads `jointIndex`.

Geometry for each kind

src/shapes/ShapeLayer.ts · lines 121–143

```
/** Three.js builds every shape centered on its origin; ours rest on the floor or a belt. */
function place(mesh: THREE.Object3D, size: Size, x: number, z: number, rotationY: number,
baseY: number) {
  mesh.position.set(x, baseY + size.y / 2, z);
  mesh.rotation.y = rotationY * DEG;
}

function geometryKey(kind: ShapeKind, s: Size): string {
  return `${kind}:${s.x}:${s.y}:${s.z}`;
}

function geometryFor(kind: ShapeKind, s: Size): THREE.BufferGeometry {
  switch (kind) {
    case "box":
      return new THREE.BoxGeometry(s.x, s.y, s.z);
    case "cylinder":
      return new THREE.CylinderGeometry(s.x / 2, s.x / 2, s.y, 40);
    case "cone":
      return new THREE.ConeGeometry(s.x / 2, s.y, 40);
    case "sphere":
      return new THREE.SphereGeometry(s.x / 2, 40, 20);
  }
}
```

Three.js builds geometry centered on the origin, so `place` lifts every shape by half its height, plus the height of whatever it stands on: 0 for the floor, a belt's top for a shape on a conveyor. `geometryFor` is a `switch` over the union of kinds; because TypeScript knows every case returns, adding a fifth kind without handling it here is a compile error.

PART 1.14 · `src/shapes/controller.ts`

The Shapes panel

The panel and the mouse placement, in one module that main.ts calls with three functions.

State

src/shapes/controller.ts · lines 1–72

```
/**
 * The Shapes panel: create shapes, or pick a part from the inventory, place them on the
 * floor with the mouse, move and delete them. Shapes are saved through the inventory API, so
they're in the database, and the
 * panel polls every few seconds so changes from another browser appear here too.
 *
 * main.ts owns the pointer and keyboard and asks this module first: handleClick and
 * handleKey return true when they used the event.
 */
import * as THREE from "three";
import type { ArmDimensions } from "../arm/kinematics";
import { shapesApi, ShapesApiError } from "./api";
import { reachOf, reachRadiusAt, REACH_LABEL, type Reach } from "./reach";
import { ShapeLayer } from "./ShapeLayer";
import { lookFor, placedBySku, type InventoryPart } from "./fromPart";
import { SHAPE_KINDS, type NewShape, type Shape, type ShapeKind, type Size } from "./types";

const POLL_MS = 4000;
const FLOOR_LIMIT = 3; // meters, same as the API
const DEFAULT_COLORS: Record<ShapeKind, string> = { box: "#f59e0b", cylinder: "#22c55e",
sphere: "#e11d48", cone: "#a855f7" };

type Mode = { kind: "idle" } | { kind: "placing" } | { kind: "moving"; id: string };

interface Draft {
  kind: ShapeKind;
```

```
  /** centimeters, as typed */
  w: number; h: number; d: number;
  color: string;
  name: string;
  rotationY: number;
  /** centimeters */
  x: number; z: number;
}

/**
 * What's under the floor plane's way: conveyor belts, provided by the conveyor controller.
 * support(x, z) is the height a shape at (x, z) stands on; pick() is where the mouse ray
 * hits the top of a belt, so you can place a shape on one.
 */
export interface Surface {
  support(x: number, z: number): number;
  pick(raycaster: THREE.Raycaster): { x: number; z: number } | null;
}

interface Options {
  root: HTMLElement;
  scene: THREE.Scene;
  camera: THREE.Camera;
  canvas: HTMLCanvasElement;
  arm: ArmDimensions;
}

export function createShapes({ root, scene, camera, canvas, arm }: Options) {
  const layer = new ShapeLayer(reachRadiusAt(0, arm));
  scene.add(layer.root);

  let shapes: Shape[] = [];
  let parts: InventoryPart[] = [];
  /** "basic": a plain shape; "part": a shape standing for an inventory part. */
  let source: "basic" | "part" = "basic";
  let partSku: string | null = null;
  let partSearch = "";
  let selected: string | null = null;
  let mode: Mode = { kind: "idle" };
  let online = false;
  let busy = false;
```

```
  let message: { text: string; error: boolean } | null = null;
  let hover: { x: number; z: number } | null = null;
  /** True while the belts run: shapes move on their own, so editing waits until Stop. */
  let running = false;
  let surface: Surface = { support: () => 0, pick: () => null };
  const reach = (s: Pick<Shape, "kind" | "size" | "position">) => reachOf(s, arm,
surface.support(s.position.x, s.position.z));
  const draft: Draft = { kind: "box", w: 20, h: 20, d: 20, color: DEFAULT_COLORS.box, name: "",
rotationY: 0, x: 60, z: 0 };
```

The panel's state is a handful of local variables inside `createShapes`, closed over by the functions below. `Mode` is a discriminated union: idle, placing a new shape, or moving the shape with a given id. A moving mode without an id can't be represented, so it can't happen.

Talking to the API

src/shapes/controller.ts · lines 74–149

```
// ---- Talking to the API ----

async function load() {
  try {
    const [list, stock, info] = await Promise.all([shapesApi.list(), shapesApi.parts(), online
? null : shapesApi.info()]);
    if (info) dbLabel.textContent = info.db;
    online = true;
    shapes = list;
    parts = stock;
    layer.sync(shapes);
    if (message?.error && message.text.startsWith("Can't reach")) message = null;
  } catch (err) {
    online = false;
    message = { text: (err as Error).message, error: true };
  }
  render();
}

async function act(work: () => Promise<unknown>, success?: string) {
  busy = true;
  render();
  try {
    await work();
```

```
    message = success ? { text: success, error: false } : null;
    await load();
  } catch (err) {
    const e = err as ShapesApiError;
    message = { text: e.problems?.length ? e.problems.join("; ") : e.message, error: true };
  } finally {
    busy = false;
    render();
  }
}

function sizeOf(d: Draft): Size {
  const cm = (n: number) => Math.round(n) / 100;
  if (d.kind === "sphere") return { x: cm(d.w), y: cm(d.w), z: cm(d.w) };
  if (d.kind === "cylinder" || d.kind === "cone") return { x: cm(d.w), y: cm(d.h), z: cm(d.w)
};
  return { x: cm(d.w), y: cm(d.h), z: cm(d.d) };
}

function newShape(x: number, z: number): NewShape {
  const count = shapes.filter((s) => s.kind === draft.kind).length + 1;
  const sku = source === "part" ? partSku : null;
  return {
    name: draft.name.trim() || `${draft.kind[0].toUpperCase()}${draft.kind.slice(1)} ${count}`,
    kind: draft.kind, size: sizeOf(draft), color: draft.color, position: { x, z }, rotationY:
draft.rotationY, sku,
  };
}

function create(x: number, z: number) {
  if (source === "part" && !partSku) {
    message = { text: "Pick a part from the inventory list first.", error: true };
    render();
    return;
  }
  const shape = newShape(x, z);
  void act(() => shapesApi.create(shape), `Added ${shape.name}${shape.sku ? ` (${shape.sku})` :
""}.`);
  if (source === "basic") draft.name = "";
}
```

```
/** Fills the form from a part: its look (kind, size, color) and its name. */
function pickPart(sku: string) {
  const part = parts.find((p) => p.sku === sku);
  if (!part) return;
  partSku = sku;
  const look = lookFor(part);
  const cm = (m: number) => Math.round(m * 1000) / 10;
  Object.assign(draft, { kind: look.kind, w: cm(look.size.x), h: cm(look.size.y), d:
cm(look.size.z), color: look.color, name: part.name.slice(0, 40) });
  render();
}

// Poll so changes from other browsers (or the API directly) show up. Skipped mid-placement.
setInterval(() => {
  if (mode.kind === "idle" && !busy && !running) void load();
}, POLL_MS);
```

`load` reads shapes and parts together with `Promise.all`. `act` wraps every change: mark busy, call the API, show the API's own problems if it refuses, then reload. Reloading after every change, rather than patching the local list, means the screen always shows what the database holds.

Sizes are typed in centimeters and sent in meters. `pickPart` fills the form from a part's look, so a part can still be resized or recolored before it's placed. The poll every 4 s is skipped mid-placement, so a refresh never moves the ghost under the mouse.

Placing with the mouse

src/shapes/controller.ts · lines 151–241

```
// ---- Pointer: placing and moving on the floor ----

const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();
const floorPlane = new THREE.Plane(new THREE.Vector3(0, 1, 0), 0);
const hitPoint = new THREE.Vector3();

function aim(e: PointerEvent) {
  const rect = canvas.getBoundingClientRect();
  pointer.set(((e.clientX - rect.left) / rect.width) * 2 - 1, -((e.clientY - rect.top) /
rect.height) * 2 + 1);
  raycaster.setFromCamera(pointer, camera);
}
```

```
/**
 * Where the pointer meets the top of a belt, or else the floor, rounded to the centimeter;
 * null if off the floor area.
 */
function floorPoint(e: PointerEvent): { x: number; z: number } | null {
  aim(e);
  let at = surface.pick(raycaster);
  if (!at) {
    if (!raycaster.ray.intersectPlane(floorPlane, hitPoint)) return null;
    at = { x: hitPoint.x, z: hitPoint.z };
  }
  const x = Math.round(at.x * 100) / 100;
  const z = Math.round(at.z * 100) / 100;
  return Math.abs(x) <= FLOOR_LIMIT && Math.abs(z) <= FLOOR_LIMIT ? { x, z } : null;
}

function ghostSource(): { kind: ShapeKind; size: Size; color: string; rotationY: number } |
null {
  if (mode.kind === "placing") return { kind: draft.kind, size: sizeOf(draft), color:
draft.color, rotationY: draft.rotationY };
  if (mode.kind === "moving") {
    const id = mode.id;
    const s = shapes.find((x) => x.id === id);
    return s ? { kind: s.kind, size: s.size, color: s.color, rotationY: s.rotationY } : null;
  }
  return null;
}

/** Called on every pointer move. Returns true if the cursor is ours to set. */
function handleHover(e: PointerEvent): boolean {
  const src = ghostSource();
  if (src) {
    hover = floorPoint(e);
    if (hover) layer.showGhost(src.kind, src.size, src.color, hover.x, hover.z, src.rotationY);
    else layer.hideGhost();
    canvas.style.cursor = hover ? "crosshair" : "not-allowed";
    renderHint();
    return true;
  }
  aim(e);
  if (layer.pick(raycaster)) {
```

```
    canvas.style.cursor = "pointer";
    return true;
  }
  return false;
}

/** Called on a click (not a drag). Returns true if it was used. */
function handleClick(e: PointerEvent): boolean {
  if (mode.kind === "placing" || mode.kind === "moving") {
    const at = floorPoint(e);
    if (!at) return true;
    if (mode.kind === "placing") create(at.x, at.z);
    else {
      const id = mode.id;
      void act(() => shapesApi.update(id, { position: at }), "Moved.");
    }
    setMode({ kind: "idle" });
    return true;
  }
  aim(e);
  const id = layer.pick(raycaster);
  if (id) {
    setSelected(id === selected ? null : id);
    return true;
  }
  return false;
}

function handleKey(e: KeyboardEvent): boolean {
  if (e.key === "Escape" && mode.kind !== "idle") {
    setMode({ kind: "idle" });
    return true;
  }
  if ((e.key === "Delete" || e.key === "Backspace") && selected && mode.kind === "idle" &&
!running) {
    remove(selected);
    return true;
  }
  return false;
}
```

Placing needs the point where the mouse ray meets the floor. `Ray.intersectPlane` with the plane y = 0 gives it directly, with no mesh to raycast against, and it works even where the floor mesh doesn't reach. The point is rounded to the centimeter so typed and placed positions look alike.

`handleClick` and `handleKey` return true when they used the event, the contract main.ts relies on to decide whether the arm gets it.

The panel's HTML and events

src/shapes/controller.ts · lines 265–307

```
// ---- The panel ----

root.innerHTML = `
  <header>
    <h1>Shapes</h1>
    <p class="sub">Saved in the inventory database · <span class="db">connecting…</span></p>
  </header>
  <section>
    <h2>Add to the scene</h2>
    <div class="sources" role="group" aria-label="What to add">
      <button type="button" data-source="basic">Basic shape</button>
      <button type="button" data-source="part">Inventory part</button>
    </div>
    <div class="part-picker" hidden>
      <input type="search" class="part-search" placeholder="Search SKU or name" aria-
label="Search inventory parts">
      <ul class="part-list" role="listbox" aria-label="Inventory parts"></ul>
    </div>
    <div class="kinds" role="group" aria-label="Kind">
      ${SHAPE_KINDS.map((k) => `<button type="button" data-kind="${k}">${k}
</button>`).join("")}
    </div>
    <div class="dims"></div>
    <div class="row">
      <label>Color <input type="color" data-field="color"></label>
      <label>Turn ° <input type="number" data-field="rotationY" min="-360" max="360" step="5">
</label>
    </div>
    <label class="wide">Name <input type="text" data-field="name" maxlength="40"
placeholder="Automatic"></label>
    <div class="row">
      <label>X cm <input type="number" data-field="x" min="-300" max="300" step="5"></label>
```

```
      <label>Z cm <input type="number" data-field="z" min="-300" max="300" step="5"></label>
    </div>
    <p class="reach-line">At X, Z: <span class="reach"></span></p>
    <div class="actions">
      <button type="button" data-action="place" class="primary">Place with mouse</button>
      <button type="button" data-action="add">Add at X, Z</button>
    </div>
    <p class="hint" hidden></p>
  </section>
  <section>
    <h2>In the scene <span class="count"></span></h2>
    <ul class="shape-list"></ul>
  </section>
  <p class="shapes-status" role="status" hidden></p>
`;
```

The panel is one template string, built once. Events are attached to the panel root and dispatched by `data-` attributes (**event delegation**), so the list can be re-rendered freely without re-attaching handlers to new buttons.

Rendering

src/shapes/controller.ts · lines 377–446

```
function renderParts() {
  const placed = placedBySku(shapes);
  const shown = parts.filter((p) => !partSearch || p.sku.toLowerCase().includes(partSearch) ||
p.name.toLowerCase().includes(partSearch));
  partList.innerHTML = shown.length === 0
    ? `<li class="empty">${parts.length ? "No parts match." : "No parts in the inventory yet."}
</li>`
    : shown.map((p) => {
        const inScene = placed.get(p.sku) ?? 0;
        return `<li role="option" data-sku="${p.sku}" aria-selected="${p.sku === partSku}"
class="${p.sku === partSku ? "on" : ""}">
          <span class="sku">${p.sku}</span>
          <span class="pname">${escapeHtml(p.name)}</span>
          <span class="stock ${inScene > p.onHand ? "over" : ""}">${p.available} avail${inScene
? ` · ${inScene} in scene` : ""}</span>
        </li>`;
      }).join("");
}
```

```
function render() {
  root.querySelectorAll<HTMLButtonElement>("[data-source]").forEach((b) =>
b.setAttribute("aria-pressed", String(b.dataset.source === source)));
  picker.hidden = source !== "part";
  kinds.hidden = source === "part"; // a part's kind comes from its category
  if (source === "part") renderParts();
  root.querySelectorAll<HTMLButtonElement>("[data-kind]").forEach((b) => b.setAttribute("aria-
pressed", String(b.dataset.kind === draft.kind)));
  if (renderedKind !== draft.kind || dims.dataset.for !== `${draft.w}:${draft.h}:${draft.d}` &&
!dims.contains(document.activeElement)) {
    // Rebuild only when the kind changes, so typing in a field doesn't lose focus.
    dims.innerHTML =
      draft.kind === "box" ? dimInput("Width cm", "w") + dimInput("Height cm", "h") +
dimInput("Depth cm", "d")
      : draft.kind === "sphere" ? dimInput("Diameter cm", "w")
      : dimInput("Diameter cm", "w") + dimInput("Height cm", "h");
    renderedKind = draft.kind;
    dims.dataset.for = `${draft.w}:${draft.h}:${draft.d}`;
  }
  for (const key of ["color", "rotationY", "name", "x", "z"] as const) {
    const input = field(key);
    if (document.activeElement !== input) input.value = String(draft[key]);
  }

  placeBtn.textContent = mode.kind === "placing" ? "Cancel placing" : "Place with mouse";
  placeBtn.setAttribute("aria-pressed", String(mode.kind === "placing"));
  placeBtn.disabled = addBtn.disabled = !online || busy || running || (source === "part" &&
!partSku);
  if (!online) dbLabel.textContent = "offline";

  list.innerHTML = shapes.length === 0
    ? `<li class="empty">${online ? "No shapes yet. Place one with the mouse." : "Start the
inventory API to add shapes."}</li>`
    : shapes.map((s) => {
        const where = reach(s);
        const part = s.sku ? parts.find((p) => p.sku === s.sku) : undefined;
        const placed = s.sku ? placedBySku(shapes).get(s.sku) ?? 0 : 0;
        const stockNote = part
          ? `<small class="${placed > part.onHand ? "over" : ""}">${s.sku} · ${placed} in
scene, ${part.onHand} on hand</small>`
```

```
          : s.sku ? `<small class="over">${s.sku} · no longer in the inventory</small>` : "";
        const moving = mode.kind === "moving" && mode.id === s.id;
        return `<li class="${s.id === selected ? "selected" : ""}">
          <button type="button" class="pick" data-id="${s.id}" aria-pressed="${s.id ===
selected}">
            <span class="swatch" style="background:${s.color}"></span>
            <span class="label"><strong>${escapeHtml(s.name)}</strong><small>${s.kind} ·
${sizeText(s)}</small>${stockNote}
              <span class="pill ${where}">${REACH_LABEL[where]}</span></span>
          </button>
          <span class="row-actions">
            <button type="button" data-id="${s.id}" data-do="move" aria-pressed="${moving}"
${running ? "disabled" : ""} title="Click the floor to move it">${moving ? "Cancel" : "Move"}
</button>
            <button type="button" data-id="${s.id}" data-do="delete" ${running ? "disabled" :
""} title="Delete (or press Delete)">x</button>
          </span>
        </li>`;
      }).join("");
  $("span.count").textContent = shapes.length ? `(${shapes.length})` : "";

  status.hidden = !message;
  status.textContent = message?.text ?? "";
  status.classList.toggle("error", !!message?.error);
  renderHint();
}
```

`render` updates everything from state, with one subtlety: inputs aren't overwritten while focused, and the size fields are rebuilt only when the kind or the values change from outside, so typing never loses focus or the caret. User text (a shape's name) goes through `escapeHtml` before entering `innerHTML`; a name like `<img onerror=…>` stays text.

Each shape row shows its reach, and for parts the count in the scene against stock on hand, in orange when the scene uses more than the shelf holds.

The API client

`src/shapes/api.ts · lines 1–45`

```
/**
 * Talks to the inventory API (Snack 5), which stores the shapes in PostgreSQL or MongoDB.
 * The API allows calls from pages on localhost (CORS), so this dev server can use it directly.
 */
```

```
import type { InventoryPart } from "./fromPart";
import type { NewShape, Shape, ShapePatch } from "./types";
import type { Conveyor, ConveyorPatch, NewConveyor } from "../conveyors/types";
import type { NewSplitter, Splitter, SplitterPatch } from "../splitters/types";
import type { Part } from "../splitters/rules";

/** Override with VITE_INVENTORY_API, e.g. http://localhost:8080 for the API without Docker. */
export const API_BASE: string = (import.meta.env.VITE_INVENTORY_API as string | undefined) ??
"http://localhost:8081";

export class ShapesApiError extends Error {
  readonly problems: string[];
  constructor(message: string, problems: string[] = []) {
    super(message);
    this.problems = problems;
  }
}

async function call<T>(method: string, path: string, body?: unknown): Promise<T> {
  let res: Response;
  try {
    res = await fetch(`${API_BASE}/api${path}`, {
      method,
      headers: body === undefined ? undefined : { "content-type": "application/json" },
      body: body === undefined ? undefined : JSON.stringify(body),
    });
  } catch {
    throw new ShapesApiError(`Can't reach the inventory API at ${API_BASE}. Is Snack 5 running?
`);
  }
  const data = (await res.json().catch(() => ({}))) as { error?: string; problems?: string[] };
  if (!res.ok) throw new ShapesApiError(data.error ?? res.statusText, data.problems ?? []);
  return data as T;
}

export const shapesApi = {
  info: () => call<{ db: string; host: string }>("GET", "/info"),
  list: () => call<Shape[]>("GET", "/shapes"),
  parts: () => call<InventoryPart[]>("GET", "/parts"),
  create: (shape: NewShape) => call<Shape>("POST", "/shapes", shape),
  update: (id: string, patch: ShapePatch) => call<Shape>("PATCH", `/shapes/${id}`, patch),
```

```
  remove: (id: string) => call<{ deleted: string }>("DELETE", `/shapes/${id}`),
};
```

The base URL defaults to the Docker setup (nginx on 8081) and can be overridden with a Vite environment variable. A failed `fetch` (nothing listening) becomes a readable message, and the API's list of problems is kept for display. This works across origins because the API sends CORS headers for localhost pages (Chapter 5).

PART 1.15 · `src/conveyors/transport.ts`

Conveyor belts: how a belt carries things

You can build conveyor belts in the scene, and shapes resting on one ride along when the line runs. This part is the motion: plain math with no Three.js, so it's unit-tested like the kinematics.

What a belt is

src/conveyors/types.ts · lines 1–21

```
/** Conveyor belts as the inventory API stores them (snack5-
inventory/api/src/domain/conveyors.ts). */
export interface Conveyor {
  id: string;
  name: string;
  /** Meters along the direction of travel. */
  length: number;
  /** Meters across. */
  width: number;
  /** Height of the belt surface, meters. */
  height: number;
  /** Meters per second along the belt's own +x; negative runs it backwards. */
  speed: number;
  position: { x: number; z: number };
  /** Degrees about the vertical axis. */
  rotationY: number;
  createdAt: string;
  updatedAt: string;
}

export type NewConveyor = Omit<Conveyor, "id" | "createdAt" | "updatedAt">;
export type ConveyorPatch = Partial<Pick<Conveyor, "name" | "position" | "rotationY" |
"speed">>;
```

A belt is a box on the floor. `height` is its top surface, which is what a shape stands on. `speed` is along the belt's own length, so turning a belt changes where things go without changing its speed. The type matches the API's field for field (Chapter 5 stores it). `ConveyorPatch` leaves out the size: a belt of a different size is a different belt, as with shapes.

The belt's own frame

src/conveyors/transport.ts · lines 1–48

```
/**
 * How belts carry things. Pure math with no Three.js, so it's unit-tested.
 *
 * The model is deliberately simple, a "kinematic" conveyor like the arm's own gripper:
 * - a shape rests on whatever is under its center: the highest belt top at or below it, or the
floor;
```

```
 * - resting on a running belt, it moves at the belt's speed along the belt;
 * - with nothing under it any more (it rode off the end), it falls with gravity, keeping the
 *   belt's forward speed like a thrown ball, until it lands on the floor (and stops) or on a
 *   lower belt (which carries it on).
 * - passing a splitter's gate, it's sent left, right or straight on by the splitter's rules
 *   (decided by the caller); sent sideways, it slides off the belt's side at an angle.
 * Shapes don't collide with each other or with a belt's sides. Snack 3 is where real contact
 * physics lives; here the point is layout and flow.
 */
import type { Conveyor } from "./types";

type Belt = Pick<Conveyor, "length" | "width" | "height" | "speed" | "position" | "rotationY">
& { id?: string };

/** A splitter's gate: on which belt, and how far from that belt's start (its -u end). */
export interface Gate {
  id: string;
  conveyorId: string;
  along: number;
}
export type Exit = "left" | "straight" | "right";

/** A diverted item leaves at this angle to the belt's direction of travel. */
export const DIVERT_ANGLE = 60 * (Math.PI / 180);

export const GRAVITY = 9.81; // m/s²
const DEG = Math.PI / 180;
const EPS = 1e-6;

/**
 * A world point in the belt's own frame: u along its length (direction of travel), v across.
 * Three.js turns local +x to (cos θ, 0, -sin θ) for a rotation θ about +y, so this is the
inverse.
 */
export function toBelt(b: Belt, x: number, z: number): { u: number; v: number } {
  const t = b.rotationY * DEG;
  const dx = x - b.position.x;
  const dz = z - b.position.z;
  return { u: dx * Math.cos(t) - dz * Math.sin(t), v: dx * Math.sin(t) + dz * Math.cos(t) };
}
```

```
export function isOver(b: Belt, x: number, z: number): boolean {
  const { u, v } = toBelt(b, x, z);
  return Math.abs(u) <= b.length / 2 + EPS && Math.abs(v) <= b.width / 2 + EPS;
}
```

A turned belt is awkward in world coordinates and trivial in its own. `toBelt` subtracts the belt's position and rotates by −θ, giving `u` along the belt and `v` across it (Figure 1.6). In that frame every belt is an axis-aligned rectangle, so “is this point on the belt?” is two comparisons.

The sign convention is the one subtle part. Three.js turns an object's +x to (cos θ, 0, −sin θ) when `rotation.y = θ`, because +y points up and positive angles turn counter-clockwise seen from above. The math copies that exactly, so the belt drawn by Three.js and the belt the math moves things along are the same belt. The first test pins it down: a belt turned 90° runs toward −z.

`EPS` lets a point exactly on an edge count as on the belt. Without it, rounding (0.15 computed as 0.15000000000000002) would make edge cases depend on luck.

What holds a shape up

src/conveyors/transport.ts · lines 50–68

```
/** The highest belt under (x, z) whose top is at or below maxY, if any. */
export function beltUnder<B extends Belt>(belts: readonly B[], x: number, z: number, maxY =
Infinity): B | null {
  let best: B | null = null;
  for (const b of belts) {
    if (b.height <= maxY + EPS && isOver(b, x, z) && (!best || b.height > best.height)) best =
b;
  }
  return best;
}

/** What a shape at (x, z) rests on: the top of the highest belt at or below maxY, or the
floor. */
export function supportHeight(belts: readonly Belt[], x: number, z: number, maxY = Infinity):
number {
  return beltUnder(belts, x, z, maxY)?.height ?? 0;
}

/** World velocity of a belt's surface. */
export function beltVelocity(b: Belt): { vx: number; vz: number } {
  const t = b.rotationY * DEG;
  return { vx: b.speed * Math.cos(t), vz: -b.speed * Math.sin(t) };
}
```

`beltUnder` returns the highest belt under a point, but only among belts whose top is at or below `maxY`. That limit is what makes belts at different heights behave: a shape falling past a high belt's end shouldn't be caught by it, and a flat plate on the floor under a raised belt shouldn't jump up onto it. The generic `<B extends Belt>` returns the caller's own type, so the controller gets a full `Conveyor` back, not just the fields this module reads.

`beltVelocity` is the speed along `u`, turned into world x and z with the same cos/−sin pair as `toBelt`.

One step of motion

src/conveyors/transport.ts · lines 70–84

```
/** A shape while the belts run: where it is, and how fast it's moving. */
export interface Body {
  /** The shape's id, passed back to the caller when the body reaches a gate. */
  id?: string;
  /** Set when a gate sent it sideways: which belt, and which side of it (+1 is the belt's +v
side). */
  divert?: { beltId: string | undefined; side: 1 | -1 };
  x: number;
  z: number;
  /** Height of the shape's base. */
  y: number;
  /** Velocity, m/s: vx, vz while falling off a belt, vy negative while falling. */
  vx: number;
  vy: number;
  vz: number;
}
```

src/conveyors/transport.ts · lines 112–169

```
/** Advances every body by dt seconds. Bodies are changed in place. */
export function step(bodies: Iterable<Body>, belts: readonly Belt[], dt: number, options:
StepOptions | number = {}): void {
  const { gates = [], route, floorLimit = 2.95 } = typeof options === "number" ? { floorLimit:
options } : options;
  for (const body of bodies) {
    // Only what's at or below the base can hold it up: a belt above it doesn't lift it.
    const support = supportHeight(belts, body.x, body.z, body.y);
    if (body.y > support + 1e-4) {
      // Semi-implicit Euler, as in Snack 3: velocity first, then position.
      body.vy -= GRAVITY * dt;
      body.y += body.vy * dt;
      move(body, body.vx, body.vz, dt, floorLimit);
      if (body.y <= support) {
        // Landed. What it landed on is found again next step, since the fall moved it.
        body.y = supportHeight(belts, body.x, body.z, support);
        body.vx = body.vy = body.vz = 0;
        body.divert = undefined;
      }
      continue;
```

```
    }
    body.y = support;
    body.vy = 0;
    const belt = beltUnder(belts, body.x, body.z, body.y);
    if (!belt || Math.abs(belt.height - body.y) > 1e-4) {
      body.vx = body.vz = 0; // resting on the floor
      body.divert = undefined;
      continue;
    }
    // Sent sideways by a gate on this belt: keep sliding off. On another belt, ride it
normally.
    const diverted = body.divert && body.divert.beltId === belt.id ? body.divert : undefined;
    if (!diverted) body.divert = undefined;
    const v = diverted ? divertVelocity(belt, diverted.side) : beltVelocity(belt);
    const before = toBelt(belt, body.x, body.z).u;
    body.vx = v.vx;
    body.vz = v.vz;
    move(body, v.vx, v.vz, dt, floorLimit);
    if (diverted || !route || belt.id === undefined) continue;

    // Did this step carry it past a gate on this belt? Gates are measured from the belt's -u
end.
    const after = toBelt(belt, body.x, body.z).u;
    for (const gate of gates) {
      if (gate.conveyorId !== belt.id) continue;
      const g = gate.along - belt.length / 2;
      const crossed = (before < g && after >= g) || (before > g && after <= g);
      if (!crossed) continue;
      const exit = route(body, gate);
      if (exit !== "straight") {
        body.divert = { beltId: belt.id, side: sideOf(exit, belt.speed) };
        break;
      }
    }
  }
}

/** Stop at the edge of the stored floor area rather than leaving it. */
function move(body: Body, vx: number, vz: number, dt: number, limit: number): void {
  body.x = Math.max(-limit, Math.min(limit, body.x + vx * dt));
```

```
  body.z = Math.max(-limit, Math.min(limit, body.z + vz * dt));
}
```

Every shape is a `Body`: where its base is, and its velocity. Each step does one of three things. (The `divert` field and the gate check at the end of the loop belong to splitters, covered in their own part below.)

1. **In the air** (above whatever is under it): fall. Velocity first, then position, the same semi-implicit Euler as Snack 3's conveyor physics. Forward speed is kept, so an item leaving a belt flies on like a thrown ball (Figure 1.7). When it reaches what's below, it lands and stops.
2. **On the floor:** nothing. The floor doesn't move things.
3. **On a belt:** move at the belt's surface velocity, and remember it as the body's velocity in case the next step finds nothing underneath.

This is a **kinematic** model: belts set velocities directly, with no forces, friction or collisions. Shapes pass through each other and through side rails. That's a deliberate trade: the point here is layout and flow (where does each item end up?), and it keeps the step to forty lines that can be tested exactly. Snack 3 is where contact physics lives.

`move` clamps to ±2.95 m, inside the API's ±3 m floor limit, so saving a position after a run can never be refused.

Testing the belts

`tests/transport.test.ts · lines 1–52`

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { beltUnder, beltVelocity, step, supportHeight, toBelt, type Body } from
"../src/conveyors/transport";

const belt = (over: Partial<{ length: number; width: number; height: number; speed: number; x:
number; z: number; rotationY: number }> = {}) => {
  const b = { length: 1, width: 0.3, height: 0.2, speed: 0.5, x: 0, z: 0, rotationY: 0, ...over
};
  return { length: b.length, width: b.width, height: b.height, speed: b.speed, position: { x:
b.x, z: b.z }, rotationY: b.rotationY };
};
const close = (a: number, b: number, tol = 1e-9) => assert.ok(Math.abs(a - b) <= tol, `${a} vs
${b}`);

function run(bodies: Body[], belts: ReturnType<typeof belt>[], seconds: number, dt = 1 / 120) {
  for (let t = 0; t < seconds - 1e-9; t += dt) step(bodies, belts, dt);
}
```

```
test("a belt's frame: u along its length, v across, following its rotation", () => {
  const b = belt({ x: 1, z: 1, rotationY: 90 }); // turned 90°: travels toward -z
  const p = toBelt(b, 1, 0.6); // 0.4 m in the direction of travel
  close(p.u, 0.4);
  close(p.v, 0);
  const v = beltVelocity(b);
  close(v.vx, 0);
  close(v.vz, -0.5);
});

test("support: the highest belt under a point, ignoring belts above a given height", () => {
  const low = belt({ height: 0.2 });
  const high = belt({ height: 0.5, width: 0.1 });
  assert.equal(supportHeight([low, high], 0, 0), 0.5);
  assert.equal(supportHeight([low, high], 0, 0.1), 0.2); // outside the narrow high belt
  assert.equal(supportHeight([low, high], 0, 0, 0.3), 0.2); // a body at 0.3 m sits on the low
one
  assert.equal(supportHeight([low, high], 2, 0), 0); // floor
  assert.equal(beltUnder([low], 0.5, 0.15), low); // edges count
});

test("a shape on a running belt rides at the belt's speed", () => {
  const b = belt({ length: 2 }); // 0.5 m/s along +x, from x = -1 to 1
  const body: Body = { x: -0.8, z: 0, y: 0.2, vx: 0, vy: 0, vz: 0 };
  run([body], [b], 1);
  close(body.x, -0.3, 1e-6);
  close(body.y, 0.2);
});

test("it falls off the end, keeping its forward speed, and stops on the floor", () => {
  const b = belt({ length: 1, height: 0.5, speed: 0.5 });
  const body: Body = { x: 0.3, z: 0, y: 0.5, vx: 0, vy: 0, vz: 0 };
  run([body], [b], 3);
  assert.equal(body.y, 0);
  // A 0.5 m fall takes sqrt(2·0.5/9.81) = 0.32 s, at 0.5 m/s forward: lands about 0.16 m past
the end.
  close(body.x, 0.5 + 0.5 * Math.sqrt(1 / 9.81), 0.01);
  assert.deepEqual([body.vx, body.vz], [0, 0]);
});
```

Each test sets up belts, steps the simulation at 120 Hz, and checks where things end up. The fall-off test checks the landing point against the projectile formula, `end + speed · √(2h/g)`, to within a centimeter: that's the test that caught the first version of `step`, which dropped items straight down and so could never hand them to another belt.

tests/transport.test.ts · lines 53–92

```
test("falling takes the time gravity says", () => {
  const body: Body = { x: 5, z: 0, y: 0.4905, vx: 0, vy: 0, vz: 0 }; // h = g t²/2 with t =
0.316 s
  let t = 0;
  while (body.y > 0) {
    step([body], [], 1 / 1000);
    t += 1 / 1000;
  }
  close(t, Math.sqrt((2 * 0.4905) / 9.81), 0.003);
});

test("a higher belt hands over to a lower one, which carries it on", () => {
  const upper = belt({ x: 0, length: 1, height: 0.4, speed: 0.4 });
  const lower = belt({ x: 0.6, length: 1, height: 0.15, speed: 0.4, rotationY: 90 }); // runs
toward -z
  const body: Body = { x: 0, z: 0, y: 0.4, vx: 0, vy: 0, vz: 0 };
  run([body], [upper, lower], 2.5);
  close(body.y, 0.15);
  assert.ok(body.z < -0.3, `carried along the lower belt to z = ${body.z}`);
});

test("shapes on the floor, or under a belt that's above them, don't move", () => {
  const b = belt({ height: 0.5 });
  const onFloor: Body = { x: 2, z: 0, y: 0, vx: 0, vy: 0, vz: 0 };
  const under: Body = { x: 0, z: 0, y: 0, vx: 0, vy: 0, vz: 0 }; // e.g. a flat plate beneath a
high belt
  run([onFloor, under], [b], 1);
  assert.deepEqual([onFloor.x, under.x, under.y], [2, 0, 0]);
});

test("a stopped belt holds things in place, and a reversed one runs backwards", () => {
  const body: Body = { x: 0, z: 0, y: 0.2, vx: 0, vy: 0, vz: 0 };
  run([body], [belt({ speed: 0 })], 1);
  assert.equal(body.x, 0);
  run([body], [belt({ speed: -0.2 })], 1);
```

```
  close(body.x, -0.2, 1e-6);
});

test("carried shapes stop at the edge of the floor area", () => {
  const body: Body = { x: 2.8, z: 0, y: 0.2, vx: 0, vy: 0, vz: 0 };
  run([body], [belt({ x: 2.5, length: 1.5, speed: 1 })], 1);
  assert.equal(body.x, 2.95);
});
```

The gravity test steps at 1 ms and compares the fall time with √(2h/g). The hand-over test is the reason the model exists: a high belt feeding a lower belt turned 90°, with the item arriving on the second belt and travelling along it toward −z.

PART 1.16 · `src/conveyors/ConveyorLayer.ts`

Drawing the belts

Belt models built from boxes, a moving chevron texture, and a see-through preview for placing.

A model in the belt's frame

src/conveyors/ConveyorLayer.ts · lines 1–33

```
import * as THREE from "three";
import { DEG } from "../arm/kinematics";
import type { Conveyor } from "./types";

/** One chevron per this many meters of belt. */
const CHEVRON_PITCH = 0.25;
const FRAME_DEPTH = 0.06; // the belt bed under the surface
const RAIL = 0.015; // side rail thickness and height above the belt

type Belt = Pick<Conveyor, "length" | "width" | "height">;

/**
 * Draws the conveyors. Each belt is a group in its own frame: x along its length (the way
 * it runs at a positive speed), z across, y up from the floor. The group's rotation about y
 * is the stored rotationY, which is the same convention transport.ts uses, so what you see
 * and what moves the shapes agree.
 *
 * Like ShapeLayer, sync() makes the scene match the API's list, keyed by id.
 */
```

```
export class ConveyorLayer {
  readonly root = new THREE.Group();
  private readonly belts = new Map<string, { group: THREE.Group; key: string; surface:
THREE.Mesh; texture: THREE.CanvasTexture; frame: THREE.MeshStandardMaterial; speed: number }>
();
  private readonly ghost = new THREE.Group();
  private ghostKey = "";
  private readonly chevrons = chevronCanvas();
  private selectedId: string | null = null;

  constructor() {
    this.root.name = "Conveyors";
    this.ghost.visible = false;
    this.root.add(this.ghost);
  }
```

Each belt is a `Group` whose children are built in the belt's own frame: x along, z across, y up from the floor. The group's `rotation.y` is the stored angle, so a belt's model and its motion share one convention (previous part).

src/conveyors/ConveyorLayer.ts · lines 120–158

```
/** The model: a bed, the moving surface on top, side rails, end rollers and legs. */
function buildBelt(b: Belt, frame: THREE.Material, belt: THREE.Material): { group: THREE.Group;
surface: THREE.Mesh } {
  const group = new THREE.Group();
  const add = (geometry: THREE.BufferGeometry, material: THREE.Material, x: number, y: number,
z: number) => {
    const mesh = new THREE.Mesh(geometry, material);
    mesh.position.set(x, y, z);
    mesh.castShadow = true;
    mesh.receiveShadow = true;
    group.add(mesh);
    return mesh;
  };

  const bed = Math.min(FRAME_DEPTH, b.height);
  add(new THREE.BoxGeometry(b.length, bed, b.width), frame, 0, b.height - bed / 2, 0);

  // The surface is a flat plane a hair above the bed, facing up. Its UVs run along x, so
  // the chevron texture repeats along the belt's length.
  const surface = add(new THREE.PlaneGeometry(b.length, b.width).rotateX(-Math.PI / 2), belt,
```

```
0, b.height + 0.001, 0);
  surface.castShadow = false;

  for (const side of [-1, 1]) {
    add(new THREE.BoxGeometry(b.length, RAIL * 2, RAIL), frame, 0, b.height + RAIL / 2, side *
(b.width / 2 + RAIL / 2));
  }
  for (const end of [-1, 1]) {
    const roller = add(new THREE.CylinderGeometry(bed / 2, bed / 2, b.width,
20).rotateX(Math.PI / 2), frame, end * (b.length / 2), b.height - bed / 2, 0);
    roller.castShadow = false;
  }

  const legHeight = b.height - bed;
  if (legHeight > 0.01) {
    const inset = Math.min(0.1, b.length / 4);
    for (const end of [-1, 1]) {
      for (const side of [-1, 1]) {
        add(new THREE.BoxGeometry(0.03, legHeight, 0.03), frame, end * (b.length / 2 - inset),
legHeight / 2, side * (b.width / 2 - 0.02));
      }
    }
  }
  return { group, surface };
}
```

The bed is at most 6 cm deep, hanging below the surface. Below that, legs run to the floor if there's room, so a 5 cm belt is a flat slab and a 1 m belt stands on four legs. The moving surface is a separate plane a millimeter above the bed: it carries the texture, and it's the mesh the mouse ray tests when you place a shape on a belt.

Sync, and the moving chevrons

`src/conveyors/ConveyorLayer.ts · lines 34–79`

```
sync(list: readonly Conveyor[]): void {
  const keep = new Set(list.map((b) => b.id));
  for (const [id, belt] of this.belts) {
    if (!keep.has(id)) {
      this.root.remove(belt.group);
      disposeTree(belt.group);
      belt.texture.dispose();
```

```
      this.belts.delete(id);
    }
  }
  for (const b of list) {
    const key = `${b.length}:${b.width}:${b.height}`;
    let belt = this.belts.get(b.id);
    if (belt && belt.key !== key) {
      // A different size is a different model; rebuild it (sizes don't change from this app
today).
      this.root.remove(belt.group);
      disposeTree(belt.group);
      belt.texture.dispose();
      belt = undefined;
    }
    if (!belt) {
      const texture = new THREE.CanvasTexture(this.chevrons);
      texture.wrapS = THREE.RepeatWrapping;
      texture.repeat.set(b.length / CHEVRON_PITCH, 1);
      texture.colorSpace = THREE.SRGBColorSpace;
      const frame = new THREE.MeshStandardMaterial({ color: 0x4b5563, roughness: 0.6,
metalness: 0.4 });
      const { group, surface } = buildBelt(b, frame, new THREE.MeshStandardMaterial({ map:
texture, roughness: 0.9 }));
      group.traverse((o) => (o.userData.beltId = b.id));
      belt = { group, key, surface, texture, frame, speed: b.speed };
      this.belts.set(b.id, belt);
      this.root.add(group);
    }
    belt.speed = b.speed;
    belt.group.name = b.name;
    belt.group.position.set(b.position.x, 0, b.position.z);
    belt.group.rotation.y = b.rotationY * DEG;
  }
  this.select(this.selectedId && keep.has(this.selectedId) ? this.selectedId : null);
}

/** Slides each belt's chevrons along at its speed. Call every frame while running. */
animate(dt: number): void {
  // With the texture repeated over the belt, moving the texture's offset by -d moves
  // the pattern +d·pitch along the belt.
  for (const belt of this.belts.values()) belt.texture.offset.x -= (belt.speed * dt) /
```

```
CHEVRON_PITCH;
}
```

`sync` reconciles meshes with the API's list, as ShapeLayer does. Changing a belt's angle or speed only touches the group's transform; a new size rebuilds the model.

The chevron texture is drawn once on a 64 × 64 canvas, and each belt gets a `CanvasTexture` on it with `RepeatWrapping` and `repeat.x = length / 0.25`: one chevron every 25 cm, whatever the belt's length. `animate` slides the texture's offset. A texture coordinate is `uv · repeat + offset`, so decreasing the offset by d moves the pattern forward d / repeat of the belt, which is d · 0.25 m. Setting d = speed · dt / 0.25 makes the chevrons move at exactly the belt's speed, matching the shapes riding on it.

src/conveyors/ConveyorLayer.ts · lines 160–187

```
/** A dark belt with one light chevron pointing along +x (the direction of travel). */
function chevronCanvas(): HTMLCanvasElement {
  const c = document.createElement("canvas");
  c.width = 64;
  c.height = 64;
  const g = c.getContext("2d")!;
  g.fillStyle = "#1f2937";
  g.fillRect(0, 0, 64, 64);
  g.strokeStyle = "#9ca3af";
  g.lineWidth = 7;
  g.lineCap = "round";
  g.beginPath();
  g.moveTo(22, 10);
  g.lineTo(42, 32);
  g.lineTo(22, 54);
  g.stroke();
  return c;
}

function disposeTree(object: THREE.Object3D): void {
  object.traverse((o) => {
    if (o instanceof THREE.Mesh) {
      o.geometry.dispose();
      // The texture is disposed by the caller; materials are per belt.
      (Array.isArray(o.material) ? o.material : [o.material]).forEach((m: THREE.Material) =>
m.dispose());
    }
```

```
  });
}
```

A canvas texture costs nothing to download and scales to any belt. `disposeTree` releases each mesh's geometry and material when a belt is deleted, since GPU memory isn't garbage-collected.

Picking

src/conveyors/ConveyorLayer.ts · lines 80–118

```
  select(id: string | null): void {
    this.selectedId = id;
    for (const [beltId, belt] of this.belts) belt.frame.emissive.set(beltId === id ? 0x1e3a8a :
0x000000);
  }

  /** The belt under the ray, if any. */
  pick(raycaster: THREE.Raycaster): string | null {
    const hit = raycaster.intersectObjects([...this.belts.values()].map((b) => b.group), true)
[0];
    return (hit?.object.userData.beltId as string | undefined) ?? null;
  }

  /** Where the ray first hits a belt's top surface, for placing shapes on belts. */
  surfacePoint(raycaster: THREE.Raycaster): { x: number; z: number } | null {
    const hit = raycaster.intersectObjects([...this.belts.values()].map((b) => b.surface),
false)[0];
    return hit ? { x: hit.point.x, z: hit.point.z } : null;
  }

  showGhost(b: Belt, x: number, z: number, rotationY: number): void {
    const key = `${b.length}:${b.width}:${b.height}`;
    if (key !== this.ghostKey) {
      for (const child of [...this.ghost.children]) {
        this.ghost.remove(child);
        disposeTree(child);
      }
      const see = () => new THREE.MeshStandardMaterial({ color: 0x60a5fa, transparent: true,
opacity: 0.4, depthWrite: false });
      const { group } = buildBelt(b, see(), see());
      this.ghost.add(group);
```

```
      this.ghostKey = key;
    }
    this.ghost.position.set(x, 0, z);
    this.ghost.rotation.y = rotationY * DEG;
    this.ghost.visible = true;
  }

  hideGhost(): void {
    this.ghost.visible = false;
  }
}
```

`pick` raycasts against whole groups (the `true` argument recurses into children) and reads the belt id every child was given in `userData`. `surfacePoint` tests only the top planes, which is what the Shapes panel asks for: the point on a belt's top under the mouse, so a shape dropped there lands where the cursor is, not where the ray would have met the floor beneath.

PART 1.17 · `src/conveyors/controller.ts`

The Conveyors tab, and running the line

The panel that builds belts, and the Run / Stop loop that moves the shapes and saves where they end up.

State, and telling the shapes about belts

src/conveyors/controller.ts · lines 1–69

```
/**
 * The Conveyors tab: build belts, place them with the mouse, turn, reverse and delete them,
 * and Run / Stop the line. Belts are saved through the inventory API like the shapes.
 *
 * While running, transport.ts moves the shapes every frame. Nothing is saved until Stop,
 * when each shape's new position is written back with one PATCH, so a run is just a
 * preview until you stop it.
 */
import * as THREE from "three";
import { conveyorsApi, ShapesApiError } from "../shapes/api";
import type { createShapes } from "../shapes/controller";
import { ConveyorLayer } from "./ConveyorLayer";
import { step, supportHeight, type Body, type Exit, type Gate } from "./transport";
import type { Conveyor, NewConveyor } from "./types";
```

```
type Shapes = ReturnType<typeof createShapes>;
type Mode = { kind: "idle" } | { kind: "placing" } | { kind: "moving"; id: string };

const POLL_MS = 4000;
const FLOOR_LIMIT = 3;
/** Physics steps are at most this long, however slow the frame. */
const MAX_STEP = 1 / 120;

/** What the splitters tell the running line: where their gates are, and where each item goes.
*/
export interface Gates {
  list(): readonly Gate[];
  /** Asked once when a shape reaches a gate. */
  route(shapeId: string, gate: Gate): Exit;
}

interface Draft {
  /** centimeters (speed in cm/s), as typed */
  length: number; width: number; height: number; speed: number;
  rotationY: number; name: string; x: number; z: number;
}

interface Options {
  root: HTMLElement;
  runButton: HTMLButtonElement;
  scene: THREE.Scene;
  camera: THREE.Camera;
  canvas: HTMLCanvasElement;
  shapes: Shapes;
}

export function createConveyors({ root, runButton, scene, camera, canvas, shapes }: Options) {
  const layer = new ConveyorLayer();
  scene.add(layer.root);

  let belts: Conveyor[] = [];
  let selected: string | null = null;
  let mode: Mode = { kind: "idle" };
  let online = false;
  let busy = false;
```

```
  let message: { text: string; error: boolean } | null = null;
  let hover: { x: number; z: number } | null = null;
  const draft: Draft = { length: 150, width: 30, height: 30, speed: 20, rotationY: 0, name: "",
x: 70, z: -60 };

  /** Set while running: one body per shape, keyed by shape id. */
  let bodies: Map<string, Body> | null = null;
  let started: Map<string, { x: number; z: number }> = new Map();
  let gates: Gates = { list: () => [], route: () => "straight" };
  const listeners: ((belts: readonly Conveyor[]) => void)[] = [];

  // Shapes stand on whichever belt is under them, and can be placed onto one.
  shapes.setSurface({
    support: (x, z) => supportHeight(belts, x, z),
    pick: (raycaster) => layer.surfacePoint(raycaster),
  });
```

The panel mirrors the Shapes panel: a few closed-over variables, a `Mode` union, a draft in centimeters. Two variables are new: `bodies` is non-null only while the line runs, which makes “running” a fact about the data rather than a separate flag that could disagree with it, and `started` remembers where each shape was when Run was pressed.

`shapes.setSurface` is how the two panels cooperate without importing each other's internals. The Shapes panel asks a `Surface` two questions, what height is under (x, z) and where the mouse ray hits a belt's top, and this module answers with the transport math and the layer. Shapes now sit on belts, show reach from the top of the belt, and can be placed onto one.

src/shapes/controller.ts · lines 473–515

```
  // ---- For the conveyor controller ----

  /** Belts changed: shapes may now stand on (or fall off) one. */
  function setSurface(next: Surface) {
    surface = next;
    layer.supportAt = (x, z) => surface.support(x, z);
    refresh();
  }

  function refresh() {
    layer.sync(shapes);
    render();
  }
```

```
  function setRunning(on: boolean) {
    running = on;
    if (on) setMode({ kind: "idle" });
    else render();
  }

  /** After a run: keep where the belts left each shape, here and in the database. */
  function saveMoves(moves: readonly { id: string; x: number; z: number }[]) {
    const byId = new Map(moves.map((m) => [m.id, m]));
    shapes = shapes.map((s) => {
      const m = byId.get(s.id);
      return m ? { ...s, position: { x: m.x, z: m.z } } : s;
    });
    refresh();
    if (moves.length === 0) return;
    void act(async () => {
      // One request per shape; a few dozen at most, so a bulk endpoint isn't worth it.
      for (const m of moves) await shapesApi.update(m.id, { position: { x: m.x, z: m.z } });
    }, `Saved where ${moves.length} shape${moves.length === 1 ? "" : "s"} ended up.`);
  }

  return {
    layer, handleClick, handleHover, handleKey, setSurface, setRunning, saveMoves, refresh,
    /** The shapes as last loaded (or saved). */
    get shapes(): readonly Shape[] {
      return shapes;
    },
  };
}
```

Run and Stop

src/conveyors/controller.ts · lines 128–170

```
// ---- Running ----

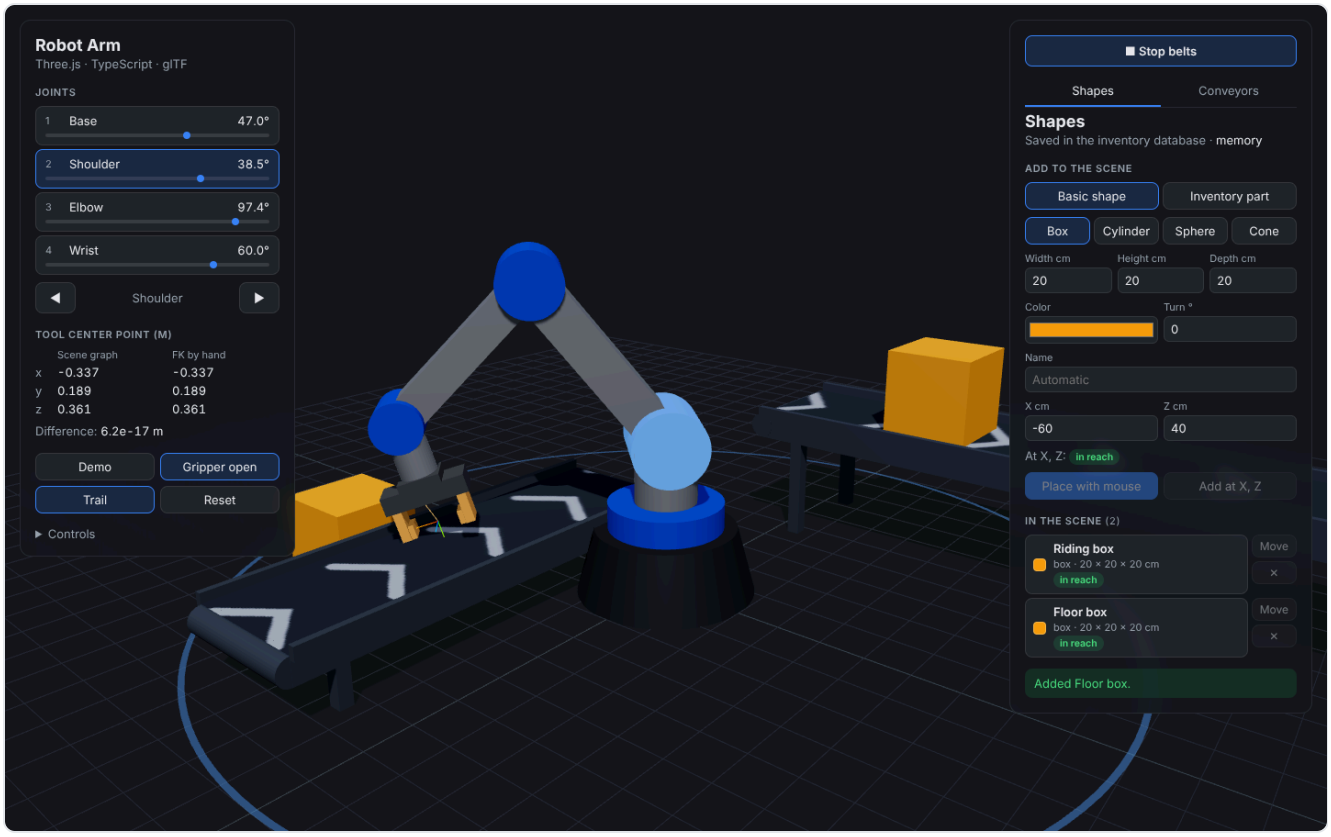
function start() {
  if (bodies) return;
  setMode({ kind: "idle" });
  bodies = new Map();
  started = new Map();
  for (const s of shapes.shapes) {
```

```
    bodies.set(s.id, { id: s.id, x: s.position.x, z: s.position.z, y: supportHeight(belts,
s.position.x, s.position.z), vx: 0, vy: 0, vz: 0 });
    started.set(s.id, { ...s.position });
  }
  shapes.setRunning(true);
  message = null;
  render();
}

function stop() {
  if (!bodies) return;
  const moves: { id: string; x: number; z: number }[] = [];
  for (const [id, b] of bodies) {
    const from = started.get(id)!;
    // Millimeters are plenty, and the API keeps whatever we send.
    const x = Math.round(b.x * 1000) / 1000;
    const z = Math.round(b.z * 1000) / 1000;
    if (Math.hypot(x - from.x, z - from.z) >= 0.001) moves.push({ id, x, z });
  }
  bodies = null;
  shapes.setRunning(false);
  shapes.saveMoves(moves);
  if (moves.length === 0) message = { text: "Nothing moved. Shapes ride a belt when placed on
top of it.", error: false };
  render();
}

/** Called every frame by main.ts. */
function update(dt: number) {
  if (!bodies) return;
  layer.animate(dt);
  const options = { gates: gates.list(), route: (b: Body, g: Gate) => gates.route(b.id!, g) };
  for (let left = dt; left > 1e-9; left -= MAX_STEP) step(bodies.values(), belts,
Math.min(MAX_STEP, left), options);
  for (const [id, b] of bodies) shapes.layer.pose(id, b.x, b.y, b.z);
}

runButton.addEventListener("click", () => (bodies ? stop() : start()));
```



The line running: the box on the back belt rides toward its far end, the Run button has become Stop, and the Shapes panel's edit buttons are disabled until the run ends.

Start makes a body for every shape, standing on whatever is under it, and freezes the Shapes panel: its poll stops and its edit buttons disable, because a refresh mid-run would snap shapes back to their saved positions.

update runs every frame from main.ts. A frame can take 16 ms or, on a slow machine, 50 ms; the physics always steps at most 1/120 s, as many times as needed. Big steps would make falls land late and could carry a fast item past a short belt in one jump. Then each shape's mesh is posed directly, with no database involved.

Stop compares each body with where it started and saves only the shapes that moved, rounded to the millimeter. Until Stop, a run is a preview: reload the page mid-run and everything is back where it was.

main.ts caps each frame's dt at 50 ms, so a tab that was hidden for a minute doesn't fast-forward. The same cap means a very slow machine runs the belts in slow motion rather than skipping: the test browser, drawing 3 frames a second, moved shapes at about a third of real speed.

Placing, moving and picking belts

`src/conveyors/controller.ts · lines 172–274`

```
// ---- Pointer ----
```

```
const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();
const floorPlane = new THREE.Plane(new THREE.Vector3(0, 1, 0), 0);
const hitPoint = new THREE.Vector3();

function aim(e: PointerEvent) {
  const rect = canvas.getBoundingClientRect();
  pointer.set(((e.clientX - rect.left) / rect.width) * 2 - 1, -((e.clientY - rect.top) /
rect.height) * 2 + 1);
  raycaster.setFromCamera(pointer, camera);
}

function floorPoint(e: PointerEvent): { x: number; z: number } | null {
  aim(e);
  if (!raycaster.ray.intersectPlane(floorPlane, hitPoint)) return null;
  const x = Math.round(hitPoint.x * 100) / 100;
  const z = Math.round(hitPoint.z * 100) / 100;
  return Math.abs(x) <= FLOOR_LIMIT && Math.abs(z) <= FLOOR_LIMIT ? { x, z } : null;
}

function ghostBelt(): { length: number; width: number; height: number; rotationY: number } |
null {
  if (mode.kind === "placing") return { length: draft.length / 100, width: draft.width / 100,
height: draft.height / 100, rotationY: draft.rotationY };
  if (mode.kind === "moving") {
    const id = mode.id;
    return belts.find((b) => b.id === id) ?? null;
  }
  return null;
}

/** True while placing or moving a belt: main.ts gives this module the pointer first then. */
const isActive = () => mode.kind !== "idle";

function handleHover(e: PointerEvent): boolean {
  const g = ghostBelt();
  if (!g) return false;
  hover = floorPoint(e);
  if (hover) layer.showGhost(g, hover.x, hover.z, g.rotationY);
  else layer.hideGhost();
  canvas.style.cursor = hover ? "crosshair" : "not-allowed";
```

```
  renderHint();
  return true;
}

/** Placing or moving: the click puts the belt down. */
function handleClick(e: PointerEvent): boolean {
  if (mode.kind === "idle") return false;
  const at = floorPoint(e);
  if (!at) return true;
  if (mode.kind === "placing") create(at.x, at.z);
  else {
    const id = mode.id;
    void act(() => conveyorsApi.update(id, { position: at }), "Moved.");
  }
  setMode({ kind: "idle" });
  return true;
}

/** Idle: a click on a belt selects it. Asked after the shapes, which sit on top. */
function handlePick(e: PointerEvent): boolean {
  aim(e);
  const id = layer.pick(raycaster);
  if (!id) return false;
  setSelected(id === selected ? null : id);
  showTab();
  return true;
}

function hoversBelt(e: PointerEvent): boolean {
  aim(e);
  return layer.pick(raycaster) !== null;
}

function handleKey(e: KeyboardEvent): boolean {
  if (e.key === "Escape" && mode.kind !== "idle") {
    setMode({ kind: "idle" });
    return true;
  }
  if ((e.key === "Delete" || e.key === "Backspace") && selected && mode.kind === "idle" &&
!bodies) {
    remove(selected);
```

```
    return true;
  }
  return false;
}

function setMode(next: Mode) {
  mode = next;
  hover = null;
  if (mode.kind === "idle") {
    layer.hideGhost();
    canvas.style.cursor = "";
  }
  render();
}

function setSelected(id: string | null) {
  selected = id;
  layer.select(id);
  render();
}

/** Clicking a belt in the scene opens its tab; main.ts wires the tabs. */
let showTab = () => {};
```

Belts are placed on the floor plane like shapes. While a belt (or a splitter) is being placed, main.ts gives that module the pointer first. Otherwise shapes are asked first, then splitter gates, then belts: whatever sits on top wins the click.

src/main.ts · lines 189–206

```
canvas.addEventListener("pointerup", (e) => {
  if (Math.hypot(e.clientX - pressX, e.clientY - pressY) > 4) return; // that was an orbit drag
  // Whoever is placing something gets the click; then shapes (they sit on top of belts), then
  // splitter gates (they stand over belts), then belts.
  if (splitters.isActive() && splitters.handleClick(e)) return;
  if (belts.isActive() && belts.handleClick(e)) return;
  if (shapes.handleClick(e)) return; // placing, moving or selecting a shape
  if (splitters.handlePick(e)) return;
  if (belts.handlePick(e)) return;
  const hit = housingAt(e);
  if (hit) select(hit.userData.jointIndex as number);
});
canvas.addEventListener("pointermove", (e) => {
  if (e.buttons !== 0) return;
  if (splitters.isActive() ? splitters.handleHover(e) : belts.isActive() ? belts.handleHover(e)
: shapes.handleHover(e)) return;
  canvas.style.cursor = housingAt(e) || splitters.hoversGate(e) || belts.hoversBelt(e) ?
"pointer" : "";
});
```

The panel

src/conveyors/controller.ts · lines 276–345

```
// ---- The panel ----

root.innerHTML = `
  <section>
    <h2>Build a belt</h2>
    <div class="dims">
      <label>Length cm <input type="number" data-field="length" min="30" max="400" step="10">
</label>
      <label>Width cm <input type="number" data-field="width" min="10" max="120" step="5">
</label>
      <label>Height cm <input type="number" data-field="height" min="5" max="120" step="5">
</label>
    </div>
    <div class="row">
      <label>Speed cm/s <input type="number" data-field="speed" min="-100" max="100" step="5">
</label>
```

```
      <label>Turn ° <input type="number" data-field="rotationY" min="-360" max="360" step="15">
</label>
    </div>
    <label class="wide">Name <input type="text" data-field="name" maxlength="40"
placeholder="Automatic"></label>
    <div class="row">
      <label>X cm <input type="number" data-field="x" min="-300" max="300" step="5"></label>
      <label>Z cm <input type="number" data-field="z" min="-300" max="300" step="5"></label>
    </div>
    <div class="actions">
      <button type="button" data-action="place" class="primary">Place with mouse</button>
      <button type="button" data-action="add">Add at X, Z</button>
    </div>
    <p class="hint" hidden></p>
  </section>
  <section>
    <h2>Belts <span class="count"></span></h2>
    <ul class="shape-list belt-list"></ul>
  </section>
  <p class="shapes-status" role="status" hidden></p>
`;
const $ = <T extends HTMLElement>(sel: string) => root.querySelector(sel) as T;
const list = $("ul.belt-list");
const status = $("p.shapes-status");
const hint = $("p.hint");
const placeBtn = $<HTMLButtonElement>('[data-action="place"]');
const addBtn = $<HTMLButtonElement>('[data-action="add"]');
const field = (name: keyof Draft) => root.querySelector(`[data-field="${name}"]`) as
HTMLInputElement;

root.addEventListener("input", (e) => {
  const input = e.target as HTMLInputElement;
  const key = input.dataset.field as keyof Draft | undefined;
  if (!key) return;
  if (key === "name") draft.name = input.value;
  else if (input.value !== "") draft[key] = Number(input.value);
  if (mode.kind === "placing" && hover) layer.showGhost(ghostBelt()!, hover.x, hover.z,
draft.rotationY);
});
placeBtn.addEventListener("click", () => setMode(mode.kind === "placing" ? { kind: "idle" } : {
kind: "placing" }));
```

```
addBtn.addEventListener("click", () => create(draft.x / 100, draft.z / 100));
list.addEventListener("click", (e) => {
  const btn = (e.target as HTMLElement).closest<HTMLElement>("[data-id]");
  if (!btn) return;
  const id = btn.dataset.id!;
  const belt = belts.find((b) => b.id === id);
  if (!belt) return;
  switch (btn.dataset.do) {
    case "delete":
      return remove(id);
    case "move":
      setSelected(id);
      return setMode(mode.kind === "moving" && mode.id === id ? { kind: "idle" } : { kind:
"moving", id });
    case "turn":
      return void act(() => conveyorsApi.update(id, { rotationY: (belt.rotationY + 90) % 360
}));
    case "reverse":
      return void act(() => conveyorsApi.update(id, { speed: -belt.speed }));
    default:
      setSelected(id === selected ? null : id);
  }
});
```

Turn and reverse are one PATCH each: `rotationY + 90` modulo 360, and `-speed`. Every button that changes something is disabled while the line runs, in the list's HTML (`${dis}`) as well as in the click handler's state, so the screen never offers what the code would refuse.

src/conveyors/controller.ts · lines 346–406

```
function render() {
  for (const key of ["length", "width", "height", "speed", "rotationY", "name", "x", "z"] as
const) {
    const input = field(key);
    if (document.activeElement !== input) input.value = String(draft[key]);
  }
  const running = !!bodies;
  const locked = !online || busy || running;
  placeBtn.textContent = mode.kind === "placing" ? "Cancel placing" : "Place with mouse";
  placeBtn.setAttribute("aria-pressed", String(mode.kind === "placing"));
  placeBtn.disabled = addBtn.disabled = locked;
```

```
  runButton.textContent = running ? "■ Stop belts" : "▶ Run belts";
  runButton.setAttribute("aria-pressed", String(running));
  runButton.disabled = !online || busy || (!running && belts.length === 0);
  runButton.title = belts.length === 0 ? "Add a conveyor first" : running ? "Stop and save
where the shapes ended up" : "Start the belts";

  list.innerHTML = belts.length === 0
    ? `<li class="empty">${online ? "No belts yet. Place one with the mouse." : "Start the
inventory API to add belts."}</li>`
    : belts.map((b) => {
        const moving = mode.kind === "moving" && mode.id === b.id;
        const dis = running ? "disabled" : "";
        const dir = b.speed === 0 ? "stopped" : `${Math.abs(Math.round(b.speed * 100))} cm/s
${b.speed > 0 ? "forward" : "reversed"}`;
        return `<li class="${b.id === selected ? "selected" : ""}">
          <button type="button" class="pick" data-id="${b.id}" aria-pressed="${b.id ===
selected}">
            <span class="swatch belt-swatch"></span>
            <span class="label"><strong>${escapeHtml(b.name)}</strong>
              <small>${cm(b.length)} × ${cm(b.width)} cm, ${cm(b.height)} cm high</small>
              <small>${dir} · turned ${Math.round(b.rotationY)}°</small></span>
          </button>
          <span class="row-actions">
            <button type="button" data-id="${b.id}" data-do="move" aria-pressed="${moving}"
${dis}>${moving ? "Cancel" : "Move"}</button>
            <button type="button" data-id="${b.id}" data-do="turn" ${dis} title="Turn 90°">↻
90°</button>
            <button type="button" data-id="${b.id}" data-do="reverse" ${dis} title="Run the
other way">⇄</button>
            <button type="button" data-id="${b.id}" data-do="delete" ${dis} title="Delete (or
press Delete)">×</button>
          </span>
        </li>`;
      }).join("");
  $("span.count").textContent = belts.length ? `(${belts.length})` : "";

  status.hidden = !message;
  status.textContent = message?.text ?? "";
  status.classList.toggle("error", !!message?.error);
  renderHint();
}
```

```
function renderHint() {
  if (mode.kind === "idle") {
    hint.hidden = !bodies;
    hint.textContent = "Running. Shapes on a belt ride along and drop off the end. Stop saves
where they are.";
    return;
  }
  hint.hidden = false;
  const verb = mode.kind === "placing" ? "place it" : "move it there";
  hint.textContent = hover
    ? `x ${(hover.x * 100).toFixed(0)} cm, z ${(hover.z * 100).toFixed(0)} cm. Click to
${verb}.`
    : `Click the floor to ${verb}. Esc cancels.`;
}

render();
void load();
```

Wiring: tabs and the frame loop

src/main.ts · lines 27–56

```
// Shapes on the floor, stored in the inventory database (Snack 5's API).
const shapes = createShapes({ root: document.getElementById("shapes-panel")!, scene, camera,
canvas, arm: ARM });

// Conveyor belts, also stored there. Shapes on a running belt ride along.
const belts = createConveyors({
  root: document.getElementById("conveyors-panel")!,
  runButton: document.getElementById("btn-run") as HTMLButtonElement,
  scene, camera, canvas, shapes,
});

// Splitters: gates on the belts that send each item left, right or straight on, by your rules.
const splitters = createSplitters({ root: document.getElementById("splitters-panel")!, scene,
camera, canvas, shapes, conveyors: belts });

// The right-hand panel's tabs.
const tabs = {
  shapes: document.getElementById("tab-shapes")!,
  conveyors: document.getElementById("tab-conveyors")!,
```

```
  splitters: document.getElementById("tab-splitters")!,
};
function showTab(name: keyof typeof tabs) {
  for (const [key, tab] of Object.entries(tabs)) {
    const on = key === name;
    tab.setAttribute("aria-selected", String(on));
    document.getElementById(tab.getAttribute("aria-controls")!)!.hidden = !on;
  }
}
tabs.shapes.addEventListener("click", () => showTab("shapes"));
tabs.conveyors.addEventListener("click", () => showTab("conveyors"));
tabs.splitters.addEventListener("click", () => showTab("splitters"));
belts.onShowTab(() => showTab("conveyors"));
```

The right-hand panel became two tabs. Showing one hides the other's panel through the `aria-controls` link, which is also what a screen reader uses to announce them. Clicking a belt in the scene switches to the Conveyors tab.

src/main.ts · lines 238–239

```
gripper?.update(dt);
belts.update(dt); // moves shapes riding the belts while running
```

One line in the frame loop drives the whole line: `belts.update(dt)`, which does nothing unless it's running.

PART 1.18 · `src/splitters/rules.ts`

Splitters: rules that decide where items go

A splitter is a gate on a belt that sends each item left, right or straight on. What it separates is up to you: an ordered list of rules, stored in the database as data. This part is the decision itself, pure functions with no Three.js.

A rule list as data

src/splitters/types.ts · lines 1–58

```
/** Splitters as the inventory API stores them (snack5-inventory/api/src/domain/splitters.ts).
*/
export const EXITS = ["left", "straight", "right"] as const;
export type Exit = (typeof EXITS)[number];
```

```
export type FieldType = "text" | "number" | "boolean";

/** What a condition can test. Sizes are meters in the API; the panel shows centimeters. */
export const FIELDS = {
  kind: { type: "text", label: "Shape kind" },
  color: { type: "text", label: "Color" },
  name: { type: "text", label: "Shape name" },
  sku: { type: "text", label: "SKU" },
  width: { type: "number", label: "Width (cm)" },
  height: { type: "number", label: "Height (cm)" },
  depth: { type: "number", label: "Depth (cm)" },
  isPart: { type: "boolean", label: "Is an inventory part" },
  "part.category": { type: "text", label: "Part category" },
  "part.location": { type: "text", label: "Part location" },
  "part.name": { type: "text", label: "Part name" },
  "part.onHand": { type: "number", label: "Part on hand" },
  "part.available": { type: "number", label: "Part available" },
  "part.low": { type: "boolean", label: "Part low on stock" },
} as const satisfies Record<string, { type: FieldType; label: string }>;
export type Field = keyof typeof FIELDS;

export type Op = "eq" | "ne" | "in" | "contains" | "lt" | "lte" | "gt" | "gte";
export const OPS: Record<FieldType, readonly Op[]> = {
  text: ["eq", "ne", "in", "contains"],
  number: ["eq", "ne", "lt", "lte", "gt", "gte"],
  boolean: ["eq"],
};
export const OP_LABEL: Record<Op, string> = { eq: "is", ne: "is not", in: "is one of",
contains: "contains", lt: "<", lte: "≤", gt: ">", gte: "≥" };

export interface Condition {
  field: Field;
  op: Op;
  value: string | number | boolean | string[];
}

export interface Rule {
  conditions: Condition[];
  exit: Exit;
}
```

```
export interface Splitter {
  id: string;
  name: string;
  conveyorId: string;
  /** Meters from the start of the belt (its -x end before turning). */
  along: number;
  rules: Rule[];
  otherwise: Exit;
  createdAt: string;
  updatedAt: string;
}

export type NewSplitter = Omit<Splitter, "id" | "createdAt" | "updatedAt">;
export type SplitterPatch = Partial<Pick<Splitter, "name" | "along" | "rules" | "otherwise">>;
```

A rule is a list of conditions that must all hold, and an exit. A condition is three values: a **field**, an **operator** and a **value**. Keeping rules this plain is what lets them live in the database and be edited from anywhere: the panel, the API, or SQL. Nothing about a rule is code.

`FIELDS` gives each field a type, and `OPS` says which operators each type allows: text can be equal, not equal, one of a list, or contain something; numbers compare; true/false fields can only be equal. The editor builds its dropdowns from these two tables, so a rule like “kind > box” can't even be expressed. `satisfies` checks the table's shape without widening its type, so `FIELDS[f].type` stays the exact literal text, number or boolean.

Fields starting with `part.` describe the **inventory part** a shape stands for: its category, location and stock.

Reading a field, and matching one condition

src/splitters/rules.ts · lines 1–62

```
/**
 * Which way does a splitter send an item? Pure functions over a shape and the inventory
 * part it stands for, so they're unit-tested and the panel can preview every decision.
 *
 * Part fields are looked up from the latest inventory list when the item reaches the gate,
 * so changing a part in the database (its category, location or stock) changes where its
 * items go on the next pass, without touching the splitter.
 */
import type { Shape } from "../shapes/types";
import type { InventoryPart } from "../shapes/fromPart";
import type { Condition, Exit, Field, Splitter } from "./types";
```

```
export type Part = InventoryPart & { location?: string; onHand: number; available: number;
low?: boolean; reorderLevel?: number };

/** The value of one field for a shape, or undefined when it doesn't apply (part fields for a
plain shape). */
export function fieldValue(field: Field, shape: Shape, part: Part | undefined): string | number
| boolean | undefined {
  switch (field) {
    case "kind": return shape.kind;
    case "color": return shape.color;
    case "name": return shape.name;
    case "sku": return shape.sku ?? undefined;
    case "width": return shape.size.x;
    case "height": return shape.size.y;
    case "depth": return shape.size.z;
    case "isPart": return shape.sku !== null;
    case "part.category": return part?.category;
    case "part.location": return part?.location;
    case "part.name": return part?.name;
    case "part.onHand": return part?.onHand;
    case "part.available": return part?.available;
    case "part.low": return part ? (part.low ?? part.available <= (part.reorderLevel ?? 0)) :
undefined;
  }
}

/** Sizes are stored in meters but typed in centimeters; compare to the nearest millimeter so
20 cm equals 0.2 m. */
const close = (a: number, b: number) => Math.abs(a - b) < 0.0005;

export function matches(c: Condition, shape: Shape, part: Part | undefined): boolean {
  const v = fieldValue(c.field, shape, part);
  // A condition about a part never matches a shape that isn't one, even "is not".
  if (v === undefined) return false;
  if (typeof v === "number") {
    const x = Number(c.value);
    switch (c.op) {
      case "eq": return close(v, x);
      case "ne": return !close(v, x);
      case "lt": return v < x && !close(v, x);
      case "lte": return v < x || close(v, x);
```

```
      case "gt": return v > x && !close(v, x);
      case "gte": return v > x || close(v, x);
      default: return false;
    }
  }
  if (typeof v === "boolean") return c.op === "eq" && v === c.value;
  const text = v.toLowerCase();
  switch (c.op) {
    case "eq": return text === String(c.value).toLowerCase();
    case "ne": return text !== String(c.value).toLowerCase();
    case "contains": return text.includes(String(c.value).toLowerCase());
    case "in": return Array.isArray(c.value) && c.value.some((s) => s.toLowerCase() === text);
    default: return false;
  }
```

`fieldValue` is one `switch` that maps a field name to a value from the shape or its part. It returns `undefined` when the field doesn't apply, a part field for a plain shape. `matches` then treats undefined as “no match” for every operator, including “is not”. Otherwise “part category is not plate” would be true for a plain red box, which has no category at all: true by a technicality, and surprising.

Sizes are compared to the millimeter. The panel takes centimeters and the API stores meters, and `20 / 100` is exactly 0.2, but a size computed elsewhere might be 0.20000000000000004. Exact equality on floating-point numbers is a classic bug, so `close` allows half a millimeter.

Text comparisons ignore case, so “Plate”, “plate” and “PLATE” are the same category. The API also lowercases text values when it saves a rule.

First match wins

src/splitters/rules.ts · lines 64–69

```
/** The first rule whose conditions all hold decides; otherwise the splitter's default. rule is
-1 for the default. */
export function route(splitter: Pick<Splitter, "rules" | "otherwise">, shape: Shape, part: Part
| undefined): { exit: Exit; rule: number } {
  const i = splitter.rules.findIndex((r) => r.conditions.length > 0 && r.conditions.every((c)
=> matches(c, shape, part)));
  return i >= 0 ? { exit: splitter.rules[i].exit, rule: i } : { exit: splitter.otherwise, rule:
-1 };
}
```

`route` is the whole decision: the first rule whose conditions all hold picks the exit; if none does, the splitter's `otherwise` exit. It returns the rule's index too, so the panel can say *why* an item went where it did.

Order matters, as in a firewall or a mail filter. A rule “plates → left” above “anything taller than 10 cm → right” sends a tall plate left; swap them and it goes right. That's why the editor has ↑ ↓ buttons, and why the API stores the rules as one ordered list rather than as separate rows.

Testing the rules

tests/splitters.test.ts · lines 1–51

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { matches, route, type Part } from "../src/splitters/rules";
import { step, sideOf, type Body, type Gate } from "../src/conveyors/transport";
import type { Shape } from "../src/shapes/types";
import type { Condition, Rule } from "../src/splitters/types";

const shape = (over: Partial<Shape> = {}): Shape => ({
  id: "s1", name: "Crate", kind: "box", size: { x: 0.2, y: 0.15, z: 0.2 }, color: "#e11d48",
  position: { x: 0, z: 0 }, rotationY: 0, sku: null, createdAt: "", updatedAt: "", ...over,
});
const plate: Part = { sku: "PL-200", name: "Mounting plate, 200 mm", category: "plate", unit:
"plate", onHand: 45, available: 41, location: "B-01-2", reorderLevel: 10 };
const c = (field: Condition["field"], op: Condition["op"], value: Condition["value"]):
Condition => ({ field, op, value });

test("text conditions ignore case; 'is one of' takes a list", () => {
  assert.ok(matches(c("color", "eq", "#E11D48"), shape(), undefined));
  assert.ok(matches(c("kind", "in", ["sphere", "BOX"]), shape(), undefined));
  assert.ok(matches(c("name", "contains", "rat"), shape(), undefined));
  assert.ok(!matches(c("kind", "ne", "box"), shape(), undefined));
});

test("sizes compare in meters, to the millimeter", () => {
  assert.ok(matches(c("height", "eq", 0.15), shape({ size: { x: 0.2, y: 0.15000000000000002, z:
0.2 } }), undefined));
  assert.ok(matches(c("height", "gte", 0.15), shape(), undefined));
  assert.ok(!matches(c("height", "gt", 0.15), shape(), undefined));
  assert.ok(matches(c("width", "lt", 0.25), shape(), undefined));
});
```

```
test("part conditions read the inventory part, and never match a plain shape", () => {
  const p = shape({ sku: "PL-200" });
  assert.ok(matches(c("part.category", "eq", "plate"), p, plate));
  assert.ok(matches(c("part.available", "gt", 40), p, plate));
  assert.ok(matches(c("isPart", "eq", true), p, plate));
  assert.ok(matches(c("isPart", "eq", false), shape(), undefined));
  assert.ok(!matches(c("part.category", "ne", "plate"), shape(), undefined)); // not a part: no
part.category at all
  assert.ok(matches(c("part.low", "eq", false), p, plate));
  assert.ok(matches(c("part.low", "eq", true), p, { ...plate, available: 10 }));
});

test("the first matching rule wins; otherwise the default", () => {
  const rules: Rule[] = [
    { conditions: [c("part.category", "eq", "plate")], exit: "left" },
    { conditions: [c("color", "eq", "#e11d48"), c("height", "gt", 0.1)], exit: "right" },
  ];
  const s = { rules, otherwise: "straight" as const };
  assert.deepEqual(route(s, shape({ sku: "PL-200" }), plate), { exit: "left", rule: 0 });
  assert.deepEqual(route(s, shape(), undefined), { exit: "right", rule: 1 });
  assert.deepEqual(route(s, shape({ size: { x: 0.2, y: 0.05, z: 0.2 } }), undefined), { exit:
"straight", rule: -1 });
  // Changing the part in the database changes the decision, with the same rules.
  assert.deepEqual(route(s, shape({ sku: "PL-200", color: "#000000" }), { ...plate, category:
"gripper" }), { exit: "straight", rule: -1 });
});
```

The last test is the one that matters for “I want to change it by changing the database”: the same rules give a different answer when only the part's category is different. The arm app doesn't cache decisions; it asks `route` with the latest inventory each time an item reaches a gate.

PART 1.19 · `src/conveyors/transport.ts`

Gates on the running line

How the belts' motion code asks a splitter what to do, and what “send it left” means physically.

Gates, sides and the push-off velocity

src/conveyors/transport.ts · lines 19–28

```
/** A splitter's gate: on which belt, and how far from that belt's start (its -u end). */
export interface Gate {
  id: string;
  conveyorId: string;
  along: number;
}
export type Exit = "left" | "straight" | "right";

/** A diverted item leaves at this angle to the belt's direction of travel. */
export const DIVERT_ANGLE = 60 * (Math.PI / 180);
```

To the transport code, a gate is just a belt id and a distance along the belt. It doesn't know about rules: it asks a `route` callback, so the same motion code works whatever decides the exits.

src/conveyors/transport.ts · lines 86–110

```
export interface StepOptions {
  gates?: readonly Gate[];
  /** Asked once when a body crosses a gate: where should it go? */
  route?: (body: Body, gate: Gate) => Exit;
  floorLimit?: number;
}

/**
 * Which side of a belt is "left" depends on which way it runs. Seen from above, facing the
 * direction of travel, right is the belt's +v side when it runs forward (+u), and -v when
 * it runs backwards.
 */
export function sideOf(exit: "left" | "right", speed: number): 1 | -1 {
  const forward = speed >= 0 ? 1 : -1;
  return (exit === "right" ? forward : -forward) as 1 | -1;
}

/** World velocity of an item pushed off a belt's side at DIVERT_ANGLE to its travel. */
export function divertVelocity(b: Belt, side: 1 | -1): { vx: number; vz: number } {
  const t = b.rotationY * DEG;
  const du = b.speed * Math.cos(DIVERT_ANGLE);
  const dv = side * Math.abs(b.speed) * Math.sin(DIVERT_ANGLE);
  // u points along (cos θ, -sin θ) in world x, z, and v along (sin θ, cos θ): see toBelt.
```

```
  return { vx: du * Math.cos(t) + dv * Math.sin(t), vz: -du * Math.sin(t) + dv * Math.cos(t) };
}
```

Left and right are taken facing the way the belt runs, as a person standing on the line would say them. Seen from above with y up, facing +x, your right hand points to +z, which is the belt's +v. Running backwards you face −u, and right becomes −v. `sideOf` is that rule, and a test checks both directions (Figure 1.8).

A diverted item leaves at 60° to the travel direction at the belt's speed: `cos 60°` of it still along the belt, `sin 60°` sideways. The result is turned into world x and z with the same (cos θ, −sin θ) and (sin θ, cos θ) axes as `toBelt`. Once it's past the side rail nothing holds it up, and the existing falling code takes over: it drops to the floor, or onto a belt running alongside.

Crossing a gate

src/conveyors/transport.ts · lines 139–163

```
    // Sent sideways by a gate on this belt: keep sliding off. On another belt, ride it
normally.
    const diverted = body.divert && body.divert.beltId === belt.id ? body.divert : undefined;
    if (!diverted) body.divert = undefined;
    const v = diverted ? divertVelocity(belt, diverted.side) : beltVelocity(belt);
    const before = toBelt(belt, body.x, body.z).u;
    body.vx = v.vx;
    body.vz = v.vz;
    move(body, v.vx, v.vz, dt, floorLimit);
    if (diverted || !route || belt.id === undefined) continue;

    // Did this step carry it past a gate on this belt? Gates are measured from the belt's -u
end.
    const after = toBelt(belt, body.x, body.z).u;
    for (const gate of gates) {
      if (gate.conveyorId !== belt.id) continue;
      const g = gate.along - belt.length / 2;
      const crossed = (before < g && after >= g) || (before > g && after <= g);
      if (!crossed) continue;
      const exit = route(body, gate);
      if (exit !== "straight") {
        body.divert = { beltId: belt.id, side: sideOf(exit, belt.speed) };
        break;
      }
    }
```

```
  }
}
```

A gate is a line across the belt, so “reached the gate” means **crossed** it during this step: the item's `u` was before the gate and is now at or past it (or the reverse on a backwards belt). Checking for “exactly at the gate” would almost never be true, because items move a few millimeters per step.

Each item is asked about once per gate. A diverted item is skipped (it's leaving), and an item sent straight has passed the line and won't cross it again. The gate is measured from the belt's start (its −u end), so `along - length / 2` converts it to the belt-centered `u` that `toBelt` returns.

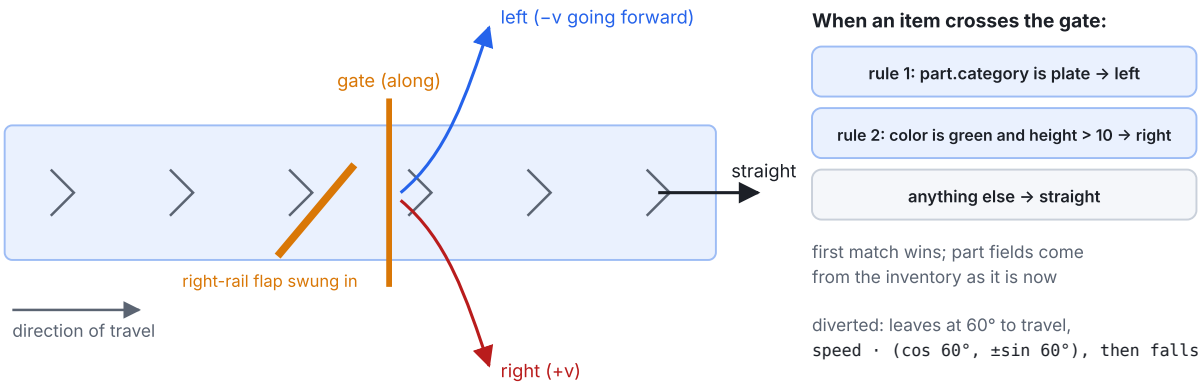


Figure 1.8. A splitter seen from above. Left and right are taken facing the way the belt runs, so they swap when the belt is reversed.

Testing the gates

`tests/splitters.test.ts · lines 53–100`

```
// ---- Gates on a running belt ----

const belt = { id: "b1", length: 2, width: 0.4, height: 0.3, speed: 0.5, position: { x: 0, z: 0
}, rotationY: 0 };
const gate: Gate = { id: "g1", conveyorId: "b1", along: 1 }; // the middle of the belt
const body = (id: string): Body => ({ id, x: -0.8, z: 0, y: 0.3, vx: 0, vy: 0, vz: 0 });
function run(bodies: Body[], belts: typeof belt[], exits: Record<string, "left" | "right" |
"straight">, seconds: number, asked: string[] = []) {
  const route = (b: Body, g: Gate) => (asked.push(`${b.id}@${g.id}`), exits[b.id!]);
  for (let t = 0; t < seconds; t += 1 / 120) step(bodies, belts, 1 / 120, { gates: [gate],
route });
}
```

```
test("facing the way the belt runs, right is +v going forward and -v going backward", () => {
  assert.equal(sideOf("right", 0.5), 1);
  assert.equal(sideOf("left", 0.5), -1);
  assert.equal(sideOf("right", -0.5), -1);
});

test("a gate sends items left, right or straight, and asks once per item", () => {
  const [l, r, s] = [body("l"), body("r"), body("s")];
  const asked: string[] = [];
  run([l, r, s], [belt], { l: "left", r: "right", s: "straight" }, 6, asked);
  assert.deepEqual(asked, ["l@g1", "r@g1", "s@g1"]);
  // Belt runs +x; seen from above facing +x, right is +z.
  assert.equal(l.y, 0);
  assert.ok(l.z < -0.2, `left went to -z: ${l.z}`);
  assert.equal(r.y, 0);
  assert.ok(r.z > 0.2, `right went to +z: ${r.z}`);
  assert.ok(Math.abs(s.z) < 1e-9 && s.x > 1, `straight rode off the end: ${s.x}`);
  // Diverted items leave near the gate, not at the end.
  assert.ok(l.x > 0 && l.x < 0.6 && r.x > 0 && r.x < 0.6, `${l.x}, ${r.x}`);
});

test("on a reversed belt, left and right follow the direction of travel", () => {
  const b = { ...belt, speed: -0.5 };
  const it: Body = { id: "x", x: 0.8, z: 0, y: 0.3, vx: 0, vy: 0, vz: 0 };
  for (let t = 0; t < 4; t += 1 / 120) step([it], [b], 1 / 120, { gates: [gate], route: () =>
"right" });
  assert.ok(it.z < -0.2, `facing -x, right is -z: ${it.z}`);
});

test("gates only act on their own belt, and a diverted item lands on a belt beside it", () => {
  const side = { id: "b2", length: 2, width: 0.4, height: 0.15, speed: 0.5, position: { x: 0.2,
z: 0.45 }, rotationY: 0 };
  const it = body("p");
  for (let t = 0; t < 2.5; t += 1 / 120) step([it], [belt, side], 1 / 120, { gates: [gate, {
id: "g2", conveyorId: "b2", along: 0.1 }], route: () => "right" });
  assert.equal(it.y, 0.15); // caught by the lower belt on the right
  assert.ok(it.divert === undefined, "riding normally again");
  const x = it.x;
  for (let t = 0; t < 0.5; t += 1 / 120) step([it], [belt, side], 1 / 120, { gates: [gate],
route: () => "right" });
```

```
  assert.ok(it.x > x + 0.2, "carried along the second belt");
});
```

Three items cross one gate: the route callback is asked exactly three times, in order, and each ends up on the right side of the belt, or off its end. The diverted ones leave near the gate rather than at the end, which shows the push happened at the right place. The last test chains belts: an item pushed right lands on a lower belt alongside and is carried on, with its `divert` cleared.

PART 1.20 · `src/splitters/controller.ts`

Drawing gates, and the Splitters tab

The gate model with its swinging flaps, and the panel where rules are written.

A gate in its own frame

src/splitters/SplitterLayer.ts · lines 1–29

```
import * as THREE from "three";
import { DEG } from "../arm/kinematics";
import type { Conveyor } from "../conveyors/types";
import type { Exit, Splitter } from "./types";

const FLAP_ANGLE = 50 * DEG; // how far a flap swings into the belt
const SWING_RATE = 10; // 1/s: flaps ease toward their target, fast but visibly
const HOLD = 0.9; // seconds a flap stays out after an item was sent its way

interface Gate {
  group: THREE.Group;
  /** Hinged on the +z (right, going forward) and -z (left) rails of the gate's own frame. */
  flaps: { right: THREE.Object3D; left: THREE.Object3D };
  /** Current and target swing, 0..1, and when each flap may close again. */
  swing: { right: number; left: number };
  until: { right: number; left: number };
  beam: THREE.MeshStandardMaterial;
  key: string;
}

/**
 * Draws each splitter as a diverter gate on its belt: an amber gantry across the belt at the
 * gate's position, and two flaps hinged on the side rails. When an item is sent left, the flap
 * on the right swings across the belt and guides it off the left side, and the other way
```

```
round.
 *
 * Each gate is built in its own frame: x along the direction of travel (so the gate is turned
 * round on a reversed belt), z to the right, y up. Then "right" is always +z, whichever way
 * the belt is turned or running.
 */
```

src/splitters/SplitterLayer.ts · lines 133–166

```
function placeOnBelt(group: THREE.Object3D, belt: Conveyor, along: number): void {
  const t = belt.rotationY * DEG;
  const u = along - belt.length / 2;
  group.position.set(belt.position.x + u * Math.cos(t), 0, belt.position.z - u * Math.sin(t));
  group.rotation.y = t + (belt.speed < 0 ? Math.PI : 0);
}

function buildGate(belt: Pick<Conveyor, "width" | "height">, beam: THREE.Material, flap:
THREE.Material) {
  const group = new THREE.Group();
  const w = belt.width;
  const top = belt.height + 0.22;
  const add = (geo: THREE.BufferGeometry, mat: THREE.Material, x: number, y: number, z: number,
parent: THREE.Object3D = group) => {
    const m = new THREE.Mesh(geo, mat);
    m.position.set(x, y, z);
    m.castShadow = true;
    parent.add(m);
    return m;
  };
  // Gantry: two posts outside the rails and a beam across, so the gate is easy to spot and
click.
  for (const side of [-1, 1]) add(new THREE.BoxGeometry(0.03, top, 0.03), beam, 0, top / 2,
side * (w / 2 + 0.04));
  add(new THREE.BoxGeometry(0.04, 0.04, w + 0.11), beam, 0, top, 0);

  // Flaps: hinged at the rail just upstream of the gate, lying along the rail when idle.
  const length = w * 0.9;
  const hingeX = -length * Math.cos(FLAP_ANGLE) * 0.5;
  const make = (side: 1 | -1) => {
    const hinge = new THREE.Group();
    hinge.position.set(hingeX, belt.height + 0.04, side * (w / 2 - 0.01));
    add(new THREE.BoxGeometry(length, 0.06, 0.012), flap, length / 2, 0, 0, hinge);
```

```
    add(new THREE.CylinderGeometry(0.012, 0.012, 0.08, 12), beam, 0, 0, 0, hinge);
    group.add(hinge);
    return hinge;
  };
  return { group, flaps: { right: make(1), left: make(-1) } };
```

Each gate is a group built in a frame where +x is the direction of travel and +z is the right-hand side. `placeOnBelt` turns the group by the belt's angle, plus 180° when the belt runs backwards. After that, “the right-hand flap” is always the one at +z, whatever the belt is doing, and the swing code never needs to think about the belt's direction.

Each flap sits inside its own hinge group, positioned at the rail. Rotating the hinge swings the flap about its end, the same trick as the arm's joints: put the pivot in a parent object and rotate that.

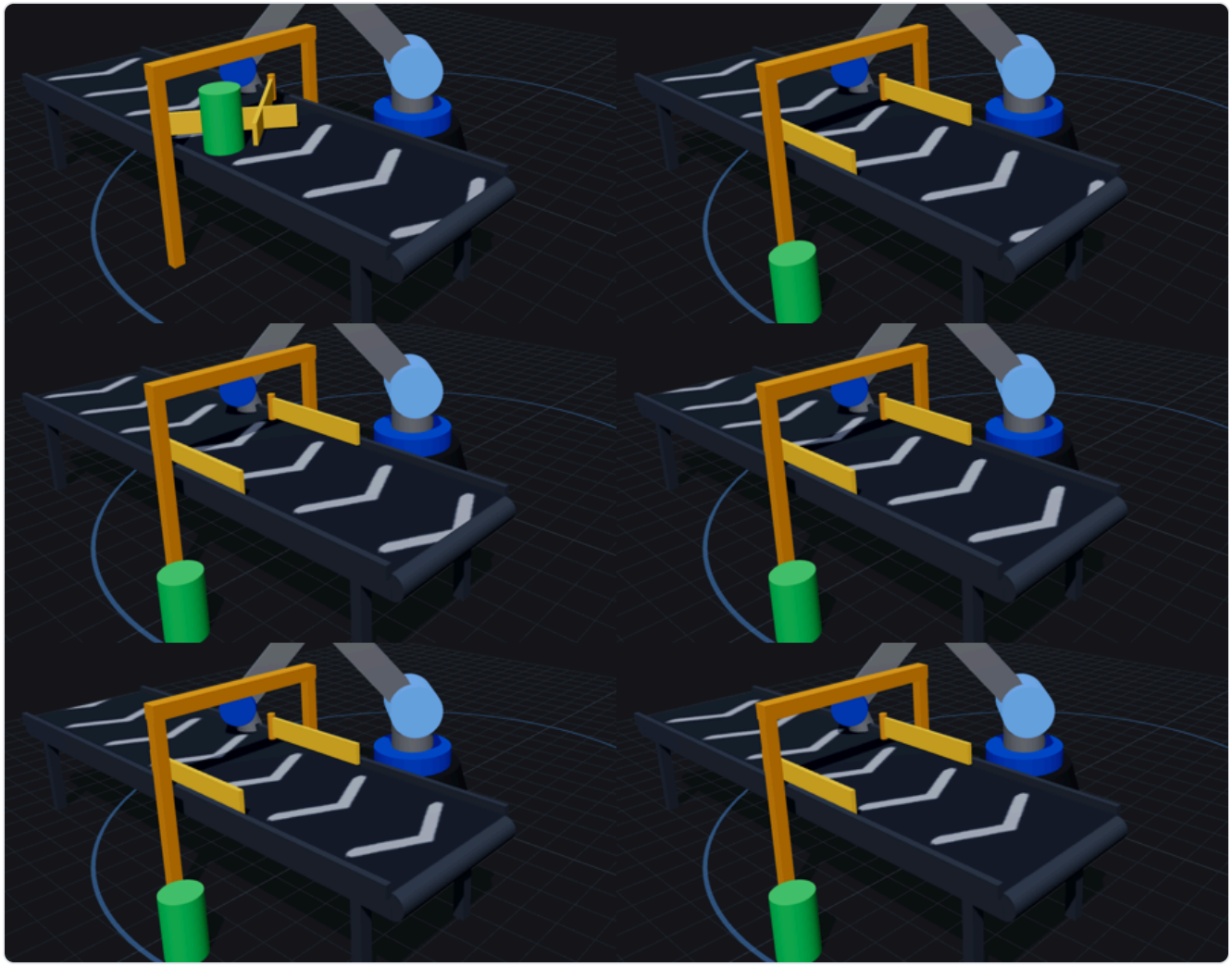
Swinging the flaps

src/splitters/SplitterLayer.ts · lines 80–100

```
/** An item was routed at this gate: swing the flap that guides it out. */
trigger(id: string, exit: Exit): void {
  const g = this.gates.get(id);
  if (!g || exit === "straight") return;
  // Sending left needs the right-hand flap across the belt, and the other way round.
  const flap = exit === "left" ? "right" : "left";
  g.until[flap] = this.time + HOLD;
}

update(dt: number): void {
  this.time += dt;
  const k = 1 - Math.exp(-SWING_RATE * dt);
  for (const g of this.gates.values()) {
    for (const side of ["right", "left"] as const) {
      const target = this.time < g.until[side] ? 1 : 0;
      g.swing[side] += (target - g.swing[side]) * k;
      // Positive rotation about y turns +x toward -z: the right flap swings left, into the
belt.
      g.flaps[side].rotation.y = (side === "right" ? 1 : -1) * FLAP_ANGLE * g.swing[side];
    }
  }
}
```

To send an item **left**, the flap on the **right** rail swings across and guides it out: `trigger` records until when that flap should stay out. `update` eases each flap toward out or in with the same frame-rate-independent factor as the demo motion, `1 - e^(-rate·dt)`. A positive rotation about y turns +x toward −z, so the right flap swings positive and the left flap negative. The flaps are only drawn: the motion code has already diverted the item, so the picture follows the decision rather than causing it.



A run filmed from in front, panels hidden. The can matches “kind is cylinder → right”, so the flap on the far (left) rail swings across and the can drops off the near side.

The splitter's state, and plugging into the line

`src/splitters/controller.ts · lines 1–71`

```
/**
 * The Splitters tab: put diverter gates on belts and write the rules that decide where each
 * item goes. Rules are saved through the inventory API, so they can also be changed from the
 * API or the database directly; the tab re-reads them, and the inventory, every few seconds,
 * including while the line runs. A change takes effect for the next item that reaches the
gate.
```

```
 */
import * as THREE from "three";
import { splittersApi, ShapesApiError } from "../shapes/api";
import type { createShapes } from "../shapes/controller";
import type { createConveyors } from "../conveyors/controller";
import { toBelt } from "../conveyors/transport";
import type { Conveyor } from "../conveyors/types";
import { SHAPE_KINDS, type Shape } from "../shapes/types";
import { route, type Part } from "./rules";
import { SplitterLayer } from "./SplitterLayer";
import { EXITS, FIELDS, OP_LABEL, OPS, type Condition, type Exit, type Field, type Rule, type
Splitter } from "./types";

const POLL_MS = 3000;
const EXIT_LABEL: Record<Exit, string> = { left: "← left", straight: "↑ straight", right:
"right →" };
/** Size fields are meters in the API and centimeters on screen. */
const IN_CM = new Set<Field>(["width", "height", "depth"]);

interface Options {
  root: HTMLElement;
  scene: THREE.Scene;
  camera: THREE.Camera;
  canvas: HTMLCanvasElement;
  shapes: ReturnType<typeof createShapes>;
  conveyors: ReturnType<typeof createConveyors>;
}

export function createSplitters({ root, scene, camera, canvas, shapes, conveyors }: Options) {
  const layer = new SplitterLayer();
  scene.add(layer.root);

  let splitters: Splitter[] = [];
  let parts: Part[] = [];
  let selected: string | null = null;
  let placing = false;
  let online = false;
  let busy = false;
  let message: { text: string; error: boolean } | null = null;
  const draft = { name: "", beltId: "", along: 50 };
  /** The selected splitter's rules as being edited, and whether they differ from what's saved.
```

```
*/
  let edit: { rules: Rule[]; otherwise: Exit } | null = null;
  let dirty = false;
  /** The latest decision at each gate, shown in the panel while the line runs. */
  const lastRouted = new Map<string, { shape: string; exit: Exit; rule: number }>();

  const belts = () => conveyors.belts;
  const partFor = (s: Shape) => (s.sku ? parts.find((p) => p.sku === s.sku) : undefined);

  // The running line asks here when an item reaches a gate. Parts are the latest loaded,
  // so a change in the inventory applies from the next item on.
  conveyors.setGates({
    list: () => splitters.filter((s) => belts().some((b) => b.id === s.conveyorId)),
    route: (shapeId, gate) => {
      const s = splitters.find((x) => x.id === gate.id);
      const shape = shapes.shapes.find((x) => x.id === shapeId);
      if (!s || !shape) return "straight";
      const decision = route(s, shape, partFor(shape));
      lastRouted.set(s.id, { shape: shape.name, ...decision });
      layer.trigger(s.id, decision.exit);
      if (s.id === selected) renderLive();
      return decision.exit;
    },
  });
  conveyors.onBeltsChange(() => {
    layer.sync(splitters, belts());
    render();
  });
```

`conveyors.setGates` is how the splitters reach the running line without the line knowing about rules. The line asks for the gates and, when an item crosses one, calls `route` with the shape's id. This module finds the shape, looks up its part in the latest inventory list, applies the rules, starts the flap and returns the exit.

Here is where “change the database, change the sorting” happens. Splitters and parts are re-read every 3 s, **including while the belts run**, unlike shapes and belts, which are frozen during a run. The next item through a gate is judged on the inventory as it is now.

Loading, saving, and edits that aren't lost

src/splitters/controller.ts · lines 73–144

```
// ---- API ----

async function load() {
  try {
    const [list, stock] = await Promise.all([splittersApi.list(), splittersApi.parts()]);
    online = true;
    splitters = list;
    parts = stock;
    layer.sync(splitters, belts());
    if (selected && !splitters.some((s) => s.id === selected)) select(null);
    // Someone else changed the rules (API, database, another browser): show theirs unless
we're mid-edit.
    if (selected && !dirty) edit = editable(splitters.find((s) => s.id === selected)!);
    if (message?.error && message.text.startsWith("Can't reach")) message = null;
  } catch (err) {
    online = false;
    message = { text: (err as Error).message, error: true };
  }
  render();
}

async function act(work: () => Promise<unknown>, success?: string) {
  busy = true;
  render();
  try {
    await work();
    message = success ? { text: success, error: false } : null;
    await load();
  } catch (err) {
    const e = err as ShapesApiError;
    message = { text: e.problems?.length ? e.problems.join("; ") : e.message, error: true };
  } finally {
    busy = false;
    render();
  }
}

setInterval(() => {
```

```
  if (!busy && !placing) void load();
}, POLL_MS);

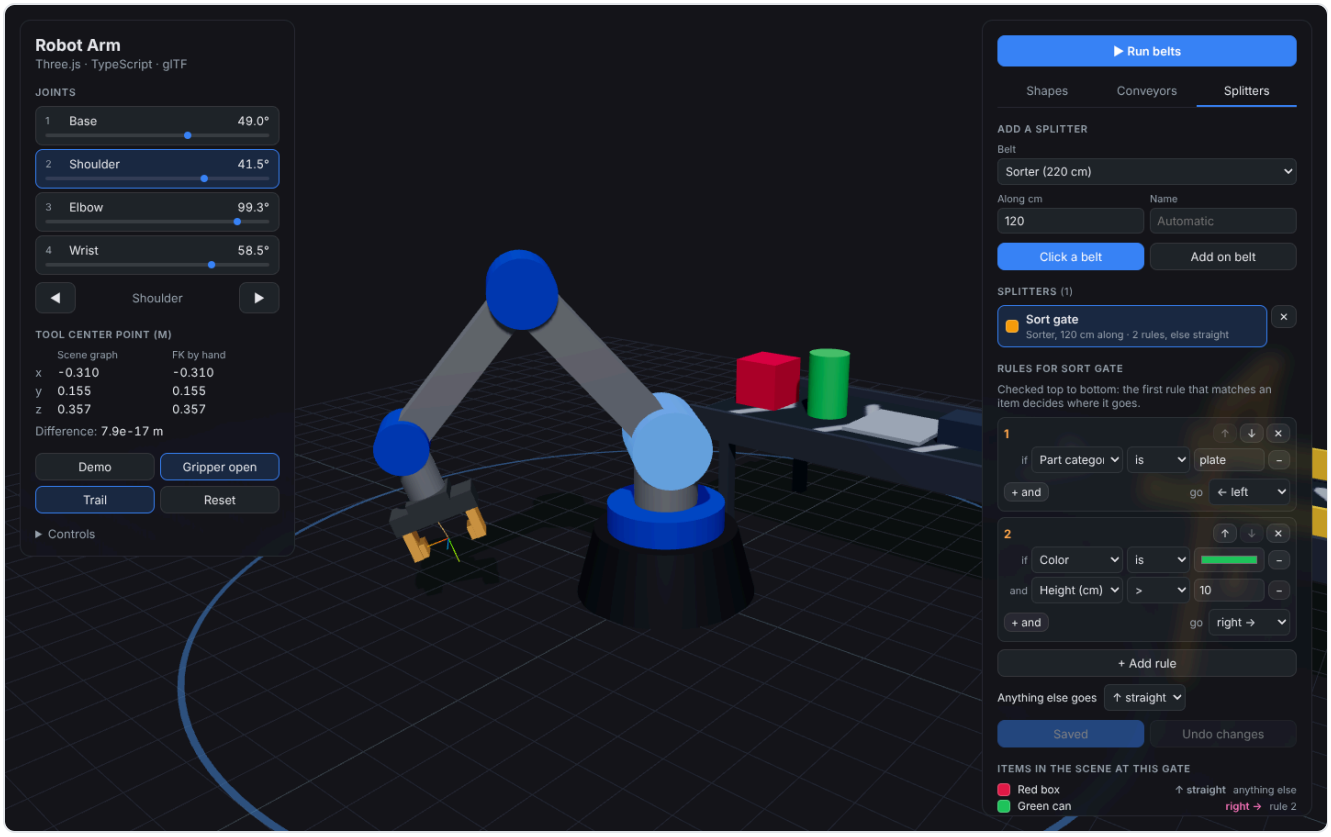
function create(beltId: string, along: number) {
  const belt = belts().find((b) => b.id === beltId);
  if (!belt) {
    message = { text: "Pick a belt first.", error: true };
    return render();
  }
  const name = draft.name.trim() || `Splitter ${splitters.length + 1}`;
  void act(async () => {
    const s = await splittersApi.create({ name, conveyorId: beltId, along:
Math.min(belt.length, Math.max(0, along)), rules: [], otherwise: "straight" });
    select(s.id);
  }, `Added ${name}. Now add rules to say what goes where.`);
  draft.name = "";
}

function select(id: string | null) {
  selected = id;
  dirty = false;
  const s = splitters.find((x) => x.id === id);
  edit = s ? editable(s) : null;
  layer.select(id);
  render();
}

function save() {
  if (!selected || !edit) return;
  const id = selected;
  const body = { rules: edit.rules.map(toApi), otherwise: edit.otherwise };
  void act(async () => {
    await splittersApi.update(id, body);
    dirty = false;
  }, "Rules saved.");
}
```

The rule editor works on a copy, `edit`, with a `dirty` flag. A poll replaces the copy with the saved rules only when nothing is being edited, so changes made elsewhere (another browser, the API, SQL) appear without wiping what you're typing. Save sends the whole ordered list; Undo goes back to what's saved. Switching to another splitter with unsaved changes asks first.

The rule editor



Two rules being edited. The list at the bottom previews where each item in the scene would go at this gate, before saving.

src/splitters/controller.ts · lines 285–378

```
root.addEventListener("input", (e) => {
  const el = e.target as HTMLInputElement | HTMLSelectElement;
  const key = el.dataset.draft as keyof typeof draft | undefined;
  if (key === "along") draft.along = Number(el.value || 0);
  else if (key === "name" || key === "beltId") draft[key] = el.value;
  if (key) return;
  if (!edit) return;
  // Rule editor: data-r is the rule, data-c the condition, data-k what's being changed.
  if (el.dataset.otherwise !== undefined) {
    edit.otherwise = el.value as Exit;
    return changed();
  }
  const r = Number(el.dataset.r);
  const rule = edit.rules[r];
  if (!rule) return;
  if (el.dataset.k === "exit") rule.exit = el.value as Exit;
  else {
```

```
    const cond = rule.conditions[Number(el.dataset.c)];
    if (!cond) return;
    if (el.dataset.k === "field") {
      cond.field = el.value as Field;
      const type = FIELDS[cond.field].type;
      if (!OPS[type].includes(cond.op)) cond.op = OPS[type][0];
      cond.value = defaultValue(cond.field);
      return changed(true);
    }
    if (el.dataset.k === "op") {
      cond.op = el.value as Condition["op"];
      if (cond.op === "in" && !Array.isArray(cond.value)) cond.value = String(cond.value) ?
[String(cond.value)] : [];
      if (cond.op !== "in" && Array.isArray(cond.value)) cond.value = cond.value.join(", ");
      return changed(true);
    }
    if (el.dataset.k === "value") cond.value = readValue(cond, el.value);
  }
  changed();
});

/** Re-draws the preview at once, and the editor's rows only when their layout changed (so
typing keeps focus). */
function changed(relayout = false) {
  dirty = true;
  if (relayout) renderRules();
  renderPreview();
  renderButtons();
}

root.addEventListener("click", (e) => {
  const btn = (e.target as HTMLElement).closest<HTMLButtonElement>("button");
  if (!btn || !root.contains(btn)) return;
  if (btn === placeBtn) return setPlacing(!placing);
  if (btn === addBtn) return create(draft.beltId, draft.along / 100);
  const id = btn.dataset.id;
  if (id) {
    if (btn.dataset.do === "delete") {
      const s = splitters.find((x) => x.id === id);
      if (id === selected) select(null);
      return void act(() => splittersApi.remove(id), s ? `Deleted ${s.name}.` : undefined);
```

```
    }
    if (id !== selected && !confirmDiscard()) return;
    return select(id === selected ? null : id);
  }
  if (!edit) return;
  const r = Number(btn.dataset.r);
  switch (btn.dataset.do) {
    case "add-rule":
      edit.rules.push({ conditions: [{ field: "kind", op: "eq", value: "box" }], exit: "left"
});
      return changed(true);
    case "add-cond":
      if (edit.rules[r].conditions.length < 5) edit.rules[r].conditions.push({ field: "color",
op: "eq", value: "#e11d48" });
      return changed(true);
    case "del-cond": {
      const cs = edit.rules[r].conditions;
      cs.splice(Number(btn.dataset.c), 1);
      if (cs.length === 0) edit.rules.splice(r, 1);
      return changed(true);
    }
    case "del-rule":
      edit.rules.splice(r, 1);
      return changed(true);
    case "up":
    case "down": {
      const to = btn.dataset.do === "up" ? r - 1 : r + 1;
      if (to < 0 || to >= edit.rules.length) return;
      [edit.rules[r], edit.rules[to]] = [edit.rules[to], edit.rules[r]];
      return changed(true);
    }
    case "save":
      return save();
    case "revert":
      dirty = false;
      edit = editable(splitters.find((s) => s.id === selected)!);
      return render();
  }
});
```

One `input` listener and one `click` listener handle the whole editor, by event delegation. Each control carries its rule index (`data-r`), condition index (`data-c`) and what it changes (`data-k`). Changing a condition's field resets its operator and value to ones that fit the new field's type. Changing a value only updates the data, without re-rendering the rows, so typing never loses the cursor.

src/splitters/controller.ts · lines 462–498

```
function conditionRow(c: Condition, r: number, i: number): string {
  const type = FIELDS[c.field].type;
  const fieldSel = `<select data-r="${r}" data-c="${i}" data-k="field" aria-
label="Field">${(Object.keys(FIELDS) as Field[])
    .map((f) => `<option value="${f}" ${f === c.field ? "selected" : ""}>${FIELDS[f].label}
</option>`).join("")}</select>`;
  const opSel = `<select data-r="${r}" data-c="${i}" data-k="op" aria-
label="Comparison">${OPS[type]
    .map((o) => `<option value="${o}" ${o === c.op ? "selected" : ""}>${OP_LABEL[o]}
</option>`).join("")}</select>`;
  const attrs = `data-r="${r}" data-c="${i}" data-k="value" aria-label="Value"`;
  let input: string;
  if (type === "boolean") {
    input = `<select ${attrs}><option value="true" ${c.value === true ? "selected" :
""}>yes</option><option value="false" ${c.value === false ? "selected" : ""}>no</option>
</select>`;
  } else if (type === "number") {
    const v = IN_CM.has(c.field) ? Math.round(Number(c.value) * 1000) / 10 : Number(c.value);
    input = `<input type="number" step="any" value="${v}" ${attrs}>`;
  } else if (c.field === "color" && (c.op === "eq" || c.op === "ne")) {
    input = `<input type="color" value="${esc(String(c.value))}" ${attrs}>`;
  } else {
    const list = { kind: "dl-kind", "part.category": "dl-category", sku: "dl-sku",
"part.location": "dl-location" }[c.field as string];
    const v = Array.isArray(c.value) ? c.value.join(", ") : String(c.value);
    input = `<input type="text" value="${esc(v)}" ${list ? `list="${list}"` : ""}
placeholder="${c.op === "in" ? "a, b, c" : ""}" ${attrs}>`;
  }
  return `<div class="cond">${i > 0 ? `<span class="and">and</span>` : `<span
class="and">if</span>`}${fieldSel}${opSel}${input}
    <button type="button" data-do="del-cond" data-r="${r}" data-c="${i}" title="Remove
condition" aria-label="Remove condition">–</button></div>`;
}
```

```
/** Where every shape would go at this gate under the rules being edited, saved or not. */
function renderPreview() {
  if (!edit) return;
  const items = shapes.shapes;
  preview.innerHTML = items.length === 0
    ? `<li class="empty">No shapes in the scene.</li>`
    : items.map((s) => {
        const d = route(edit!, s, partFor(s));
        return `<li><span class="swatch" style="background:${esc(s.color)}"></span><span
class="pname">${esc(s.name)}</span>
          <span class="exit ${d.exit}">${EXIT_LABEL[d.exit]}</span><small>${d.rule >= 0 ? `rule
${d.rule + 1}` : "anything else"}</small></li>`;
      }).join("");
}
```

The value control follows the field: yes/no for true/false fields, a number box in centimeters for sizes, a color picker for “color is”, and a text box with suggestions for the rest. The suggestions (`<datalist>`) come from the inventory as it is now, such as the categories and locations that exist.

`renderPreview` runs `route` on every shape with the rules **as edited**, saved or not. That makes the editor a tool for thinking: change a rule and immediately see which items would go where.

CHAPTER 1

Interview questions

Q1 **What's the difference between an object's local and world transform, and how does Three.js get from one to the other?**

Local is relative to the parent; world is relative to the scene. Three.js stores position, quaternion and scale, builds a local 4×4 matrix from them, and computes the world matrix as the parent's world matrix times the local one, recursively from the root. It does this lazily before rendering, or on demand with `updateWorldMatrix`. In the arm, each joint's local transform is just "up by the link length, rotated by my angle"; the hierarchy does the composing.

Q2 **Three.js already knows where the tool is. Why write forward kinematics by hand?**

Three reasons. It's an independent check: if the scene graph were built wrong, the two answers would disagree, and the panel shows the difference live. It runs without a renderer, so tests, IK and clearance checks can run in plain Node. And it documents the conventions (zero pose, signs, axis order) in a dozen readable lines, which is what everyone else on a team builds on.

Q3 **How would FK change if you added a wrist roll joint?**

The planar shortcut breaks as soon as a joint axis isn't parallel to the others, so in general you'd switch to the matrix chain: one rotation and translation per joint, multiplied in order (or DH parameters). In this specific case, a roll about the tool's own axis doesn't move the TCP, only its orientation, so the position formula survives but the orientation needs the full chain.

Q4 **A model looks inside-out after export. What do you check?**

Winding order (front faces must be counter-clockwise from outside), normals pointing outward, and negative scale on a node, which flips winding because the transform's determinant is negative. Setting the material to `DoubleSide` is a useful diagnostic, since the faces reappear, but it's not a fix: it doubles fill cost and lighting will still be wrong if the normals are.

Q5 **Why does glTF have buffers, buffer views and accessors instead of just arrays of numbers?**

Separation of storage from interpretation. Buffers are raw bytes, views are GPU-uploadable ranges (possibly interleaved with a stride), accessors describe types, counts and bounds. Many meshes can share one blob, loaders can upload views directly without parsing each number, and features like sparse accessors and quantization fit in without changing the container.

Q6 **How do you make animation frame-rate independent? What's wrong with** `x = lerp(x, target, 0.1)` **every frame?**

Scale everything by dt. The lerp-per-frame version converges in a number of frames, not a number of seconds, so it's twice as fast at 120 fps. Using a factor of `1 − e^(−λ·dt)` makes the decay exponential in real time and identical at any frame rate. Also clamp dt, so a backgrounded tab doesn't produce one enormous step.

Q7 **How did you test 3D math?**

In layers. Known poses pin down conventions (zero pose straight up, +90° on the shoulder along −X). An independent implementation, the full matrix chain, is compared on 1,000 seeded random poses. Invariants that must always hold (never beyond full reach) are checked on many poses. And a scenario test samples ten minutes of the demo to prove joint limits and floor clearance.

Q8 **What would you change to show fifty arms in a factory layout?**

Share geometry and materials between arms (today each housing has its own material so it can glow; selection could instead be an outline pass or a single swapped material). Use instancing for repeated parts, turn off matrix auto-updates for static scenery, keep the shadow camera tight or bake shadows, and add level of detail for distant arms.

Q9 **How would you show objects from a database in a Three.js scene, and keep them current?**

Treat the API's list as the state and reconcile the scene to it: add meshes for new ids, update changed ones, remove and dispose missing ones, rebuilding geometry only when the shape's size or kind changed. Poll or subscribe for changes, and skip refreshes during interactions like placing. The ShapeLayer does exactly this, keyed by shape id.

Q10 **Your belts are turned at arbitrary angles. How do you tell whether a shape is on one?**

Transform the shape's position into the belt's frame: subtract the belt's center and rotate by minus its angle. Then it's a rectangle check, $|u| \le \text{length}/2$ and $|v| \le \text{width}/2$. The rotation uses the same convention as Three.js's rotation.y, and a test pins it: a belt turned 90° carries things toward −z.

Q11 **Why not use a physics engine for the conveyors?**

The question the feature answers is where items flow, and a kinematic model answers it exactly, with deterministic tests: ride at belt speed, fall as a projectile off the end, land on whatever is below. A physics engine adds friction, contact and tipping, which matter for jams and transfers, at the cost of tuning and non-exact results. I'd add one when those effects are the question, as Snack 3 does.

Q12 **The belts move shapes every frame. Why aren't positions saved continuously?**

A run is a preview. Writing sixty positions a second per shape would flood the API and the database with intermediate states nobody needs, and another browser polling would see shapes jitter. Saving once on Stop writes only the result, and only for shapes that moved.

Q13 **How does a splitter decide, and how would you let operators change that without a deploy?**

Each splitter has an ordered list of rules stored in the database: conditions on the item or its inventory part, and an exit. When an item crosses the gate, the first rule whose conditions all hold picks left, right or straight; otherwise a default. Because rules are data with a fixed set of fields and operators, they can be edited from the UI, the API or SQL, validated on the way in, and previewed against the items in the scene before saving.

Q14 **What do “left” and “right” mean on a belt that can be turned and reversed?**

Relative to the direction of travel, as for a person standing on the line. In the belt's frame that's +v for right when it runs forward and −v when it runs backwards; one small function holds that rule, and the gate model is built in a frame that's turned with the belt's direction, so the drawn flaps and the motion always agree.

CHAPTER 1

Exercises

1.1 Add a wrist roll

Add a fifth joint that rotates about the wrist link's own axis (local Y), between `Joint4` and `toolMount`. Update `JOINTS`, `RobotArm.build`, `forwardKinematics` and the matrix-chain test.

Hint. The TCP lies on the roll axis, so its position shouldn't change at all. Write that as a test first.

1.2 Test the limits

Write a test that `setAngle(1, 500)` returns 100 and that the arm's TCP matches FK of the clamped pose.

Hint. Three.js's core runs in Node without a renderer: you can construct a `RobotArm` and call `tcp.getWorldPosition` in a test.

1.3 Map the workspace

Print an ASCII map of the vertical plane (say 4 cm cells) marking every cell the TCP can reach within the joint limits.

Hint. Don't search target points. Sample joint space densely, run FK, and mark the cell each result lands in.

1.4 Extend the gripper model

Add a small camera bracket to the side of the palm, with a third material, then check the file with the Khronos glTF Validator.

Hint. One new material, one `addMesh`, one new node. Gripper.ts shouldn't need to change, which is the point of finding fingers by name.

1.5 A ring-buffer trail

Replace `copyWithin` with a true ring buffer and measure 10,000 `push` calls before and after with `performance.now()`.

Hint. A ring buffer's points are out of order in memory. Drawing it needs two ranges, for example two `Line` objects sharing the buffer.

1.6 Snap to the grid

Add a toggle that snaps placed and moved shapes to the 10 cm floor grid, and rotation to 15° steps.

Hint. Round in floorPoint, before showing the ghost, so the preview shows exactly where the shape will land.

1.7 Collisions on the belt

Make items queue up behind each other instead of passing through when the one in front stops at a rail or the end.

Hint. Sort the bodies on each belt by u, and limit each one's advance so it stays at least its own length behind the next. That keeps the model kinematic: no forces, just a rule about spacing.

1.8 Count what each exit receives

Show a running count per exit on each splitter (left 3, straight 5, right 2), reset when the line starts.

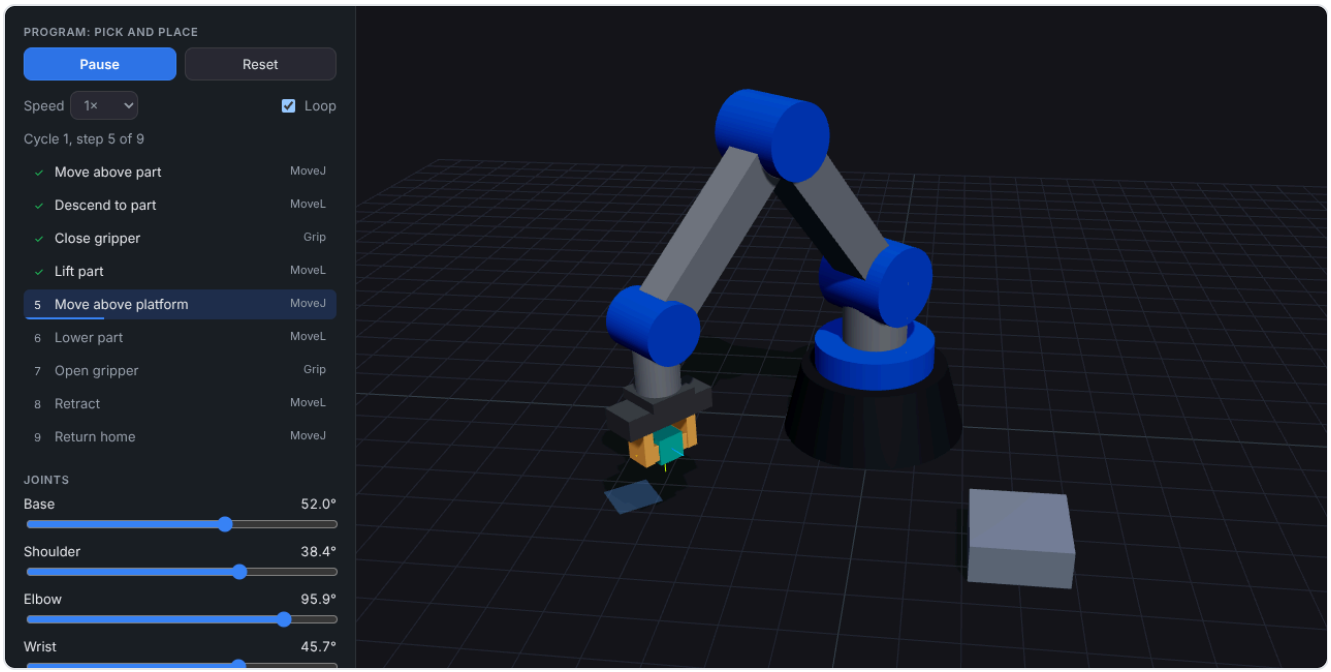
Hint. The route callback already sees every decision. Keep a Map from splitter id to counts, increment it there, and render it in the list. Bonus: store the counts on Stop, as a run summary in the database.

CHAPTER 2 · `vention/snack2-control-panel`

The React Control Panel

A React panel drives a pick-and-place program through inverse kinematics, while the 3D view animates at full frame rate without re-rendering React every frame.

React · Three.js · TypeScript



Mid-run: the part has been picked off the floor and the arm is moving it to the platform (step 5 of 9).

The one idea to take away

The simulation lives outside React. Commands flow in from the panel; the 3D view reads the live state every frame:

`panel command` → `Simulation` → `SequencePlayer` → `sim.state` → `Viewer (every frame)`

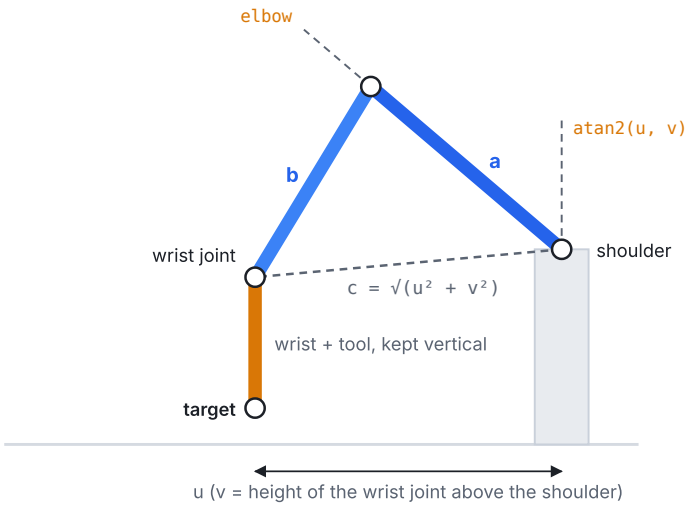
React only gets an immutable snapshot, immediately when the step changes, otherwise at most 15 times a second:

`Simulation.publish()` → `snapshot` → `useSyncExternalStore` → `panel render (≤15×/s)`

CONCEPTS · 2.1

Inverse kinematics

Inverse kinematics (IK) runs FK backwards: given where the tool should be, find the joint angles. It's harder because the answer isn't unique. There may be **none** (out of reach), **a few** (elbow up or elbow down; base turned 180° with the shoulder reversed), or **infinitely many**, for arms with more joints than the task needs.




```
1 base = atan2(z, −x)
    face the target from above

2 wrist joint = target + (wrist+tool)·up
    u = its reach, v = its height

3 cos e = (u² + v² − a² − b²) / 2ab
    |cos e| > 1: no triangle, return null

4 shoulder = atan2(u, v)
    − atan2(b·sin e, a + b·cos e)

5 wrist = 180° − shoulder − elbow
    pitch angles add up to straight down
```

Figure 2.1. Closed-form inverse kinematics. Fixing the tool vertical turns the problem into a triangle with two known sides, solved by the law of cosines.

Solvers come in two families:

- **Analytical** (closed-form), like this one: exact, constant time, no iterations, but derived for one arm's geometry. Most industrial six-axis arms have a “spherical wrist” precisely so that a closed form exists; it yields up to eight solutions per pose.
- **Numerical**: start somewhere and iterate. Cyclic coordinate descent (CCD) turns each joint in turn to point the tool at the target; Jacobian methods (transpose, pseudo-inverse, damped least squares) step along the direction that reduces the error; FABRIK works on joint positions directly. They handle any arm but can be slow, jittery or stuck in local minima.

The trick that makes Snack 2's solver closed-form is **fixing the orientation**: the tool must point straight down. That uses up the fourth joint, leaving a two-link problem in a plane, solved by the law of cosines. Joint limits are checked afterwards, because geometry alone doesn't know about them.

CONCEPTS · 2.2

Joint space and Cartesian space

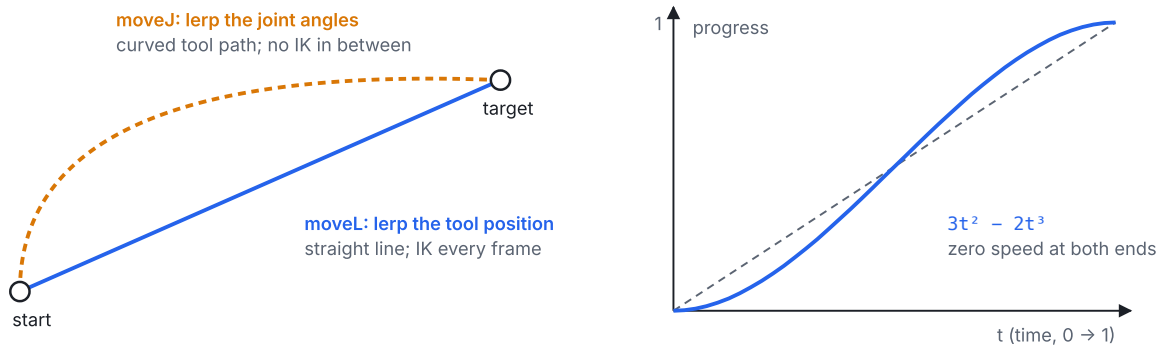


Figure 2.2. Left: the two robot moves between the same two points. Right: smoothstep easing, used for both, so every move starts and stops gently.

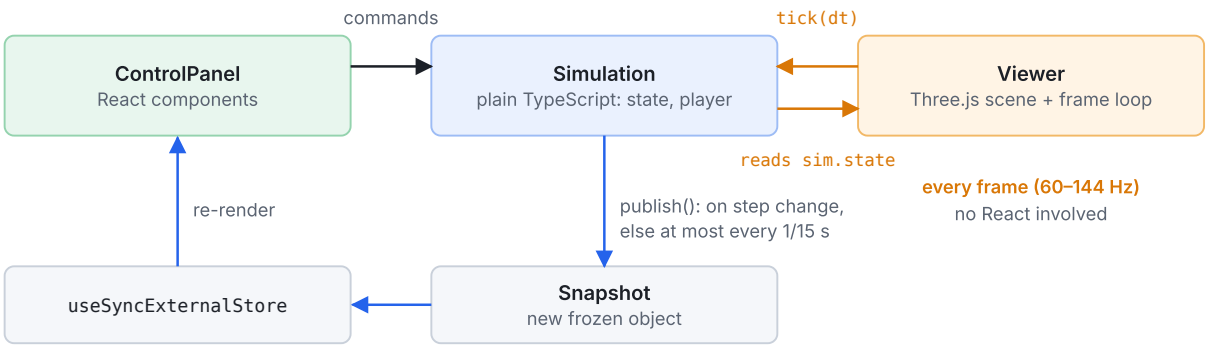
- **moveJ** interpolates the joint angles. All joints start and stop together, it's the fastest way between two poses, and it can never pass through a singularity. The tool's path is a curve that's hard to predict, so it's for open space.
- **moveL** interpolates the tool position and solves IK at every step. It's needed for approach and retreat, where a curved path would knock the part over. It can fail halfway (a point on the line unreachable, or a singularity), so real controllers validate the whole path before moving.

Both use **smoothstep** easing, $3t^2 - 2t^3$, whose speed $6t(1 - t)$ is zero at both ends, so moves don't start or stop with a jerk in velocity. Acceleration still jumps at the ends; real controllers use trapezoidal or S-curve velocity profiles limited by each joint's maximum speed and acceleration, and size each move by its slowest joint.

CONCEPTS · 2.3

How React renders, and why the simulation lives outside it

A React **render** is a call to your component functions, which return a description of the UI. React compares it with the previous description (**reconciliation**) and applies the differences to the DOM (the **commit**). A render happens when state, props or context change. Each one is cheap, but it runs on the same main thread as the 3D frame loop, inside the same 16.7 ms budget at 60 fps.



Fast path (orange): the 3D view reads mutable state directly. Slow path (blue): React sees an immutable snapshot, throttled.

Figure 2.3. Snack 2's data flow. The simulation lives outside React, so animation never waits for a React render.

So Snack 2 keeps the fast-changing state in a plain TypeScript class. The render loop mutates and reads it directly, 60 or more times a second. React receives an **immutable snapshot**: immediately on discrete events (step changed, paused), and at most 15 times a second for continuous values like joint angles. That's the same pattern state libraries such as Zustand use, and the advice react-three-fiber gives: mutate in the frame loop, don't call `setState` per frame.

CONCEPTS · 2.4

useSyncExternalStore, and tearing

With concurrent rendering, React can pause a render partway and continue later. If components read a mutable store directly while rendering, two of them could see different values in the same commit: **tearing**. `useSyncExternalStore` prevents it. React reads the store only through `getSnapshot`, checks it again before committing, and re-renders synchronously if it changed.

- `subscribe(listener)` registers a callback and returns a function that unregisters it.
- `getSnapshot()` must return the **same object** (by `Object.is`) until the data changes. Returning a fresh object each call makes React think the data changed every time, and it loops until it gives up with an error.

CONCEPTS · 2.5

Effects, cleanup and StrictMode

`useEffect` runs after React commits, and its cleanup function runs before the effect runs again and when the component unmounts. In development, StrictMode deliberately mounts, unmounts and remounts every component once, so a missing cleanup shows up immediately instead of months later.

For a Three.js view, cleanup means: stop the animation loop, remove event listeners, dispose geometries, materials, textures and controls, and dispose the renderer. Browsers cap the number of live WebGL contexts (Chrome allows around 16) and silently drop the oldest, so leaks become broken canvases. Async work started before unmount, like loading the gripper, must check a `disposed` flag when it finishes.

CONCEPTS · 2.6

Re-parenting with attach

Picking up the part is a change of parent. `parent.add(child)` keeps the child's **local** transform, so its world position jumps to wherever that offset lands under the new parent. `parent.attach(child)` keeps its **world** transform: it computes the new local matrix as `inverse(parent.matrixWorld) × child.matrixWorld`. Grasp is `gripper.attach(part)`, release is `scene.attach(part)`, and the part never moves at the moment of either.

CHAPTER 2

Line by line

12 parts, in the order data flows through the code.

PART 2.1 · `src/sim/program.ts`

The robot program

What the robot should do, written as data. No logic runs here; it's the script the player follows.

The step types

src/sim/program.ts · lines 3–18

```
/**
 * The robot program: a list of steps, in the style of an industrial robot's
 * teach pendant or Vention's MachineLogic.
 *
 * - moveJ:   joint move. Solve IK for the end point once, then interpolate each
 *            joint. Fast and smooth, but the tool's path in space is a curve.
 * - moveL:   linear move. Interpolate the tool position in a straight line and
 *            solve IK every frame. Used for approaching and leaving the part.
 * - gripper: open or close the fingers to a gap (meters).
 * - home:    joint move back to the home pose.
 */
export type Step =
  | { kind: "moveJ"; label: string; to: Vec3; duration: number }
  | { kind: "moveL"; label: string; to: Vec3; duration: number }
  | { kind: "gripper"; label: string; gap: number; duration: number }
  | { kind: "home"; label: string; duration: number };
```

A **discriminated union**: every step has a `kind` field, and TypeScript uses it to know which other fields exist. A `moveJ` step has a `to` point; a `gripper` step has a `gap` instead. When the player does `switch (step.kind)`, TypeScript narrows the type in each branch, so `step.to` is only allowed where it actually exists.

All four kinds share `label` (shown in the panel) and `duration` (seconds at 1× speed).

The doc comment explains the difference that matters most: **moveJ** interpolates the joint angles, so the tool follows a curve; **moveL** interpolates the tool position, so the tool travels in a straight line. Real robot languages have the same two moves with the same names.

The work-cell layout

src/sim/program.ts · lines 20–30

```
// ---- Work cell layout (meters) ----

export const PART_SIZE = 0.05; // cube edge
export const GAP_OPEN = 0.096; // distance between the grip pads
export const GAP_CLOSED = 0.006;

export const PLATFORM = { x: 0.3, z: 0.45, size: 0.18, height: 0.075 };

/** Where the part's center sits when resting at each station. */
export const FLOOR_SPOT: Vec3 = { x: -0.45, y: PART_SIZE / 2, z: 0.35 };
export const PLATFORM_SPOT: Vec3 = { x: PLATFORM.x, y: PLATFORM.height + PART_SIZE / 2, z:
PLATFORM.z };
```

- `PART_SIZE`: the cube is 5 cm on each side.
- `GAP_OPEN` / `GAP_CLOSED`: how far apart the grip pads are when fully open (9.6 cm) or fully closed (6 mm). These match the fingers' travel in the Snack 1 gripper model.
- `PLATFORM`: an 18 cm square block, 7.5 cm tall, centered at x = 0.3, z = 0.45.
- `FLOOR_SPOT` and `PLATFORM_SPOT`: where the **center** of the part sits when it's resting at each station. That's half the part's height above the surface, which is why both add `PART_SIZE / 2`.

Positions are the tool center point targets too: the TCP sits between the fingertips, so putting it on the part's center means the fingers close around the middle of the cube.

Helpers

src/sim/program.ts · lines 32–35

```
const CLEARANCE = 0.15; // approach height above a station
const RELEASE_DROP = 0.002; // let go 2 mm above the surface so the part settles

const above = (p: Vec3, height = CLEARANCE): Vec3 => ({ ...p, y: p.y + height });
```

`CLEARANCE` is how high above a station the arm approaches from (15 cm). `RELEASE_DROP` makes the robot let go 2 mm above the surface, so the part settles under gravity instead of being pushed into the platform.

`above()` returns a copy of a point raised by some height. `{ ...p, y: ... }` copies `x` and `z` and replaces `y`, without changing the original point. The default argument means `above(p)` uses `CLEARANCE`.

One pick-and-place cycle

src/sim/program.ts · lines 37–51

```
/** One pick-and-place cycle, ending back at home. */
export function pickAndPlace(from: Vec3, to: Vec3, fromName: string, toName: string): Step[] {
  const release = above(to, RELEASE_DROP);
  return [
    { kind: "moveJ", label: `Move above ${fromName}`, to: above(from), duration: 2 },
    { kind: "moveL", label: "Descend to part", to: from, duration: 1.2 },
    { kind: "gripper", label: "Close gripper", gap: PART_SIZE, duration: 0.6 },
    { kind: "moveL", label: "Lift part", to: above(from), duration: 1 },
    { kind: "moveJ", label: `Move above ${toName}`, to: above(release), duration: 2.2 },
    { kind: "moveL", label: "Lower part", to: release, duration: 1.2 },
    { kind: "gripper", label: "Open gripper", gap: GAP_OPEN, duration: 0.5 },
    { kind: "moveL", label: "Retract", to: above(release), duration: 0.9 },
    { kind: "home", label: "Return home", duration: 2 },
  ];
}
```

This builds the nine steps of one cycle from two stations. Reading it top to bottom is reading the robot's job:

1. **moveJ** to 15 cm above the part: the big, fast move, where the exact path doesn't matter.
2. **moveL** straight down onto the part. A straight line matters here: a curved path could knock the part over.
3. Close the gripper to exactly `PART_SIZE`, so the pads stop on the part's faces.
4. **moveL** straight back up.
5. **moveJ** across to above the drop-off point.
6. **moveL** down to the release height.
7. Open the gripper. The part is released and drops 2 mm.
8. **moveL** straight up, clear of the part.
9. Return to the home pose.

Template strings like `` `Move above ${fromName}` `` give each step a readable label for the panel.

Looping without running out of work

src/sim/program.ts · lines 53–58

```
/** Even cycles move the part floor → platform, odd cycles bring it back, so looping never runs
out of work. */
export function programForCycle(cycle: number): Step[] {
  return cycle % 2 === 0
    ? pickAndPlace(FLOOR_SPOT, PLATFORM_SPOT, "part", "platform")
    : pickAndPlace(PLATFORM_SPOT, FLOOR_SPOT, "part", "floor spot");
}
```

If the program always went floor → platform, a second loop would try to pick from an empty floor. Instead, even cycles (0, 2, 4…) carry the part to the platform and odd cycles carry it back, so **Loop** can run forever. `cycle % 2 === 0` is the remainder after dividing by 2: 0 for even numbers.

The player calls this function to get each cycle's steps, so the program can depend on the cycle number without the player knowing anything about stations.

PART 2.2 · `src/arm/ik.ts`

Inverse kinematics

Forward kinematics (Snack 1) answers “given these joint angles, where is the tool?”. Inverse kinematics answers the reverse: “I want the tool here, what angles do I need?”. It's what lets the program be written as points in space.

The signature and the idea

src/arm/ik.ts · lines 4–16

```
/**
 * Inverse kinematics for a top-down grasp: find the joint angles that put the
 * tool center point on `target` with the gripper pointing straight down.
 *
 * Returns [base, shoulder, elbow, wrist] in degrees, or null if out of reach.
 *
 * Fixing the tool orientation makes this solvable in closed form:
 *  1. The base yaw is just the compass direction to the target.
 *  2. With the tool vertical, the wrist joint sits straight above the target,
 *     so shoulder + elbow form a two-link arm reaching for that point.
 *  3. The wrist takes whatever angle makes the three pitch angles sum to 180°.
 */
export function solveTopDown(d: ArmDimensions, target: Vec3): number[] | null {
```

General IK has many solutions or none, and usually needs an iterative solver. This one is solvable directly because it **fixes the tool orientation**: pointing straight down, which is what you want for picking things up from above.

It returns degrees (matching the rest of the app) or `null` when the point is out of reach, so callers have to handle failure.

Step 1: base yaw

src/arm/ik.ts · lines 17–20

```
// 1. Base yaw. The arm reaches along its local -X, so FK gives
//    x = -reach·cos(base), z = reach·sin(base).
const reach = Math.hypot(target.x, target.z);
const base = Math.atan2(target.z, -target.x);
```

Seen from above, the base just needs to turn to face the target. `Math.hypot(x, z)` is the horizontal distance from the base axis ($\sqrt{x^2 + z^2}$).

`Math.atan2(y, x)` gives the angle of a direction, handling all four quadrants. The arguments are `(z, -x)` rather than `(z, x)` because of how the arm was built in Snack 1: it reaches along its local **−X**, and Snack 1's forward kinematics gives `x = -reach·cos(base)` and `z = reach·sin(base)`. Solving those two for `base` gives this line.

Step 2: reduce to a two-link arm

src/arm/ik.ts · lines 22–27

```
// 2. Two-link problem in the arm's vertical plane, measured from the shoulder joint:
//    u = horizontal distance, v = height of the wrist joint above the shoulder.
const u = reach;
const v = target.y + d.wrist + d.tool - (d.baseHeight + d.shoulderHeight);
const a = d.upperArm;
const b = d.forearm;
```

With the gripper pointing straight down, the wrist joint must be directly above the target, higher by the length of the wrist link plus the tool (`d.wrist + d.tool`). So instead of solving for the tool, solve for the wrist joint.

That leaves only the upper arm (`a`) and forearm (`b`) to place, in the arm's vertical plane. `u` is the horizontal distance to the wrist joint and `v` is its height above the shoulder joint (the shoulder sits at `baseHeight + shoulderHeight`). This is the classic two-link planar arm problem.

The elbow, by the law of cosines

src/arm/ik.ts · lines 29–32

```
// Law of cosines gives the elbow bend. Outside [-1, 1] means the point is too far or too
close.
const cosElbow = (u * u + v * v - a * a - b * b) / (2 * a * b);
if (cosElbow < -1 || cosElbow > 1) return null;
const elbow = Math.acos(cosElbow); // 0..180°, the "elbow up" solution
```

The upper arm, forearm and the straight line from shoulder to wrist form a triangle with sides `a`, `b` and $\sqrt{(u^2 + v^2)}$. The law of cosines relates the elbow bend to those sides, and this line solves it for the cosine of the bend.

A cosine must be between −1 and 1. If it isn't, no triangle exists: the target is farther than the arm can stretch, or closer than it can fold. That's the `return null`.

`Math.acos` returns 0 to 180°. Taking the positive answer picks the “elbow up” pose; the negative angle would be the mirror-image “elbow down” solution, which reaches the same point.

The shoulder

src/arm/ik.ts · lines 34–36

```
// Shoulder angle = direction to the wrist joint, minus the angle the elbow bend adds.
// Angles here are measured from vertical, matching how the joints are built.
const shoulder = Math.atan2(u, v) - Math.atan2(b * Math.sin(elbow), a + b * Math.cos(elbow));
```

Two angles, subtracted:

- `Math.atan2(u, v)`: the direction from the shoulder to the wrist joint, measured from vertical (the arm's zero pose points straight up, so angles are measured from vertical throughout).
- `Math.atan2(b·sin(elbow), a + b·cos(elbow))`: how far the bent elbow swings the wrist away from the upper arm's own direction.

Aim at the target, then back off by the amount the elbow bend adds, and the upper arm is at the right angle.

Step 3: the wrist keeps the tool vertical

src/arm/ik.ts · lines 38–42

```
  // 3. Keep the tool vertical: shoulder + elbow + wrist = 180°.
  const wrist = Math.PI - shoulder - elbow;

  return [base, shoulder, elbow, wrist].map((angle) => angle / DEG);
}
```

The three pitch joints are parallel, so their angles simply add up. The tool points straight down when the total is 180° (π radians), so the wrist takes whatever is left over.

The angles were computed in radians, since that's what `Math` uses. The last line converts all four to degrees: `DEG` is π/180, so dividing by it converts radians to degrees.

Joint limits

src/arm/ik.ts · lines 44–46

```
export function withinLimits(specs: readonly JointSpec[], anglesDeg: readonly number[]):
boolean {
  return specs.every((s, i) => anglesDeg[i] >= s.min && anglesDeg[i] <= s.max);
}
```

IK only knows geometry. It can return an elbow angle of 150° even though the joint stops at 140°. `every` returns true only if each angle is within its joint's min and max, and the player checks this before using any IK result.

GOING DEEPER

A numerical solver in a paragraph

Cyclic coordinate descent, for comparison: for each joint from the wrist back to the base, rotate that joint so the line from it to the tool points at the target (clamped to its limits); repeat the sweep until the tool is within a millimeter or a maximum number of sweeps is reached. It works for any chain and any number of joints, but tends to curl the last joints first, gives no control over which solution you get, and can take many sweeps. The closed form is exact in one pass, which is why it's preferred whenever the geometry allows one.

PART 2.3 · `src/sim/SequencePlayer.ts`

Running the program

The player steps through the program, turning each step into joint angles and a gripper gap, frame by frame. It imports no Three.js or React, so the tests can run the whole program in plain Node.

The state it writes

src/sim/SequencePlayer.ts · lines 6–12

```
/** The commanded state of the robot. Joints in degrees, gripper gap in meters. */
export interface ArmState {
  joints: number[];
  gap: number;
}

export type PlayerStatus = "idle" | "playing" | "paused" | "done" | "error";
```

`ArmState` is the **commanded** robot: four joint angles in degrees and the gap between the grip pads in meters. The player writes into it; the 3D viewer reads from it. It's a plain object, deliberately not a Three.js object.

`PlayerStatus` is a union of string literals, so a typo like `"plaing"` is a compile error.

Easing and interpolation

src/sim/SequencePlayer.ts · lines 14–17

```
/** Smoothstep: starts and ends with zero velocity, so moves don't jerk. */
const ease = (t: number) => t * t * (3 - 2 * t);
const lerp = (a: number, b: number, t: number) => a + (b - a) * t;
const format = (p: Vec3) => `(${p.x.toFixed(2)}, ${p.y.toFixed(2)}, ${p.z.toFixed(2)})`;
```

- `ease` is **smoothstep**: for t from 0 to 1 it also goes from 0 to 1, but starts and ends slowly. Real robots accelerate and decelerate rather than jumping to full speed, and this gives the same look.
- `lerp` (linear interpolation) returns the value a fraction `t` of the way from `a` to `b`. At t = 0 it's `a`, at t = 1 it's `b`.
- `format` turns a point into readable text for error messages.

Fields

src/sim/SequencePlayer.ts · lines 23–41

```
export class SequencePlayer {
  status: PlayerStatus = "idle";
  steps: Step[];
  index = 0;
  elapsed = 0; // seconds into the current step (scaled by speed)
  cycle = 0;
  loop = true;
  speed = 1;
  error: string | null = null;

  private readonly dims: ArmDimensions;
  private readonly specs: readonly JointSpec[];
  private readonly program: (cycle: number) => Step[];

  // Captured when a step begins, so the step interpolates from where the arm really is.
  private startJoints: number[] = [];
  private startGap = 0;
  private startTcp: Vec3 = { x: 0, y: 0, z: 0 };
  private targetJoints: number[] | null = null;
```

The public fields are what the panel shows: status, the current cycle's steps, which step it's on, how far into it (`elapsed`), the cycle number, and the loop, speed and error settings.

The `start…` fields are captured **when each step begins**: where the joints, gripper and tool were at that moment. Every step interpolates from where the arm actually is, never from where the program assumed it would be. `targetJoints` holds the IK answer for joint moves.

Constructor and progress

src/sim/SequencePlayer.ts · lines 43–54

```
constructor(dims: ArmDimensions, specs: readonly JointSpec[], program: (cycle: number) =>
Step[]) {
  this.dims = dims;
  this.specs = specs;
  this.program = program;
  this.steps = program(0);
}

/** 0..1 through the current step. */
get progress(): number {
  const step = this.steps[this.index];
  return step ? Math.min(1, this.elapsed / step.duration) : 0;
}
```

The player is given the arm's dimensions and joint limits (for IK and FK) and a **function** that returns the steps for a cycle. Passing `programForCycle` in, rather than importing it, keeps the player reusable: a test could give it any program.

`get progress()` is a getter: read like a property (`player.progress`) but computed each time. It's how far through the current step the player is, from 0 to 1, for the panel's progress bar.

Play, pause and stop

src/sim/SequencePlayer.ts · lines 56–77

```
/**
 * Idle or error: start from cycle 0. Done: start the next cycle. Paused: resume.
 * In every case the current step restarts from the arm's current pose, so if
 * someone moved a joint while paused, the arm glides back instead of jumping.
 */
play(state: ArmState): void {
  if (this.status === "playing") return;
  if (this.status === "idle" || this.status === "error") this.rewind();
  this.error = null;
  this.status = "playing";
  this.beginStep(state);
}

pause(): void {
  if (this.status === "playing") this.status = "paused";
}

stop(): void {
  this.rewind();
  this.status = "idle";
  this.error = null;
}
```

`play` handles three situations:

- From **idle** or **error**, it rewinds to cycle 0 and starts fresh.
- From **done** (one cycle finished, loop off), it leaves the cycle counter alone, so the next play runs the return trip.
- From **paused**, it resumes the same step.

All three end with `beginStep(state)`, which re-captures the start pose. So if you dragged a slider while paused, the step re-plans from your new pose and the arm glides back on course instead of jumping.

`pause` only changes the status, so the arm freezes where it is. `stop` rewinds and goes idle.

Advancing one frame

src/sim/SequencePlayer.ts · lines 79–85

```
/** Advances by dt seconds, writing the new joint angles and gap into `state`. */
update(dt: number, state: ArmState): void {
  if (this.status !== "playing") return;
  const step = this.steps[this.index];
  this.elapsed += dt * this.speed;
  const t = Math.min(1, this.elapsed / step.duration);
  const s = ease(t);
```

The 3D viewer calls this every frame with `dt`, the seconds since the last frame. Multiplying by `speed` makes 2× speed advance the program twice as far per frame. `t` is the fraction of the step completed (capped at 1), and `s` is that fraction eased.

Joint moves

src/sim/SequencePlayer.ts · lines 87–95

```
switch (step.kind) {
  case "moveJ":
  case "home": {
    const target = this.targetJoints!;
    for (let i = 0; i < state.joints.length; i++) {
      state.joints[i] = lerp(this.startJoints[i], target[i], s);
    }
    break;
  }
```

For `moveJ` and `home`, each joint goes from its start angle to its target angle, all with the same eased `s`, so all joints start and finish together.

The `!` after `this.targetJoints` tells TypeScript “this isn't null here”. That's safe because `beginStep` always sets it for these two kinds, or puts the player in the error state so `update` never gets here.

The loop writes into the existing `state.joints` array instead of creating a new one: no allocation every frame.

Linear moves

src/sim/SequencePlayer.ts · lines 96–109

```
case "moveL": {
  const p = {
    x: lerp(this.startTcp.x, step.to.x, s),
    y: lerp(this.startTcp.y, step.to.y, s),
    z: lerp(this.startTcp.z, step.to.z, s),
  };
  const q = solveTopDown(this.dims, p);
  if (!q || !withinLimits(this.specs, q)) {
    this.fail(`Can't reach ${format(p)} during "${step.label}"`);
    return;
  }
  for (let i = 0; i < q.length; i++) state.joints[i] = q[i];
  break;
}
```

For `moveL`, the player interpolates the **tool position** (x, y and z each lerped from the start to the target), then asks IK for the joint angles that put the tool there. Doing this every frame is what makes the path a straight line.

If a point along the line is unreachable or breaks a joint limit, the player stops with a readable error rather than moving the arm into an impossible pose.

Gripper steps

src/sim/SequencePlayer.ts · lines 110–113

```
  case "gripper":
    state.gap = lerp(this.startGap, step.gap, s);
    break;
}
```

The gap goes from where it was to the step's target. The joints don't change during a gripper step.

Finishing a step

src/sim/SequencePlayer.ts · lines 115–130

```
  if (t < 1) return;

  // Step finished: move to the next one, or the next cycle.
  this.index++;
  if (this.index >= this.steps.length) {
    this.cycle++;
    this.steps = this.program(this.cycle);
    this.index = 0;
    if (!this.loop) {
      this.status = "done";
      this.elapsed = 0;
      return;
    }
  }
  this.beginStep(state);
}
```

Until `t` reaches 1, there's nothing more to do this frame. Once it does, move to the next step. Past the last step, the cycle is over: bump the cycle number and ask for the next cycle's steps (which, from program.ts, go the other direction).

With loop off, stop here with status **done**. Otherwise, or if there are more steps, `beginStep` sets up the next one immediately, so there's no idle frame between steps.

Setting up a step

src/sim/SequencePlayer.ts · lines 132–157

```
private rewind(): void {
  this.cycle = 0;
  this.steps = this.program(0);
  this.index = 0;
  this.elapsed = 0;
}

private beginStep(state: ArmState): void {
  const step = this.steps[this.index];
  this.elapsed = 0;
  this.startJoints = [...state.joints];
  this.startGap = state.gap;
```

```
  this.startTcp = forwardKinematics(this.dims, state.joints.map((a) => a * DEG));
  this.targetJoints = null;

  if (step.kind === "moveJ") {
    const q = solveTopDown(this.dims, step.to);
    if (!q || !withinLimits(this.specs, q)) {
      this.fail(`Can't reach ${format(step.to)} for "${step.label}"`);
      return;
    }
    this.targetJoints = q;
  } else if (step.kind === "home") {
    this.targetJoints = this.specs.map((s) => s.home);
  }
}
```

`rewind` returns to the start of cycle 0. `beginStep` snapshots the starting point: a copy of the joint angles (`[...state.joints]` makes a new array so later changes don't affect it), the gap, and the tool position from forward kinematics (which takes radians, hence `a * DEG`).

For `moveJ`, IK runs **once**, here, for the end point; the frames in between only interpolate. For `home`, the target is just each joint's home angle. Either failure becomes an error.

Errors

src/sim/SequencePlayer.ts · lines 159–162

```
private fail(message: string): void {
  this.status = "error";
  this.error = message;
}
```

One place to enter the error state, so the status and message always change together.

GOING DEEPER

Time-based steps make the robot testable

The player never looks at the clock. It's given `dt`, advances elapsed time, and computes progress as elapsed ÷ duration. Pause is simply “ignore dt”, and speed multiplies it. Because nothing depends on rendering, a test can drive the player in plain Node with a fixed dt and check things a person can't easily see: that the gripper closes exactly on the part, and that over two looping cycles it never touches the floor or the platform.

PART 2.4 · `src/sim/Simulation.ts`

The store between React and Three.js

This is the heart of Snack 2's design. The simulation lives outside React. The 3D view reads it every frame; React gets an occasional snapshot.

What React sees

src/sim/Simulation.ts · lines 6–19

```
/** What React sees: an immutable copy of the simulation, replaced (never mutated) on each
publish. */
export interface Snapshot {
  joints: readonly number[];
  gap: number;
  tcp: Vec3;
  status: PlayerStatus;
  steps: readonly Step[];
  stepIndex: number;
  stepProgress: number;
  cycle: number;
  speed: number;
  loop: boolean;
  error: string | null;
}
```

A `Snapshot` is a frozen copy of everything the panel displays. `readonly` stops components from modifying it. Each publish creates a **new** snapshot object rather than changing the old one, which is how React knows something changed: it compares the object references.

Tuning constants

src/sim/Simulation.ts · lines 21–22

```
const PUBLISH_INTERVAL = 1 / 15; // seconds: React re-renders at most ~15x/s while things move
const GRIPPER_SPEED = 0.25; // m/s for manual open/close
```

`PUBLISH_INTERVAL` caps how often React hears about continuous motion: 1/15 s, so at most about 15 panel renders a second while the arm moves, however fast the screen refreshes. `GRIPPER_SPEED` is for the manual Open/Close buttons.

The live state

src/sim/Simulation.ts · lines 24–49

```
/**
 * The single source of truth for the robot, living outside React.
 *
 * The 3D viewer calls tick() every frame (60+ times a second) and reads `state`
 * directly. React reads `snapshot` through useSyncExternalStore, and only gets a
 * new one when something changes, throttled while the arm is moving. So the
 * scene animates at full frame rate while the panel re-renders a few times a second.
 */
export class Simulation {
  /** Live, mutable state. The viewer reads this every frame. */
  readonly state: ArmState = { joints: JOINTS.map((j) => j.home), gap: GAP_OPEN };
  readonly player = new SequencePlayer(ARM, JOINTS, programForCycle);
  /** Bumped on reset, so the viewer knows to put the part back. */
  resetVersion = 0;
  /** Frames simulated so far, shown in the panel's stats line. */
  frames = 0;

  private manualGap = GAP_OPEN;
  private snapshot: Snapshot;
  private readonly listeners = new Set<() => void>();
  private sincePublish = 0;
  private wasMoving = false;

  constructor() {
    this.snapshot = this.makeSnapshot();
  }
```

`state` is the live, mutable robot that the player writes and the viewer reads, 60 or more times a second. React never reads it directly.

`resetVersion` is a counter instead of an event. Reset increments it; the viewer remembers the last value it saw, and when they differ it puts the part back. No callbacks to register or clean up.

`manualGap` is where the manual gripper buttons want the gap to be. `listeners` holds React's subscription callbacks. `sincePublish` and `wasMoving` drive the throttling in `tick`.

The external-store contract

src/sim/Simulation.ts · lines 51–59

```
// ---- The external-store contract for useSyncExternalStore ----
// Arrow functions so `this` is bound and React gets the same function every time.

subscribe = (listener: () => void): (() => void) => {
  this.listeners.add(listener);
  return () => this.listeners.delete(listener);
};

getSnapshot = (): Snapshot => this.snapshot;
```

React's `useSyncExternalStore` hook needs exactly two functions from a store:

- `subscribe(listener)`: remember this callback, call it whenever the data changes, and return a function that unsubscribes.
- `getSnapshot()`: return the current data. It must return the **same object** until something changes; otherwise React would re-render forever.

They're written as arrow-function properties rather than methods. An arrow function captures `this`, so React can call `sim.subscribe` on its own without losing which simulation it belongs to. And because they're created once, React receives the same function on every render and doesn't resubscribe.

Commands from the panel

src/sim/Simulation.ts · lines 61–103

```
// ---- Commands from the UI ----

play(): void {
  this.player.play(this.state);
  this.publish();
}

pause(): void {
  this.interrupt();
  this.publish();
}

reset(): void {
  this.player.stop();
```

```
  JOINTS.forEach((j, i) => (this.state.joints[i] = j.home));
  this.state.gap = this.manualGap = GAP_OPEN;
  this.resetVersion++;
  this.publish();
}

/** Manual jog. Takes control away from the program, like a robot's teach pendant. */
setJoint(index: number, degrees: number): void {
  this.interrupt();
  const spec = JOINTS[index];
  this.state.joints[index] = clamp(degrees, spec.min, spec.max);
  this.publish();
}

setGripper(open: boolean): void {
  this.interrupt();
  this.manualGap = open ? GAP_OPEN : GAP_CLOSED;
  this.publish();
}

setSpeed(speed: number): void {
  this.player.speed = speed;
  this.publish();
}

setLoop(loop: boolean): void {
  this.player.loop = loop;
  this.publish();
}
```

Every button and slider in the panel calls one of these. Each changes the state, then calls `publish()` so the panel updates immediately.

- `play` / `pause` pass through to the player.
- `reset` stops the program, puts the joints and gripper back to home, and bumps `resetVersion` for the part.
- `setJoint` is a manual jog. It first **interrupts** the program, the way grabbing a real robot's teach pendant takes control away from the running program, then sets the angle clamped to the joint's limits.
- `setGripper` also interrupts, then sets the manual gap target. The gap itself moves gradually in `tick`.

- `setSpeed` / `setLoop` change player settings.

tick: once per frame

src/sim/Simulation.ts · lines 105–122

```
// ---- Called by the viewer once per frame ----

tick(dt: number): void {
  this.frames++;
  const { status, index } = this.player;
  let moving = false;

  if (this.player.status === "playing") {
    this.player.update(dt, this.state);
    moving = true;
  } else if (this.state.gap !== this.manualGap) {
    const step = GRIPPER_SPEED * dt;
    this.state.gap =
      this.state.gap < this.manualGap
        ? Math.min(this.manualGap, this.state.gap + step)
        : Math.max(this.manualGap, this.state.gap - step);
    moving = true;
  }
```

The viewer calls this every frame. It notes the status and step before doing anything, so it can tell afterwards whether they changed.

If the program is playing, the player advances. Otherwise, if the manual gripper target differs from the current gap, the gap moves toward it at `GRIPPER_SPEED`, without overshooting (`Math.min` / `Math.max`). Either way, `moving` records that something changed this frame.

Deciding when to tell React

src/sim/Simulation.ts · lines 124–131

```
  // Publish right away when the step or status changes, otherwise at most
  // every PUBLISH_INTERVAL while moving, plus once more when motion stops.
  this.sincePublish += dt;
  const discreteChange = status !== this.player.status || index !== this.player.index;
  const stopped = this.wasMoving && !moving;
  if (discreteChange || stopped || (moving && this.sincePublish >= PUBLISH_INTERVAL))
this.publish();
  this.wasMoving = moving;
}
```

This is the throttle, and the reason the panel doesn't re-render 60 times a second. It publishes when:

- a **discrete** change happened (a new step started, or the status changed), immediately, so the step list never lags;
- motion just **stopped**, once more, so the final values shown are exact;
- something is **moving** and at least `PUBLISH_INTERVAL` has passed since the last publish.

When nothing moves, nothing publishes: an idle app causes zero React renders.

Internals

src/sim/Simulation.ts · lines 133–164

```
// ---- Internals ----

/** If the program is running, pause it and hold the gripper where it is. */
private interrupt(): void {
  if (this.player.status === "playing") {
    this.player.pause();
    this.manualGap = this.state.gap;
  }
}

private publish(): void {
  this.snapshot = this.makeSnapshot();
  this.sincePublish = 0;
  this.listeners.forEach((listener) => listener());
}
```

```
private makeSnapshot(): Snapshot {
  const p = this.player;
  return {
    joints: [...this.state.joints],
    gap: this.state.gap,
    tcp: forwardKinematics(ARM, this.state.joints.map((a) => a * DEG)),
    status: p.status,
    steps: p.steps,
    stepIndex: p.index,
    stepProgress: p.progress,
    cycle: p.cycle,
    speed: p.speed,
    loop: p.loop,
    error: p.error,
  };
}
```

`interrupt` pauses the program if it's running, and sets the manual gripper target to the current gap so the fingers stay put.

`publish` builds a fresh snapshot, resets the throttle timer and calls every listener. Calling a listener is how the store tells React “ask me again”; React then calls `getSnapshot`, sees a new object and re-renders the subscribed components.

`makeSnapshot` copies what the panel needs. `[...this.state.joints]` makes a new array, so later frames mutating `state` can't change a snapshot React is holding. The TCP is computed with forward kinematics here, so the panel doesn't need Three.js to show it.

GOING DEEPER

The alternatives, and why not

- **React context** re-renders every consumer on every change, which is the problem this design avoids.
- **Zustand** is this exact pattern as a library: a store outside React, `useSyncExternalStore` underneath, and transient subscriptions for per-frame reads.
- **Signals** (Preact, Solid, Angular) track dependencies per value and update only what read it.
- **Redux** works but adds actions and reducers for what is essentially one object.

The hand-written store is about 40 lines with no dependency, and shows exactly what the libraries do internally.

PART 2.5 · `src/ui/useSimulation.ts`

The hook

One line connects React to the store.

useSyncExternalStore

src/ui/useSimulation.ts · lines 1–10

```
import { useSyncExternalStore } from "react";
import type { Simulation, Snapshot } from "../sim/Simulation";

/**
 * Subscribes a component to the simulation. React re-renders it only when the
 * simulation publishes a new snapshot, not on every animation frame.
 */
export function useSimulation(sim: Simulation): Snapshot {
  return useSyncExternalStore(sim.subscribe, sim.getSnapshot);
}
```

Any component that calls `useSimulation(sim)` gets the current snapshot, and re-renders when the simulation publishes a new one, and only then.

`useSyncExternalStore` is React's built-in way to read data that lives outside React (browser APIs, or stores like Redux and Zustand). Compared with copying the data into `useState`, it can't show a stale or torn value during rendering, and there's no subscription code to write in each component.

PART 2.6 · `src/ui/Viewport.tsx`

Mounting the 3D view

The only place React touches Three.js.

A ref and an effect

src/ui/Viewport.tsx · lines 1–19

```
import { useEffect, useRef } from "react";
import { Viewer } from "../scene/Viewer";
import type { Simulation } from "../sim/Simulation";

/**
 * React's only contact with the 3D scene: it gives the Viewer a DOM element to
 * draw into, and disposes it on unmount. This component never re-renders for
 * simulation updates; the Viewer runs its own frame loop.
 */
export function Viewport({ sim }: { sim: Simulation }) {
  const containerRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    const viewer = new Viewer(containerRef.current!, sim);
    return () => viewer.dispose();
  }, [sim]);

  return <div ref={containerRef} className="viewport" aria-label="3D view of the robot work
cell" />;
}
```

`useRef` gives a handle to the real `<div>` once React has put it on the page. `containerRef.current` is that element (the `!` says it can't be null by the time the effect runs).

`useEffect` runs after the component appears on screen. It creates the `Viewer`, which builds the whole 3D scene inside the div. The function it returns is the **cleanup**, which React calls when the component goes away.

The dependency list `[sim]` means “only redo this if `sim` changes”, which it never does. So the viewer is created once, and this component never re-renders for simulation updates: it doesn't call `useSimulation`.

> In development, React's StrictMode deliberately mounts, unmounts and remounts every component once, to catch effects that don't clean up. That's why the viewer's `dispose()` has to undo everything; see stage.ts below.

PART 2.7 · `src/scene/Viewer.ts`

The 3D side

Owns the Three.js scene and the frame loop, and is the only thing that advances the simulation.

Fields

src/scene/Viewer.ts · lines 14–23

```
export class Viewer {
  private readonly stage: ReturnType<typeof createStage>;
  private readonly sim: Simulation;
  private readonly arm = new RobotArm(ARM, JOINTS);
  private readonly workcell: Workcell;
  private readonly tcp = new THREE.Vector3();
  private gripper: Gripper | null = null;
  private seenReset: number;
  private lastTime = performance.now();
  private disposed = false;
```

`ReturnType<typeof createStage>` means “whatever type `createStage` returns”, so the type doesn't have to be written out twice.

The arm is the same `RobotArm` class as Snack 1, reused unchanged. `tcp` is one `Vector3` reused every frame instead of allocating a new one. `seenReset` is the last `resetVersion` this viewer has handled.

Constructor

src/scene/Viewer.ts · lines 25–44

```
constructor(container: HTMLElement, sim: Simulation) {
  this.sim = sim;
  this.stage = createStage(container);
  this.stage.scene.add(this.arm.root);
  this.workcell = new Workcell(this.stage.scene);
  this.seenReset = sim.resetVersion;

  Gripper.load(`${import.meta.env.BASE_URL}models/gripper.glb`)
    .catch((err: unknown) => {
      console.error("Using a placeholder gripper:", err);
      return Gripper.placeholder();
    })
    .then((gripper) => {
      if (this.disposed) return; // unmounted while the model was loading
      this.gripper = gripper;
      this.arm.toolMount.add(gripper.object);
    });

  this.stage.renderer.setAnimationLoop(this.frame);
}
```

It builds the stage (renderer, camera, lights), adds the arm and the work cell, and starts loading the gripper model.

Loading is asynchronous, so it might finish **after** the viewer was disposed (for example, during StrictMode's remount). The `if (this.disposed) return;` check stops a dead viewer from attaching a gripper to a scene nobody is drawing.

`setAnimationLoop(this.frame)` asks Three.js to call `frame` before every screen refresh.

The frame

src/scene/Viewer.ts · lines 46–65

```
private frame = (time: number) => {
  const dt = Math.min((time - this.lastTime) / 1000, 0.05);
  this.lastTime = time;

  const { sim, arm } = this;
  sim.tick(dt);

  if (sim.resetVersion !== this.seenReset) {
    this.seenReset = sim.resetVersion;
    this.workcell.reset();
  }

  sim.state.joints.forEach((angle, i) => arm.setAngle(i, angle));
  arm.tcp.getWorldPosition(this.tcp);
  const gap = this.workcell.update(dt, this.tcp, sim.state.gap, this.gripper?.object ?? null);
  this.gripper?.setGap(gap);

  this.stage.controls.update();
  this.stage.renderer.render(this.stage.scene, this.stage.camera);
};
```

Every frame, in order:

1. Work out `dt`, capped at 50 ms so a tab returning from the background doesn't make the arm leap.
2. `sim.tick(dt)` advances the program (and decides whether to tell React).
3. If Reset was pressed since the last frame, put the part back.
4. Copy the four joint angles from `sim.state` onto the Three.js joints. This is the hand-off from plain data to the scene graph.
5. Read the tool's world position and let the work cell decide whether the part is grabbed, released or falling; it returns the finger gap to draw.
6. Draw.

React isn't anywhere in this list. The scene runs at the display's frame rate however busy React is.

`frame` is an arrow-function property so that `this` still means the viewer when Three.js calls it.

Dispose

src/scene/Viewer.ts · lines 67–72

```
  dispose(): void {
    this.disposed = true;
    this.stage.renderer.setAnimationLoop(null);
    this.stage.dispose();
  }
}
```

Stops the loop, then tears down the stage. Setting `disposed` first protects the pending gripper load.

PART 2.8 · `src/scene/Workcell.ts`

Picking up the part

The platform, the floor marker and the part, plus the rules for grabbing, releasing and dropping.

Constants and fields

src/scene/Workcell.ts · lines 4–17

```
const GRAVITY = 9.81;
const GRAB_DISTANCE = 0.02; // the part's center must be this close to the TCP to be grabbed

/**
 * Everything the robot works on: the platform, a marker for the floor spot, and
 * the part. Handles picking up (re-parenting the part to the gripper), letting
 * go, and the part dropping onto whatever is under it.
 */
export class Workcell {
  readonly part: THREE.Mesh;
  private readonly scene: THREE.Scene;
  private holder: THREE.Object3D | null = null;
  private fallSpeed = 0;
  private readonly partWorld = new THREE.Vector3();
```

`GRAB_DISTANCE`: the part counts as “between the fingers” when its center is within 2 cm of the tool center point. `holder` is the object holding the part (the gripper) or `null`. `fallSpeed` is for gravity. `partWorld` is another reused vector.

Building the scene objects

src/scene/Workcell.ts · lines 19–48

```
constructor(scene: THREE.Scene) {
  this.scene = scene;

  const platform = new THREE.Mesh(
    new THREE.BoxGeometry(PLATFORM.size, PLATFORM.height, PLATFORM.size),
    new THREE.MeshStandardMaterial({ color: 0x8a93a3, metalness: 0.3, roughness: 0.6 }),
  );
  platform.position.set(PLATFORM.x, PLATFORM.height / 2, PLATFORM.z);
  platform.castShadow = true;
  platform.receiveShadow = true;
  scene.add(platform);

  const marker = new THREE.Mesh(
    new THREE.PlaneGeometry(0.09, 0.09).rotateX(-Math.PI / 2),
    new THREE.MeshBasicMaterial({ color: 0x3b82f6, transparent: true, opacity: 0.35 }),
  );
  marker.position.set(FLOOR_SPOT.x, 0.002, FLOOR_SPOT.z);
  marker.rotation.y = yawAt(FLOOR_SPOT.x, FLOOR_SPOT.z);
  scene.add(marker);

  this.part = new THREE.Mesh(
    new THREE.BoxGeometry(PART_SIZE, PART_SIZE, PART_SIZE),
    new THREE.MeshStandardMaterial({ color: 0x14b8a6, metalness: 0.1, roughness: 0.5 }),
  );
  this.part.name = "Part";
  this.part.castShadow = true;
  this.part.receiveShadow = true;
  scene.add(this.part);
  this.reset();
}
```

Three meshes: the platform box (positioned up by half its height, since a box's origin is its center), a semi-transparent square marking the floor spot, and the 5 cm teal part. All cast and receive shadows except the marker.

`.rotateX(-Math.PI / 2)` on the plane geometry lays it flat; planes are created standing up, facing +Z.

Reset

src/scene/Workcell.ts · lines 50–57

```
/** Puts the part back on the floor spot, turned to line up with the gripper's fingers. */
reset(): void {
  if (this.holder) this.scene.attach(this.part);
  this.holder = null;
  this.fallSpeed = 0;
  this.part.position.set(FLOOR_SPOT.x, FLOOR_SPOT.y, FLOOR_SPOT.z);
  this.part.rotation.set(0, yawAt(FLOOR_SPOT.x, FLOOR_SPOT.z), 0);
}
```

If the part is being held, `scene.attach` takes it back from the gripper first. Then it's placed on the floor spot, rotated to the same yaw the base will use to reach it, so its faces line up with the fingers.

Grab and release

src/scene/Workcell.ts · lines 59–80

```
/**
 * Called every frame. Grabs or releases the part based on the commanded finger
 * gap, and returns the gap to draw: never narrower than the part while it's
 * between the fingers, so the fingers stop at the part instead of passing through it.
 */
update(dt: number, tcp: THREE.Vector3, commandedGap: number, gripper: THREE.Object3D | null):
number {
  this.part.getWorldPosition(this.partWorld);
  const between = this.holder !== null || this.partWorld.distanceTo(tcp) < GRAB_DISTANCE;

  if (!this.holder && gripper && between && commandedGap <= PART_SIZE + 0.001) {
    // attach() re-parents while keeping the world transform (Unity: SetParent(parent, true)).
    gripper.attach(this.part);
    this.holder = gripper;
    this.fallSpeed = 0;
  } else if (this.holder && commandedGap > PART_SIZE + 0.002) {
    this.scene.attach(this.part);
    this.holder = null;
  }

  if (!this.holder) this.settle(dt);
  return between ? Math.max(commandedGap, PART_SIZE) : commandedGap;
}
```

Called every frame with the tool position and the **commanded** gap from the simulation.

- The part is **between** the fingers if it's already held, or its center is close enough to the TCP.
- **Grab** when it's between the fingers, not yet held, and the commanded gap has closed to the part's width.
- **Release** when it's held and the gap opens wider than the part. The small tolerances (1 mm, 2 mm) keep it from grabbing and releasing on alternate frames.

Grabbing is **re-parenting**. `gripper.attach(part)` makes the part a child of the gripper while keeping its position and rotation in the world exactly where they were, so from then on it moves with the hand. `scene.attach(part)` does the reverse on release. In Unity terms, this is `SetParent(parent, worldPositionStays: true)`; plain `add()` would keep the local offset instead and make the part jump.

The return value is the gap to **draw**: if the part is between the fingers, never less than its width. The simulation can command a fully closed gripper, but the fingers visibly stop on the part instead of passing through it.

Gravity

src/scene/Workcell.ts · lines 82–98

```
  /** Falls under gravity until resting on the platform or the floor. */
  private settle(dt: number): void {
    const p = this.part.position; // the part is a direct child of the scene here, so this is
world space
    const onPlatform =
      Math.abs(p.x - PLATFORM.x) < PLATFORM.size / 2 && Math.abs(p.z - PLATFORM.z) <
PLATFORM.size / 2;
    const restY = (onPlatform ? PLATFORM.height : 0) + PART_SIZE / 2;

    if (p.y > restY) {
      this.fallSpeed += GRAVITY * dt;
      p.y = Math.max(restY, p.y - this.fallSpeed * dt);
    }
    if (p.y <= restY) {
      p.y = restY;
      this.fallSpeed = 0;
    }
  }
}
```

When nothing holds the part, it falls until it rests on something. `restY` is where its center sits when resting: on the platform if it's over the platform's footprint, otherwise on the floor.

Each frame, falling speed increases by g·dt (9.81 m/s² × the frame time) and the part moves down by speed·dt, without passing the resting height. This simple step-by-step integration is called Euler integration, and it's plenty for a 2 mm drop.

Lining up with the fingers

src/scene/Workcell.ts · lines 100–103

```
/** The base yaw the arm uses to reach (x, z): lines a part up with the fingers. */
function yawAt(x: number, z: number): number {
  return Math.atan2(z, -x);
}
```

The same formula as the base yaw in ik.ts. Turning the part to this angle means its faces are square to the fingers when the arm arrives.

GOING DEEPER

What a real cell would do

A real gripper reports whether it's holding something, through position feedback (fingers stopped before closing fully) or motor current, often with a part-presence sensor as well. In simulation, the honest version is a physics engine with friction between pads and part, like Snack 3. Re-parenting is the **kinematic attach** that many digital-twin and offline-programming tools use, because it's fast and never drops the part by accident.

PART 2.9 · `src/scene/stage.ts`

What changed in the stage

Mostly the same lights, camera and floor as Snack 1. Three changes make it work inside a React app.

It owns its canvas

src/scene/stage.ts · lines 9–16

```
export function createStage(container: HTMLElement) {
  // The stage owns its canvas, so a remount (React StrictMode does one in development)
  // always starts from a fresh WebGL context.
  const canvas = document.createElement("canvas");
  container.append(canvas);

  const renderer = new THREE.WebGLRenderer({ canvas, antialias: true });
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
```

In Snack 1 the canvas was in index.html. Here the stage creates a fresh `<canvas>` inside the container and removes it on dispose. After StrictMode's remount, the new viewer gets a brand-new canvas and WebGL context, with no leftovers from the first one.

It sizes itself to its container

src/scene/stage.ts · lines 57–67

```
const resize = () => {
  const width = container.clientWidth;
  const height = container.clientHeight;
  if (width === 0 || height === 0) return;
  renderer.setSize(width, height, false);
  camera.aspect = width / height;
  camera.updateProjectionMatrix();
};
const observer = new ResizeObserver(resize);
observer.observe(container);
resize();
```

Snack 1 listened to the window's `resize` event because the canvas filled the window. Here the 3D view shares the page with the panel, so a `ResizeObserver` watches the container itself: it fires when the container changes size for any reason, such as the window resizing or the layout switching to the stacked mobile view.

The `false` in `setSize` tells Three.js not to set the canvas's CSS size; the stylesheet already stretches it to fill the container.

It cleans up after itself

src/scene/stage.ts · lines 69–85

```
  function dispose() {
    observer.disconnect();
    controls.dispose();
    scene.traverse((object) => {
      if (object instanceof THREE.Mesh || object instanceof THREE.LineSegments) {
        object.geometry.dispose();
        const materials = Array.isArray(object.material) ? object.material : [object.material];
        materials.forEach((m: THREE.Material) => m.dispose());
      }
    });
    renderer.dispose();
    renderer.forceContextLoss(); // free the GPU context now rather than waiting for GC
    canvas.remove();
  }

  return { renderer, scene, camera, controls, dispose };
}
```

Undoes everything the stage created: stops watching size, removes the orbit controls' event listeners, frees every geometry and material on the GPU (meshes and line objects such as the grid), releases the renderer and its WebGL context, and removes the canvas. Without this, every remount would leave a running GPU context behind.

PART 2.10 · `src/arm/Gripper.ts`

What changed in the gripper

Same glTF model and loader as Snack 1; the animation moved out of the gripper.

setGap

src/arm/Gripper.ts · lines 4–5

```
/** Distance from a finger's origin to the inner face of its grip pad (see Snack 1's build-
gripper.ts). */
const PAD_INSET = 0.022;
```

src/arm/Gripper.ts · lines 59–64

```
/** Sets the distance between the grip pads, in meters. */
setGap(gap: number): void {
  const x = gap / 2 + PAD_INSET;
  this.left.position.x = -x;
  this.right.position.x = x;
}
```

In Snack 1 the gripper animated itself between open and closed. Now the simulation owns the gap, as part of the robot's state, and the viewer passes the gap to draw (from the work cell) to `setGap` each frame.

Each finger's origin is its root, and its grip pad's inner face is 2.2 cm in from there (from the geometry in Snack 1's build-gripper.ts). So for a gap `g` between the pads, each finger sits at `g/2 + 0.022` from the center.

PART 2.11 · `src/ui/ControlPanel.tsx`

The panel

Plain React components. They read the snapshot and call simulation commands; they never touch Three.js.

Setup and the render counter

src/ui/ControlPanel.tsx · lines 7–17

```
const SPEEDS = [0.5, 1, 2, 4];
const KIND_LABEL: Record<Step["kind"], string> = { moveJ: "MoveJ", moveL: "MoveL", gripper:
"Grip", home: "MoveJ" };

/** Counts committed renders of the panel, outside React state so counting doesn't cause
renders. */
let panelCommits = 0;

export function ControlPanel({ sim }: { sim: Simulation }) {
  const snap = useSimulation(sim);
  useEffect(() => {
    panelCommits++;
  });
```

`KIND_LABEL` maps each step kind to the badge shown in the step list. `Record<Step["kind"], string>` means “an object with one string for every possible `kind`”, so adding a new step kind without a label is a compile error.

`panelCommits` counts how many times the panel has actually rendered. It's a plain module variable on purpose: storing it in React state would itself cause a render each time it changed. A `useEffect` with no dependency list runs after every render, so it increments once per render.

The panel component

src/ui/ControlPanel.tsx · lines 19–32

```
  return (
    <aside className="panel">
      <header>
        <h1>Robot Control Panel</h1>
        <p className="sub">React · Three.js · TypeScript</p>
      </header>
      <ProgramSection sim={sim} snap={snap} />
      <JointsSection sim={sim} snap={snap} />
      <GripperSection sim={sim} />
      <TcpSection snap={snap} />
      <Stats sim={sim} />
    </aside>
  );
}
```

`useSimulation(sim)` subscribes the panel to the store. The panel passes the snapshot and the simulation down to small section components. When a new snapshot arrives, the panel re-renders, and the 3D view isn't involved.

Program controls

src/ui/ControlPanel.tsx · lines 34–64

```
function ProgramSection({ sim, snap }: { sim: Simulation; snap: Snapshot }) {
  const playing = snap.status === "playing";
  const active = playing || snap.status === "paused";

  return (
    <section>
      <h2>Program: pick and place</h2>
      <div className="transport">
        <button type="button" className="primary" onClick={() => (playing ? sim.pause() :
```

```
sim.play())}>
          {playing ? "Pause" : snap.status === "paused" ? "Resume" : snap.status === "done" ?
"Run next cycle" : "Play"}
        </button>
        <button type="button" onClick={() => sim.reset()}>
          Reset
        </button>
      </div>

      <div className="options">
        <label>
          Speed{" "}
          <select value={String(snap.speed)} onChange={(e) =>
sim.setSpeed(Number(e.currentTarget.value))}>
            {SPEEDS.map((s) => (
              <option key={s} value={String(s)}>
                {s}×
              </option>
            ))}
          </select>
        </label>
        <label>
          <input type="checkbox" checked={snap.loop} onChange={(e) =>
sim.setLoop(e.currentTarget.checked)} /> Loop
        </label>
      </div>
```

The main button's label and action depend on the status: Pause while playing, Resume when paused, “Run next cycle” after a cycle finished with loop off, otherwise Play.

The speed `<select>` and loop checkbox are **controlled inputs**: their displayed value always comes from the snapshot, and changes go through the simulation. The `<select>` works with strings, hence `String()` and `Number()`. `{" "}` is how JSX adds a space between “Speed” and the dropdown.

Status line and step list

src/ui/ControlPanel.tsx · lines 66–85

```
      <p className={snap.status === "error" ? "status error" : "status"} role="status">
        {statusText(snap)}
      </p>

      <ol className="steps">
        {snap.steps.map((step, i) => {
          const current = active && i === snap.stepIndex;
          const done = active && i < snap.stepIndex;
          return (
            <li key={i} className={current ? "current" : done ? "done" : undefined}>
              <span className="label">{step.label}</span>
              <span className="kind">{KIND_LABEL[step.kind]}</span>
              {current && <span className="progress" style={{ width: `${snap.stepProgress *
100}%` }} />}
            </li>
          );
        })}
      </ol>
    </section>
  );
}
```

`role="status"` tells screen readers to announce changes to this line.

The step list highlights the current step and ticks off finished ones, but only while a run is active, so the idle list looks clean. The current step gets a thin progress bar whose width is the step's progress as a percentage.

`key={i}` helps React match list items between renders. Using the index is fine here because the list is never reordered.

Status text

src/ui/ControlPanel.tsx · lines 87–101

```
function statusText(snap: Snapshot): string {
  const where = `Cycle ${snap.cycle + 1}, step ${snap.stepIndex + 1} of ${snap.steps.length}`;
  switch (snap.status) {
    case "idle":
      return "Ready. Press Play to run the program.";
    case "playing":
      return where;
    case "paused":
      return `Paused at ${where.toLowerCase()}. Moving a joint pauses the program.`;
    case "done":
      return `Cycle ${snap.cycle} finished.`;
    case "error":
      return snap.error ?? "Error";
  }
}
```

A `switch` covering every status. Because `status` is a union type, TypeScript can tell all five cases are handled, so the function needs no fallback `return`.

Joint sliders

src/ui/ControlPanel.tsx · lines 103–123

```
function JointsSection({ sim, snap }: { sim: Simulation; snap: Snapshot }) {
  return (
    <section>
      <h2>Joints</h2>
      {JOINTS.map((joint, i) => (
        <label key={joint.name} className="slider">
          <span className="name">{joint.name}</span>
          <output>{snap.joints[i].toFixed(1)}°</output>
          <input
            type="range"
            min={joint.min}
            max={joint.max}
            step="any"
            value={snap.joints[i]}
            onChange={(e) => sim.setJoint(i, e.currentTarget.valueAsNumber)}
          />
        </label>
      ))}
    </section>
  );
}
```

One slider per joint, built from the same `JOINTS` list the arm is built from. The value comes from the snapshot, so during playback the sliders move by themselves (updated up to 15 times a second).

Dragging one calls `sim.setJoint`, which pauses the program and moves the joint. `valueAsNumber` reads the slider as a number directly. `step="any"` allows any value, so the program's fractional angles aren't rounded to a slider step.

Gripper buttons and TCP readout

src/ui/ControlPanel.tsx · lines 125–154

```
function GripperSection({ sim }: { sim: Simulation }) {
  return (
    <section>
      <h2>Gripper</h2>
      <div className="transport">
        <button type="button" onClick={() => sim.setGripper(true)}>
          Open
```

```
        </button>
        <button type="button" onClick={() => sim.setGripper(false)}>
          Close
        </button>
      </div>
    </section>
  );
}

function TcpSection({ snap }: { snap: Snapshot }) {
  const { x, y, z } = snap.tcp;
  return (
    <section>
      <h2>Tool center point (m)</h2>
      <p className="tcp">
        <span>x {x.toFixed(3)}</span>
        <span>y {y.toFixed(3)}</span>
        <span>z {z.toFixed(3)}</span>
        <span>gap {(snap.gap * 1000).toFixed(0)} mm</span>
      </p>
    </section>
  );
}
```

Open and Close call `setGripper`. The readout shows the tool position (from the snapshot's forward kinematics) and the gap in millimeters.

Stats without renders

src/ui/ControlPanel.tsx · lines 156–172

```
/**
 * Frames rendered vs. times React rendered the panel. Updated by writing to the
 * DOM directly twice a second, so showing the numbers doesn't add renders.
 */
function Stats({ sim }: { sim: Simulation }) {
  const ref = useRef<HTMLParagraphElement>(null);
  useEffect(() => {
    const id = window.setInterval(() => {
      if (ref.current) {
        ref.current.textContent =
          `3D frames: ${sim.frames.toLocaleString()} · panel renders:
${panelCommits.toLocaleString()}`;
      }
    }, 500);
    return () => window.clearInterval(id);
  }, [sim]);
  return <p ref={ref} className="stats" />;
}
```

This line proves the design works: it shows how many 3D frames have been drawn next to how many times the panel has rendered.

Showing those numbers through React state would cause renders of its own, ruining the measurement. Instead, a timer writes them straight into the paragraph's text twice a second through a ref. The cleanup function clears the timer when the component goes away.

PART 2.12 · `src/App.tsx`

Putting it together

One simulation, two consumers

src/App.tsx · lines 1–16

```
import { useState } from "react";
import { Simulation } from "./sim/Simulation";
import { ControlPanel } from "./ui/ControlPanel";
import { Viewport } from "./ui/Viewport";

export default function App() {
  // One simulation for the app's lifetime, shared by the panel and the 3D view.
  const [sim] = useState(() => new Simulation());

  return (
    <div className="app">
      <ControlPanel sim={sim} />
      <Viewport sim={sim} />
    </div>
  );
}
```

`useState(() => new Simulation())` creates the simulation once and keeps it for the app's lifetime. Passing a function (rather than `useState(new Simulation())`) means it's only constructed on the first render, not thrown away on every render.

Both the panel and the viewport receive the same `sim`. They never talk to each other directly; each only talks to the simulation.

CHAPTER 2

Interview questions

Q1 **Why not keep the simulation's state in `useState` and update it every frame?**

Every update would re-render the panel at 60–144 Hz on the same thread as the render loop, for numbers a person reads a few times a second. Keeping the state in a plain object lets the 3D loop mutate and read it directly, and React gets an immutable snapshot only when a discrete event happens or at most 15 times a second.

Q2 **What does `getSnapshot` have to guarantee?**

That it returns the same object, by `Object.is`, as long as the data hasn't changed, and a new object when it has. Returning a fresh object each call looks like a change every time, so React re-renders in a loop and errors. Returning the same object after mutating it hides changes. Hence: build a new frozen snapshot on publish, and return the cached one in between.

Q3 **What is tearing?**

With concurrent rendering, a render can be paused and resumed. If components read mutable external data during render, some can see the old value and some the new one in the same commit, so the UI is inconsistent. `useSyncExternalStore` re-checks the snapshot before committing and re-renders synchronously if it changed.

Q4 **Walk me through your IK. What happens when the target can't be reached?**

Base yaw from atan2 of the target's horizontal position. With the tool fixed vertical, the wrist joint must be directly above the target, so solve the two-link arm to the wrist joint: law of cosines for the elbow, then the shoulder as the direction to the wrist minus the angle the bent elbow adds. The wrist takes whatever makes the pitch angles sum to 180°. If the cosine falls outside [−1, 1] there is no triangle and it returns null; a separate check rejects solutions outside the joint limits.

Q5 **moveJ or moveL: when do you use each, and what can go wrong with moveL?**

moveJ for big moves in open space: fastest, always feasible between two valid poses, but the tool's path curves. moveL near parts, for approach and retreat, where the path matters. moveL can hit an unreachable point or a singularity halfway, so the path should be validated before running it.

Q6 **Your component works in production but loads twice and leaks in development. Why?**

StrictMode mounts, unmounts and remounts components in development to expose effects without proper cleanup. The fix isn't to disable StrictMode; it's a cleanup that stops the loop, disposes resources and the renderer, and a guard so async work that finishes after unmount doesn't touch a dead scene.

Q7 What's the difference between `add` and `attach` in Three.js?

`add` keeps the child's local transform, so the object jumps if the new parent is somewhere else. `attach` keeps the world transform by setting local = inverse(parent world) × child world. It's Unity's `SetParent(parent, worldPositionStays: true)`.

Q8 How do you test a simulation that normally runs in a browser?

Keep the simulation pure TypeScript with time passed in. Then tests run in Node: FK(IK(p)) = p across the work area, unreachable points return null, a full cycle ends at home, the gripper closes exactly on the part, nothing touches the floor over two loops, pause and resume, and speed scaling.

Q9 How would you let users edit the program?

The program is already data: a discriminated union of steps. An editor would edit that array immutably (cheap undo and redo), validate each step on change by running IK along it, and show unreachable steps in the panel before anything runs. Serialization is just JSON.

CHAPTER 2

Exercises

2.1 Elbow down

Add an option for the “elbow down” solution and a test that FK of both solutions reaches the same point.

Hint. Negate the elbow angle; the shoulder formula changes sign in one place. Expect different joint-limit failures for each branch.

2.2 Validate before moving

Before running, check every moveL by sampling 20 points along its line through IK and the limits, and show any failure in the panel.

Hint. This belongs in the simulation, not the component. The panel just renders a list of problems from the snapshot.

2.3 A third station

Add a third platform and a program that cycles the part through all three.

Hint. `pickAndPlace(from, to)` already builds a cycle from two stations; the loop logic needs a modulo 3 instead of even/odd.

2.4 Smarter publishing

Replace the fixed 15 Hz throttle with “publish when a displayed value changes at display precision” (0.1°, 1 mm). Count renders with the React Profiler.

Hint. Compare against the last published snapshot, rounded the way the panel rounds.

2.5 Real velocity profiles

Replace smoothstep with a trapezoidal profile per joint, given a maximum speed and acceleration, and size each moveJ by its slowest joint.

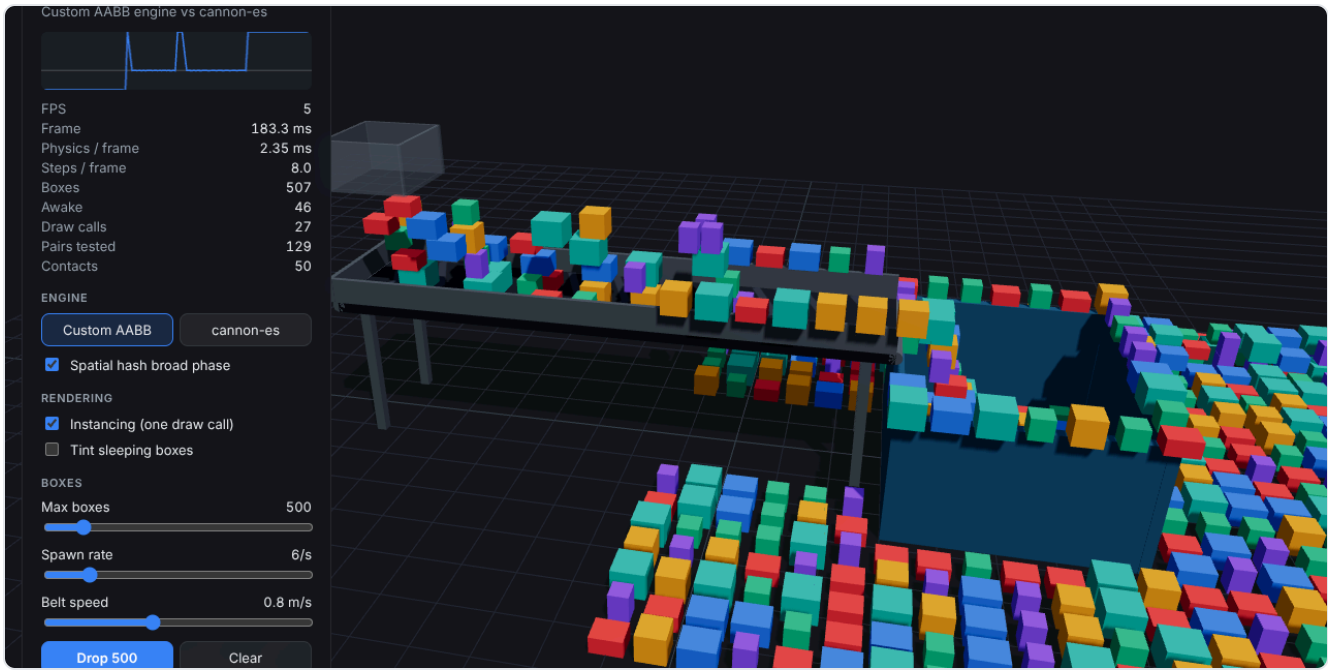
Hint. Duration = the largest of each joint's time to travel its distance. Every joint then follows the same normalized profile.

CHAPTER 3 · `vention/snack3-conveyor-physics`

Conveyor + Physics

A physics engine written from scratch, the same scene on cannon-es, and the optimisations that let thousands of boxes run at 60 fps in a browser.

Three.js · cannon-es · TypeScript



After a 500-box drop: boxes still ride the belt into the bin while the pile around it settles.

The one idea to take away

Physics runs at a fixed 120 Hz, decoupled from the frame rate, and rendering blends the last two states:

`frame time` → `FixedStepper.advance` → `N × step(1/120 s)` → `alpha` → `BoxRenderer: 1 draw call`

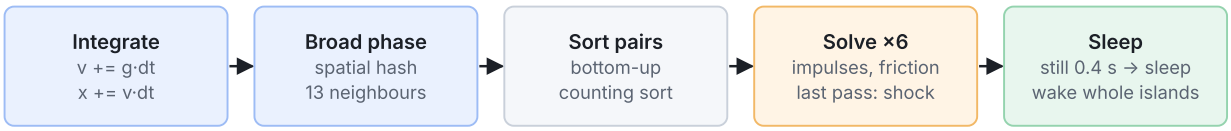
Each step is a handful of stages, each with one optimisation that keeps it cheap:

`integrate` → `broad phase (spatial hash)` → `sort bottom-up` → `6 solver passes, last with shock` → `sleep`

CONCEPTS · 3.1

What a physics engine does in one step

A rigid-body engine keeps each body's position and velocity (and, for bodies that rotate, orientation, angular velocity and an inertia tensor) and advances them by one small time step at a time. Every step has the same shape:



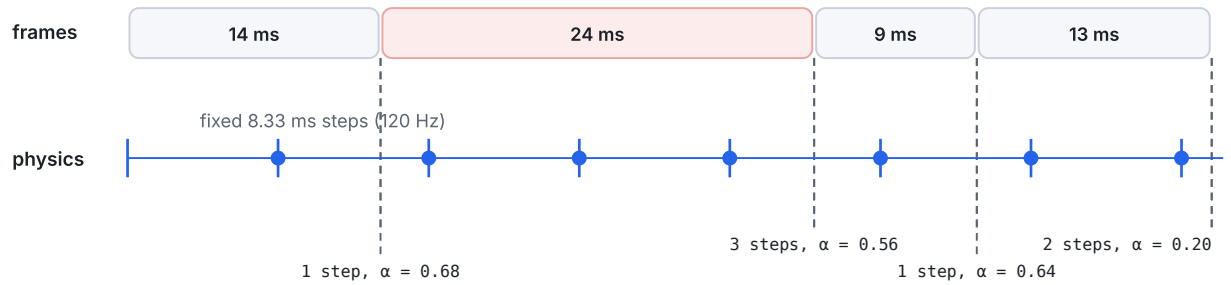
One fixed step of the custom engine (AabbWorld.step). Sleeping boxes skip integration and solving, so a settled pile costs almost nothing.

Figure 3.1. The four stages of a physics step, plus the sort that makes stacks stable.

The custom engine's big simplification is that boxes **never rotate**. Every box is an axis-aligned bounding box (AABB), so a contact normal is always one of the three axes and no contact produces torque. That cuts the code to a few hundred lines while keeping the parts that matter for a conveyor: stacking, friction, sleeping and scale.

CONCEPTS · 3.2

Time: fixed steps and interpolation



Each frame banks its time and spends it in whole steps; the leftover fraction α blends the last two physics states for drawing.

Figure 3.2. Uneven frames, identical physics steps. The 24 ms frame (a hitch) runs three steps to catch up.

- **Stability** depends on the step size. A solver tuned for 1/120 s may explode at 1/30 s. Fixed steps mean it only ever sees one size.
- **Determinism**: same inputs and same steps give the same results, so tests and benchmarks are reproducible.

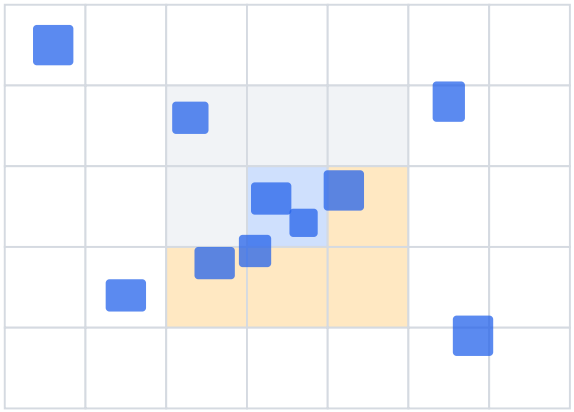
- **Interpolation** draws the moment between the last two physics states, using α. The cost is up to one step (8.3 ms) of extra latency; the alternative, extrapolating ahead, has no latency but visibly overshoots on collisions.

Integration is **semi-implicit Euler**: update velocity from forces first, then position from the new velocity. It costs the same as plain Euler but conserves energy far better, which is why almost every game engine uses it.

CONCEPTS · 3.3

Finding pairs: the broad phase

Testing every pair of n boxes is n(n − 1)/2 checks: 4.5 million for 3,000 boxes, every step. The **broad phase** cheaply finds the few pairs that might touch, and the **narrow phase** tests only those.



Cell size ≥ the largest box (+ margin), so two touching boxes are in the same or adjacent cells.

In 2D that's 8 neighbours; checking only the 4 "after" it finds each pair once. In 3D: 26 and 13.

Each bucket is a linked list in two typed arrays:



`next[7] = 3, next[3] = 12, next[12] = −1`

No list objects: rebuilding the grid each step is two array writes per box, and allocates nothing.

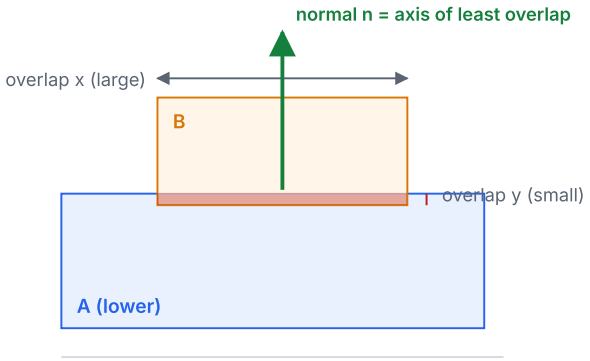
blue: the box's cell · orange: neighbours it checks · grey: neighbours that check it

Figure 3.3. The broad phase. A uniform grid stored in a hash table turns the n^2 all-pairs check into roughly n.

- **Uniform grid / spatial hash** (used here): O(n) when objects are similar in size. Struggles when sizes vary a lot.
- **Sweep and prune** (cannon-es): sort bounds along an axis and compare neighbours. Fast when objects move a little each step, because the sort is nearly sorted already.
- **Dynamic AABB tree** (Box2D, Bullet, PhysX): a hierarchy of bounding boxes. Handles any sizes and scales well; more complex to maintain.

CONCEPTS · 3.4

Contacts and impulses



`vn = (v_B − v_A) · n`
closing speed along the normal (< 0 = approaching)

`j = −(1 + e) · vn / (invA + invB)`
impulse that stops (or bounces) the approach

`|jt| ≤ μ · j`
Coulomb: friction can't exceed μ × the push

`v_rel = v_box − v_surface`
a belt's surface moves, so friction drags boxes

`push out 80% of (overlap − slop)`
fix overlap gently, without jitter

Figure 3.4. One contact between two axis-aligned boxes: find the normal, cancel the closing speed, cap friction, and remove the overlap.

Each contact is a constraint: “these two may not approach along the normal”. Solving all of them at once is a large linear problem, so engines use **sequential impulses** (projected Gauss–Seidel): fix each contact in turn, then sweep again, because fixing one disturbs its neighbours. More iterations give stiffer stacks at more cost; six is the trade-off here.

- **Restitution** only above 0.5 m/s, so resting contacts don't bounce forever on tiny gravity-driven speeds.
- **Position correction** pushes out most of the overlap beyond a small **slop**. Correcting to exactly zero makes resting boxes jitter.
- **Coulomb friction** caps the sideways impulse at μ times the normal one. Below the cap the slide stops (static friction); above it, the box keeps sliding.
- **Warm starting**, which real engines add, reuses last step's impulses as the starting guess, so fewer iterations converge. It's an exercise below.

CONCEPTS · 3.5

Stacks, sleeping and islands

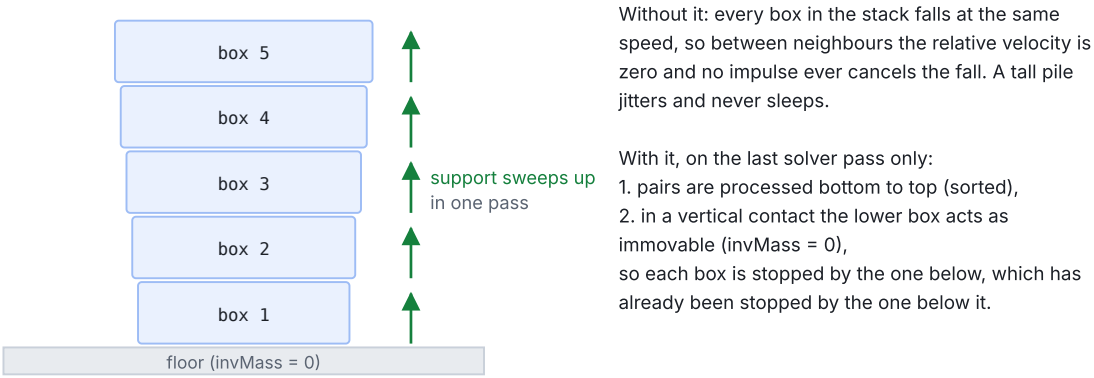


Figure 3.5. Shock propagation: the trick that lets a 3,000-box pile settle and sleep within about two seconds.

Sleeping is the other half of scaling: a box that has been nearly still for 0.4 s stops being simulated. The danger is waking: if something is removed from under a sleeping pile, the boxes above must wake, or they'd hang in the air. Waking therefore spreads to everything touching, then everything touching those, the whole connected **island**. Two bugs found while building this were both about waking: boxes falling asleep on a stopped belt and ignoring it when it restarted, and a wake stack that overwrote itself while spreading.

CONCEPTS · 3.6

Performance in a browser

- **Structure of arrays**: one typed array per property (`px`, `py`, `vx`…) instead of one object per box. Loops read contiguous memory, which suits CPU caches, and the garbage collector has nothing to scan.
- **No allocation while running**: pools, preallocated pair buffers, scratch objects. Allocation-free code has no GC pauses.
- **One draw call for all boxes**: `InstancedMesh` sends one geometry plus a buffer of per-box matrices and colors. With ~500 boxes visible, unticking instancing took the scene from 27 draw calls to 484.
- **Measure, don't guess**: time each phase with `performance.now()`, warm the JIT up before benchmarking, fix the random seed. 3,000 boxes cost about 1.6 ms per physics step while falling and 0.8 ms once settled.

CHAPTER 3

Line by line

12 parts, in the order data flows through the code.

PART 3.1 · `src/layout.ts`

The work cell as data

Every dimension in one file. The physics engines build their colliders from it and the renderer draws the same boxes, so what you see is exactly what collides.

Simulation constants

src/layout.ts · lines 6–16

```
export const GRAVITY = -9.81;
export const FIXED_STEP = 1 / 120; // physics runs at 120 Hz whatever the display does
export const MAX_SUBSTEPS = 8; // per frame; beyond this we drop time instead of spiralling
export const MAX_BODIES = 4096;
export const DENSITY = 500; // kg/m³, roughly cardboard boxes with something in them

export interface V3 {
  x: number;
  y: number;
  z: number;
}
```

- `GRAVITY` is in m/s². Negative because +Y is up.
- `FIXED_STEP` is 1/120 s: physics always advances in steps this size, whatever the display's frame rate.
- `MAX_SUBSTEPS` caps how many steps one frame may run before it gives up on catching up (see FixedStepper).
- `MAX_BODIES` sizes every per-box array once, up front. Nothing grows while the app runs.
- `DENSITY` turns a box's volume into its mass, so big boxes are heavier than small ones.
- `V3` is a plain {x, y, z}. Three.js's `Vector3` fits it, but so does any object, which keeps the physics free of Three.js.

Static colliders

src/layout.ts · lines 18–26

```
export interface StaticCollider {
  name: string;
  min: V3;
  max: V3;
  /** The conveyor belt: its surface drags boxes along at the belt speed through friction. */
  conveyor?: boolean;
  /** Draw it? The floor is drawn separately as a grid. */
  visible?: boolean;
}
```

A static collider is an axis-aligned box given by its `min` and `max` corners, and it never moves. The `?` marks optional fields:

- `conveyor` marks the belt, whose surface drags boxes along.
- `visible: false` hides the floor, since the grid is drawn instead.

Belt, bin and spawner

src/layout.ts · lines 28–30

```
export const BELT = { minX: -1.6, maxX: 0.6, halfWidth: 0.25, topY: 0.6, thickness: 0.05 };
export const BIN = { minX: 0.6, maxX: 1.5, halfDepth: 0.55, height: 0.5, wall: 0.04 };
export const SPAWN = { x: -1.45, y: 0.95, zJitter: 0.14 };
```

The belt runs along X from −1.6 m to 0.6 m, 50 cm wide, with its top 60 cm above the floor. The bin starts exactly where the belt ends, so boxes falling off the end land inside it. Boxes appear at `SPAWN`, slightly above the belt's start, with a little random sideways offset (`zJitter`).

The floor and the belt

src/layout.ts · lines 35–43

```
export const STATICS: readonly StaticCollider[] = [
  // A thick floor, so fast-falling boxes can't step right through it in one physics step.
  { name: "floor", min: { x: -8, y: -1, z: -8 }, max: { x: 8, y: 0, z: 8 }, visible: false },
  {
    name: "belt",
    min: { x: BELT.minX, y: BELT.topY - BELT.thickness, z: -BELT.halfWidth },
    max: { x: BELT.maxX, y: BELT.topY, z: BELT.halfWidth },
    conveyor: true,
  },
```

The floor is a whole meter thick even though only its top matters. A box falling at 8 m/s moves about 7 cm per physics step, so a thin floor could be stepped over completely between two checks (“tunnelling”). A thick one can't.

The belt is just a thin slab whose top is at `BELT.topY`, flagged `conveyor: true`. The rails, the backstop and the four bin walls that follow are built the same way.

Box sizes

src/layout.ts · lines 82–94

```
/**
 * A handful of box sizes (half extents). Using presets rather than random sizes
 * lets the cannon-es backend share one shape per size, and gives each size a color.
 */
export const SIZE_PRESETS: readonly V3[] = [
  { x: 0.05, y: 0.05, z: 0.05 },
  { x: 0.06, y: 0.04, z: 0.05 },
  { x: 0.04, y: 0.04, z: 0.04 },
  { x: 0.055, y: 0.03, z: 0.045 },
  { x: 0.035, y: 0.05, z: 0.035 },
  { x: 0.06, y: 0.05, z: 0.06 },
];
export const MAX_HALF_EXTENT = 0.06;
```

Six fixed sizes (half extents, so the first is a 10 cm cube). Using presets rather than any random size means cannon-es can share one shape object per size, and each size gets its own color. `MAX_HALF_EXTENT` is the biggest half size; the spatial hash uses it to choose its cell size.

Stress-test area and world bounds

src/layout.ts · lines 96–100

```
/** Where a stress-test burst rains down: over the belt end, the bin and the floor around it.
*/
export const BURST_AREA = { minX: -0.2, maxX: 2.6, minZ: -1.4, maxZ: 1.4, baseY: 1.0, spacing:
0.14 };

/** Boxes outside this are removed (fell off the world, or got flung away). */
export const BOUNDS = { minY: -2, maxAbsXZ: 6 };
```

A burst lays boxes out in a grid of layers starting 1 m up, 14 cm apart. The largest box is 12 cm, so none of them start overlapping. Anything that falls below −2 m or drifts more than 6 m out is removed.

PART 3.2 · `src/physics/FixedStepper.ts`

The fixed-timestep loop

Frames arrive at uneven intervals. Physics shouldn't care: this turns whatever time a frame took into a whole number of identical 1/120 s steps.

State

src/physics/FixedStepper.ts · lines 10–22

```
export class FixedStepper {
  readonly step: number;
  readonly maxSteps: number;
  accumulator = 0;
  /** How far between the previous and current physics state this frame is (0..1). */
  alpha = 0;
  /** Simulated time thrown away because a frame needed more than maxSteps. */
  droppedTime = 0;

  constructor(step: number, maxSteps: number) {
    this.step = step;
    this.maxSteps = maxSteps;
  }
```

`accumulator` holds simulated time that has built up but not been stepped yet. `alpha` is the fraction of a step left over after the last one, from 0 to 1, which the renderer uses to blend. `droppedTime` counts time thrown away because a frame was too slow.

Running the steps

src/physics/FixedStepper.ts · lines 24–32

```
/** Runs as many fixed steps as `frameDt` calls for. Returns how many ran. */
advance(frameDt: number, stepOnce: () => void): number {
  this.accumulator += frameDt;
  let steps = 0;
  while (this.accumulator >= this.step && steps < this.maxSteps) {
    stepOnce();
    this.accumulator -= this.step;
    steps++;
  }
```

Add the frame's time to the bank, then spend it a step at a time. On a 60 Hz display each frame adds 1/60 s and two 1/120 s steps run; on a 144 Hz display most frames run zero or one. Either way every step is the same size, so the simulation behaves identically. `stepOnce` is a callback, so this class knows nothing about the engine it drives.

Avoiding the spiral of death

src/physics/FixedStepper.ts · lines 33–38

```
// If physics can't keep up, drop the backlog rather than trying to catch up
// next frame, which would take even longer (the "spiral of death").
if (this.accumulator >= this.step) {
  this.droppedTime += this.accumulator - (this.accumulator % this.step);
  this.accumulator %= this.step;
}
```

If a frame was very slow, the bank holds more than `maxSteps` worth. Trying to run it all next frame would make that frame slower still, which leaves even more to run the frame after: the “spiral of death”. Instead, whole steps beyond the cap are dropped (and counted), keeping only the fraction of a step. `%` is the remainder after division.

Interpolation factor

src/physics/FixedStepper.ts · lines 39–41

```
  this.alpha = this.accumulator / this.step;
  return steps;
}
```

The leftover fraction of a step. If `alpha` is 0.3, the moment being drawn is 30% of the way from the previous physics state to the current one, so each box is drawn 30% of the way between its two positions. Without this, motion would visibly stutter whenever the frame rate and the physics rate don't line up.

GOING DEEPER

How far does determinism go?

Fixed steps plus a seeded random generator make a run repeatable on one machine and one build. Across browsers or CPUs, results can still drift: `Math.sin` and friends aren't required to round identically everywhere, and different JIT decisions can reorder floating-point operations. Networked games that simulate in lockstep therefore use fixed-point math or carefully specified math libraries; for a single-player demo and its tests, same-machine determinism is enough.

PART 3.3 · `src/sim/SlotPool.ts`

Object pooling

Boxes are created and removed constantly. Instead of allocating objects, every box is a slot number into preallocated arrays.

Three arrays

src/sim/SlotPool.ts · lines 13–31

```
export class SlotPool {
  readonly capacity: number;
  /** Live slots, packed: dense[0..count-1]. */
  readonly dense: Int32Array;
  count = 0;
  /** slot -> its index in `dense`, or -1 if free. */
  private readonly where: Int32Array;
  private readonly free: Int32Array;
  private freeTop: number;

  constructor(capacity: number) {
    this.capacity = capacity;
    this.dense = new Int32Array(capacity);
    this.where = new Int32Array(capacity).fill(-1);
    this.free = new Int32Array(capacity);
    // Stack the free slots so slot 0 comes out first.
    for (let i = 0; i < capacity; i++) this.free[i] = capacity - 1 - i;
    this.freeTop = capacity;
  }
```

- `dense` lists the live slots packed at the front, so loops run over exactly `count` entries and never skip gaps.
- `where` maps a slot back to its position in `dense` (−1 when free). That's what makes removal O(1).
- `free` is a stack of unused slots. The constructor fills it backwards so slot 0 is handed out first.

`Int32Array` is a fixed-size array of 32-bit integers: compact, fast, and never resized or garbage-collected.

Taking a slot

src/sim/SlotPool.ts · lines 33–40

```
/** Returns a free slot, or -1 if the pool is full. */
alloc(): number {
  if (this.freeTop === 0) return -1;
  const slot = this.free[--this.freeTop];
  this.where[slot] = this.count;
  this.dense[this.count++] = slot;
  return slot;
}
```

Pop a slot off the free stack, append it to the end of `dense`, and record where it went. `--this.freeTop` decreases the counter first and then reads; `this.count++` reads the old value and then increases it.

Giving it back: swap-remove

src/sim/SlotPool.ts · lines 42–50

```
release(slot: number): void {
  const k = this.where[slot];
  if (k < 0) return; // already free
  const last = this.dense[--this.count];
  this.dense[k] = last; // move the last live slot into the hole
  this.where[last] = k;
  this.where[slot] = -1;
  this.free[this.freeTop++] = slot;
}
```

To remove from the middle of `dense` without shifting everything after it, move the **last** live slot into the hole and shrink the count. One copy instead of up to 4,000. The cost is that `dense` isn't kept in spawn order, which nothing relies on. The slot goes back on the free stack for reuse.

PART 3.4 · `src/physics/types.ts`

One interface, two engines

The rest of the app only ever talks to this interface, which is what makes the engines swappable and the benchmark fair.

Data types

src/physics/types.ts · lines 11–28

```
/** A box to add: center and half extents, in meters. */
export interface BodyDesc {
  x: number;
  y: number;
  z: number;
  hx: number;
  hy: number;
  hz: number;
}

export interface PhysicsStats {
  /** Pairs checked in the narrow phase this step (custom engine only; -1 when not available).
*/
  pairsTested: number;
  /** Touching pairs this step. */
  contacts: number;
  /** Bodies not asleep. */
  awake: number;
}
```

`BodyDesc` describes a new box: its center and half extents. `PhysicsStats` is what the panel shows. cannon-es doesn't report how many pairs it tested, so it reports −1, and the panel shows “n/a”.

PhysicsBackend

src/physics/types.ts · lines 30–48

```
/**
 * What the rest of the app needs from a physics engine. The custom engine and
 * cannon-es both implement it, so they can be swapped at runtime and benchmarked
 * on exactly the same scene.
 */
export interface PhysicsBackend {
  readonly name: string;
  readonly stats: PhysicsStats;
  beltSpeed: number;
  add(slot: number, desc: BodyDesc): void;
  remove(slot: number): void;
  /** Remember the current state as "previous", for interpolation. Call before each step. */
  savePrevious(pool: SlotPool): void;
  step(dt: number, pool: SlotPool): void;
  getPosition(slot: number, out: V3): void;
  /** Position and rotation blended `alpha` of the way from the previous step to the current
one. */
  readTransform(slot: number, alpha: number, pos: V3, quat: Quat): void;
  isAsleep(slot: number): boolean;
}
```

Everything an engine must do, keyed by slot number: add and remove boxes, step, report positions, and say whether a box is asleep. `savePrevious` plus `readTransform(slot, alpha, …)` is the interpolation contract: the engine keeps the last two states and blends them for the renderer.

`beltSpeed` is a property, so the custom engine can implement it with a getter and setter (it wakes everything on change).

PART 3.5 · `src/physics/SpatialHash.ts`

The spatial hash

The broad phase answers “which pairs of boxes might be touching?”. Checking every pair is n^2 work; a grid makes it roughly n.

Layout in memory

src/physics/SpatialHash.ts · lines 12–33

```
export class SpatialHash {
  readonly cellSize: number;
  readonly head: Int32Array;
  readonly next: Int32Array;
  readonly cx: Int32Array;
  readonly cy: Int32Array;
  readonly cz: Int32Array;
  private readonly inv: number;
  private readonly mask: number;

  constructor(capacity: number, cellSize: number) {
    this.cellSize = cellSize;
    this.inv = 1 / cellSize;
    let size = 1;
    while (size < capacity * 2) size <<= 1; // power of two, so `& mask` replaces `%`
    this.mask = size - 1;
    this.head = new Int32Array(size);
    this.next = new Int32Array(capacity);
    this.cx = new Int32Array(capacity);
    this.cy = new Int32Array(capacity);
    this.cz = new Int32Array(capacity);
  }
```

Space is divided into cubic cells, and each box belongs to the cell containing its center. Since there could be any number of cells, they're stored in a hash table with a fixed number of buckets (at least twice the number of boxes, so buckets are rarely shared).

Each bucket is a linked list built from two arrays: `head[bucket]` is the first box in it and `next[box]` is the box after that, with −1 meaning the end. No list objects, no allocation. `cx`, `cy`, `cz` remember each box's cell.

Rounding the table size up to a power of two (`size <<= 1` doubles it) means `& mask` can replace the slower `%` when turning a hash into a bucket number.

Hashing a cell

src/physics/SpatialHash.ts · lines 35–38

```
/** Bucket for integer cell coordinates. Large primes spread neighboring cells apart. */
bucket(x: number, y: number, z: number): number {
  return (Math.imul(x, 73856093) ^ Math.imul(y, 19349663) ^ Math.imul(z, 83492791)) &
this.mask;
}
```

Multiplies each cell coordinate by a large prime, mixes the three with XOR (`^`), and keeps the low bits. Neighboring cells land in scattered buckets. `Math.imul` does 32-bit integer multiplication, wrapping around as C would, instead of producing a huge floating-point number.

Filling the grid

src/physics/SpatialHash.ts · lines 40–56

```
/** Integer cell coordinate along one axis. */
cell(v: number): number {
  return Math.floor(v * this.inv);
}

build(list: Int32Array, count: number, px: Float64Array, py: Float64Array, pz: Float64Array):
void {
  this.head.fill(-1);
  for (let k = 0; k < count; k++) {
    const i = list[k];
    const x = (this.cx[i] = this.cell(px[i]));
    const y = (this.cy[i] = this.cell(py[i]));
    const z = (this.cz[i] = this.cell(pz[i]));
    const b = this.bucket(x, y, z);
    this.next[i] = this.head[b];
    this.head[b] = i;
  }
}
```

`cell` turns a coordinate into an integer cell index (multiplying by `1/cellSize` is cheaper than dividing). `build` empties every bucket, then for each live box records its cell and pushes it onto the front of its bucket's list. This runs once per physics step.

Half the neighbors

src/physics/SpatialHash.ts · lines 59–72

```
/**
 * Half of the 26 neighbor offsets: the ones "after" (0,0,0) in x, then y, then z
 * order. Checking only these (plus the body's own cell) visits every neighboring
 * pair exactly once.
 */
export const HALF_NEIGHBORS: readonly (readonly [number, number, number])[] = (() => {
  const out: [number, number, number][] = [];
  for (let dx = -1; dx <= 1; dx++)
    for (let dy = -1; dy <= 1; dy++)
      for (let dz = -1; dz <= 1; dz++) {
        if (dx > 0 || (dx === 0 && dy > 0) || (dx === 0 && dy === 0 && dz > 0)) out.push([dx,
dy, dz]);
      }
  return out;
})();
```

Cells are at least as wide as the largest box, so two touching boxes are always in the same or adjacent cells: 27 cells to look at. But looking in all 26 neighbors from every box finds each pair twice (A finds B, then B finds A).

So each box only looks at the 13 neighbors that come “after” it in x, then y, then z order, plus later boxes in its own cell. Every adjacent pair is then found exactly once. The `(() => { … })()` runs once at startup to build the list.

GOING DEEPER

Choosing the cell size

Too small, and a box overlaps many cells, so it must be inserted into all of them or neighbours missed. Too large, and many boxes share a cell, sliding back toward n^2. Here the cell is the largest box plus the broad-phase margin, and each box is inserted by its center only, which is exactly enough to guarantee touching boxes are in neighbouring cells.

The three multipliers in `bucket()` (73856093, 19349663, 83492791) are the classic primes from Teschner et al.'s 2003 paper on spatial hashing for collision detection. Different cells can still collide in one bucket, which the code handles by checking each box's stored cell coordinates.

PART 3.6 · `src/physics/AabbWorld.ts`

The custom engine

About 500 lines that do what a physics engine does, for boxes that never rotate. The four stages of a step are integration, broad phase, narrow phase and sleeping.

Tuning constants

src/physics/AabbWorld.ts · lines 6–21

```
// ---- Tuning ----
const ITERATIONS = 6; // solver passes per step; more = stiffer stacks, more cost
const RESTITUTION = 0.1; // bounciness for real impacts
const BOUNCE_THRESHOLD = 0.5; // m/s; slower contacts don't bounce, which stops resting jitter
const FRICTION = 0.5;
const SLOP = 0.0005; // allowed overlap (m) before position correction kicks in
const CORRECTION = 0.8; // fraction of the overlap removed per pass
const BROAD_MARGIN = 0.005; // m; pairs this close become candidates for the step
const MAX_SPEED = 8; // m/s; caps how far a box can move in one step (anti-tunnelling)
const SLEEP_SPEED_SQ = 0.06 * 0.06;
const SLEEP_TIME = 0.4; // s below SLEEP_SPEED before a body sleeps
const WAKE_SPEED = 0.25; // m/s impact that wakes a sleeping body
const WAKE_MARGIN = 0.01; // m; sleeping bodies this close to a woken one wake too
const MAX_PAIRS_PER_BODY = 16;
const LAYER_HEIGHT = 0.05; // m; pairs are ordered bottom-up in layers this thick
const LAYERS = 128; // covers y from -1 m to 5.4 m; anything higher goes in the top layer
```

Each has a comment saying what it trades off. A few worth knowing:

- `ITERATIONS`: contacts are solved 6 times per step. Each pass fixes some overlap and disturbs neighbors a little; repeating converges.
- `SLOP`: up to 0.5 mm of overlap is allowed before correction. Correcting to exactly zero makes resting boxes jitter.
- `BOUNCE_THRESHOLD`: only impacts faster than 0.5 m/s bounce. Otherwise resting boxes would keep hopping on tiny gravity-driven contacts.
- `MAX_SPEED`: capping speed caps how far a box moves per step, the other half of the anti-tunnelling defence.

Structure of arrays

src/physics/AabbWorld.ts · lines 47–82

```
export class AabbWorld implements PhysicsBackend {
  readonly name = "Custom AABB";
  readonly stats: PhysicsStats = { pairsTested: 0, contacts: 0, awake: 0 };
  /** Off = test every pair of bodies (O(n²)), to show what the spatial hash saves. */
  useSpatialHash = true;

  readonly px: Float64Array;
  readonly py: Float64Array;
  readonly pz: Float64Array;
  readonly vx: Float64Array;
  readonly vy: Float64Array;
  readonly vz: Float64Array;
  readonly hx: Float64Array;
  readonly hy: Float64Array;
  readonly hz: Float64Array;
  readonly invMass: Float64Array;
  readonly asleep: Uint8Array;
  private readonly alive: Uint8Array;
  private readonly prevX: Float64Array;
  private readonly prevY: Float64Array;
  private readonly prevZ: Float64Array;
  private readonly sleepTime: Float64Array;
  /** Set during a step for bodies touching the belt: those must not fall asleep while it runs.
*/
  private readonly onBelt: Uint8Array;
  private speed = 0.8;

  private readonly statics: StaticBox[];
  private readonly hash: SpatialHash;
  private readonly neighborOffsets: Int32Array;
  private pairs: Int32Array;
  private sortedPairs: Int32Array;
  private readonly layerCounts = new Int32Array(LAYERS + 1);
  private pairCount = 0;
  private readonly wakeStack: Int32Array;
  private hashBuilt = false;
  private lastPool: SlotPool | null = null;
```

There's no `Box` class. Each property of every box is its own typed array indexed by slot: `px[7]` is box 7's x position. Loops that touch one property read contiguous memory, which is much faster for the CPU's cache than chasing objects scattered around the heap, and there is nothing for the garbage collector to scan. This is how engines and ECS frameworks lay out data.

`invMass` stores 1/mass, because the solver divides by mass constantly, and an immovable object is simply `invMass = 0`.

Constructor

src/physics/AabbWorld.ts · lines 84–112

```
constructor(capacity: number, statics: readonly StaticCollider[] = STATICS) {
  const f64 = () => new Float64Array(capacity);
  this.px = f64(); this.py = f64(); this.pz = f64();
  this.vx = f64(); this.vy = f64(); this.vz = f64();
  this.hx = f64(); this.hy = f64(); this.hz = f64();
  this.prevX = f64(); this.prevY = f64(); this.prevZ = f64();
  this.invMass = f64();
  this.sleepTime = f64();
  this.asleep = new Uint8Array(capacity);
  this.alive = new Uint8Array(capacity);
  this.onBelt = new Uint8Array(capacity);

  this.statics = statics.map((s) => ({
    cx: (s.min.x + s.max.x) / 2,
    cy: (s.min.y + s.max.y) / 2,
    cz: (s.min.z + s.max.z) / 2,
    hx: (s.max.x - s.min.x) / 2,
    hy: (s.max.y - s.min.y) / 2,
    hz: (s.max.z - s.min.z) / 2,
    conveyor: s.conveyor === true,
  }));

  // Cells as wide as the biggest box (plus margin), so touching boxes are always in
neighboring cells.
  this.hash = new SpatialHash(capacity, 2 * MAX_HALF_EXTENT + BROAD_MARGIN);
  this.neighborOffsets = Int32Array.from(HALF_NEIGHBORS.flat());
  this.pairs = new Int32Array(capacity * MAX_PAIRS_PER_BODY * 2);
  this.sortedPairs = new Int32Array(capacity * MAX_PAIRS_PER_BODY * 2);
  this.wakeStack = new Int32Array(capacity);
}
```

Allocates everything once at full capacity: positions, velocities, sizes, previous positions for interpolation, sleep timers and flags. Static colliders are converted from min/max corners to center + half size, which is what the collision math wants.

The hash's cell size is the largest box plus the broad-phase margin, which is what guarantees touching boxes are in neighboring cells. The pair buffer holds up to 16 pairs per box, again allocated once.

Belt speed

src/physics/AabbWorld.ts · lines 114–129

```
get beltSpeed(): number {
  return this.speed;
}

/** Changing the belt speed wakes everything: sleeping boxes on the belt have to start (or
stop) moving. */
set beltSpeed(v: number) {
  if (v === this.speed) return;
  this.speed = v;
  if (this.lastPool) {
    const { dense, count } = this.lastPool;
    for (let k = 0; k < count; k++) {
      this.asleep[dense[k]] = 0;
      this.sleepTime[dense[k]] = 0;
    }
  }
}
```

A getter/setter pair, so `world.beltSpeed = 0.5` runs code. Changing the speed wakes every box: a sleeping box on the belt isn't being simulated, so without this it would ignore the belt starting up again.

Adding and removing

src/physics/AabbWorld.ts · lines 131–149

```
add(slot: number, d: BodyDesc): void {
  this.px[slot] = this.prevX[slot] = d.x;
  this.py[slot] = this.prevY[slot] = d.y;
  this.pz[slot] = this.prevZ[slot] = d.z;
  this.vx[slot] = this.vy[slot] = this.vz[slot] = 0;
  this.hx[slot] = d.hx;
  this.hy[slot] = d.hy;
  this.hz[slot] = d.hz;
  this.invMass[slot] = 1 / (8 * d.hx * d.hy * d.hz * DENSITY);
  this.asleep[slot] = 0;
  this.sleepTime[slot] = 0;
  this.alive[slot] = 1;
}

remove(slot: number): void {
  this.alive[slot] = 0;
  // Whatever was resting on this box has lost its support: wake it (and its pile).
  this.drainWake(this.pushSleepingNeighbors(slot, 0));
}
```

`add` writes the new box's values into its slot. Previous position = current position, so interpolation doesn't draw a streak from wherever the slot's last box was. Mass is volume × density; `8·hx·hy·hz` is the volume, since the full sides are twice the half extents.

`remove` marks the slot dead, then wakes anything that was resting on it (explained under Sleeping).

One step

src/physics/AabbWorld.ts · lines 151–180

```
savePrevious(pool: SlotPool): void {
  const { dense, count } = pool;
  for (let k = 0; k < count; k++) {
    const i = dense[k];
    this.prevX[i] = this.px[i];
    this.prevY[i] = this.py[i];
    this.prevZ[i] = this.pz[i];
  }
}
```

```
step(dt: number, pool: SlotPool): void {
  this.lastPool = pool;
  this.integrate(dt, pool);
  this.broadPhase(pool);
  this.sortPairsBottomUp();

  this.stats.contacts = 0;
  for (let k = 0; k < pool.count; k++) this.onBelt[pool.dense[k]] = 0;
  for (let it = 0; it < ITERATIONS; it++) {
    const first = it === 0;
    const shock = it === ITERATIONS - 1;
    for (let k = 0; k < pool.count; k++) {
      const i = pool.dense[k];
      if (!this.asleep[i]) this.solveStatics(i, first);
    }
    for (let p = 0; p < this.pairCount; p++) this.solvePair(this.pairs[2 * p], this.pairs[2 * p
+ 1], first, shock);
  }

  this.updateSleep(dt, pool);
}
```

`savePrevious` copies positions aside before the step, for interpolation. `step` is the whole engine in a dozen lines:

1. **Integrate**: move everything by gravity and velocity.
2. **Broad phase**: find the pairs that might touch.
3. **Sort** those pairs from bottom to top.
4. **Solve**, 6 times: each awake box against the statics, then every pair. The first pass counts contacts; the last uses shock propagation.
5. **Sleep**: put still boxes to sleep.

`onBelt` is cleared first and set again during solving, so it's true only for boxes touching the belt this step.

Reading results

src/physics/AabbWorld.ts · lines 182–198

```
getPosition(slot: number, out: V3): void {
  out.x = this.px[slot];
  out.y = this.py[slot];
  out.z = this.pz[slot];
}

readTransform(slot: number, alpha: number, pos: V3, quat: Quat): void {
  pos.x = this.prevX[slot] + (this.px[slot] - this.prevX[slot]) * alpha;
  pos.y = this.prevY[slot] + (this.py[slot] - this.prevY[slot]) * alpha;
  pos.z = this.prevZ[slot] + (this.pz[slot] - this.prevZ[slot]) * alpha;
  quat.x = quat.y = quat.z = 0; // axis-aligned: never rotates
  quat.w = 1;
}

isAsleep(slot: number): boolean {
  return this.asleep[slot] === 1;
}
```

`readTransform` blends previous and current positions by `alpha` (the same lerp formula as Snack 2). Rotation is always the identity quaternion (0, 0, 0, 1), because these boxes never rotate.

1. Integration

src/physics/AabbWorld.ts · lines 208–228

```
// ---- 1. Integration ----

private integrate(dt: number, pool: SlotPool): void {
  const { px, py, pz, vx, vy, vz, asleep } = this;
  const { dense, count } = pool;
  for (let k = 0; k < count; k++) {
    const i = dense[k];
    if (asleep[i]) continue;
    vy[i] += GRAVITY * dt;
    const speedSq = vx[i] * vx[i] + vy[i] * vy[i] + vz[i] * vz[i];
    if (speedSq > MAX_SPEED * MAX_SPEED) {
      const s = MAX_SPEED / Math.sqrt(speedSq);
      vx[i] *= s;
      vy[i] *= s;
      vz[i] *= s;
    }
    px[i] += vx[i] * dt;
    py[i] += vy[i] * dt;
    pz[i] += vz[i] * dt;
  }
}
```

Semi-implicit Euler: update velocity first (gravity), then move by the **new** velocity. Doing it in that order, rather than moving by the old velocity, is what keeps simple game physics stable. Sleeping boxes are skipped entirely.

Copying the arrays into local constants (`const { px, … } = this`) saves a property lookup on every access in the loop. Speeds above `MAX_SPEED` are scaled back down.

2. Broad phase

src/physics/AabbWorld.ts · lines 230–267

```
// ---- 2. Broad phase ----

/** Fills `pairs` with every pair of live bodies whose boxes are within BROAD_MARGIN. */
private broadPhase(pool: SlotPool): void {
  this.pairCount = 0;
  this.stats.pairsTested = 0;
  const { dense, count } = pool;
```

```
  if (!this.useSpatialHash) {
    for (let a = 0; a < count; a++)
      for (let b = a + 1; b < count; b++) this.candidate(dense[a], dense[b]);
    this.hashBuilt = false;
    return;
  }

  const h = this.hash;
  h.build(dense, count, this.px, this.py, this.pz);
  this.hashBuilt = true;
  const { head, next, cx, cy, cz } = h;
  const offsets = this.neighborOffsets;

  for (let k = 0; k < count; k++) {
    const i = dense[k];
    const x = cx[i], y = cy[i], z = cz[i];
    // Same cell: only bodies after i in the chain, so each pair is seen once.
    for (let j = next[i]; j !== -1; j = next[j]) {
      if (cx[j] === x && cy[j] === y && cz[j] === z) this.candidate(i, j);
    }
    // The 13 "forward" neighbor cells.
    for (let o = 0; o < offsets.length; o += 3) {
      const qx = x + offsets[o], qy = y + offsets[o + 1], qz = z + offsets[o + 2];
      for (let j = head[h.bucket(qx, qy, qz)]; j !== -1; j = next[j]) {
        // Different cells can share a bucket; check it really is the cell we asked for.
        if (cx[j] === qx && cy[j] === qy && cz[j] === qz) this.candidate(i, j);
      }
    }
  }
}
```

With the hash turned off, the double loop tries every pair: for 3,000 boxes that's 4.5 million checks per step. With it, the grid is rebuilt and each box checks its own cell's later boxes and its 13 forward neighbors.

Different cells can hash to the same bucket, so each box found is checked against the exact cell asked for. Without that check, a bucket shared by two of a box's neighbor cells would be scanned twice and report pairs twice.

Sorting pairs bottom-up

src/physics/AabbWorld.ts · lines 269–292

```
/**
 * Reorders the pairs from lowest to highest (counting sort on height layers, O(n)).
 * Solving bottom-up lets support travel up a stack within a single pass.
 */
private sortPairsBottomUp(): void {
  const counts = this.layerCounts;
  counts.fill(0);
  const { py, pairs } = this;
  const layerOf = (p: number) => {
    const y = Math.min(py[pairs[2 * p]], py[pairs[2 * p + 1]]);
    return Math.min(LAYERS - 1, Math.max(0, Math.floor((y + 1) / LAYER_HEIGHT)));
  };
  for (let p = 0; p < this.pairCount; p++) counts[layerOf(p) + 1]++;
  for (let l = 1; l <= LAYERS; l++) counts[l] += counts[l - 1]; // running totals = where each
layer starts
  const out = this.sortedPairs;
  for (let p = 0; p < this.pairCount; p++) {
    const at = counts[layerOf(p)]++;
    out[2 * at] = pairs[2 * p];
    out[2 * at + 1] = pairs[2 * p + 1];
  }
  // Swap buffers instead of copying back.
  this.sortedPairs = pairs;
  this.pairs = out;
}
```

A **counting sort**: instead of comparing pairs, it buckets them by height in 5 cm layers. It counts how many pairs fall in each layer, turns the counts into running totals (where each layer starts), then drops each pair into its place. That's O(n), faster than a general sort's O(n log n).

Rather than copying the sorted pairs back, the two buffers swap roles. Why sort at all is explained under shock propagation below.

Candidate test

src/physics/AabbWorld.ts · lines 294–307

```
private candidate(i: number, j: number): void {
  if (this.asleep[i] && this.asleep[j]) return; // two sleepers can't affect each other
  this.stats.pairsTested++;
  if (
    Math.abs(this.px[j] - this.px[i]) < this.hx[i] + this.hx[j] + BROAD_MARGIN &&
    Math.abs(this.py[j] - this.py[i]) < this.hy[i] + this.hy[j] + BROAD_MARGIN &&
    Math.abs(this.pz[j] - this.pz[i]) < this.hz[i] + this.hz[j] + BROAD_MARGIN &&
    2 * this.pairCount < this.pairs.length
  ) {
    this.pairs[2 * this.pairCount] = i;
    this.pairs[2 * this.pairCount + 1] = j;
    this.pairCount++;
  }
}
```

Two sleeping boxes can't push each other, so the pair is skipped before it's even counted. Otherwise, two boxes are a candidate if they overlap (or nearly, within `BROAD_MARGIN`) on all three axes: the gap between centers is less than the sum of their half sizes. This one test is what an AABB (axis-aligned bounding box) check is.

3. Narrow phase: is there a contact, and which way?

src/physics/AabbWorld.ts · lines 311–334

```
/**
 * Box vs box. The normal points from i to j along the axis where they overlap least.
 *
 * `shock` (the last pass) is shock propagation: in a vertical contact the lower box
 * is treated as immovable, so a stack's upper boxes can't push it back down. Combined
 * with bottom-up order, the floor's support reaches the top of a stack in one sweep.
 * Without it, every box in a tall stack keeps a shared downward velocity that never
 * cancels, and the stack never comes to rest.
 */
private solvePair(i: number, j: number, first: boolean, shock: boolean): void {
  const { px, py, pz, vx, vy, vz, hx, hy, hz } = this;
  const dx = px[j] - px[i], dy = py[j] - py[i], dz = pz[j] - pz[i];
  const ox = hx[i] + hx[j] - Math.abs(dx);
  if (ox <= 0) return;
  const oy = hy[i] + hy[j] - Math.abs(dy);
  if (oy <= 0) return;
  const oz = hz[i] + hz[j] - Math.abs(dz);
  if (oz <= 0) return;
  if (first) this.stats.contacts++;

  let nx = 0, ny = 0, nz = 0, depth: number;
  if (ox <= oy && ox <= oz) { depth = ox; nx = dx < 0 ? -1 : 1; }
  else if (oy <= oz) { depth = oy; ny = dy < 0 ? -1 : 1; }
  else { depth = oz; nz = dz < 0 ? -1 : 1; }
```

Overlap on each axis is the sum of half sizes minus the distance between centers; if any is ≤ 0, they aren't touching. The contact **normal** is the axis with the **least** overlap: the direction that separates them with the smallest push. A box resting on another overlaps a little vertically and a lot horizontally, so the normal is vertical, as it should be.

Waking, and shock propagation

src/physics/AabbWorld.ts · lines 336–349

```
const rvx = vx[j] - vx[i], rvy = vy[j] - vy[i], rvz = vz[j] - vz[i];
const vn = rvx * nx + rvy * ny + rvz * nz; // < 0: closing in

// A sleeping box acts as immovable, unless it's hit hard enough to wake.
if (this.asleep[i] && vn < -WAKE_SPEED) this.wake(i);
if (this.asleep[j] && vn < -WAKE_SPEED) this.wake(j);
let wi = this.asleep[i] ? 0 : this.invMass[i];
let wj = this.asleep[j] ? 0 : this.invMass[j];
if (shock && ny !== 0) {
  if (ny > 0) wi = 0; // i is below j
  else wj = 0;
}
const w = wi + wj;
if (w === 0) return;
```

`vn` is how fast the boxes approach each other along the normal (negative means closing in). A sleeping box acts as an immovable object, unless it's hit faster than `WAKE_SPEED`, which wakes it.

Then **shock propagation**, on the last pass only: in a vertical contact, the lower box gets an inverse mass of 0, as if it were the floor. Combined with the bottom-up order, the floor's support travels from the bottom of a stack to the top in one sweep.

> Why this was needed: without it, every box in a tall stack shared the same downward velocity. Between neighbours the relative velocity was zero, so no impulse ever cancelled it. A 3,000-box pile stayed awake and jittering forever; with it, the pile is asleep within about two seconds.

Pushing apart

src/physics/AabbWorld.ts · lines 351–356

```
// Push apart, split by inverse mass (the lighter box moves more).
const push = (Math.max(depth - SLOP, 0) * CORRECTION) / w;
px[i] -= nx * push * wi; py[i] -= ny * push * wi; pz[i] -= nz * push * wi;
px[j] += nx * push * wj; py[j] += ny * push * wj; pz[j] += nz * push * wj;

if (vn >= 0) return; // already separating
```

Remove most (80%) of the overlap beyond the allowed slop, shared in proportion to inverse mass, so a light box moves more than a heavy one and a sleeping or shocked box doesn't move at all. If they're already moving apart, there's nothing more to do.

Normal impulse

src/physics/AabbWorld.ts · lines 358–362

```
// Normal impulse: cancel the closing speed (plus a little bounce for real impacts).
const e = vn < -BOUNCE_THRESHOLD ? RESTITUTION : 0;
const jn = (-(1 + e) * vn) / w;
vx[i] -= nx * jn * wi; vy[i] -= ny * jn * wi; vz[i] -= nz * jn * wi;
vx[j] += nx * jn * wj; vy[j] += ny * jn * wj; vz[j] += nz * jn * wj;
```

An **impulse** is an instant change in momentum. This one is sized to exactly cancel the closing speed (or reverse a little of it, if the impact is hard enough to bounce), split between the two boxes by inverse mass. Equal and opposite: one box loses what the other gains, as Newton's third law says.

Friction

src/physics/AabbWorld.ts · lines 364–372

```
  // Friction: oppose sliding, but no more than FRICTION × the normal impulse (Coulomb).
  let tx = rvx - vn * nx, ty = rvy - vn * ny, tz = rvz - vn * nz;
  const slide = Math.sqrt(tx * tx + ty * ty + tz * tz);
  if (slide < 1e-9) return;
  const jt = Math.min(slide / w, FRICTION * jn);
  tx /= slide; ty /= slide; tz /= slide;
  vx[i] += tx * jt * wi; vy[i] += ty * jt * wi; vz[i] += tz * jt * wi;
  vx[j] -= tx * jt * wj; vy[j] -= ty * jt * wj; vz[j] -= tz * jt * wj;
}
```

`t` is the sliding part of the relative velocity (with the normal part removed). Friction pushes against the slide, but **Coulomb's law** caps it at `FRICTION` × the normal impulse: pressing harder allows more grip. If the cap isn't reached, the slide stops completely (static friction); if it is, the box keeps sliding, slower (kinetic friction).

Boxes against statics: the conveyor

src/physics/AabbWorld.ts · lines 374–412

```
/** Box vs every static collider. Statics never move, so the box takes the whole correction. */
private solveStatics(i: number, first: boolean): void {
  const { px, py, pz, vx, vy, vz } = this;
  for (const s of this.statics) {
    const dx = px[i] - s.cx, dy = py[i] - s.cy, dz = pz[i] - s.cz;
    const ox = this.hx[i] + s.hx - Math.abs(dx);
    if (ox <= 0) continue;
```

```
    const oy = this.hy[i] + s.hy - Math.abs(dy);
    if (oy <= 0) continue;
    const oz = this.hz[i] + s.hz - Math.abs(dz);
    if (oz <= 0) continue;
    if (first) this.stats.contacts++;
    if (s.conveyor) this.onBelt[i] = 1;

    let nx = 0, ny = 0, nz = 0, depth: number;
    if (ox <= oy && ox <= oz) { depth = ox; nx = dx < 0 ? -1 : 1; }
    else if (oy <= oz) { depth = oy; ny = dy < 0 ? -1 : 1; }
    else { depth = oz; nz = dz < 0 ? -1 : 1; }

    const push = Math.max(depth - SLOP, 0) * CORRECTION;
    px[i] += nx * push; py[i] += ny * push; pz[i] += nz * push;

    // Velocity relative to the surface. The belt's surface moves, so friction drags boxes
along.
    const rvx = vx[i] - (s.conveyor ? this.speed : 0), rvy = vy[i], rvz = vz[i];
    const vn = rvx * nx + rvy * ny + rvz * nz;
    if (vn >= 0) continue;

    const e = vn < -BOUNCE_THRESHOLD ? RESTITUTION : 0;
    const dvn = -(1 + e) * vn;
    vx[i] += nx * dvn; vy[i] += ny * dvn; vz[i] += nz * dvn;

    let tx = rvx - vn * nx, ty = rvy - vn * ny, tz = rvz - vn * nz;
    const slide = Math.sqrt(tx * tx + ty * ty + tz * tz);
    if (slide < 1e-9) continue;
    const dvt = Math.min(slide, FRICTION * dvn);
    tx /= slide; ty /= slide; tz /= slide;
    vx[i] -= tx * dvt; vy[i] -= ty * dvt; vz[i] -= tz * dvt;
  }
}
```

The same steps, simplified: a static never moves, so the box takes the whole correction and impulses become plain velocity changes.

The one difference is line 397. Relative velocity is measured against the **surface**, and the belt's surface moves at `speed`. A box sitting still on a running belt is sliding backwards relative to it, so friction pushes it forward until it matches the belt. The belt collider itself never moves: there's no special conveyor code anywhere else, and jams and slipping just happen.

4. Sleeping

src/physics/AabbWorld.ts · lines 414–437

```
// ---- 4. Sleeping ----

private updateSleep(dt: number, pool: SlotPool): void {
  const { vx, vy, vz, asleep, sleepTime } = this;
  const { dense, count } = pool;
  let awake = 0;
  for (let k = 0; k < count; k++) {
    const i = dense[k];
    if (asleep[i]) continue;
    const beltHolds = this.onBelt[i] === 1 && this.speed !== 0; // jammed on a running belt:
stay awake
    if (!beltHolds && vx[i] * vx[i] + vy[i] * vy[i] + vz[i] * vz[i] < SLEEP_SPEED_SQ) {
      sleepTime[i] += dt;
      if (sleepTime[i] >= SLEEP_TIME) {
        asleep[i] = 1;
        vx[i] = vy[i] = vz[i] = 0;
        continue;
      }
    } else {
      sleepTime[i] = 0;
    }
    awake++;
  }
  this.stats.awake = awake;
}
```

A box that stays slower than 6 cm/s for 0.4 s goes to sleep: its velocity is zeroed and it's skipped by integration, solving and (for sleeper–sleeper pairs) the broad phase. A settled pile then costs almost nothing.

The exception is a box touching a **running** belt. It might be jammed and motionless, but the belt is still pulling on it, so it has to stay simulated.

Waking whole piles

src/physics/AabbWorld.ts · lines 439–460

```
/**
 * Wakes a body and, transitively, every sleeping body touching it: the whole
 * resting pile it belongs to (an "island"). Otherwise a box could stay asleep
 * floating in mid-air after the box under it was knocked away.
 */
private wake(start: number): void {
  if (this.asleep[start] !== 1) return;
  this.asleep[start] = 2;
  this.wakeStack[0] = start;
  this.drainWake(1);
}

/** Wakes everything on the stack, pushing each one's sleeping neighbors in turn (depth-first
flood fill). */
private drainWake(top: number): void {
  const stack = this.wakeStack;
  while (top > 0) {
    const i = stack[--top];
    this.asleep[i] = 0;
    this.sleepTime[i] = 0;
    top = this.pushSleepingNeighbors(i, top);
  }
}
```

Waking one box isn't enough. If a box is knocked out of a pile, the sleeping boxes on top of it would hang in mid-air, because sleeping boxes aren't simulated. So waking spreads to every sleeping box touching the woken one, and then to those touching them, until the whole connected pile (an **island**) is awake. Engines like Box2D and PhysX do the same.

It's a depth-first flood fill with an explicit stack array instead of recursion, so a 3,000-box pile can't overflow the call stack. While queued, a box is marked `2`, so it can't be pushed twice.

Finding sleeping neighbors

src/physics/AabbWorld.ts · lines 466–498

```
private pushSleepingNeighbors(i: number, top: number): number {
  const stack = this.wakeStack;
  const near = (j: number) =>
    j !== i &&
```

```
    this.alive[j] === 1 &&
    this.asleep[j] === 1 &&
    Math.abs(this.px[j] - this.px[i]) < this.hx[i] + this.hx[j] + WAKE_MARGIN &&
    Math.abs(this.py[j] - this.py[i]) < this.hy[i] + this.hy[j] + WAKE_MARGIN &&
    Math.abs(this.pz[j] - this.pz[i]) < this.hz[i] + this.hz[j] + WAKE_MARGIN;
  const mark = (j: number) => {
    this.asleep[j] = 2;
    stack[top++] = j;
  };

  if (this.hashBuilt) {
    // Sleeping bodies haven't moved since the hash was built, so their cells are still right.
    const h = this.hash;
    const x = h.cell(this.px[i]);
    const y = h.cell(this.py[i]);
    const z = h.cell(this.pz[i]);
    for (let dx = -1; dx <= 1; dx++)
      for (let dy = -1; dy <= 1; dy++)
        for (let dz = -1; dz <= 1; dz++) {
          for (let j = h.head[h.bucket(x + dx, y + dy, z + dz)]; j !== -1; j = h.next[j]) {
            if (h.cx[j] === x + dx && h.cy[j] === y + dy && h.cz[j] === z + dz && near(j))
mark(j);
          }
        }
  } else if (this.lastPool) {
    const { dense, count } = this.lastPool;
    for (let k = 0; k < count; k++) if (near(dense[k])) mark(dense[k]);
  }
  return top;
}
```

Uses the spatial hash from the last step to look in the 27 cells around the box. That's still accurate, because sleeping boxes haven't moved since the grid was built. Without the hash (the all-pairs mode), it scans every live box instead.

The two small arrow functions (`near`, `mark`) keep the loops readable. They're created per call, which is fine here: waking happens a few times a second, not millions of times a step.

GOING DEEPER

What a full engine adds

- **Rotation**: orientation, angular velocity, inertia tensors, and impulses applied at contact points so they create torque.
- **Oriented collision tests**: separating-axis tests (SAT) for boxes, GJK/EPA for general convex shapes, and contact **manifolds** of up to four points per pair, so a box can rest flat instead of rocking on one corner.
- **Warm starting** and caching of contacts between steps.
- **Continuous collision detection** for fast, thin objects, instead of capping speed and thickening the floor.
- **Joints**: hinges, sliders, motors, all as more constraints in the same solver.

PART 3.7 · `src/physics/CannonWorld.ts`

The cannon-es adapter

The same interface, implemented with a real rigid-body engine: rotation, tumbling, and constraint-based contacts.

Setup

src/physics/CannonWorld.ts · lines 29–38

```
constructor(capacity: number, statics: readonly StaticCollider[] = STATICS) {
  this.world = new CANNON.World({ gravity: new CANNON.Vec3(0, GRAVITY, 0), allowSleep: true });
  this.world.broadphase = new CANNON.SAPBroadphase(this.world);
  (this.world.solver as CANNON.GSSolver).iterations = 8;
  this.world.defaultContactMaterial.friction = 0.5;
  this.world.defaultContactMaterial.restitution = 0.1;

  this.bodies = new Array<CANNON.Body | null>(capacity).fill(null);
  this.prev = new Float64Array(capacity * 7);
```

A cannon-es `World` with gravity and sleeping on. **Sweep-and-prune** (SAP) is cannon's broad phase: it sorts bodies along an axis and only compares neighbours, a different route to the same O(n)-ish goal as the spatial hash. Friction and bounciness match the custom engine so the comparison is fair.

Statics and the kinematic belt

src/physics/CannonWorld.ts · lines 39–59

```
  let belt: CANNON.Body | null = null;
  for (const s of statics) {
    const body = new CANNON.Body({
      mass: 0,
      // The belt is kinematic: it has a velocity, so friction carries boxes along,
      // but we put it back in place every step so it never actually moves.
      type: s.conveyor ? CANNON.Body.KINEMATIC : CANNON.Body.STATIC,
      shape: new CANNON.Box(new CANNON.Vec3((s.max.x - s.min.x) / 2, (s.max.y - s.min.y) / 2,
(s.max.z - s.min.z) / 2)),
      position: new CANNON.Vec3((s.min.x + s.max.x) / 2, (s.min.y + s.max.y) / 2, (s.min.z +
s.max.z) / 2),
    });
    if (s.conveyor) {
      body.allowSleep = false;
      belt = body;
    }
    this.world.addBody(body);
  }
  if (!belt) throw new Error("The layout has no conveyor belt");
  this.belt = belt;
  this.beltHome = belt.position.clone();
  this.beltSpeed = 0.8;
}
```

Each static collider becomes a mass-zero cannon body with a box shape (cannon also takes half extents). The belt is **kinematic**: it has a velocity but isn't pushed around by anything. cannon's friction uses that velocity, so boxes on it are carried along, which is the same surface-velocity idea as the custom engine. The step function puts it back in place each step, so it never actually travels.

Adding boxes

src/physics/CannonWorld.ts · lines 70–89

```
add(slot: number, d: BodyDesc): void {
  const key = `${d.hx}|${d.hy}|${d.hz}`;
  let shape = this.shapes.get(key);
  if (!shape) {
    shape = new CANNON.Box(new CANNON.Vec3(d.hx, d.hy, d.hz));
    this.shapes.set(key, shape);
  }
  const body = new CANNON.Body({
    mass: 8 * d.hx * d.hy * d.hz * DENSITY,
    shape,
    position: new CANNON.Vec3(d.x, d.y, d.z),
    sleepSpeedLimit: 0.06,
    sleepTimeLimit: 0.4,
    linearDamping: 0.01,
    angularDamping: 0.05,
  });
  this.world.addBody(body);
  this.bodies[slot] = body;
  this.writePrev(slot, body);
}
```

Shapes are cached by size in a `Map`, so all boxes of one size share a `CANNON.Box`. Bodies are still created per spawn, because cannon owns its body objects. That allocation is part of what the comparison shows.

Removing, and waking

src/physics/CannonWorld.ts · lines 91–99

```
remove(slot: number): void {
  const body = this.bodies[slot];
  if (!body) return;
  this.world.removeBody(body);
  this.bodies[slot] = null;
  // cannon-es doesn't wake bodies resting on a removed one, so do it ourselves,
  // spreading through anything touching (the same island idea as the custom engine).
  this.wakeAround(body);
}
```

src/physics/CannonWorld.ts · lines 168–185

```
/** Wakes sleeping bodies whose bounding spheres touch `start`'s, and so on outward. */
private wakeAround(start: CANNON.Body): void {
  const pool = this.lastPool;
  if (!pool) return;
  const queue: CANNON.Body[] = [start];
  while (queue.length > 0) {
    const a = queue.pop()!;
    for (let k = 0; k < pool.count; k++) {
      const b = this.bodies[pool.dense[k]];
      if (!b || b === a || b.sleepState !== CANNON.Body.SLEEPING) continue;
      const reach = a.boundingRadius + b.boundingRadius + WAKE_MARGIN;
      if (a.position.distanceSquared(b.position) < reach * reach) {
        b.wakeUp();
        queue.push(b);
      }
    }
  }
}
```

cannon-es doesn't wake bodies that were resting on a removed one, so the adapter does it itself, with the same island idea: wake sleeping bodies whose bounding spheres touch, then theirs, and so on. Bodies can be rotated here, so it uses bounding spheres (`boundingRadius`) rather than axis-aligned boxes.

Stepping and interpolation

src/physics/CannonWorld.ts · lines 109–122

```
step(dt: number, pool: SlotPool): void {
  this.lastPool = pool;
  this.belt.position.copy(this.beltHome);
  this.world.step(dt); // one fixed step; our FixedStepper does the accumulating

  this.stats.contacts = this.world.contacts.length;
  let awake = 0;
  const { dense, count } = pool;
  for (let k = 0; k < count; k++) {
    const body = this.bodies[dense[k]];
    if (body && body.sleepState !== CANNON.Body.SLEEPING) awake++;
  }
  this.stats.awake = awake;
}
```

src/physics/CannonWorld.ts · lines 131–151

```
readTransform(slot: number, alpha: number, pos: V3, quat: Quat): void {
  const body = this.bodies[slot]!;
  const o = slot * 7;
  const prev = this.prev;
  pos.x = prev[o] + (body.position.x - prev[o]) * alpha;
  pos.y = prev[o + 1] + (body.position.y - prev[o + 1]) * alpha;
  pos.z = prev[o + 2] + (body.position.z - prev[o + 2]) * alpha;

  // Normalized lerp between the two rotations: close enough to slerp over one 1/120 s step.
  const q = body.quaternion;
  const sign = prev[o + 3] * q.x + prev[o + 4] * q.y + prev[o + 5] * q.z + prev[o + 6] * q.w <
0 ? -1 : 1;
  const x = prev[o + 3] + (sign * q.x - prev[o + 3]) * alpha;
  const y = prev[o + 4] + (sign * q.y - prev[o + 4]) * alpha;
  const z = prev[o + 5] + (sign * q.z - prev[o + 5]) * alpha;
  const w = prev[o + 6] + (sign * q.w - prev[o + 6]) * alpha;
  const len = Math.hypot(x, y, z, w) || 1;
  quat.x = x / len;
  quat.y = y / len;
  quat.z = z / len;
  quat.w = w / len;
}
```

`world.step(dt)` with a single argument runs exactly one fixed step; the shared `FixedStepper` does the accumulating, exactly as for the custom engine.

Positions interpolate as before. Rotations use a **normalized lerp** of the two quaternions: blend the four numbers, then rescale to length 1. If the dot product is negative, one quaternion is flipped first. `q` and `−q` are the same rotation, and without the flip the blend would spin the long way round.

GOING DEEPER

Reading the comparison fairly

cannon-es does more work per body than the custom engine: rotation, oriented contacts, an object per body, and a general solver. A side-by-side comparison measures the **cost of generality** in this specific scene. It isn't evidence that one engine is “better”: change the scene to tumbling objects and only one of them can represent it at all.

PART 3.8 · `src/sim/BoxFlow.ts`

Spawning and recycling

Decides which boxes exist. Knows nothing about rendering, and works with either engine.

Per-frame update

src/sim/BoxFlow.ts · lines 36–54

```
/** Call once per frame. */
update(dt: number): void {
  this.clock += dt;
  while (this.pool.count > this.target) this.retireOldest();

  if (this.target > 0) {
    // Accumulate fractional boxes so e.g. 6/s spawns evenly whatever the frame rate.
    this.spawnDebt += this.rate * dt;
    while (this.spawnDebt >= 1) {
      this.spawnDebt -= 1;
      if (this.pool.count >= this.target) this.retireOldest(); // recycle: oldest out, new one
in
      this.spawn(SPAWN.x + (this.random() - 0.5) * 0.04, SPAWN.y, (this.random() * 2 - 1) *
SPAWN.zJitter);
    }
  } else {
    this.spawnDebt = 0;
  }

  this.retireOutOfBounds();
}
```

If the target was lowered, the oldest boxes are removed until it's met. Then spawning uses a **debt** counter: at 6 boxes/s and 60 fps, each frame adds 0.1, and a box appears every time it passes 1. That keeps the rate exact at any frame rate.

At the target, each new box first retires the oldest one, so the scene keeps flowing instead of stopping when full.

The stress-test burst

src/sim/BoxFlow.ts · lines 56–73

```
/** Drops `n` boxes at once in layers above the bin and the floor around it. Returns the new
target. */
burst(n: number): number {
  const a = BURST_AREA;
  const cols = Math.floor((a.maxX - a.minX) / a.spacing) + 1;
  const rows = Math.floor((a.maxZ - a.minZ) / a.spacing) + 1;
  const room = Math.min(n, this.pool.capacity - this.pool.count);
  for (let k = 0; k < room; k++) {
    const layer = Math.floor(k / (cols * rows));
    const inLayer = k % (cols * rows);
    this.spawn(
      a.minX + (inLayer % cols) * a.spacing,
      a.baseY + layer * a.spacing,
      a.minZ + Math.floor(inLayer / cols) * a.spacing,
    );
  }
  this.target = Math.max(this.target, this.pool.count);
  return this.target;
}
```

Lays `n` boxes out in a grid: `cols × rows` per layer, then more layers above. `%` and `Math.floor(k / …)` turn a running count `k` into a column, row and layer. It raises the target too, so the burst isn't immediately recycled away.

Switching engines mid-run

src/sim/BoxFlow.ts · lines 80–91

```
/** Moves every live box into another engine, at its current position, so engines can be
switched mid-run. */
switchBackend(next: PhysicsBackend): void {
  const { dense, count } = this.pool;
  next.beltSpeed = this.backend.beltSpeed;
  for (let k = 0; k < count; k++) {
    const slot = dense[k];
    this.backend.getPosition(slot, this.tmp);
    const h = SIZE_PRESETS[this.preset[slot]];
    next.add(slot, { x: this.tmp.x, y: this.tmp.y, z: this.tmp.z, hx: h.x, hy: h.y, hz: h.z });
  }
  this.backend = next;
}
```

Copies every live box into the new engine at its current position, keeping its slot number, so the renderer and the pool don't notice. Velocities and rotations aren't carried over, which is fine for a demo.

Spawning and retiring

src/sim/BoxFlow.ts · lines 93–106

```
private spawn(x: number, y: number, z: number): void {
  const slot = this.pool.alloc();
  if (slot < 0) return; // pool full
  const p = Math.floor(this.random() * SIZE_PRESETS.length);
  const h = SIZE_PRESETS[p];
  this.preset[slot] = p;
  this.spawnedAt[slot] = this.clock;
  this.backend.add(slot, { x, y, z, hx: h.x, hy: h.y, hz: h.z });
}

private retire(slot: number): void {
  this.backend.remove(slot);
  this.pool.release(slot);
}
```

src/sim/BoxFlow.ts · lines 108–126

```
/** O(n) scan; at most a few dozen times a second, so simpler than keeping a queue. */
private retireOldest(): void {
  const { dense, count } = this.pool;
  if (count === 0) return;
  let oldest = dense[0];
  for (let k = 1; k < count; k++) if (this.spawnedAt[dense[k]] < this.spawnedAt[oldest]) oldest
= dense[k];
  this.retire(oldest);
}

private retireOutOfBounds(): void {
  const { dense } = this.pool;
  // Walk backwards: release() swaps the last live slot into the hole, and we've already
checked that one.
  for (let k = this.pool.count - 1; k >= 0; k--) {
    const slot = dense[k];
    this.backend.getPosition(slot, this.tmp);
    const p = this.tmp;
    if (p.y < BOUNDS.minY || Math.abs(p.x) > BOUNDS.maxAbsXZ || Math.abs(p.z) >
BOUNDS.maxAbsXZ) this.retire(slot);
  }
}
```

`retireOldest` is a simple O(n) scan for the smallest spawn time. It runs a few dozen times a second at most, so the simplicity wins over maintaining a queue. Out-of-bounds removal walks the list backwards, because swap-remove moves the last element into the hole: walking backwards, that element has already been checked.

Repeatable randomness

src/sim/BoxFlow.ts · lines 128–132

```
/** Small seeded generator, so runs (and benchmarks) are repeatable. */
private random(): number {
  this.seed = (Math.imul(this.seed, 1664525) + 1013904223) >>> 0;
  return this.seed / 2 ** 32;
}
```

A tiny **linear congruential generator** instead of `Math.random()`, seeded with a fixed number, so every run and every benchmark spawns the same sequence of boxes. `>>> 0` keeps the result an unsigned 32-bit integer; dividing by 2^{32} maps it into [0, 1).

PART 3.9 · `src/render/BoxRenderer.ts`

Drawing thousands of boxes

The rendering half of the performance story: one draw call for every box, instead of one each.

Two ways to draw

src/render/BoxRenderer.ts · lines 10–39

```
/**
 * Draws every box, two ways:
 *
 * - "instanced": one InstancedMesh. All boxes share one geometry and one material,
 *   and each box is just a matrix and a color in a GPU buffer, so 3,000 boxes are a
 *   single draw call (plus one for the shadow map).
 * - "meshes": one THREE.Mesh per box, the straightforward way. Same picture, but
 *   one draw call per box per pass. Here for comparison.
 *
 * Both reuse their objects frame to frame; nothing is allocated per frame.
 */
export class BoxRenderer {
  readonly group = new THREE.Group();
  mode: RenderMode = "instanced";
  tintSleeping = false;

  private readonly geometry = new THREE.BoxGeometry(1, 1, 1);
  private readonly instanced: THREE.InstancedMesh;
  private readonly colors = PALETTE.map((c) => new THREE.Color(c));
  private readonly sleepingColor = new THREE.Color(SLEEPING);
  private readonly materials = PALETTE.map((c) => new THREE.MeshStandardMaterial({ color: c,
roughness: 0.7 }));
  private readonly sleepingMaterial = new THREE.MeshStandardMaterial({ color: SLEEPING,
roughness: 0.7 });
  private readonly meshes: THREE.Mesh[] = [];
  private meshesShown = 0;

  // Scratch objects, reused every frame.
  private readonly matrix = new THREE.Matrix4();
  private readonly pos = new THREE.Vector3();
  private readonly quat = new THREE.Quaternion();
  private readonly scale = new THREE.Vector3();
```

A **draw call** is one request from the CPU to the GPU: “draw this geometry with this material”. Each has a fixed overhead, so thousands of tiny ones are slow even when each draws only 12 triangles.

An `InstancedMesh` draws one geometry many times in a single call, reading each copy's matrix (position, rotation, scale) and color from a buffer on the GPU. The scratch `matrix`, `pos`, `quat` and `scale` objects are reused for every box, every frame.

Setting up the instanced mesh

src/render/BoxRenderer.ts · lines 41–57

```
constructor(capacity: number) {
  // White base color: the per-instance color multiplies it.
  this.instanced = new THREE.InstancedMesh(
    this.geometry,
    new THREE.MeshStandardMaterial({ color: 0xffffff, roughness: 0.7 }),
    capacity,
  );
  this.instanced.instanceMatrix.setUsage(THREE.DynamicDrawUsage);
  this.instanced.setColorAt(0, this.colors[0]); // creates the instanceColor buffer
  this.instanced.instanceColor!.setUsage(THREE.DynamicDrawUsage);
  this.instanced.count = 0;
  this.instanced.castShadow = true;
  this.instanced.receiveShadow = true;
  // Instances move every frame, so a bounding sphere computed once would soon be wrong.
  this.instanced.frustumCulled = false;
  this.group.add(this.instanced);
}
```

One 1 m cube, scaled per box. The material is white because each instance's color multiplies it. `DynamicDrawUsage` tells the GPU driver these buffers change every frame. Calling `setColorAt` once creates the color buffer.

Frustum culling normally skips objects outside the camera's view, using a bounding sphere. For an instanced mesh that sphere is computed once and goes stale as boxes move, so culling is turned off rather than risk boxes vanishing.

The instanced path

src/render/BoxRenderer.ts · lines 59–77

```
update(flow: BoxFlow, alpha: number): void {
  const { dense, count } = flow.pool;
  const backend = flow.backend;

  if (this.mode === "instanced") {
    this.hideMeshes(0);
    this.instanced.visible = true;
    for (let k = 0; k < count; k++) {
      const slot = dense[k];
      this.composeMatrix(flow, slot, alpha);
      this.instanced.setMatrixAt(k, this.matrix);
      const sleeping = this.tintSleeping && backend.isAsleep(slot);
      this.instanced.setColorAt(k, sleeping ? this.sleepingColor :
this.colors[flow.preset[slot]]);
    }
    this.instanced.count = count;
    this.instanced.instanceMatrix.needsUpdate = true;
    this.instanced.instanceColor!.needsUpdate = true;
    return;
  }
```

For each live box: get its interpolated transform from whichever engine is running, build its matrix, and write the matrix and color into position `k`. Setting `count` tells Three.js to draw only the first `count` instances; `needsUpdate` tells it to upload the changed buffers. Coloring every box every frame is cheap, and it means swap-removal reordering never shows.

The one-mesh-per-box path

src/render/BoxRenderer.ts · lines 79–91

```
  this.instanced.visible = false;
  for (let k = 0; k < count; k++) {
    const slot = dense[k];
    const mesh = this.meshes[k] ?? this.createMesh();
    const sleeping = this.tintSleeping && backend.isAsleep(slot);
    mesh.material = sleeping ? this.sleepingMaterial : this.materials[flow.preset[slot]];
    backend.readTransform(slot, alpha, mesh.position, mesh.quaternion);
    const h = SIZE_PRESETS[flow.preset[slot]];
    mesh.scale.set(2 * h.x, 2 * h.y, 2 * h.z);
    mesh.visible = true;
  }
  this.hideMeshes(count);
}
```

src/render/BoxRenderer.ts · lines 100–113

```
private createMesh(): THREE.Mesh {
  const mesh = new THREE.Mesh(this.geometry, this.materials[0]);
  mesh.castShadow = true;
  mesh.receiveShadow = true;
  this.meshes.push(mesh);
  this.group.add(mesh);
  return mesh;
}

/** Hides pooled meshes from index `from` up to however many were shown last frame. */
private hideMeshes(from: number): void {
  for (let k = from; k < this.meshesShown; k++) this.meshes[k].visible = false;
  this.meshesShown = from;
}
```

The straightforward way, kept for comparison. Meshes are pooled too: created only when more are needed than ever before, and hidden rather than deleted when not. `readTransform` writes straight into each mesh's `position` and `quaternion`, since they're compatible with the engine's plain objects.

Unticking **Instancing** with ~500 boxes visible took the scene from 27 draw calls to 484.

GOING DEEPER

What the GPU actually receives

`instanceMatrix` is a buffer attribute with a divisor of 1: the vertex shader reads a new matrix per instance instead of per vertex. The whole pile is one `drawElementsInstanced` call. What remains per frame is uploading the changed buffers: at the 4,096-box capacity that's 256 KB of matrices if every one changed. Beyond that, the next steps are writing matrices straight into the array rather than composing them, and eventually moving the simulation to the GPU with compute shaders.

PART 3.10 · `src/render/Workcell.ts`

The conveyor and bin

Visuals built from the same collider list the physics uses, plus a scrolling texture to show the belt moving.

Drawing the colliders

src/render/Workcell.ts · lines 11–31

```
export function buildWorkcell(scene: THREE.Scene) {
  const steel = new THREE.MeshStandardMaterial({ color: 0x9aa3ae, metalness: 0.7, roughness:
0.35 });
  const dark = new THREE.MeshStandardMaterial({ color: 0x2a2e35, metalness: 0.4, roughness: 0.6
});
  const binMat = new THREE.MeshStandardMaterial({ color: 0x3b6e8f, metalness: 0.3, roughness:
0.6 });

  const beltTexture = makeBeltTexture();
  beltTexture.repeat.set((BELT.maxX - BELT.minX) / STRIPE_PERIOD, 1);
  const beltTop = new THREE.MeshStandardMaterial({ map: beltTexture, roughness: 0.85 });

  for (const s of STATICS) {
    if (s.visible === false) continue;
    const size = new THREE.Vector3(s.max.x - s.min.x, s.max.y - s.min.y, s.max.z - s.min.z);
    // BoxGeometry face order is +x, -x, +y, -y, +z, -z: only the belt's top face gets the
texture.
    const material = s.conveyor ? [dark, dark, beltTop, dark, dark, dark] :
s.name.startsWith("bin") ? binMat : steel;
    const mesh = new THREE.Mesh(new THREE.BoxGeometry(size.x, size.y, size.z), material);
    mesh.position.set((s.min.x + s.max.x) / 2, (s.min.y + s.max.y) / 2, (s.min.z + s.max.z) /
2);
    mesh.castShadow = true;
    mesh.receiveShadow = true;
    mesh.name = s.name;
    scene.add(mesh);
  }
```

Every visible static becomes a box mesh of exactly its size and position, so the picture can't drift from the physics. A box mesh can take an array of six materials, one per face (+x, −x, +y, −y, +z, −z), so only the belt's top face gets the striped texture.

A belt that looks like it moves

src/render/Workcell.ts · lines 73–78

```
return {
  update(dt: number, beltSpeed: number) {
    // Scrolling the texture makes the belt look like it moves; the physics belt never moves.
    beltTexture.offset.x = (beltTexture.offset.x - (beltSpeed * dt) / STRIPE_PERIOD) % 1;
  },
};
```

src/render/Workcell.ts · lines 83–101

```
  const canvas = document.createElement("canvas");
  canvas.width = 64;
  canvas.height = 128;
  const g = canvas.getContext("2d")!;
  g.fillStyle = "#23272e";
  g.fillRect(0, 0, 64, 128);
  g.strokeStyle = "#3a414d";
  g.lineWidth = 10;
  g.beginPath();
  g.moveTo(10, 0);
  g.lineTo(44, 64);
  g.lineTo(10, 128);
  g.stroke();
  const texture = new THREE.CanvasTexture(canvas);
  texture.wrapS = texture.wrapT = THREE.RepeatWrapping;
  texture.colorSpace = THREE.SRGBColorSpace;
  texture.anisotropy = 4;
  return texture;
}
```

The physics belt never moves, so neither does its mesh. Instead the texture slides: shifting `offset.x` by speed × time ÷ stripe spacing each frame moves the chevrons at exactly the belt speed. `% 1` keeps the number small, since the texture repeats.

The chevron is drawn on an off-screen `<canvas>` and used as a texture, so there's no image file to load. `RepeatWrapping` tiles it along the belt.

PART 3.11 · `src/main.ts`

The frame loop

Wires everything together and measures it.

Setup

src/main.ts · lines 14–24

```
const canvas = document.getElementById("scene") as HTMLCanvasElement;
const { renderer, scene, camera, controls } = createStage(canvas);
const workcell = buildWorkcell(scene);

const boxes = new BoxRenderer(MAX_BODIES);
scene.add(boxes.group);

const flow = new BoxFlow(new AabbWorld(MAX_BODIES));
const stepper = new FixedStepper(FIXED_STEP, MAX_SUBSTEPS);
let spatialHash = true;
let switching = false;
```

Scene, conveyor visuals, box renderer, the flow (starting on the custom engine), and the fixed stepper. `MAX_BODIES` is passed everywhere so all the arrays match.

Controls

src/main.ts · lines 26–44

```
const hud = createHud({
  onEngine: (kind) => void setEngine(kind),
  onSpatialHash: (on) => {
    spatialHash = on;
    if (flow.backend instanceof AabbWorld) flow.backend.useSpatialHash = on;
  },
  onInstancing: (on) => (boxes.mode = on ? "instanced" : "meshes"),
  onTintSleeping: (on) => (boxes.tintSleeping = on),
  onTarget: (n) => (flow.target = n),
  onRate: (r) => (flow.rate = r),
  onBeltSpeed: (v) => (flow.backend.beltSpeed = v),
  onBurst: () => hud.setTarget(flow.burst(BURST_SIZE)),
  onClear: () => {
    flow.clear();
    flow.target = 0;
    hud.setTarget(0);
  },
});
hud.setEngine("custom");
```

Each panel control maps to one assignment. `void setEngine(kind)` calls an async function without waiting for it; `void` says that's deliberate. **Drop 500** returns the new target so the slider can show it.

Switching engine, with cannon-es on demand

src/main.ts · lines 46–71

```
/**
 * Swaps the physics engine mid-run, carrying every box over at its current position.
 * cannon-es is loaded on demand (a separate chunk), so the page starts without it.
 */
async function setEngine(kind: EngineKind) {
  if (switching) return;
  switching = true;
  try {
    let next: PhysicsBackend;
    if (kind === "cannon") {
      const { CannonWorld } = await import("./physics/CannonWorld");
      next = new CannonWorld(MAX_BODIES);
```

```
    } else {
      const custom = new AabbWorld(MAX_BODIES);
      custom.useSpatialHash = spatialHash;
      next = custom;
    }
    flow.switchBackend(next);
    hud.setEngine(kind);
  } catch (err) {
    console.error(err);
    hud.showMessage("Couldn't load cannon-es. Run npm install, then reload.");
  } finally {
    switching = false;
  }
}
```

`await import("./physics/CannonWorld")` is a **dynamic import**: Vite puts cannon-es in a separate file that's only downloaded the first time someone picks it, so the page starts faster. `switching` stops a double click from starting two switches. If loading fails, the panel says so instead of breaking.

Each frame

src/main.ts · lines 78–96

```
renderer.setAnimationLoop((now) => {
  const frameMs = now - last;
  last = now;
  const dt = Math.min(frameMs / 1000, 0.25); // after a hidden tab, don't try to simulate
minutes

  flow.update(dt);

  // Physics: fixed 1/120 s steps, timed so the panel can show what physics costs per frame.
  const t0 = performance.now();
  const steps = stepper.advance(dt, () => {
    flow.backend.savePrevious(flow.pool);
    flow.backend.step(FIXED_STEP, flow.pool);
  });
  const physicsMs = performance.now() - t0;

  workcell.update(dt, flow.backend.beltSpeed);
  boxes.update(flow, stepper.alpha); // draw between the last two physics states
  controls.update();
  renderer.render(scene, camera);
```

In order: work out the frame time (capped at 0.25 s so a tab returning from the background doesn't try to simulate minutes), spawn or retire boxes, run the fixed physics steps (timed with `performance.now()`), scroll the belt, update the boxes using `alpha`, and draw. Each physics step first saves the previous state, so interpolation always has the last two.

Stats without overhead

src/main.ts · lines 98–118

```
hud.pushFrame(frameMs);
sample.time += dt;
sample.frames++;
sample.physics += physicsMs;
sample.steps += steps;
if (sample.time >= STATS_INTERVAL) {
  const s = flow.backend.stats;
  hud.showStats({
    fps: sample.frames / sample.time,
    frameMs: (sample.time * 1000) / sample.frames,
    physicsMs: sample.physics / sample.frames,
    stepsPerFrame: sample.steps / sample.frames,
    boxes: flow.count,
    awake: s.awake,
    drawCalls: renderer.info.render.calls,
    pairsTested: s.pairsTested,
    contacts: s.contacts,
    droppedMs: stepper.droppedTime * 1000,
  });
  sample = { time: 0, frames: 0, physics: 0, steps: 0 };
}
```

The graph gets every frame's time. The numbers are averaged over a quarter second and written only then, because updating text 144 times a second is wasted work nobody can read. Draw calls come from `renderer.info`, which Three.js fills in during `render()`.

PART 3.12 · `src/ui/hud.ts`

The frame-time graph

The panel is plain DOM; the one interesting piece is the graph.

A ring buffer, drawn every frame

src/ui/hud.ts · lines 87–116

```
// Frame-time graph: a ring buffer of the last 120 frames, drawn every frame.
const graph = byId<HTMLCanvasElement>("graph");
```

```
const g = graph.getContext("2d")!;
const history = new Float32Array(120);
let head = 0;

return {
  /** Every frame: feeds the graph. */
  pushFrame(ms: number) {
    history[head] = ms;
    head = (head + 1) % history.length;
    const { width, height } = graph;
    g.clearRect(0, 0, width, height);
    const y = (v: number) => height - (Math.min(v, GRAPH_MAX_MS) / GRAPH_MAX_MS) * height;
    g.strokeStyle = "rgba(255,255,255,0.15)"; // 60 fps budget line
    g.beginPath();
    g.moveTo(0, y(1000 / 60));
    g.lineTo(width, y(1000 / 60));
    g.stroke();
    g.strokeStyle = "#3b82f6";
    g.lineWidth = 1.5;
    g.beginPath();
    for (let k = 0; k < history.length; k++) {
      const x = (k / (history.length - 1)) * width;
      const v = history[(head + k) % history.length];
      if (k === 0) g.moveTo(x, y(v));
      else g.lineTo(x, y(v));
    }
    g.stroke();
  },
```

The last 120 frame times live in a fixed `Float32Array` used as a **ring buffer**: `head` is where the next value goes, wrapping around with `%`, so the oldest value is always overwritten and nothing is ever shifted or allocated.

Drawing starts at `head` (the oldest) and walks forward, so the line reads left to right in time. A faint line marks 16.7 ms, the budget for 60 fps: any spike above it was a dropped frame. Times are capped at 50 ms so one huge hitch doesn't flatten the rest of the graph.

CHAPTER 3

Interview questions

Q1 **Why a fixed timestep? What's the spiral of death?**

A fixed step keeps the solver stable and the simulation reproducible whatever the frame rate. Frames bank their time and spend it in whole steps; α interpolates for drawing. The spiral of death is when a slow frame leaves more steps to run, which makes the next frame slower still. Capping the steps per frame and dropping the excess breaks the loop: the simulation slows down instead of freezing.

Q2 **Explain broad phase versus narrow phase.**

The broad phase cheaply finds pairs that might touch, using bounding boxes and a spatial structure (a hash grid here, sweep and prune in cannon-es). The narrow phase does the exact test on those candidates and produces the contact: normal, depth and the data the solver needs. The split is what makes thousands of bodies affordable.

Q3 **Why does the solver iterate? What happens with too few iterations?**

Contacts are coupled: fixing one disturbs its neighbours, so a single pass leaves errors. Each sweep reduces them (Gauss–Seidel). Too few iterations and stacks feel soft: they sink, jitter, or never come to rest. More iterations cost linearly. Warm starting is the usual way to get the same stiffness with fewer passes.

Q4 **Tall stacks wouldn't settle. Why, and how did you fix it?**

Every box in the stack had the same downward velocity, so the relative velocity between neighbours was zero and the solver never applied an impulse that stopped the fall: only the bottom contact did any work. Sorting pairs bottom-up and, on the last pass, treating the lower box of each vertical contact as immovable (shock propagation) lets the floor's support travel to the top in one sweep.

Q5 **How does the conveyor move boxes when its collider never moves?**

Friction is computed from relative velocity at the contact. For the belt, the surface's velocity is the belt speed, so a box at rest is sliding backwards relative to it and friction accelerates it up to belt speed. No special conveyor code exists; jams and slipping fall out of the friction model.

Q6 **What's structure of arrays, and why is it faster in JavaScript?**

One typed array per property instead of one object per entity. Hot loops touch contiguous numbers, which caches and prefetchers love, typed arrays avoid boxing and hidden-class checks, and there are no per-entity objects for the garbage collector. It's the layout data-oriented engines and ECS frameworks use.

Q7 **What's a draw call, and what did instancing change?**

A draw call is one CPU-to-GPU request to draw some geometry with some state, each with fixed overhead. One mesh per box meant one call per box: 484 for ~500 boxes. One `InstancedMesh` draws all of them in a single call with per-instance matrices and colors: 27 calls for the whole scene.

Q8 **How do sleeping and islands work, and what went wrong?**

Bodies slower than a threshold for 0.4 s sleep and are skipped. Removing or hitting a body wakes everything transitively touching it (its island), using the spatial hash to find neighbours. Bugs found: boxes slept on a stopped belt and ignored it restarting (fixed by an on-belt flag and waking all bodies when the speed changes), and the wake stack overwrote itself while spreading (fixed by draining it properly).

Q9 **How did you benchmark fairly?**

Same seeded scene for every run, a JIT warm-up before measuring, a headless Node benchmark so rendering doesn't interfere, per-phase timing, and reporting falling and settled separately, since sleeping changes the cost by half.

CHAPTER 3

Exercises

3.1 Warm starting

Store each pair's normal impulse keyed by its two slots and apply most of it at the start of the next step. Measure how few iterations the ten-box tower test then needs.

Hint. Key pairs with `i * MAX_BODIES + j` in a Map. Clear entries for pairs that weren't found this step.

3.2 A tunnelling lab

Make the floor 1 cm thick and raise `MAX_SPEED`. Find the impact speed at which boxes start falling through, then fix it with a swept test against the floor.

Hint. A box moves v·dt per step. It tunnels when that exceeds the floor thickness plus its own height.

3.3 Sweep and prune

Implement SAP behind the same interface the spatial hash offers and compare pair counts and step time in `npm run bench`.

Hint. Keep the sorted order between steps and re-sort with insertion sort: nearly-sorted input makes it close to O(n).

3.4 A pusher

Add a moving static box that sweeps across the belt and pushes boxes off the side.

Hint. Treat it like the belt: a surface with a velocity. Its velocity is (position − previous position) ÷ dt.

3.5 Iterations versus settling

Plot (or print) the time the ten-box tower takes to fall asleep for ITERATIONS from 2 to 12, with and without shock propagation.

Hint. Make ITERATIONS a constructor option for the experiment; the tests can keep the default.

CHAPTER 4 · `vention/snack4-refactor-kata`

The Legacy Refactor Kata

A messy 180-line quote calculator made small, typed and pure in 13 commits, with characterization tests proving exactly what changed and what didn't.

TypeScript · node:test

```
> npm run demo

Quote Q-1762525200000

  40x40 extrusion 1200mm              4 ×     $35.10      $140.40
  40x80 extrusion 2400mm black        2 ×    $136.96      $273.92
  Plate PL-200                        4 ×     $32.00      $128.00
  Fasteners (pack of 50)              3 ×      $6.75       $20.25
  Linear actuator                     2 ×    $420.00      $840.00
  Motion controller                   1 ×    $650.00      $650.00
  Assembly (8 h)                      1 ×    $280.00      $280.00

  Subtotal                                              $2,332.57
  Discount                                               -$205.26
  Shipping                                                $43.01
  Tax                                                    $282.14
  Total                                                 $2,452.46

  Ships in 17 working days
```

npm run demo: a quote for a sample machine design, from the refactored code.

The one idea to take away

Never change behaviour and structure at the same time. Every refactoring step goes through the same loop:

`freeze legacy` → `characterize: 1,002 cases + 21 specs` → `test the tests` → `refactor a step` → `all green` → `commit`

Then, and only then, each bug is fixed on its own, test first:

`write the correct rule as a test` → `see it fail` → `fix` → `only explained cases change` → `commit`

CONCEPTS · 4.1

What makes code “legacy”

Michael Feathers' working definition is the useful one: **legacy code is code without tests**. Age doesn't matter; what matters is that you can't change it with confidence. That creates a loop: to change code safely you want tests, and to add tests you often have to change the code.

The quote calculator breaks the loop the cheapest way: it's already a callable function, so tests can be written **around it from the outside** before a single line changes. When code isn't callable like that (hard-wired databases, globals, UI), the first steps are small, mechanical changes to create **seams**: places where behaviour can be observed or substituted.

CONCEPTS · 4.2

Characterization tests and the golden master

A **characterization test** records what the code does now, not what it should do. A **golden master** does it in bulk: generate many inputs, run them through the frozen original, and require the new code to give identical outputs.

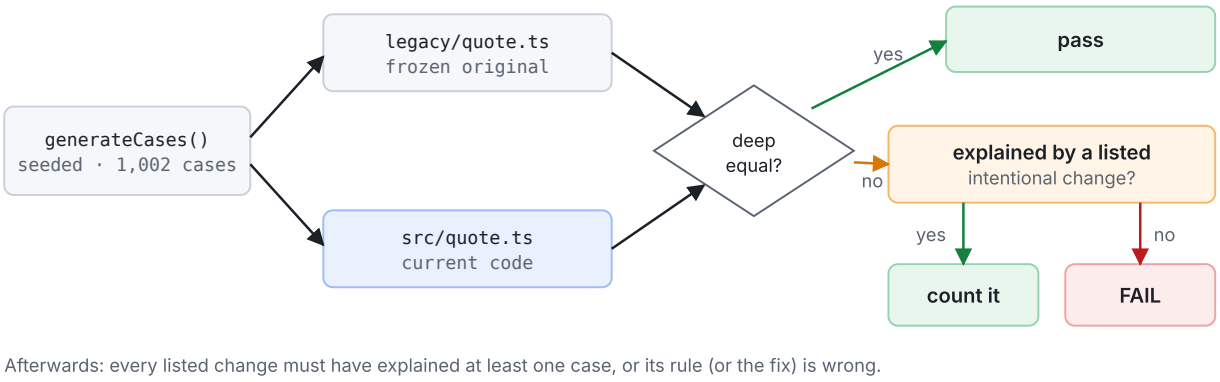


Figure 4.1. The golden master with intentional changes. Any difference must be explained, and every explanation must be used.

- **Inputs decide everything.** A seeded generator gives 1,002 varied, awkward designs that are identical on every run.
- **Check the inputs reach the code.** A coverage test asks for at least two cases through each branch worth protecting. It's how the unreachable rush lead-time check was found.
- **Test the tests.** Plant a mistake (a one-cent change to a fee) and confirm the net catches it. That's mutation testing by hand; tools like Stryker automate it.

- **Know the weakness.** A golden master locks in bugs along with everything else, and only covers what the inputs exercise. The `=== true` near-miss in commit 8 was exactly an input gap, closed in commit 9.

CONCEPTS · 4.3

Refactoring in small, green steps

Martin Fowler defines refactoring as changing the structure of code **without changing its behaviour**, in small steps. The steps used here have names in his catalog: replace magic literals with named constants, extract function, replace conditionals with table lookups, introduce types, split a calculation from its side effects.

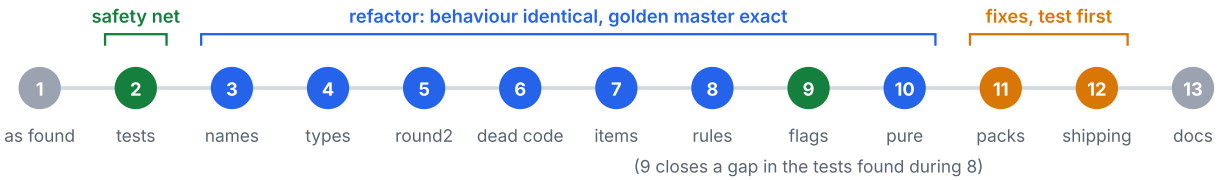


Figure 4.2. The 13 commits. Every one leaves all tests green; behaviour changes are confined to 11 and 12.

Small commits are a feature, not tidiness. Each can be reviewed in minutes, reverted alone, and found by `git bisect` if something slips through. Mixing a refactor and a fix in one commit makes the fix invisible in a sea of moved lines, and makes it impossible to revert one without the other.

CONCEPTS · 4.4

A pure core with side effects at the edge

The legacy function read the clock, wrote a global and logged. The refactored `createQuote(design, customer, now)` does none of that: same inputs, same output, always. Gary Bernhardt calls this "functional core, imperative shell": keep decisions in pure functions that are trivial to test, and push I/O to a thin outer layer.

The old entry point survives as a deprecated wrapper that adds the old side effects back. Existing callers keep working, new callers use the pure function, and the wrapper can be deleted once nobody calls it: the strangler-fig pattern at the scale of one function.

CONCEPTS · 4.5

Changing behaviour on purpose

A bug fix is a behaviour change, so it gets the opposite treatment from a refactor: write the **correct** rule as a test, watch it fail for the right reason, then fix. The golden master stays, but now with a written list of intentional changes. Each says which cases it may affect, any other difference fails, and each must explain at least one case, which proves the list is honest.

In production, the same idea scales up as **shadow mode** (GitHub's Scientist library is the well-known example): run old and new code side by side on real traffic, serve the old result, and log every mismatch until there are none you can't explain.

CONCEPTS · 4.6

Money and floating point

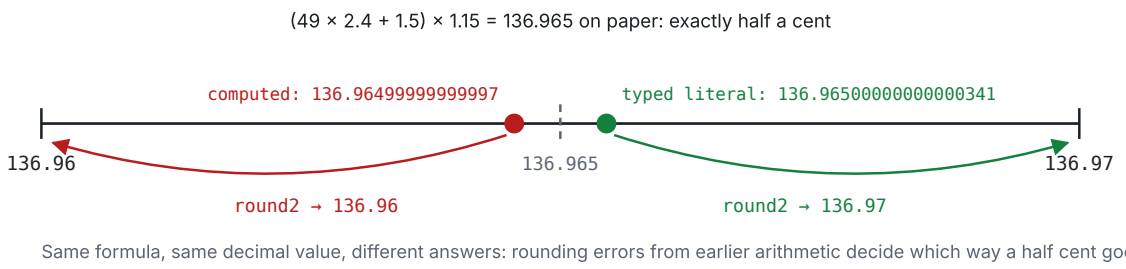


Figure 4.3. Why the demo shows 136.96. Binary floating point can't hold most decimal fractions exactly, so a computed half cent can land on either side.

JavaScript numbers are IEEE-754 doubles. They represent integers exactly up to 2^{53} but most decimal fractions only approximately, so a computed half cent can land either side of the line. The robust fixes are integer cents (or a decimal library), an explicit rounding mode (half-up, or banker's half-even), and an explicit rule for whether tax is rounded per line or on the total. All of those change invoices, which is why the refactor documented the issue instead of fixing it.

> Correction to the separate Snack 4 walkthrough: it said 136.965 itself is stored as 136.96499…. In fact the literal 136.965 is stored slightly **above** the half cent and `round2` gives 136.97. The demo's 136.96 comes from the arithmetic that produces it, as the figure shows. REFACTORING.md in the repo has the same wording.

CHAPTER 4

Line by line

11 parts, in the order data flows through the code.

PART 4.1 · `legacy/quote.ts`

The code as found

The frozen original, kept in the repo so the tests can compare against it. Reading it chunk by chunk, with the problems a reviewer would flag. None of them are fixed yet; first, the behaviour gets pinned down.

Header and globals

legacy/quote.ts · lines 5–11

```
// quote.ts - machine quote calculator
// TODO: clean this up (2019)
// DO NOT TOUCH THE ROUNDING - finance relies on it

export var lastQuote: any = null;

var PLATES: any = { "PL-100": 18.5, "PL-200": 32, "PL-300": 54.25, "BR-90": 7.8 };
```

- A five-year-old TODO and a warning not to touch the rounding: signs that people were afraid to change this file. That fear is what tests remove.
- `export var lastQuote`: **global mutable state**. Any caller can read (or overwrite) the last quote, so the function's result depends on more than its arguments, and two quotes in a row interfere.
- `any`: the type checker is switched off for everything this file touches.
- Prices hard-coded in the middle of a code file, with no name for what they are.

Signature and accumulators

legacy/quote.ts · lines 13–25

```
export function calculateQuote(design: any, customer: any) {
  var lines: any[] = [];
  var warnings: string[] = [];
  var subtotal = 0;
  var labor = 0;
  var weight = 0;
  var hours = 0;
  var hasMotion = false, hasController = false, hasActuator = false;
  var longest = 0;

  if (!design || !design.items || design.items.length == 0) {
    return { error: "Empty design" };
  }
```

Two `any` parameters, ten mutable `var`s declared up front, and three boolean flags that the loop sets and code far below reads. To follow any one number you have to keep all of them in your head. The error case returns a completely different shape from a normal quote, which callers have to know about.

Extrusions: the if-ladder

legacy/quote.ts · lines 27–49

```
for (var i = 0; i < design.items.length; i++) {
  var it = design.items[i];
  if (it.type == "extrusion") {
    var len = it.length;
    if (len < 50) {
      warnings.push("Extrusion " + it.sku + " too short (" + len + "mm), skipped");
      continue;
    }
    var perM = 0;
    var kg = 0;
    if (it.profile == "20x20") {
      perM = 12.5;
      kg = 0.5;
    } else if (it.profile == "40x40") {
      perM = 28;
      kg = 1.5;
    } else if (it.profile == "40x80") {
      perM = 49;
      kg = 2.8;
    } else {
      warnings.push("Unknown profile " + it.profile + " for " + it.sku);
      continue;
    }
```

One `for` loop and one long `if/else` chain on `it.type` handle every kind of part. Within it, another ladder picks a price and weight per profile. The numbers (12.5, 0.5, 28, 1.5, …) have no names, and `==` is used instead of `===` (loose equality, which converts types before comparing).

Extrusions: the price

legacy/quote.ts · lines 50–62

```
var price = perM * (len / 1000) + 1.5;
if (it.finish == "black") price = price * 1.15;
if (len > 3000) {
  warnings.push("Extrusion " + it.sku + " exceeds 3000mm stock; will be spliced");
  price = price + 8 + 1.5;
}
price = Math.round(price * 100) / 100;
var t = price * it.qty;
lines.push({
  sku: it.sku,
  desc: it.profile + " extrusion " + len + "mm" + (it.finish == "black" ? " black" : ""),
  qty: it.qty,
  unit: price,
```

The pricing rule is real business logic, but it's expressed only as arithmetic on unnamed numbers. Is 1.5 a cut fee? A weight? The same `1.5` is also the US shipping rate further down. Rounding happens here, mid-calculation, and again later.

Plates

legacy/quote.ts · lines 70–78

```
var p = PLATES[it.sku];
if (p == undefined) {
  warnings.push("Unknown plate " + it.sku);
  continue;
}
lines.push({ sku: it.sku, desc: "Plate " + it.sku, qty: it.qty, unit: p, total: p * it.qty });
subtotal += p * it.qty;
weight += 1.2 * it.qty;
hours += 0.25 * it.qty;
```

`PLATES[it.sku]` is an ordinary object lookup with a user-supplied key. A SKU of `"constructor"` would find the built-in `Object` constructor instead of `undefined`, and the quote would silently turn into `NaN`. Unlikely, but it's the kind of thing typed lookups prevent.

Fasteners: bug number one

legacy/quote.ts · lines 79–82

```
} else if (it.type == "fastener") {
  var packs = Math.round(it.qty / 50);
  if (packs == 0) packs = 1;
  lines.push({ sku: it.sku, desc: "Fasteners (pack of 50)", qty: packs, unit: 6.75, total:
packs * 6.75 });
```

Fasteners are sold in packs of 50, and `Math.round` rounds to the **nearest** pack. Ordering 120 bolts gives 2.4 packs, which rounds to 2: the customer is billed for 100 bolts and receives 100, twenty short. It should round **up**. The refactor keeps this behaviour (it's what the code does today) and fixes it at the end, in its own commit.

Assembly

legacy/quote.ts · lines 113–119

```
if (hasMotion && !hasController) warnings.push("No controller in design");

if (design.assembly) {
  if (hours < 2) hours = 2;
  labor = hours * 35;
  lines.push({ sku: "LAB", desc: "Assembly (" + hours + " h)", qty: 1, unit: labor, total:
labor });
}
```

Labour is billed at 35/hour with a 2-hour minimum. Note `hours` was accumulated in every branch of the loop above. This works, but only because each branch remembered to add to it.

Discounts

legacy/quote.ts · lines 121–132

```
// discounts
var disc = 0;
var tierPct = 0;
if (customer.tier == "partner") tierPct = 0.1;
if (customer.tier == "enterprise") tierPct = 0.15;
var tierDisc = subtotal * tierPct;
var promoDisc = 0;
if (customer.code == "SPRING10") promoDisc = (subtotal + labor) * 0.1;
if (promoDisc > tierDisc) disc = promoDisc;
else disc = tierDisc;
if (customer.orders > 10) disc = disc + subtotal * 0.02;
disc = Math.round(disc * 100) / 100;
```

The rules are reasonable (the larger of the tier discount or the promo code, plus a loyalty bonus), but they're spread over twelve lines of reassignments. The promo code is compared case-sensitively, so `spring10` silently does nothing. That's arguably a bug too, but nobody has asked for it to change, so it's pinned by a test and left alone.

Shipping: bug number two

legacy/quote.ts · lines 150–155

```
var ship = weight * rate;
if (ship < min) ship = min;
if ((customer.region == "CA" || customer.region == "US") && subtotal >= 2500) ship = 0;
if (customer.code == "FREESHIP") ship = 0;
if (design.rush) ship = ship * 1.5;
ship = Math.round(ship * 100) / 100;
```

Line 152 decides free shipping by comparing `subtotal` (the parts total **before** any discount) with 2,500. An enterprise customer whose order is 2,793 before and 2,374 after their 15% discount still gets free shipping, which isn't what the business intends. Again, pinned first, fixed last.

Tax

legacy/quote.ts · lines 157–168

```
// tax
var taxRate = 0;
if (customer.region == "CA") {
  if (customer.province == "ON") taxRate = 0.13;
  else if (customer.province == "QC") taxRate = 0.14975;
  else if (customer.province == "BC") taxRate = 0.12;
  else if (customer.province == "AB") taxRate = 0.05;
  else taxRate = 0.05;
} else if (customer.region == "EU") taxRate = 0.2;
var taxable = subtotal + labor - disc + ship;
var tax = Math.round(taxable * taxRate * 100) / 100;
var total = Math.round((taxable + tax) * 100) / 100;
```

Another ladder, this time for provinces. The last two branches (`AB` and the default) give the same 5%, so it's easy to misread which provinces are covered. Tax applies to the discounted goods plus shipping.

Lead time, and dead code

legacy/quote.ts · lines 170–178

```
// lead time
var lead = 5;
if (longest > 2000) lead += 3;
if (hasActuator) lead += 7;
if (design.assembly) lead += 2;
if (design.rush) {
  lead = Math.ceil(lead / 2);
  if (lead < 2) lead = 2;
}
```

Lead time starts at 5 days and only ever goes up. Halving it for a rush gives at least 3, so line 177 (`if (lead < 2) lead = 2`) can never run. Nobody noticed by reading. It was found by the coverage test described below, which asked for cases at the 2-day minimum and got none.

Side effects

legacy/quote.ts · lines 180–194

```
  var q = {
    quoteId: "Q-" + Date.now(),
    lines: lines,
    subtotal: Math.round((subtotal + labor) * 100) / 100,
    discount: disc,
    shipping: ship,
    tax: tax,
    total: total,
    leadTimeDays: lead,
    warnings: warnings,
  };
  lastQuote = q;
  console.log("quote " + q.quoteId + " total " + total);
  return q;
}
```

Three things that make the function hard to test and reuse:

- `Date.now()` in the ID: the same inputs give a different result every millisecond.
- `lastQuote = q`: the global is overwritten on every call.
- `console.log`: every quote writes to the log, whether the caller wants it or not.

GOING DEEPER

Reading code you're afraid of

Before testing, it helps to understand. Two techniques from Feathers: **scratch refactoring**, where you refactor freely on a throwaway branch just to learn the code, then delete it; and **effect sketches**, a quick diagram of which values each part of the function reads and writes. Both are how the list of problems in this part was found.

PART 4.2 · `git log`

The plan, as commits

Each step is its own commit, and every commit leaves all tests passing. The whole history ships with the project in `history/refactor-steps.bundle`.

Safety net first

1. **Add the legacy quote calculator, as found.** The starting point.
2. **Add characterization tests before touching anything.** Freeze a copy, build the golden master and spec tests, and check that the net actually catches a planted mistake.

Refactor: change structure, never behaviour

3. **Name the magic numbers:** every price, weight and rate moves into `catalog.ts`.
4. **Introduce types; let/const and a switch** instead of `any`, `var` and if-chains.
5. **Extract `round2()`** for the six copies of the rounding formula.
6. **Remove the unreachable minimum rush lead time.**
7. **Price each item type in its own function** (`items.ts`).
8. **Extract the pricing rules** (`rules.ts`).
9. **Golden master: also send non-boolean flags.** This closes a gap found in the previous step.
10. **Separate the pure calculation from its side effects:** `createQuote()` is pure, and `calculateQuote()` becomes a deprecated wrapper.

After each one: `npm test` (golden master + spec tests) and `tsc --strict`, then commit.

Fix: change behaviour, on purpose, one bug at a time

11. **Fix: fasteners are sold in whole packs, rounded up.** Test first, seen failing, then the fix.
12. **Fix: judge the free-shipping threshold after discounts.** Same pattern.

Keeping refactors and fixes in separate commits means each can be reviewed on its own and reverted on its own. A reviewer can see that commits 3–10 change no numbers at all, and that 11 and 12 each change exactly one rule.

PART 4.3 · `tests/cases.ts`

Generating the golden-master cases

Hand-written examples only check what you think of. A generator produces a thousand varied designs, including the awkward ones.

Repeatable randomness

tests/cases.ts · lines 14–27

```
/** Seeded generator, so the case list is identical on every run. */
function makeRandom(seed: number) {
  let s = seed >>> 0;
  const next = () => {
    s = (Math.imul(s, 1664525) + 1013904223) >>> 0;
    return s / 2 ** 32;
  };
  return {
    next,
    int: (min: number, max: number) => min + Math.floor(next() * (max - min + 1)),
    pick: <T>(items: readonly T[]): T => items[Math.floor(next() * items.length)],
    chance: (p: number) => next() < p,
  };
}
```

A tiny linear congruential generator with a fixed seed, so the same 1,002 cases come out on every run and on every machine. `Math.random()` would make a failure impossible to reproduce. `int`, `pick` and `chance` are the three helpers everything else uses.

Choosing values that find edges

tests/cases.ts · lines 29–46

```
const PROFILES = ["20x20", "40x40", "40x80", "40x80", "30x30"] as const;
const LENGTHS = [30, 49, 50, 120, 300, 750, 1000, 1500, 2000, 2001, 2500, 2999, 3000, 3001,
4200] as const;
const PLATES = ["PL-100", "PL-200", "PL-300", "BR-90", "PL-999"] as const;
const TIERS = ["standard", "partner", "enterprise", undefined] as const;
const REGIONS = ["CA", "CA", "US", "EU", "OTHER", "JP"] as const;
const PROVINCES = ["ON", "QC", "BC", "AB", "NS", undefined] as const;
const CODES = [undefined, undefined, "SPRING10", "spring10", "FREESHIP"] as const;

export function generateCases(count = 1000, seed = 2024): Case[] {
  const r = makeRandom(seed);
  const cases: Case[] = [
    { name: "empty design", design: { items: [] }, customer: { region: "CA" } },
    { name: "no design", design: null, customer: { region: "CA" } },
  ];

  // Callers don't always send real booleans; the legacy code used truthiness, so test that
too.
  const flag = (p: number): unknown => (r.chance(0.05) ? r.pick([1, "yes", 0, ""]) :
r.chance(p));
```

The value lists are where the thinking goes. Lengths include both sides of every boundary in the code (49/50 for the minimum, 2000/2001 for the long-piece surcharge, 2999/3000/3001 for splicing). There's an unknown profile (`30x30`), an unknown plate (`PL-999`), the lower-case promo code, an unknown region (`JP`) and province (`NS`), plus two edge cases up front: an empty design and no design at all.

`flag` is the fix from commit 9: 5% of the time, `assembly` and `rush` get a non-boolean value like `1` or `""`, because the legacy code tested truthiness and real callers might rely on that.

PART 4.4 · `tests/golden.test.ts`

The golden master

The core safety net. There's no spec, so the legacy behaviour is the spec: every case must give the same answer from the old code and the new.

Setup

tests/golden.test.ts · lines 13–28

```
import { after, before, mock, test } from "node:test";
import assert from "node:assert/strict";
import { isDeepStrictEqual } from "node:util";
import { calculateQuote as legacyQuote } from "../legacy/quote";
import { calculateQuote } from "../src/quote";
import type { Customer, Design } from "../src/types";
import { generateCases, type Case } from "./cases";

const FIXED_NOW = 1_700_000_000_000;
const cases = generateCases();

before(() => {
  mock.method(Date, "now", () => FIXED_NOW); // quote IDs use the clock
  mock.method(console, "log", () => {}); // the legacy code logs every quote
});
after(() => mock.restoreAll());
```

Both implementations are imported side by side: the frozen copy as `legacyQuote`, and the code being refactored. `mock.method` from Node's built-in test runner replaces `Date.now` with a fixed value, so the quote IDs match, and silences the legacy `console.log`. `after` puts the real ones back.

Intentional changes, written down

tests/golden.test.ts · lines 33–54

```
interface IntentionalChange {
  name: string;
  /** Could this fix change this case's quote? */
  mayAffect(c: Case, before: Any, after: Any): boolean;
}

const INTENTIONAL_CHANGES: IntentionalChange[] = [
  {
    name: "fasteners round up to whole packs",
    // Only quantities where rounding to nearest and rounding up disagree: 51-74, 101-124, ...
    mayAffect: (c) =>
      itemsOf(c).some((i) => i.type === "fastener" && Math.max(1, Math.round(i.qty / 50)) !==
Math.max(1, Math.ceil(i.qty / 50))),
  },
  {
    name: "free-shipping threshold uses the discounted order value",
    // Only CA/US orders without the free-shipping code, where free shipping applied before or
applies now.
    mayAffect: (c, before, after) => {
      const cust = c.customer as Any;
      return ["CA", "US"].includes(cust.region) && cust.code !== "FREESHIP" && (before.shipping
=== 0 || after.shipping === 0);
    },
  },
];
```

Before the bug fixes, this list was empty and the test required every case to be identical. After them, each fix is listed with a rule for **which cases it could possibly affect**:

- Rounding up only differs from rounding to nearest for some quantities: 51–74, 101–124, and so on.
- The threshold fix can only matter for CA/US orders without the free-shipping code, where shipping was free before or is free now.

This turns “the numbers changed, but I think that's expected” into something checkable.

The test

tests/golden.test.ts · lines 56–68

```
test(`current code matches legacy on all ${cases.length} cases, apart from intentional
changes`, () => {
  const changedBy = new Map(INTENTIONAL_CHANGES.map((ch) => [ch.name, 0]));
  for (const c of cases) {
    const before = legacyQuote(c.design, c.customer) as Any;
    const after = calculateQuote(c.design as Design, c.customer as Customer) as Any;
    if (isDeepStrictEqual(after, before)) continue;

    const reasons = INTENTIONAL_CHANGES.filter((ch) => ch.mayAffect(c, before, after));
    assert.ok(reasons.length > 0, `${c.name} changed, and no intentional change explains it`);
    for (const r of reasons) changedBy.set(r.name, changedBy.get(r.name)! + 1);
  }
  for (const [name, count] of changedBy) assert.ok(count > 0, `"${name}" changed no cases at
all`);
});
```

For each case, compare old and new with `isDeepStrictEqual`: every field, every line and warning, every number to the last bit. If they differ, at least one intentional change must explain it, or the test fails with the case's name. Finally, each listed fix must have changed at least one case. A fix that changed nothing means its rule, or the fix itself, is wrong.

Before the fixes this passed with all 1,002 identical. After them, 226 cases differ, and every one is accounted for.

Testing the tests

tests/golden.test.ts · lines 70–76

```
test("the cases exercise every branch worth protecting", () => {
  const seen = new Map<string, number>();
  const hit = (key: string) => seen.set(key, (seen.get(key) ?? 0) + 1);

  for (const c of cases) {
    const q = legacyQuote(c.design, c.customer) as Any;
    if ("error" in q) {
```

tests/golden.test.ts · lines 96–102

```
  const expected = [
    "error", "too short", "spliced", "unknown profile", "unknown plate", "unknown type", "no
controller",
    "assembly", "black finish", "discount", "free shipping", "free-shipping threshold", "promo
code", "tax",
  ];
  for (const key of expected) assert.ok((seen.get(key) ?? 0) >= 2, `only ${seen.get(key) ?? 0}
cases hit "${key}"`);
});
```

A golden master is only as good as its cases. This second test runs the legacy code over the same cases and counts how often each interesting branch is hit (errors, splices, unknown parts, promo codes, free shipping, tax). Each must be hit at least twice.

> This is how the dead code was found: the first version of this test also asked for the 2-day minimum lead time, and no case could reach it.

The net was also checked directly: changing the cut fee from 1.50 to 1.51 makes the golden master fail at once.

GOING DEEPER

How strict is “deeply equal”?

`isDeepStrictEqual` compares primitives with `Object.is`, so `NaN` equals `NaN` but `0` and `-0` differ, and it compares object prototypes too, while ignoring key order. That's strict enough to catch a reordered floating-point sum that changes the last bit of a number, which is why the refactor kept the legacy order of additions.

PART 4.5 · `tests/spec.test.ts`

Spec tests: the rules, written down

The golden master says “nothing changed” but not what the rules are. These 21 tests are the missing documentation, each with the arithmetic in its name.

Helpers

tests/spec.test.ts · lines 14–25

```
const NOW = 1_700_000_000_000;
afterEach(() => mock.restoreAll());

type Any = Record<string, any>;
const customer = (over: Any = {}) => ({ tier: "standard", region: "US", orders: 0, ...over });
// Items are written as plain objects for readability; cast once here.
const quote = (items: Any[], design: Any = {}, cust: Any = {}) =>
  createQuote({ items, assembly: false, rush: false, ...design } as unknown as Design,
customer(cust) as Customer, NOW) as Any;

const extrusion = (profile: string, length: number, qty = 1, finish = "clear") =>
  ({ type: "extrusion", sku: "EX", profile, length, qty, finish });
const close = (a: number, b: number) => assert.ok(Math.abs(a - b) < 1e-9, `${a} ≠ ${b}`);
```

`customer()` and `quote()` fill in sensible defaults so each test only states what matters. Since commit 10 they call the pure `createQuote` with a fixed `NOW`, so no mocking is needed. Items are written as plain objects for readability, and cast once in the helper.

A rule as an example

tests/spec.test.ts · lines 29–32

```
test("extrusion: profile price per meter × length, plus a 1.50 cut fee (40x40, 1 m: 28 + 1.5 =
29.50)", () => {
  const q = quote([extrusion("40x40", 1000, 2)]);
  assert.equal(q.lines[0].unit, 29.5);
  assert.equal(q.lines[0].total, 59);
```

The name is the specification: price per meter times length, plus a 1.50 cut fee, with the arithmetic spelled out. Anyone changing prices later can read the rule here rather than reverse-engineering it.

A bug fix, test first

tests/spec.test.ts · lines 112–120

```
test("free shipping at 2,500 is judged on what the customer pays, after discounts (was before,
a bug)", () => {
  const sevenActuators = [{ type: "actuator", sku: "LA", qty: 7 }]; // 2,793 in parts, 24.5 kg
  // Standard customer: 2,793 ≥ 2,500, ships free.
  assert.equal(quote(sevenActuators).shipping, 0);
  // Enterprise: 2,793 − 418.95 (15%) = 2,374.05 < 2,500, so pays 24.5 kg × 1.50 = 36.75.
  const enterprise = quote(sevenActuators, {}, { tier: "enterprise" });
  assert.equal(enterprise.discount, 418.95);
  assert.equal(enterprise.shipping, 36.75);
});
```

Until commit 12 this test said the opposite: its name started with “BUG (fixed later)” and it asserted the wrong behaviour, free shipping, so the refactor couldn't change it by accident. To fix the bug, the test was rewritten to state the correct rule **first**, run and seen failing (`expected 36.75, actual 0`), and only then was the code changed. It covers both sides of the line.

Purity, and the old API

tests/spec.test.ts · lines 152–168

```
test("createQuote is pure: the ID comes from the `now` argument, nothing is logged or stored",
() => {
  const log = mock.method(console, "log", () => {});
  const design = { items: [{ type: "plate", sku: "PL-100", qty: 1 }] } as Design;
  const a = createQuote(design, customer(), NOW) as Any;
  assert.equal(a.quoteId, "Q-1700000000000");
  assert.deepEqual(createQuote(design, customer(), NOW), a, "same inputs, same quote");
  assert.equal(log.mock.callCount(), 0);
});

test("the deprecated calculateQuote keeps its old side effects: clock ID, global lastQuote, log
line", () => {
  mock.method(Date, "now", () => NOW);
  const log = mock.method(console, "log", () => {});
  const q = calculateQuote({ items: [{ type: "plate", sku: "PL-100", qty: 1 }] } as Design,
customer()) as Any;
  assert.equal(q.quoteId, "Q-1700000000000");
  assert.equal(lastQuote, q);
  assert.deepEqual(log.mock.calls[0].arguments, ["quote Q-1700000000000 total 48.5"]);
});
```

Two tests pin down commit 10. `createQuote` is pure: the same inputs give an equal result, and nothing is logged. `mock.method` records calls, so `callCount()` proves it. The deprecated `calculateQuote` still has every legacy side effect, including the exact log line, so existing callers can't tell the difference.

PART 4.6 · `src/catalog.ts`

Named numbers

Commit 3. Every number the business cares about gets a name, a unit, and one place to live.

Prices and weights

src/catalog.ts · lines 1–30

```
/**
 * Every price, weight, rate and threshold the quote uses, in one place.
```

```
 * Money in dollars, weight in kg, length in mm, labour in hours, lead time in days.
 */

export const PROFILES = {
  "20x20": { pricePerMeter: 12.5, kgPerMeter: 0.5 },
  "40x40": { pricePerMeter: 28, kgPerMeter: 1.5 },
  "40x80": { pricePerMeter: 49, kgPerMeter: 2.8 },
} as const;

export const EXTRUSION = {
  cutFee: 1.5,
  spliceFee: 8,
  minLengthMm: 50,
  stockLengthMm: 3000,
  blackFinishMultiplier: 1.15,
  laborHours: 0.5,
} as const;

export const PLATE_PRICES: Readonly<Record<string, number>> = {
  "PL-100": 18.5,
  "PL-200": 32,
  "PL-300": 54.25,
  "BR-90": 7.8,
};

export const PLATE = { kg: 1.2, laborHours: 0.25 } as const;

export const FASTENER_PACK = { size: 50, price: 6.75, kg: 0.4 } as const;
```

The profile ladder becomes a table: adding a 40×120 profile is now one line, not another `else if`. `as const` makes the objects read-only and keeps their exact values in the types.

Rates and thresholds

src/catalog.ts · lines 47–60

```
export const DISCOUNTS = {
  tierRates: { partner: 0.1, enterprise: 0.15 } as Readonly<Record<string, number>>,
  promoCode: "SPRING10",
  promoRate: 0.1,
  loyaltyAfterOrders: 10,
  loyaltyRate: 0.02,
} as const;

export const SHIPPING = {
  regions: {
    CA: { perKg: 1.2, min: 25 },
    US: { perKg: 1.5, min: 30 },
    EU: { perKg: 2.4, min: 45 },
  } as Readonly<Record<string, { perKg: number; min: number }>>,
```

Discount and shipping rules are configuration now. The `as Readonly<Record<...>>` on the inner tables says they're looked up by arbitrary strings (a customer's region, say), which is exactly what the next helper is for.

Safe lookups

src/catalog.ts · lines 82–85

```
/** Reads a table entry by a key from outside (user input), ignoring inherited properties like
"constructor". */
export function lookup<T>(table: Readonly<Record<string, T>>, key: unknown): T | undefined {
  return typeof key === "string" && Object.hasOwn(table, key) ? table[key] : undefined;
}
```

`Object.hasOwn` checks that the key is the table's own, not an inherited one like `constructor` or `toString`. For every real input this behaves exactly like the old `==` checks, which is what the golden master confirmed, but junk input now finds nothing instead of a function.

PART 4.7 · `src/types.ts`

Types

Commit 4. The shapes of the inputs and outputs, written down where the compiler can check them.

One interface per item type

src/types.ts · lines 9–18

```
export interface ExtrusionItem {
  type: "extrusion";
  sku: string;
  /** "20x20", "40x40" or "40x80"; anything else is skipped with a warning. */
  profile: string;
  /** mm */
  length: number;
  qty: number;
  finish?: "clear" | "black";
}
```

Comments carry the units (mm) and the rules (other profiles are skipped). `profile` is a `string`, not a union of the three valid values, on purpose: callers do send unknown profiles, and the code has to keep handling them.

A discriminated union

src/types.ts · lines 51–57

```
export type Item = ExtrusionItem | PlateItem | FastenerItem | MotorItem | ActuatorItem |
ControllerItem;

export interface Design {
  items: readonly Item[];
  assembly?: boolean;
  rush?: boolean;
}
```

`Item` is one of six interfaces, told apart by their `type` field. In a `switch (item.type)`, TypeScript narrows `item` in each `case`, so `item.length` is only allowed in the extrusion branch, and a typo like `"extrution"` is a compile error rather than a silently skipped part.

PART 4.8 · `src/money.ts`

One rounding function

Commit 5.

round2

src/money.ts · lines 1–9

```
/**
 * Rounds to the cent. This is exactly the legacy formula, so every result stays
 * identical; that's what the old "DO NOT TOUCH THE ROUNDING" comment protects.
 * (Rounding half-cents of floating-point values isn't perfect; changing it would
 * be a behaviour change for finance to sign off, not part of a refactor.)
 */
export function round2(amount: number): number {
  return Math.round(amount * 100) / 100;
}
```

Six copies of `Math.round(x * 100) / 100` become one function with exactly the same formula, so results are bit-for-bit identical.

The comment records something the refactor deliberately did **not** change. Floating-point numbers can't represent most decimals exactly. In the demo, a black 40×80 piece is (49 × 2.4 + 1.5) × 1.15, which is 136.965 on paper. In floating point the multiplications come out at 136.96499999999997, a hair under the half cent, so it rounds down to 136.96 instead of up. Moving to whole cents would fix that, but it changes invoices, and that's a decision for whoever owns billing, not a refactoring step.

PART 4.9 · `src/items.ts`

Pricing each item

Commit 7. The 90-line loop body becomes six small functions behind one dispatcher.

The shape of a result

src/items.ts · lines 7–39

```
/** What one design item adds to a quote. */
export interface PricedItem {
  line: QuoteLine;
  weightKg: number;
  laborHours: number;
  /** Notes that don't stop the item being quoted, like a splice. */
  warnings: string[];
}

/** An item that can't be quoted, and why. */
```

```
export interface SkippedItem {
  skipped: string;
}

export function priceItem(item: Item): PricedItem | SkippedItem {
  switch (item.type) {
    case "extrusion":
      return priceExtrusion(item);
    case "plate":
      return pricePlate(item);
    case "fastener":
      return priceFasteners(item);
    case "motor":
      return priceMotor(item);
    case "actuator":
      return priceActuator(item);
    case "controller":
      return priceController(item);
    default:
      // Not a known Item, but callers can still send one.
      return { skipped: "Unknown item type " + (item as { type: unknown }).type };
  }
}
```

Each pricing function returns either a `PricedItem` (its quote line, weight, labour hours and any warnings) or a `SkippedItem` with the reason. The caller checks which with `"skipped" in priced`. That replaces the legacy `continue` statements scattered through the loop.

The `default` branch covers item types TypeScript doesn't know about. They can't happen according to the types, but can happen at runtime, so the cast `(item as { type: unknown })` is explicit about that.

Extrusions, readable

`src/items.ts · lines 41–70`

```
/** Cut to length from stock: price per meter plus a cut fee; long pieces are spliced from two.
*/
function priceExtrusion(item: ExtrusionItem): PricedItem | SkippedItem {
  const len = item.length;
  if (len < EXTRUSION.minLengthMm) return { skipped: `Extrusion ${item.sku} too short
(${len}mm), skipped` };
  const profile = lookup(PROFILES, item.profile);
```

```
  if (!profile) return { skipped: `Unknown profile ${item.profile} for ${item.sku}` };

  const warnings: string[] = [];
  const black = item.finish === "black";
  let price = profile.pricePerMeter * (len / 1000) + EXTRUSION.cutFee;
  if (black) price = price * EXTRUSION.blackFinishMultiplier;
  if (len > EXTRUSION.stockLengthMm) {
    warnings.push(`Extrusion ${item.sku} exceeds 3000mm stock; will be spliced`);
    price = price + EXTRUSION.spliceFee + EXTRUSION.cutFee;
  }
  price = round2(price);

  return {
    line: {
      sku: item.sku,
      desc: `${item.profile} extrusion ${len}mm${black ? " black" : ""}`,
      qty: item.qty,
      unit: price,
      total: price * item.qty,
    },
    weightKg: profile.kgPerMeter * (len / 1000) * item.qty,
    laborHours: EXTRUSION.laborHours * item.qty,
    warnings,
  };
}
```

The same arithmetic as lines 50–62 of the legacy file, in the same order, now with names: `pricePerMeter`, `cutFee`, `blackFinishMultiplier`, `spliceFee`. Early `return`s handle the two ways an extrusion can be skipped, so the rest of the function is the normal case.

Fasteners, fixed

src/items.ts · lines 83–98

```
/** Sold in whole packs, rounded up; quantity on the line is packs, not pieces. */
function priceFasteners(item: FastenerItem): PricedItem {
  const packs = Math.max(1, Math.ceil(item.qty / FASTENER_PACK.size));
  return {
    line: {
      sku: item.sku,
      desc: "Fasteners (pack of 50)",
      qty: packs,
      unit: FASTENER_PACK.price,
      total: packs * FASTENER_PACK.price,
    },
    weightKg: FASTENER_PACK.kg * packs,
    laborHours: 0,
    warnings: [],
  };
}
```

This is the whole of fix number one: `Math.ceil` instead of `Math.round`, with `Math.max(1, …)` keeping the old “at least one pack” rule. Because the function is small and isolated, the fix is one line and obviously correct.

GOING DEEPER

switch, table or polymorphism?

- **A table** when only data differs between cases: profiles and plates are lookups in `catalog.ts`.
- **A switch over a discriminated union** when behaviour differs and the set of cases is closed: TypeScript narrows the type in each branch and can prove the switch exhaustive.
- **Polymorphism** (a class or strategy per type) when cases are many, grow often, or come from plugins. Here it would add ceremony without benefit.

PART 4.10 · `src/rules.ts`

Pricing rules

Commit 8. The business rules that apply to the whole order, each a small pure function with a doc comment.

Labour and discount

src/rules.ts · lines 5–21

```
/** Billed assembly hours (never under the minimum) and their cost. */
export function assemblyLabor(hours: number): { hours: number; cost: number } {
  const billed = hours < ASSEMBLY.minHours ? ASSEMBLY.minHours : hours;
  return { hours: billed, cost: billed * ASSEMBLY.ratePerHour };
}

/**
 * The larger of the customer's tier discount (on parts) and the promo code (on
 * parts and labour), plus a loyalty bonus on parts. Rounded to the cent.
 */
export function discount(parts: number, labor: number, customer: Customer): number {
  const tier = parts * (lookup(DISCOUNTS.tierRates, customer.tier) ?? 0);
  const promo = customer.code === DISCOUNTS.promoCode ? (parts + labor) * DISCOUNTS.promoRate :
0;
  let amount = promo > tier ? promo : tier;
  if ((customer.orders ?? 0) > DISCOUNTS.loyaltyAfterOrders) amount = amount + parts *
DISCOUNTS.loyaltyRate;
  return round2(amount);
}
```

`discount` reads almost like the sentence in its comment: the larger of tier and promo, plus loyalty, rounded. Every expression is the same as the legacy code's, in the same order, so the floating-point results match exactly.

Shipping

src/rules.ts · lines 23–37

```
/**
 * Weight × the region's rate, with a minimum charge. Free in CA/US once
 * `orderValue` reaches the threshold, or with the free-shipping code; ×1.5 for
 * rush orders. Rounded to the cent.
 */
export function shipping(weightKg: number, orderValue: number, customer: Customer, rush:
boolean): number {
  const zone = lookup(SHIPPING.regions, customer.region) ?? SHIPPING.elsewhere;
  let cost = weightKg * zone.perKg;
  if (cost < zone.min) cost = zone.min;
  const freeRegion = customer.region !== undefined &&
SHIPPING.freeShippingRegions.includes(customer.region);
  if (freeRegion && orderValue >= SHIPPING.freeShippingThreshold) cost = 0;
  if (customer.code === SHIPPING.freeShippingCode) cost = 0;
  if (rush) cost = cost * SHIPPING.rushMultiplier;
  return round2(cost);
}
```

The parameter is called `orderValue` on purpose. During the refactor, `createQuote` passed the pre-discount parts total, as the legacy code did. Fix number two changed only the argument at the call site, not this function, so the shipping rules themselves never had to change.

Lead time

src/rules.ts · lines 53–61

```
/** Working days to ship: a base, plus extra for long pieces, actuators and assembly; rush
halves it. */
export function leadTimeDays(f: LeadTimeFactors): number {
  let days: number = LEAD_TIME.baseDays;
  if (f.longestPieceMm > LEAD_TIME.longPieceMm) days += LEAD_TIME.longPieceDays;
  if (f.hasActuator) days += LEAD_TIME.actuatorDays;
  if (f.assembly) days += LEAD_TIME.assemblyDays;
  if (f.rush) days = Math.ceil(days / 2);
  return days;
}
```

Without the dead minimum, the rush rule is one line. `LeadTimeFactors` (lines 46–51) names the four things lead time depends on, instead of reaching into the design and the loop's flags.

PART 4.11 · `src/quote.ts`

The result

Commit 10, plus fix two. The entry point now reads as a summary of the process.

A pure function

src/quote.ts · lines 6–13

```
/**
 * Prices a machine design for a customer.
 *
 * Pure: no globals, no logging, and the clock is a parameter, so the same inputs
 * always produce the same quote (which is also what makes it easy to test).
 */
export function createQuote(design: Design | null | undefined, customer: Customer, now:
number): Quote | QuoteError {
  if (!design || !design.items || design.items.length === 0) return { error: "Empty design" };
```

`now` is a parameter instead of a hidden call to `Date.now()`. That one change makes the function **pure**: same inputs, same output, no side effects. Tests don't need to mock anything, and two quotes can be computed in any order, or in parallel.

1. Add up the items

src/quote.ts · lines 15–44

```
// 1. Price every item, adding up money, weight and labour as we go.
const lines: QuoteLine[] = [];
const warnings: string[] = [];
let parts = 0;
let weightKg = 0;
let laborHours = 0;
let longestPieceMm = 0;
let hasMotion = false;
let hasActuator = false;
let hasController = false;

for (const item of design.items) {
  const priced = priceItem(item);
  if ("skipped" in priced) {
    warnings.push(priced.skipped);
```

```
    continue;
  }
  lines.push(priced.line);
  warnings.push(...priced.warnings);
  parts += priced.line.total;
  weightKg += priced.weightKg;
  laborHours += priced.laborHours;

  if (item.type === "motor" || item.type === "actuator") hasMotion = true;
  if (item.type === "actuator") hasActuator = true;
  if (item.type === "controller") hasController = true;
  if (item.type === "extrusion" && item.length > longestPieceMm) longestPieceMm = item.length;
}

if (hasMotion && !hasController) warnings.push("No controller in design");
```

Each item is priced by `priceItem`; the loop only adds up money, weight and hours, and notes the flags lead time and the controller warning need. `for…of` replaces the indexed `for` loop.

2–3. Labour, discount, shipping, tax

src/quote.ts · lines 46–73

```
  // 2. Assembly labour, if asked for.
  let labor = 0;
  if (design.assembly) {
    const billed = assemblyLabor(laborHours);
    labor = billed.cost;
    lines.push({ sku: "LAB", desc: `Assembly (${billed.hours} h)`, qty: 1, unit: labor, total:
labor });
  }

  // 3. Discount, shipping, tax.
  const rush = Boolean(design.rush);
  const disc = discount(parts, labor, customer);
  // Free-shipping threshold: what the customer pays for parts and assembly, after discounts.
  const ship = shipping(weightKg, parts + labor - disc, customer, rush);
  const taxable = parts + labor - disc + ship;
  const tax = round2(taxable * taxRate(customer));

  return {
    quoteId: `Q-${now}`,
```

```
    lines,
    subtotal: round2(parts + labor),
    discount: disc,
    shipping: ship,
    tax,
    total: round2(taxable + tax),
    leadTimeDays: leadTimeDays({ longestPieceMm, hasActuator, assembly:
Boolean(design.assembly), rush }),
    warnings,
  };
}
```

Each step is a call to a named rule. Line 58 is fix number two: the free-shipping threshold is checked against what the customer pays after discounts (`parts + labor - disc`). `Boolean(design.rush)` keeps the legacy truthiness, which commit 9's extra cases now test.

The old API, still working

src/quote.ts · lines 75–90

```
// ---- Legacy API, kept so existing callers keep working ----

/** @deprecated The most recent quote from calculateQuote(). Use its return value instead. */
export let lastQuote: Quote | null = null;

/**
 * @deprecated Use createQuote(design, customer, Date.now()).
 * Same result, plus the legacy side effects: it records the quote in `lastQuote` and logs it.
 */
export function calculateQuote(design: Design | null | undefined, customer: Customer): Quote |
QuoteError {
  const quote = createQuote(design, customer, Date.now());
  if ("error" in quote) return quote;
  lastQuote = quote;
  console.log(`quote ${quote.quoteId} total ${quote.total}`);
  return quote;
}
```

`calculateQuote` keeps its exact signature and side effects (the clock, the `lastQuote` global, the log line) but is now three lines around `createQuote`. `@deprecated` makes editors strike it through, nudging callers to migrate, while nothing breaks today. Once no caller uses it, it can be deleted in a commit of its own.

The golden master still runs through this wrapper, so it covers both: the wrapper and the pure function behind it.

GOING DEEPER

Shipping a behaviour change safely

Fixes 11 and 12 change real prices. In a real business the code is the easy part: someone owns the pricing rules and signs off, customer-facing changes may need notice, and the rollout might go behind a feature flag or run in shadow mode first. The intentional-change list in the golden master doubles as the change note: it says exactly which quotes move, and why.

CHAPTER 4

Interview questions

Q1 **How do you refactor code that has no tests?**

Pin its current behaviour first with characterization tests: a golden master over many generated inputs against a frozen copy, plus readable spec tests for the important rules. Check the inputs reach every branch, and plant a mistake to prove the tests catch it. Then change structure in small commits, each green, and only afterwards change behaviour, one bug per commit, test first.

Q2 **What's a golden master, and what are its weaknesses?**

Recorded outputs of the existing code for many inputs, compared against the new code. It locks in bugs as well as features, is only as good as the inputs, gives opaque failures ("case 731 differs") and needs an explicit mechanism for intended changes. That's why it's paired with spec tests that state the rules in words.

Q3 **How did you know the generated cases were good enough?**

A second test asserts that the cases hit every branch worth protecting at least twice, and mutation by hand: a one-cent change to a fee had to fail the suite. When a near-miss showed a gap (only real booleans were ever generated), the generator was fixed before the code was trusted again.

Q4 **Why separate refactoring commits from bug-fix commits?**

Reviewability and reversibility. A reviewer can verify that commits 3–10 change no output at all, and that 11 and 12 each change exactly one rule. Either fix can be reverted alone, and `git bisect` lands on a small, single-purpose commit.

Q5 **Why keep the deprecated `calculateQuote` ?**

Existing callers depend on its side effects: the global `lastQuote`, the log line, the clock-based ID. Removing them is a breaking change. The wrapper keeps old callers working while new code uses the pure `createQuote`, and it can be deleted when the last caller migrates.

Q6 **You found the rounding issue. Why not fix it?**

Because it isn't a refactoring question. How invoices round is a business rule, owned by whoever owns billing, and changing it moves real prices. The right move is to document it precisely and raise it, not to slip it into a refactor.

Q7 What was the `=== true` near-miss?

While extracting the rules, it would have been natural to write `design.rush === true`, but the legacy code used truthiness, so `1` or `"yes"` counted as rush. The golden master didn't object because every generated case used real booleans. The fix went into the tests first (generate non-boolean flags), and only then was the code trusted.

Q8 How would you make the clock-based quote ID testable?

Inject the time: `createQuote` takes `now` as a parameter, so tests pass a fixed number. The legacy path is tested by mocking `Date.now` (and `console.log`) in the golden master, and a spec test checks the wrapper's side effects explicitly.

Q9 How would you bring this approach to a team?

Start with the safety net in its own PR, so reviewers can trust it before anything changes. Then small refactor PRs that each say “no behaviour change, golden master exact”, and separate fix PRs with the failing test first and the list of affected cases. The commit history in the repo is that process, step by step.

CHAPTER 4

Exercises

4.1 A new item type

Add a cable chain priced per meter with a minimum length. Write the spec tests first; the golden master must stay exactly as it is.

Hint. New behaviour for new inputs only: the generator doesn't produce the new type, so legacy cases can't change.

4.2 Integer cents

Convert all money to integer cents internally, add it to the intentional-change list, and report how many of the 1,002 cases change.

Hint. Write the `mayAffect` rule first. It should be narrow: cases where a computed half cent lands on the other side.

4.3 Mutation testing

Run Stryker on `src/` and add tests for any surviving mutants that matter.

Hint. Some survivors are equivalent mutants (changes that can't alter behaviour). Decide which, and say why.

4.4 Property-based tests

With fast-check, assert properties for random designs: total = subtotal − discount + shipping + tax to the cent; discount is never negative; adding an item never lowers the subtotal.

Hint. Reuse the generator's building blocks so properties see the same awkward values as the golden master.

4.5 Regions as data

Replace the region and province rules with a table of per-region strategies (rate, shipping per kg, minimum). The golden master must stay exact.

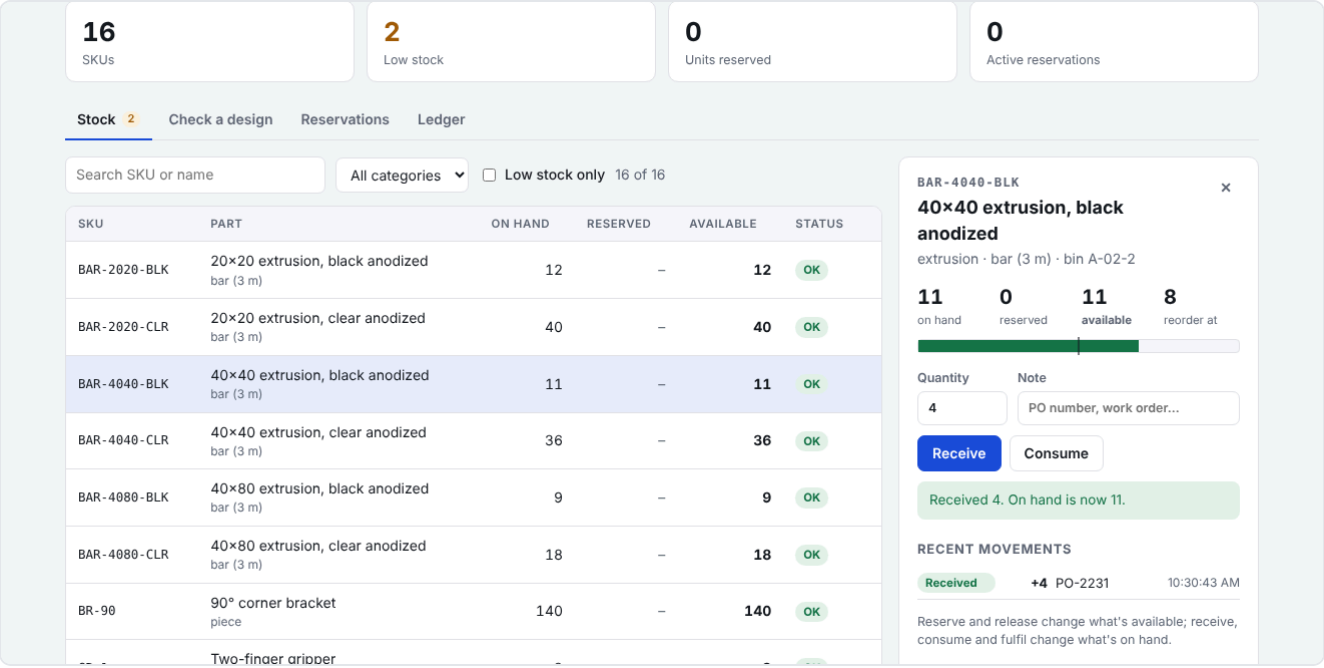
Hint. Keep the order of floating-point operations identical, or the deep-equality check will tell you.

CHAPTER 5 · `vention/snack5-inventory`

Parts Inventory

A stock system for the parts the other snacks use: check a machine design against live stock and reserve it, all or nothing, on PostgreSQL or MongoDB behind one interface.

Node.js · React · PostgreSQL or MongoDB · Docker · Kubernetes



The stock page with a 40×40 black bar open: four bars just received, logged in its movement history.

The one idea to take away

The database is the only place two API replicas share, so it's where “is there enough?” is answered:

`design` → `plan(): stock lines` → `repo.reserve()` → `locks / conditional updates` → `reservation or shortages`

One interface, three implementations, one contract suite that keeps them identical:

`InventoryRepo` → `PostgresRepo · MongoRepo · MemoryRepo` → `contractTests()` → `same 19 tests on each`

CONCEPTS · 5.1

The shape of the system

Snack 5 is a classic three-tier web app: a static front end, a stateless API, and a database. The interesting part is the seam between the API and the database: one TypeScript interface, `InventoryRepo`, with three implementations.

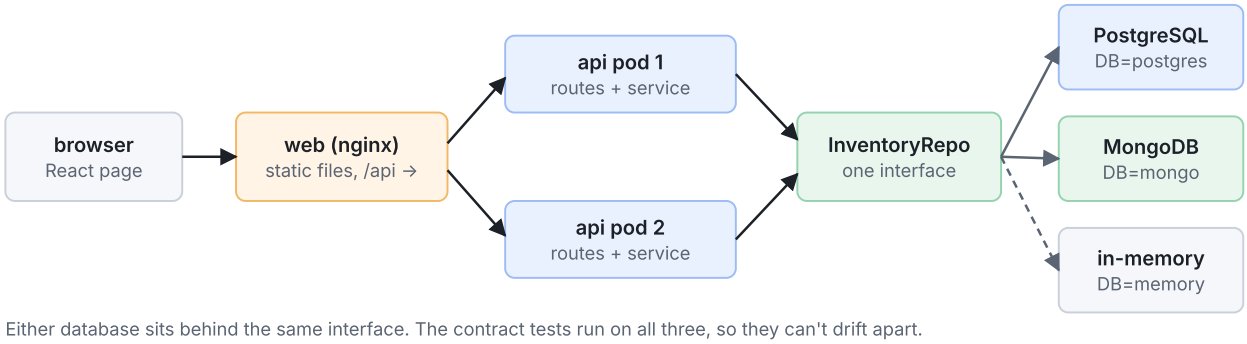


Figure 5.1. Snack 5's architecture. Two API replicas share one database; one environment variable chooses which kind.

Two API replicas run at once, in Kubernetes and potentially in Compose. That's the point: the replicas share nothing but the database, so **the database is the only place where "is there enough stock?" can be answered reliably**. Any check done in API memory would be wrong as soon as the other replica changed something.

CONCEPTS · 5.2

Relational and document data models

A **relational** database stores data in tables of rows with a fixed set of columns, and relates tables with keys. A **document** database stores self-contained JSON-like documents, which can nest arrays and objects.

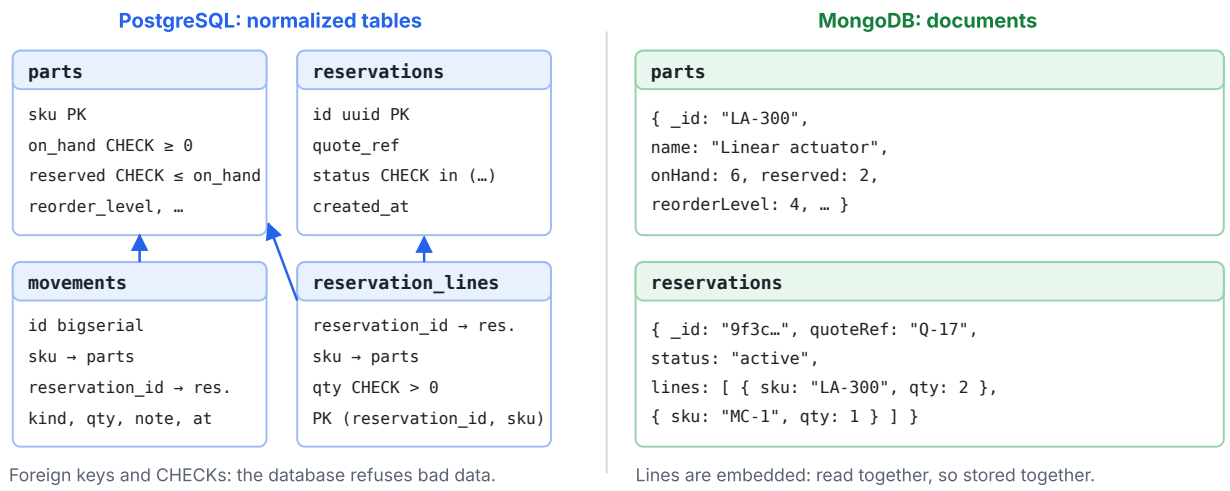


Figure 5.2. The same data in both models. Postgres splits it into related tables; MongoDB nests what's read together.

- **Normalization** (Postgres side): each fact is stored once. A reservation line is a row that points at a part and a reservation. Joins put them back together, and foreign keys guarantee every pointer is valid.
- **Embedding** (MongoDB side): data that's always read together is stored together. A reservation and its lines are one document, read and written in one operation, with no join.
- **Schema**: Postgres enforces column types and constraints on every write. MongoDB accepts any shape unless you add a JSON Schema validator; here the TypeScript types and the repository are what keep documents consistent.
- **Rule of thumb**: model relationally when data is shared and queried many ways (parts appear in many reservations and reports); embed when a piece of data belongs to one parent and is only read through it (a reservation's lines).

CONCEPTS · 5.3

Transactions, isolation and race conditions

A **transaction** groups statements so they all happen or none do (atomicity), and so concurrent transactions don't see each other's half-finished work (isolation). Together with consistency and durability that's **ACID**.



Figure 5.3. The race the database has to prevent. Without a lock, both requests act on a value that's already stale.

The left side is a **lost update**, also called check-then-act: both requests read "1 left", both decide to reserve, both write. Each request was correct on its own; together they promised a part that doesn't exist. Three tools prevent it:

- **Row locks.** `SELECT … FOR UPDATE` locks the rows until the transaction ends; a second transaction wanting them waits. Used in `inv_reserve`, where several rows must be checked together.
- **Conditional updates.** Put the rule in the update itself: `UPDATE … WHERE on_hand - reserved >= 2`. Under READ COMMITTED, Postgres re-checks that condition on the latest row version if it had to wait, so the update simply does nothing if the stock is gone. Used for single-part changes, on both databases.
- **Constraints.** `CHECK (reserved <= on_hand)` makes the database refuse any statement that would break the rule. It doesn't help you produce a nice error, but it catches every bug the other two miss.

Locks bring a new risk, **deadlock**: transaction 1 locks part A and wants B while transaction 2 holds B and wants A. Postgres detects this and kills one of them. The standard prevention is a **global lock order**: everyone locks in SKU order, so a cycle can't form. The SQL test fired 30 reservations at once, half listing the parts in reverse, with no deadlocks.

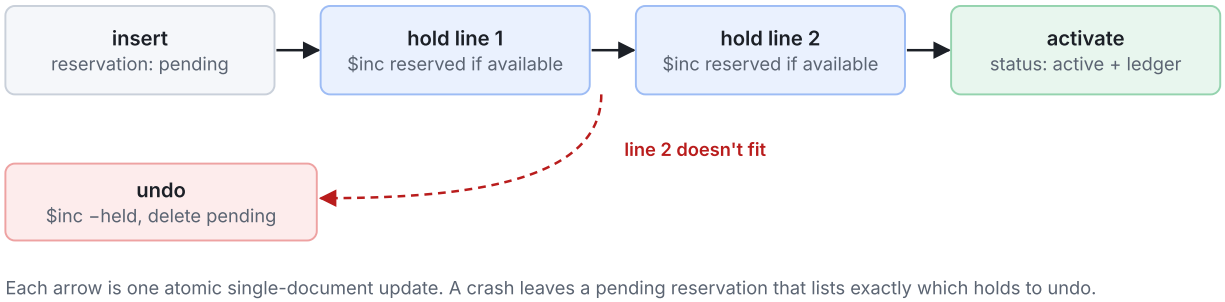
> Postgres's stricter isolation levels (REPEATABLE READ, SERIALIZABLE) can also prevent these anomalies, by aborting one of the conflicting transactions with a serialization error that the application must retry. Explicit locks avoid the retry loop, at the cost of making requests wait.

CONCEPTS · 5.4

Atomicity in MongoDB

MongoDB guarantees that a write to **one document** is atomic, including all the fields it changes and the filter it matched. Multi-document transactions exist too, but only on a replica set or sharded cluster, not a single standalone server.

So the MongoDB repository leans on single-document atomicity. Each part is one document, and each rule goes in the filter: `updateOne({ _id: sku, $expr: { $gte: [onHand − reserved, qty] } }, { $inc: { reserved: qty } })`. Either the document matched and was changed, or nothing happened. A reservation spans several documents, so it becomes a **saga**:



Each arrow is one atomic single-document update. A crash leaves a pending reservation that lists exactly which holds to undo.

Figure 5.4. The MongoDB reservation saga. Without a multi-document transaction, all-or-nothing is built from atomic steps plus compensation.

A saga gives atomicity eventually rather than instantly: between steps, other requests can see a partial hold. The invariant that matters (never promise more than is on the shelf) holds at every moment; the cost is that a racing request might be refused while a hold that's about to be undone is in place.

CONCEPTS · 5.5

Repositories and contract tests

The **repository pattern** hides storage behind an interface shaped like the domain (`reserve`, `consume`), not like the database (`query`, `find`). Routes, the service and the UI never learn which database is in use.

An interface alone promises method names, not behaviour. A **contract test suite** is what makes the promise real: one set of tests, written against the interface, run against every implementation. If MongoDB returned shortages in a different shape, or forgot to write the ledger on release, its run would fail while Postgres's passed.

Keeping an in-memory implementation is worth its 140 lines: it defines the intended behaviour with no concurrency or I/O in the way, makes the API tests fast and runnable anywhere, and lets the whole app run with no database for a demo.

CONCEPTS · 5.6

Cut lists and bin packing

How many 3 m bars does a cut list need? That's the **bin-packing problem**: pack items of given sizes into the fewest bins of fixed capacity. Finding the true minimum is NP-hard, so practical tools use heuristics.

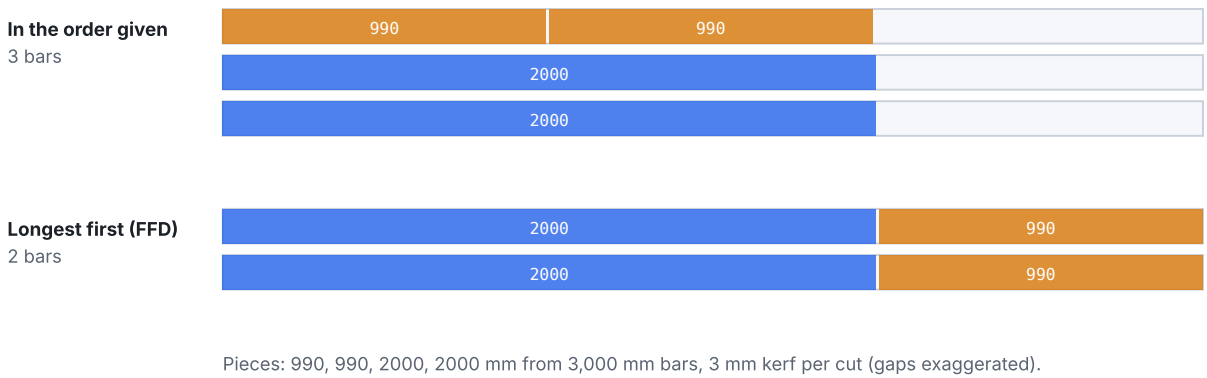


Figure 5.5. First-fit decreasing on a cut list. Placing big pieces first leaves gaps the small ones fill.

First-fit decreasing (FFD) sorts pieces longest first and puts each in the first bar with room. It's simple, fast (n log n for the sort plus a scan), and provably never needs more than about 11/9 of the optimal number of bars (plus a small constant). The kerf is what makes this real rather than textbook: every cut loses 3 mm, so two 1,500 mm pieces need two bars.

Industrial cutting software also handles multiple stock lengths, offcuts kept for later, and grouping cuts to reduce saw setups. Those are optimisation problems in their own right (the “cutting stock problem”), usually solved with linear programming.

CONCEPTS · 5.7

Choosing the database at deploy time

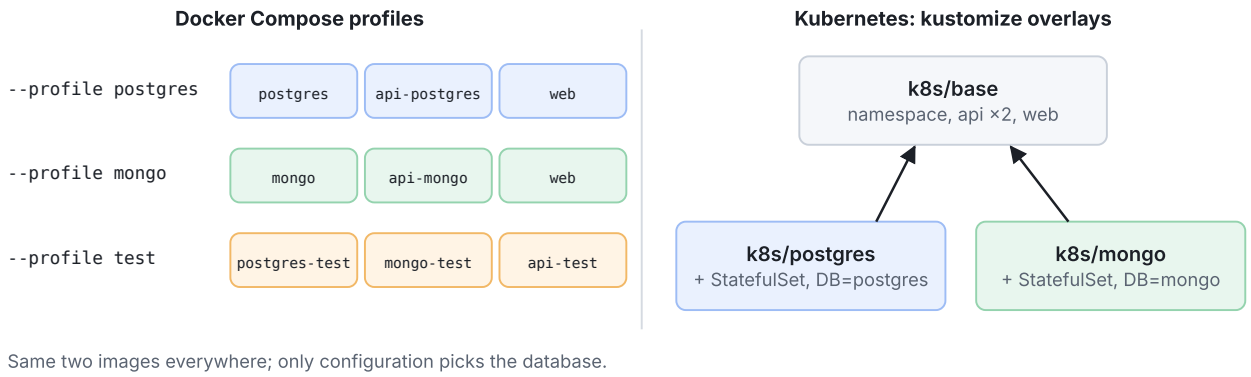


Figure 5.6. How the database is chosen at deploy time, in Compose and in Kubernetes.

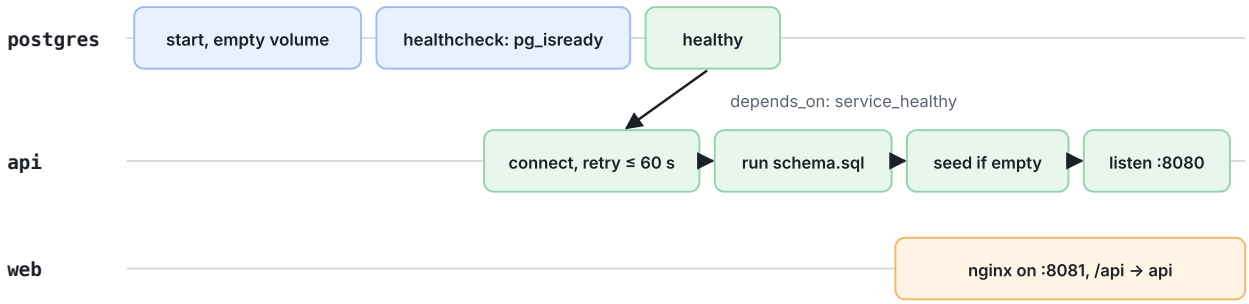
Configuration comes from the environment, the **twelve-factor** rule: the same image runs in Compose, Kubernetes and tests, and only environment variables differ. Compose selects services with **profiles**; Kubernetes selects with **kustomize overlays** that add a database StatefulSet and generate the API's settings on top of a shared base.

Databases run as **StatefulSets**, not Deployments: each pod gets a stable name (`postgres-0`) and its own persistent volume that follows it across restarts. The API's readiness probe checks the database, so Kubernetes only routes traffic to pods that can actually serve it.

CONCEPTS · 5.8

Setting up the database

There's no separate install or migration step. Docker Compose starts the database, and the API sets it up the first time it connects. The whole sequence, on an empty machine:



Every step can run again safely: tables are CREATE ... IF NOT EXISTS, and seeding only fills an empty database.

Figure 5.7. What happens on the first docker compose --profile postgres up. MongoDB follows the same sequence with mongosh as its healthcheck.

1. **Compose starts PostgreSQL** with a named volume, `pg-data`, so the data survives restarts and rebuilds. The official image creates the `inventory` user and database from its environment variables on first boot.
2. **Compose waits for the healthcheck.** `pg_isready` runs every 5 s. The API container only starts once it passes (`depends_on: condition: service_healthy`).
3. **The API connects**, retrying every 2 s for up to a minute. In Kubernetes there's no `depends_on`, so this retry loop, plus the startup probe that allows 150 s, is what makes ordering not matter.
4. **The API runs `schema.sql`**: four tables, their constraints and indexes, and the two functions. Every statement is `CREATE … IF NOT EXISTS` or `CREATE OR REPLACE`, so it's safe on every start, and two API replicas starting together don't conflict.
5. **The API seeds the stock**, only if `parts` is empty: 16 parts in one transaction. Restarting never duplicates stock, and your changes survive.
6. **The API starts listening**, the readiness probe passes, and nginx starts receiving answers on port 8081.

I ran exactly this schema and seed against a real PostgreSQL 16 and then used it the way the app does. This is the session, trimmed, with the user and database names shown as they are under Docker (`inventory`):

psql: the database after first start

```
inventory=> \dt
 Schema |       Name        | Type  | Owner
--------+-------------------+-------+-----------
 public | movements         | table | inventory
 public | parts             | table | inventory
 public | reservation_lines | table | inventory
 public | reservations      | table | inventory

inventory=> SELECT sku, on_hand, reserved, on_hand - reserved AS available, reorder_level
            FROM parts WHERE on_hand - reserved <= reorder_level;
     sku      | on_hand | reserved | available | reorder_level
--------------+---------+----------+-----------+---------------
 BAR-4040-BLK |       7 |        0 |         7 |             8
 M5-BOLT      |      22 |        0 |        22 |            25
 PL-300       |       8 |        0 |         8 |            10
```

The three low-stock parts are deliberate, so the low-stock filter has something to show on day one. Then a reservation that fits, one that doesn't, and an attempt to break the rules directly:

psql: reserving, and the safety net

```
inventory=> SELECT inv_reserve('Q-1001',
              '[{"sku":"LA-300","qty":2},{"sku":"MC-1","qty":1},{"sku":"PL-200","qty":4}]');
 {"id": "a8ca2dfa-b176-403f-9d62-71a9b00ed112", "ok": true}

inventory=> SELECT inv_reserve('Q-1002', '[{"sku":"LA-300","qty":8}]');
 {"ok": false, "shortages": [{"sku": "LA-300", "need": 8, "known": true, "available": 4}]}

inventory=> SELECT id, sku, kind, qty, note FROM movements;
 id |  sku   |  kind   | qty |        note
----+--------+---------+-----+---------------------
  1 | LA-300 | reserve |   2 | Reserved for Q-1001
  2 | MC-1   | reserve |   1 | Reserved for Q-1001
  3 | PL-200 | reserve |   4 | Reserved for Q-1001

inventory=> UPDATE parts SET reserved = 99 WHERE sku = 'MC-1';
ERROR:  new row for relation "parts" violates check constraint "reserved_within_stock"
```

The refused reservation changed nothing and wrote no ledger lines. The last statement bypasses the application entirely, and the database still refuses: that's what “rules in the database” means.

To look inside your own database while the app runs:

PowerShell, from vention/snack5-inventory

```
# PostgreSQL
docker compose --profile postgres exec postgres psql -U inventory -d inventory
#   then e.g.  SELECT * FROM movements ORDER BY id DESC LIMIT 10;

# MongoDB
docker compose --profile mongo exec mongo mongosh -u inventory -p inventory-dev-password --
authenticationDatabase admin inventory
#   then e.g.  db.parts.find({ $expr: { $lte: [{ $subtract: ["$onHand", "$reserved"] },
"$reorderLevel"] } })

# Start again from the seed stock (deletes that database's volume)
docker compose --profile postgres down -v
```

CONCEPTS · 5.9

Two apps, one database

The robot arm app (Chapter 1) now stores its scene in this database: shapes on the floor, some of them standing for inventory parts. Two front ends sharing one API is common, and it raises two questions this chapter's design already answers.

- **Where do rules live?** In the API and the database, not in either front end. The arm app checks sizes and SKUs as you type for convenience; the API checks again, and Postgres's constraints and foreign key are the final word.
- **How do browsers on other origins call it?** With CORS headers from the API, allowed for pages on localhost by default.
- **What links the data?** The SKU. A shape for a part carries it, so the arm app can show “2 in scene, 1 on hand”. Placing a part in the scene doesn't change stock: a scene is a layout, and holding stock is a reservation's job.

CHAPTER 5

Line by line

16 parts, in the order data flows through the code.

PART 5.1 · `api/src/domain/types.ts`

The data model and the one interface

Everything starts with the types: what a part, a movement and a reservation are, and the single interface every database has to implement.

Parts

api/src/domain/types.ts · lines 6–30

```
/**
 * The inventory's data model and the one interface every database implements.
 *
 * Quantities are whole stock units: bars, plates, packs or pieces, as given by `unit`.
 * `available` is what can still be promised: on hand minus what's reserved for quotes.
 */

export type Category = "extrusion" | "plate" | "fastener" | "motor" | "actuator" | "controller"
| "gripper";

export interface Part {
  sku: string;
  name: string;
  category: Category;
  /** What one unit of stock is, e.g. "bar (3 m)" or "pack of 50". */
  unit: string;
  onHand: number;
  reserved: number;
  available: number;
  /** Reorder when available falls to this level or below. */
  reorderLevel: number;
  low: boolean;
  /** Bin location in the warehouse, e.g. "A-02-1". */
  location: string;
```

```
  updatedAt: string;
}
```

A part's stock is three numbers: **on hand** (physically on the shelf), **reserved** (promised to quotes) and **available** = on hand − reserved. Available is what sales can still promise, so it's what every check uses. `Category` is a string literal union, like Snack 1's `Axis`: a typo is a compile error.

`available` and `low` are **derived**: no database stores them. They're computed in one place (`withDerived`, below), so the three implementations can't disagree about what “low stock” means.

Seed parts and movements

`api/src/domain/types.ts · lines 32–54`

```
/** A part as it's first stocked, before anything is reserved. */
export interface SeedPart {
  sku: string;
  name: string;
  category: Category;
  unit: string;
  onHand: number;
  reorderLevel: number;
  location: string;
}

export type MovementKind = "receive" | "consume" | "reserve" | "release" | "fulfil";

/** One line of the stock ledger. Every change to a part writes one. */
export interface Movement {
  id: string;
  sku: string;
  kind: MovementKind;
  qty: number;
  note: string;
  reservationId: string | null;
  at: string;
}
```

`SeedPart` is a part before anything has happened to it: no reservations, no timestamp. `Movement` is one line of the **stock ledger**. Every change writes one, and they're never edited, so the current numbers can always be explained by replaying the ledger. Accountants call this an append-only journal; event-sourced systems take the same idea further and store only the events.

Lines, reservations and shortages

api/src/domain/types.ts · lines 56–80

```
export interface Line {
  sku: string;
  qty: number;
}

export type ReservationStatus = "active" | "released" | "fulfilled";

export interface Reservation {
  id: string;
  quoteRef: string;
  status: ReservationStatus;
  lines: Line[];
  createdAt: string;
  closedAt: string | null;
}

/** Why a reservation couldn't be made: `available` is 0 and `known` false for SKUs not stocked
at all. */
export interface Shortage {
  sku: string;
  need: number;
  available: number;
  known: boolean;
}

export type ReserveResult = { ok: true; reservation: Reservation } | { ok: false; shortages:
Shortage[] };
```

A `Line` is a SKU and a quantity, the common currency of the whole system: a design becomes lines, a reservation holds lines, a shortage reports a line that didn't fit. `known: false` distinguishes “we stock it but ran out” from “we don't stock that SKU at all”, which a sales person handles very differently.

`ReserveResult` is a **discriminated union**: check `ok` and TypeScript narrows to either `reservation` or `shortages`. Running out of stock is an expected outcome, not an exception, so it's a return value.

The repository interface

api/src/domain/types.ts · lines 82–163

```
export interface PartFilter {
  /** Case-insensitive match on SKU or name. */
  search?: string;
  /** Only parts at or below their reorder level. */
  lowOnly?: boolean;
}

export type DbKind = "postgres" | "mongo" | "memory";

/**
 * Everything the API needs from a database. Postgres, MongoDB and the in-memory
 * reference implementation all pass the same contract tests (tests/contract.ts).
 *
 * Rules every implementation enforces:
 * - stock never goes negative, and reserved never exceeds on hand, even under concurrent
requests;
 * - consuming can only use available stock, never stock reserved for someone else;
 * - a reservation is all or nothing: if any line is short, nothing is reserved;
 * - every change writes a movement.
 */
export interface InventoryRepo {
  readonly kind: DbKind;
  /** Creates tables/indexes if missing. Safe to call on every start. */
  init(): Promise<void>;
  /** Inserts the seed parts if there are none yet. Returns whether it did. */
  seedIfEmpty(parts: readonly SeedPart[]): Promise<boolean>;
  /** Throws if the database can't be reached. */
  ping(): Promise<void>;
  close(): Promise<void>;

  listParts(filter?: PartFilter): Promise<Part[]>;
  getParts(skus: readonly string[]): Promise<Part[]>;
  /**
   * Adds a new part. Initial stock is written to the ledger as a receipt, so the
   * ledger still explains every unit. Throws ConflictError if the SKU already exists.
   */
  createPart(part: SeedPart): Promise<Part>;
  /** Changes name, category, location or reorder level; never stock. Throws NotFoundError for
```

```
an unknown SKU. */
  updatePart(sku: string, patch: PartPatch): Promise<Part>;
  /** Adds stock. Throws NotFoundError for an unknown SKU. */
  receive(sku: string, qty: number, note: string): Promise<Part>;
  /** Removes available stock. Throws NotFoundError, or ConflictError if not enough is
available. */
  consume(sku: string, qty: number, note: string): Promise<Part>;
  /** Newest first. */
  movements(filter: { sku?: string; limit: number }): Promise<Movement[]>;

  /** All or nothing. Lines with the same SKU are added together. */
  reserve(quoteRef: string, lines: readonly Line[]): Promise<ReserveResult>;
  /** Newest first. */
  reservations(limit: number): Promise<Reservation[]>;
  /** Gives reserved stock back. Throws NotFoundError, or ConflictError unless active. */
  release(id: string): Promise<Reservation>;
  /** Ships reserved stock: on hand and reserved both go down. Same errors as release. */
  fulfil(id: string): Promise<Reservation>;

  /** Shapes in the robot arm's scene, oldest first. */
  listShapes(): Promise<Shape[]>;
  createShape(shape: NewShape): Promise<Shape>;
  /** Throws NotFoundError for an unknown id. */
  updateShape(id: string, patch: ShapePatch): Promise<Shape>;
  /** Throws NotFoundError for an unknown id. */
  deleteShape(id: string): Promise<void>;

  /** Conveyor belts in the robot arm's scene, oldest first. */
  listConveyors(): Promise<Conveyor[]>;
  createConveyor(conveyor: NewConveyor): Promise<Conveyor>;
  /** Throws NotFoundError for an unknown id. */
  updateConveyor(id: string, patch: ConveyorPatch): Promise<Conveyor>;
  /** Throws NotFoundError for an unknown id. */
  deleteConveyor(id: string): Promise<void>;

  /** Splitters (diverter gates on belts), oldest first. Deleting a belt deletes its splitters.
*/
  listSplitters(): Promise<Splitter[]>;
  /** The caller checks the belt exists and `along` fits on it. */
  createSplitter(splitter: NewSplitter): Promise<Splitter>;
  /** Throws NotFoundError for an unknown id. */
```

```
  updateSplitter(id: string, patch: SplitterPatch): Promise<Splitter>;
  /** Throws NotFoundError for an unknown id. */
  deleteSplitter(id: string): Promise<void>;

  /** Deletes everything. Tests only. */
  reset(): Promise<void>;
}
```

This is the **repository pattern**: the rest of the code talks to `InventoryRepo`, never to a database driver. The doc comment is the real specification. Four rules hold for every implementation, and the contract tests check each of them on each database.

Everything returns a `Promise`, even in the in-memory version, which could answer synchronously. An interface that's async everywhere lets any implementation do I/O without changing a single caller.

Errors are part of the contract too: “unknown SKU” is a `NotFoundError` and “not enough” is a `ConflictError` in every implementation, so the HTTP layer maps them to 404 and 409 without knowing where they came from.

GOING DEEPER

Ledgers, event sourcing and audit

The movements table is an append-only ledger beside the current counts. **Event sourcing** goes one step further: store only the events, and compute current stock by replaying them (with snapshots for speed). It gives a perfect audit trail and time travel (“what did we have on March 3?”), at the cost of more complex reads. A ledger next to counters, as here, gets most of the audit benefit with simple queries.

PART 5.2 · `api/src/domain/errors.ts`

Errors and shared helpers

Three error classes carry their HTTP status, and two small helpers keep the implementations consistent.

Errors that know their status

api/src/domain/errors.ts · lines 1–19

```
/** Errors the HTTP layer turns into status codes. */

export class NotFoundError extends Error {
  readonly status = 404;
}

export class ConflictError extends Error {
  readonly status = 409;
}

export class ValidationError extends Error {
  readonly status = 400;
  readonly problems: string[];

  constructor(problems: string[]) {
    super(problems.join("; "));
    this.problems = problems;
  }
}
```

`readonly status = 404` is a class field, so every `NotFoundError` has it. The HTTP handler reads `err.status` and doesn't need a table of error types. `ValidationError` collects every problem with a request, so a user fixing a design sees all its mistakes at once instead of one per attempt.

Merging lines, and derived fields

api/src/domain/lines.ts · lines 1–14

```
import type { Line, Part } from "./types.ts";

/** Adds up lines with the same SKU and sorts by SKU (the order rows are locked in, to avoid
deadlocks). */
export function mergeLines(lines: readonly Line[]): Line[] {
  const total = new Map<string, number>();
  for (const { sku, qty } of lines) total.set(sku, (total.get(sku) ?? 0) + qty);
  return [...total].map(([sku, qty]) => ({ sku, qty })).sort((a, b) => (a.sku < b.sku ? -1 :
a.sku > b.sku ? 1 : 0));
}

/** Derived fields, computed the same way for every database. */
export function withDerived(p: Omit<Part, "available" | "low">): Part {
  const available = p.onHand - p.reserved;
  return { ...p, available, low: available <= p.reorderLevel };
}
```

`mergeLines` adds up repeated SKUs and **sorts by SKU**. The sort isn't cosmetic: it's the order every database locks rows in, which is what prevents deadlocks (chapter concept 5.2). Merging first also means a design listing PL-200 twice is checked against its total, not each half separately.

Postgres sorts SKUs with its own collation, which can order punctuation differently from JavaScript's `<`. That's harmless: in Postgres, every lock is taken by Postgres's own `ORDER BY sku`, so all transactions agree; in MongoDB, this JavaScript order is the only one used. What matters is one consistent order per database, not the same order everywhere.

PART 5.3 · `api/src/domain/design.ts`

From a machine design to stock

Snack 4 prices a design; this file works out what stock it consumes. Most of it is bookkeeping, but one function is a classic algorithm: packing cut pieces into stock bars.

The input format

api/src/domain/design.ts · lines 1–28

```
/**
 * Turns a machine design, in the same shape Snack 4's quote calculator takes, into the
```

```
 * stock it needs.
 *
 * Most items map one to one (4 × PL-200 plates need 4 plates). Two don't:
 * - fasteners are stocked in packs of 50, so 120 bolts need 3 packs;
 * - extrusions are cut from 3 m stock bars, so the question is how many bars the
 *   pieces fit into. That's the bin-packing problem; first-fit decreasing never uses more
 *   than about 11/9 of the optimal number of bars, and on short cut lists is usually optimal.
 */
import type { Line } from "./types.ts";
import { mergeLines } from "./lines.ts";
import { ValidationError } from "./errors.ts";

/** Same item shapes as snack4-refactor-kata/src/types.ts. */
export interface DesignItem {
  type: string;
  sku: string;
  qty: number;
  profile?: string;
  /** mm, extrusions only */
  length?: number;
  finish?: "clear" | "black";
}

export interface Design {
  items: DesignItem[];
}
```

The item shape is copied from Snack 4's `types.ts`, deliberately. The two projects talk about the same machine, so a quote and a stock check use the same vocabulary. A shared npm package would be the next step if they lived in one codebase.

Constants and results

api/src/domain/design.ts · lines 30–50

```
export const BAR_LENGTH_MM = 3000;
/** Material lost to the saw blade per cut. */
export const KERF_MM = 3;
export const MIN_PIECE_MM = 50;
export const FASTENER_PACK = 50;
export const PROFILES = ["20x20", "40x40", "40x80"];

export interface Requirement extends Line {
  /** Where the number came from, for people: "120 bolts in packs of 50". */
  detail: string;
}

export interface Plan {
  requirements: Requirement[];
  /** Items that can't be turned into stock (unknown profile, piece too short). */
  warnings: string[];
}

export function barSku(profile: string, finish: string | undefined): string {
  return `BAR-${profile.replace("x", "")}-${finish === "black" ? "BLK" : "CLR"}`;
}
```

Stock bars are 3 m. The **kerf** is the width of material the saw blade turns into chips: 3 mm per cut. Ignore it and two 1,500 mm pieces seem to fit in one bar, but the cut between them needs 3 mm the bar doesn't have.

`barSku` derives the stock SKU from the design item: a black 40×80 piece is cut from `BAR-4080-BLK`. Each `Requirement` keeps a `detail` string, “4 × 1200 mm cut from 2 × 3 m bars”, because a number nobody can explain is a number nobody trusts.

Splicing long pieces

api/src/domain/design.ts · lines 52–68

```
/**
 * First-fit decreasing: sort pieces longest first, put each into the first bar with room.
 * Each cut also loses one saw kerf. Pieces longer than a bar are spliced: whole bars
 * plus a remainder piece that's packed like any other.
 */
export function barsNeeded(piecesMm: readonly number[], barMm = BAR_LENGTH_MM, kerfMm =
KERF_MM): number {
  let wholeBars = 0;
  const pieces: number[] = [];
  for (const len of piecesMm) {
    if (len > barMm) {
      wholeBars += Math.floor(len / barMm);
      const rest = len % barMm;
      if (rest > 0) pieces.push(rest);
    } else {
      pieces.push(len);
    }
  }
```

A piece longer than a bar is joined from several. Snack 4 adds a splice fee; here, a 4,000 mm piece needs one whole bar plus a 1,000 mm remainder, and the remainder goes into the packing like any other piece, so it can share a bar.

First-fit decreasing

api/src/domain/design.ts · lines 69–79

```
  pieces.sort((a, b) => b - a);
  const free: number[] = []; // remaining length in each opened bar
  for (const len of pieces) {
    // A piece fits if the bar has its length left. Cutting it off costs a kerf, unless
    // it takes the rest of the bar, so the leftover is never negative.
    const i = free.findIndex((room) => room >= len);
    if (i >= 0) free[i] = Math.max(0, free[i] - len - kerfMm);
    else free.push(Math.max(0, barMm - len - kerfMm));
  }
  return wholeBars + free.length;
}
```

`free` holds the length left in each bar opened so far. For each piece, longest first, `findIndex` finds the first bar with room; if none has room, a new bar is opened. The answer is how many bars were opened.

The kerf rule is in one expression: a piece fits if the bar has the piece's length left, and cutting it off costs the piece plus a kerf, floored at zero. So a piece that uses exactly the rest of the bar needs no cut, which is right: there's nothing left to cut it from.

Sorting longest first is what makes the heuristic good. Big pieces are placed while there's freedom, and small pieces fill the gaps they leave. Without the sort, two 990 mm pieces can take a bar together, leaving no bar to share with a 2,000 mm piece; one test pins exactly that case.

Turning items into requirements

api/src/domain/design.ts · lines 81–111

```
export function plan(design: Design): Plan {
  const warnings: string[] = [];
  const cutLists = new Map<string, number[]>(); // bar SKU -> piece lengths
  const cutText = new Map<string, string[]>();
  const direct: Requirement[] = [];

  for (const item of design.items) {
    if (item.type === "extrusion") {
      const profile = item.profile ?? "";
      const length = item.length ?? 0;
      if (!PROFILES.includes(profile)) {
        warnings.push(`${item.sku}: unknown profile ${profile || "(none)"}, not checked`);
        continue;
      }
      if (length < MIN_PIECE_MM) {
        warnings.push(`${item.sku}: ${length} mm is below the ${MIN_PIECE_MM} mm minimum, not
checked`);
        continue;
      }
      const sku = barSku(profile, item.finish);
      const list = cutLists.get(sku) ?? [];
      for (let i = 0; i < item.qty; i++) list.push(length);
      cutLists.set(sku, list);
      cutText.set(sku, [...(cutText.get(sku) ?? []), `${item.qty} × ${length} mm`]);
    } else if (item.type === "fastener") {
      const packs = Math.ceil(item.qty / FASTENER_PACK);
      direct.push({ sku: item.sku, qty: packs, detail: `${item.qty} pcs in packs of
```

```
${FASTENER_PACK}` });
    } else {
      // plates, motors, actuators, controllers, and anything else stocked by SKU
      direct.push({ sku: item.sku, qty: item.qty, detail: `${item.qty} × ${item.type}` });
    }
  }
```

Extrusions are collected into one **cut list per stock SKU**, across items: two different design items both cut from 40×40 clear bars share bars. Fasteners round up to packs. Everything else maps one to one by SKU, including item types Snack 4 doesn't price, like a gripper, so the inventory still checks them.

Items that can't be turned into stock (an unknown profile, a piece under 50 mm) become **warnings**, not errors: the rest of the design can still be checked, and the user sees what was skipped.

api/src/domain/design.ts · lines 113–123

```
  const bars: Requirement[] = [...cutLists].map(([sku, pieces]) => {
    const qty = barsNeeded(pieces);
    return { sku, qty, detail: `${cutText.get(sku)!.join(" + ")} cut from ${qty} × 3 m bar${qty
=== 1 ? "" : "s"}` };
  });

  // Merge requirements for the same SKU, keeping every detail.
  const details = new Map<string, string[]>();
  for (const r of [...bars, ...direct]) details.set(r.sku, [...(details.get(r.sku) ?? []),
r.detail]);
  const requirements = mergeLines([...bars, ...direct]).map((l) => ({ ...l, detail:
details.get(l.sku)!.join("; ") }));
  return { requirements, warnings };
}
```

Each cut list is packed, then all requirements are merged by SKU, joining their details. The result is sorted, deduplicated, and explains itself.

Validating untrusted input

api/src/domain/design.ts · lines 125–154

```
/** Validates untrusted JSON into a Design, collecting every problem rather than stopping at
the first. */
export function parseDesign(input: unknown): Design {
  const problems: string[] = [];
  const items = (input as { items?: unknown } | null)?.items;
```

```
  if (!Array.isArray(items) || items.length === 0) throw new ValidationError(["design.items
must be a non-empty array"]);
  if (items.length > 500) throw new ValidationError(["design.items has more than 500 items"]);

  const out: DesignItem[] = [];
  items.forEach((raw: unknown, i: number) => {
    const it = raw as Record<string, unknown> | null;
    const where = `items[${i}]`;
    if (!it || typeof it !== "object") return problems.push(`${where} must be an object`);
    const { type, sku, qty } = it;
    if (typeof type !== "string" || !type) problems.push(`${where}.type must be a string`);
    if (typeof sku !== "string" || !isSku(sku)) problems.push(`${where}.sku must be a SKU like
"PL-200"`);
    if (!isQty(qty)) problems.push(`${where}.qty must be a whole number from 1 to 100000`);
    if (type === "extrusion") {
      if (typeof it.profile !== "string") problems.push(`${where}.profile is required for
extrusions`);
      if (typeof it.length !== "number" || !Number.isFinite(it.length) || it.length <= 0 ||
it.length > 20000)
        problems.push(`${where}.length must be a length in mm up to 20000`);
      if (it.finish !== undefined && it.finish !== "clear" && it.finish !== "black")
problems.push(`${where}.finish must be "clear" or "black"`);
    }
    out.push({
      type: String(type), sku: String(sku), qty: Number(qty),
      ...(type === "extrusion" ? { profile: String(it.profile), length: Number(it.length),
finish: it.finish as DesignItem["finish"] } : {}),
    });
  });
  if (problems.length) throw new ValidationError(problems);
  return { items: out };
}
```

A design arrives as JSON from a browser, so nothing about it can be assumed. `parseDesign` checks each field and **collects** problems with their path (`items[3].qty`) instead of stopping at the first. Limits matter too: at most 500 items and 100,000 per line, so one request can't make the server pack a billion pieces.

The output is built fresh from checked fields rather than passing the input through, so unexpected extra properties never reach the database.

api/src/domain/design.ts · lines 156–162

```
export function isSku(s: unknown): s is string {
  return typeof s === "string" && /^[A-Z0-9][A-Z0-9-]{1,31}$/.test(s);
}

export function isQty(n: unknown): n is number {
  return typeof n === "number" && Number.isInteger(n) && n >= 1 && n <= 100_000;
}
```

Type predicates (`s is string`) let TypeScript narrow the type after the check, so callers don't need casts.

PART 5.4 · `api/src/db/memory.ts`

The reference implementation

The in-memory repository is about 140 lines of maps and arrays. It defines what “correct” means; the database versions have to behave exactly like it.

Why it's atomic for free

api/src/db/memory.ts · lines 1–32

```
/**
 * Reference implementation: plain maps in one process. It defines the behaviour the
 * contract tests expect, runs the API with no database (DB=memory), and is what the
 * API tests use. JavaScript runs one callback at a time and nothing here awaits midway,
 * so every method is atomic for free.
 */
import { randomUUID } from "node:crypto";
import { ConflictError, NotFoundError } from "../domain/errors.ts";
import { mergeLines, withDerived } from "../domain/lines.ts";
import type {
  InventoryRepo, Line, Movement, MovementKind, Part, PartFilter, Reservation, ReserveResult,
SeedPart, Shortage,
} from "../domain/types.ts";
import type { NewShape, Shape, ShapePatch } from "../domain/shapes.ts";
import type { Conveyor, ConveyorPatch, NewConveyor } from "../domain/conveyors.ts";
import type { NewSplitter, Splitter, SplitterPatch } from "../domain/splitters.ts";
import type { PartPatch } from "../domain/newPart.ts";
```

```
interface PartRow { sku: string; name: string; category: Part["category"]; unit: string;
onHand: number; reserved: number; reorderLevel: number; location: string; updatedAt: string }

export class MemoryRepo implements InventoryRepo {
  readonly kind = "memory";
  private parts = new Map<string, PartRow>();
  private moves: Movement[] = [];
  private res = new Map<string, Reservation>();
  private seq = 0;
  private shapes = new Map<string, Shape>();
  private conveyors = new Map<string, Conveyor>();
  private splitters = new Map<string, Splitter>();

  async init(): Promise<void> {}
  async ping(): Promise<void> {}
  async close(): Promise<void> {}
```

JavaScript runs one piece of code at a time on its event loop. A method that never `await`s in the middle can't be interleaved with another request, so every method here is atomic without any locking. That's precisely what a real database has to work to provide, and why this is a good reference: it shows the intended behaviour with no concurrency noise.

Reading

api/src/db/memory.ts · lines 34–52

```
async seedIfEmpty(seed: readonly SeedPart[]): Promise<boolean> {
  if (this.parts.size > 0) return false;
  const now = new Date().toISOString();
  for (const s of seed) this.parts.set(s.sku, { ...s, reserved: 0, updatedAt: now });
  return true;
}

async listParts(filter: PartFilter = {}): Promise<Part[]> {
  const q = filter.search?.trim().toLowerCase();
  return [...this.parts.values()]
    .map(out)
    .filter((p) => !q || p.sku.toLowerCase().includes(q) || p.name.toLowerCase().includes(q))
    .filter((p) => !filter.lowOnly || p.low)
    .sort((a, b) => (a.sku < b.sku ? -1 : 1));
}

async getParts(skus: readonly string[]): Promise<Part[]> {
  return skus.flatMap((s) => (this.parts.has(s) ? [out(this.parts.get(s)!)] : []));
}
```

Seeding only happens on an empty store, so restarting the API never duplicates stock. Listing applies the same filters as the SQL and Mongo queries, and every result goes through `out()`, which adds the derived fields.

Reserving, all or nothing

api/src/db/memory.ts · lines 92–113

```
async reserve(quoteRef: string, lines: readonly Line[]): Promise<ReserveResult> {
  const want = mergeLines(lines);
  const shortages: Shortage[] = [];
  for (const { sku, qty } of want) {
    const p = this.parts.get(sku);
    const available = p ? p.onHand - p.reserved : 0;
    if (available < qty) shortages.push({ sku, need: qty, available, known: p !== undefined });
  }
  if (shortages.length) return { ok: false, shortages };

  const reservation: Reservation = {
    id: randomUUID(), quoteRef, status: "active", lines: want, createdAt: new
Date().toISOString(), closedAt: null,
  };
  this.res.set(reservation.id, reservation);
  for (const { sku, qty } of want) {
    const p = this.parts.get(sku)!;
    p.reserved += qty;
    this.touch(p);
    this.log(sku, "reserve", qty, `Reserved for ${quoteRef}`, reservation.id);
  }
  return { ok: true, reservation: structuredClone(reservation) };
}
```

Two passes: first check every line, and only if all fit, apply them all. With no `await` between the passes, no other request can change stock in between. The returned reservation is a `structuredClone`, so a caller mutating it can't reach into the store, the same isolation a database gives by returning copies of rows.

Closing a reservation

api/src/db/memory.ts · lines 204–218

```
private close_(id: string, action: "release" | "fulfil"): Reservation {
  const r = this.res.get(id);
  if (!r) throw new NotFoundError(`No reservation ${id}`);
  if (r.status !== "active") throw new ConflictError(`Reservation is already ${r.status}`);
  for (const { sku, qty } of r.lines) {
    const p = this.parts.get(sku)!;
    p.reserved -= qty;
    if (action === "fulfil") p.onHand -= qty;
    this.touch(p);
    this.log(sku, action, qty, `${action === "fulfil" ? "Shipped" : "Released"} for
${r.quoteRef}`, id);
  }
  r.status = action === "fulfil" ? "fulfilled" : "released";
  r.closedAt = new Date().toISOString();
  return structuredClone(r);
}
```

Release and fulfil share one function: both remove the reservation's hold, and fulfil also takes the stock off the shelf. Only an active reservation can be closed, so a double-click can't release the same stock twice.

PART 5.5 · `api/src/db/schema.sql`

The PostgreSQL schema

Tables, constraints and two functions. The database enforces the stock rules itself, so they hold even if the application has a bug.

Parts, with the rules as constraints

api/src/db/schema.sql · lines 1–17

```
-- Inventory schema for PostgreSQL. Run on every API start: everything is idempotent.
--
-- The database itself enforces the stock rules, so no bug or race in the API can break them:
--   on_hand >= 0, reserved >= 0, reserved <= on_hand.

CREATE TABLE IF NOT EXISTS parts (
  sku            text PRIMARY KEY,
  name           text NOT NULL,
  category       text NOT NULL,
  unit           text NOT NULL,
  on_hand        integer NOT NULL DEFAULT 0 CHECK (on_hand >= 0),
  reserved       integer NOT NULL DEFAULT 0 CHECK (reserved >= 0),
  reorder_level  integer NOT NULL DEFAULT 0 CHECK (reorder_level >= 0),
  location       text NOT NULL DEFAULT '',
  updated_at     timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT reserved_within_stock CHECK (reserved <= on_hand)
);
```

`CREATE TABLE IF NOT EXISTS` makes the script safe to run on every start. The `CHECK` constraints are the last line of defence: whatever the code does, Postgres rejects any statement that would leave stock negative or reserve more than is on hand, and the whole transaction rolls back.

Reservations, their lines, and the ledger

api/src/db/schema.sql · lines 19–46

```
CREATE TABLE IF NOT EXISTS reservations (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  quote_ref   text NOT NULL,
  status      text NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'released',
'fulfilled')),
  created_at  timestamptz NOT NULL DEFAULT clock_timestamp(),
  closed_at   timestamptz
);

CREATE TABLE IF NOT EXISTS reservation_lines (
  reservation_id  uuid NOT NULL REFERENCES reservations (id) ON DELETE CASCADE,
  sku             text NOT NULL REFERENCES parts (sku),
  qty             integer NOT NULL CHECK (qty > 0),
```

```
  PRIMARY KEY (reservation_id, sku)
);

-- The stock ledger: one row per change, never updated.
CREATE TABLE IF NOT EXISTS movements (
  id              bigserial PRIMARY KEY,
  sku             text NOT NULL REFERENCES parts (sku),
  kind            text NOT NULL CHECK (kind IN ('receive', 'consume', 'reserve', 'release',
'fulfil')),
  qty             integer NOT NULL CHECK (qty > 0),
  note            text NOT NULL DEFAULT '',
  reservation_id  uuid REFERENCES reservations (id) ON DELETE CASCADE,
  at              timestamptz NOT NULL DEFAULT clock_timestamp()
);

CREATE INDEX IF NOT EXISTS movements_by_sku ON movements (sku, id DESC);
CREATE INDEX IF NOT EXISTS reservations_by_date ON reservations (created_at DESC);
```

A reservation's lines are a separate table, the **normalized** relational design: one row per SKU per reservation, with a composite primary key so a SKU can't appear twice. Foreign keys mean a line can't reference a part that doesn't exist, and `ON DELETE CASCADE` removes lines with their reservation.

`gen_random_uuid()` is built into Postgres 13+. `clock_timestamp()` rather than `now()` because `now()` is frozen at the start of a transaction: several reservations created in one transaction would get identical timestamps and an unstable order.

The indexes match the queries: “movements for this SKU, newest first” and “reservations, newest first”. `movements_by_sku` is on `(sku, id DESC)`, so Postgres can read the newest rows for a SKU straight off the index.

Reserving: merge, validate, lock

`api/src/db/schema.sql · lines 98–124`

```
-- Reserve every line of a quote, or nothing.
--   p_lines: [{"sku": "PL-200", "qty": 4}, ...]
--   returns {"ok": true, "id": "<uuid>"} or {"ok": false, "shortages": [...]}
--
-- The rows are locked (FOR UPDATE) in SKU order before anything is checked. Two quotes
-- reserving the same parts at the same moment therefore run one after the other, and
-- always lock in the same order, so they can't deadlock.
CREATE OR REPLACE FUNCTION inv_reserve(p_quote text, p_lines jsonb) RETURNS jsonb
LANGUAGE plpgsql AS $$
```

```
DECLARE
  v_id uuid;
  v_wanted jsonb;
  v_shortages jsonb;
BEGIN
  -- Add up lines with the same SKU: [{"sku", "qty"}], sorted by SKU.
  SELECT coalesce(jsonb_agg(jsonb_build_object('sku', sku, 'qty', qty) ORDER BY sku), '[]')
    INTO v_wanted
    FROM (SELECT l ->> 'sku' AS sku, sum((l ->> 'qty')::integer) AS qty
            FROM jsonb_array_elements(p_lines) AS l GROUP BY 1) AS t;

  IF EXISTS (SELECT 1 FROM jsonb_to_recordset(v_wanted) AS w (sku text, qty integer) WHERE qty
<= 0) THEN
    RAISE EXCEPTION 'quantities must be positive' USING ERRCODE = '22023';
  END IF;

  PERFORM 1 FROM parts
    WHERE sku IN (SELECT w.sku FROM jsonb_to_recordset(v_wanted) AS w (sku text, qty integer))
    ORDER BY sku FOR UPDATE;
```

The function receives the lines as JSONB and first merges them by SKU into `v_wanted`. `jsonb_to_recordset` turns that JSON back into rows whenever it's needed, which avoids a temporary table.

Then the key statement: `SELECT … ORDER BY sku FOR UPDATE` locks every part row the reservation touches, in SKU order. Until this transaction ends, any other transaction that wants to change those rows waits. And because everyone locks in the same order, two reservations can never each hold a lock the other needs: no deadlock.

Reserving: check, then write

api/src/db/schema.sql · lines 126–150

```
  SELECT jsonb_agg(jsonb_build_object(
           'sku', w.sku, 'need', w.qty,
           'available', coalesce(p.on_hand - p.reserved, 0),
           'known', p.sku IS NOT NULL) ORDER BY w.sku)
    INTO v_shortages
    FROM jsonb_to_recordset(v_wanted) AS w (sku text, qty integer)
    LEFT JOIN parts p ON p.sku = w.sku
    WHERE p.sku IS NULL OR p.on_hand - p.reserved < w.qty;

  IF v_shortages IS NOT NULL THEN
```

```
    RETURN jsonb_build_object('ok', false, 'shortages', v_shortages);
  END IF;

  INSERT INTO reservations (quote_ref) VALUES (p_quote) RETURNING id INTO v_id;
  INSERT INTO reservation_lines (reservation_id, sku, qty)
    SELECT v_id, w.sku, w.qty FROM jsonb_to_recordset(v_wanted) AS w (sku text, qty integer);
  UPDATE parts p SET reserved = p.reserved + w.qty, updated_at = now()
    FROM jsonb_to_recordset(v_wanted) AS w (sku text, qty integer) WHERE p.sku = w.sku;
  INSERT INTO movements (sku, kind, qty, note, reservation_id)
    SELECT w.sku, 'reserve', w.qty, 'Reserved for ' || p_quote, v_id
      FROM jsonb_to_recordset(v_wanted) AS w (sku text, qty integer) ORDER BY w.sku;

  RETURN jsonb_build_object('ok', true, 'id', v_id);
END;
$$;
```

With the rows locked, the check can't go stale. A `LEFT JOIN` finds both kinds of shortage in one query: SKUs not stocked at all (`p.sku IS NULL`) and SKUs with too little available. If there are any, the function returns them and writes nothing.

Otherwise four statements create the reservation, its lines, raise `reserved`, and write the ledger. They all run in the function's transaction, so another session sees either none of it or all of it.

Closing a reservation

api/src/db/schema.sql · lines 152–187

```
-- Release (give the stock back) or fulfil (ship it) an active reservation.
-- Returns 'ok', 'not_found', or the reservation's current status if it isn't active.
CREATE OR REPLACE FUNCTION inv_close_reservation(p_id uuid, p_action text) RETURNS text
LANGUAGE plpgsql AS $$
DECLARE
  v_status text;
  v_quote text;
BEGIN
  IF p_action NOT IN ('release', 'fulfil') THEN
    RAISE EXCEPTION 'unknown action %', p_action USING ERRCODE = '22023';
  END IF;

  SELECT status, quote_ref INTO v_status, v_quote FROM reservations WHERE id = p_id FOR UPDATE;
  IF NOT FOUND THEN RETURN 'not_found'; END IF;
  IF v_status <> 'active' THEN RETURN v_status; END IF;
```

```
  PERFORM 1 FROM parts WHERE sku IN (SELECT sku FROM reservation_lines WHERE reservation_id =
p_id) ORDER BY sku FOR UPDATE;

  UPDATE parts p
     SET reserved = p.reserved - l.qty,
         on_hand = CASE WHEN p_action = 'fulfil' THEN p.on_hand - l.qty ELSE p.on_hand END,
         updated_at = now()
    FROM reservation_lines l
   WHERE l.reservation_id = p_id AND p.sku = l.sku;

  INSERT INTO movements (sku, kind, qty, note, reservation_id)
    SELECT sku, p_action, qty,
           CASE WHEN p_action = 'fulfil' THEN 'Shipped for ' ELSE 'Released for ' END ||
v_quote, p_id
      FROM reservation_lines WHERE reservation_id = p_id ORDER BY sku;

  UPDATE reservations
     SET status = CASE WHEN p_action = 'fulfil' THEN 'fulfilled' ELSE 'released' END, closed_at
= now()
   WHERE id = p_id;
  RETURN 'ok';
END;
$$;
```

Lock the reservation row first. Two simultaneous “release” requests then queue: the first changes the status; the second, once it gets the lock, sees `released` and returns that instead of releasing again. The parts are locked in SKU order, as in `inv_reserve`, before both counters change in one `UPDATE … FROM` join.

Returning a status string instead of raising an error keeps the expected outcomes (`not_found`, already closed) out of Postgres's error machinery; TypeScript turns them into the right exception.

GOING DEEPER

Why functions in the database?

Putting `inv_reserve` in PL/pgSQL makes a reservation one round trip and one transaction, with the locking right next to the data. The cost is logic in two languages and a deploy step for the database. The alternative is the same steps from TypeScript: `BEGIN`, `SELECT … FOR UPDATE`, checks, writes, `COMMIT`, on one pooled connection. Both are common; what matters is that the lock and the check happen inside one transaction.

PART 5.6 · `api/src/db/postgres-sql.ts`

The PostgreSQL repository

Every SQL statement lives in one module, and the repository is a thin layer that runs them and maps rows to the domain types.

Reading, with parameters

api/src/db/postgres-sql.ts · lines 1–14

```
/**
 * Every SQL statement PostgresRepo runs, in one dependency-free module so they can also be
 * exercised directly with psql, without the Node driver.
 */
export const Q = {
  listParts: `
    SELECT sku, name, category, unit, on_hand, reserved, reorder_level, location, updated_at
      FROM parts
     WHERE ($1::text IS NULL OR sku ILIKE '%' || $1 || '%' OR name ILIKE '%' || $1 || '%')
       AND (NOT $2::boolean OR on_hand - reserved <= reorder_level)
     ORDER BY sku`,
  getParts: `
    SELECT sku, name, category, unit, on_hand, reserved, reorder_level, location, updated_at
      FROM parts WHERE sku = ANY($1::text[]) ORDER BY sku`,
```

Every value is a **parameter** (`$1`, `$2`), never pasted into the SQL text, which is what makes SQL injection impossible: the database receives the statement and the values separately.

`($1::text IS NULL OR …)` makes a filter optional in one statement. The search uses `ILIKE`, case-insensitive `LIKE`; the repository escapes `%` and `_` first, so a user searching for “200_mm” gets that text, not a wildcard.

Changing one part: rule and ledger in one statement

api/src/db/postgres-sql.ts · lines 31–47

```
receive: `
  WITH p AS (
    UPDATE parts SET on_hand = on_hand + $2, updated_at = now() WHERE sku = $1
    RETURNING sku, name, category, unit, on_hand, reserved, reorder_level, location, updated_at
  ), m AS (
    INSERT INTO movements (sku, kind, qty, note) SELECT sku, 'receive', $2, $3 FROM p
  )
  SELECT * FROM p`,
consume: `
  WITH p AS (
    UPDATE parts SET on_hand = on_hand - $2, updated_at = now()
     WHERE sku = $1 AND on_hand - reserved >= $2
    RETURNING sku, name, category, unit, on_hand, reserved, reorder_level, location, updated_at
  ), m AS (
    INSERT INTO movements (sku, kind, qty, note) SELECT sku, 'consume', $2, $3 FROM p
  )
  SELECT * FROM p`,
```

A **CTE** (`WITH … AS`) chains statements: the `UPDATE` returns the changed row, and the `INSERT` writes a ledger line for each row it returned, zero or one. Both are one statement, so they succeed or fail together.

For consume, the rule is in the `WHERE`: `on_hand - reserved >= $2`. Under Postgres's default READ COMMITTED isolation, if two consumes race for the same row, the second waits for the first, then **re-evaluates the WHERE against the new row**. If there's no longer enough, it updates nothing. That's why no explicit lock is needed here.

Calling the functions, and reading reservations

api/src/db/postgres-sql.ts · lines 48–62

```
movements: `
  SELECT id, sku, kind, qty, note, reservation_id, at FROM movements
   WHERE ($1::text IS NULL OR sku = $1)
   ORDER BY id DESC LIMIT $2`,
reserve: `SELECT inv_reserve($1, $2::jsonb) AS result`,
close: `SELECT inv_close_reservation($1::uuid, $2) AS result`,
reservations: `
  SELECT r.id, r.quote_ref, r.status, r.created_at, r.closed_at,
         coalesce(jsonb_agg(jsonb_build_object('sku', l.sku, 'qty', l.qty) ORDER BY l.sku)
                  FILTER (WHERE l.sku IS NOT NULL), '[]') AS lines
    FROM reservations r LEFT JOIN reservation_lines l ON l.reservation_id = r.id
   WHERE ($1::uuid IS NULL OR r.id = $1)
   GROUP BY r.id
   ORDER BY r.created_at DESC, r.id
   LIMIT $2`,
```

Reserve and close are one-line calls to the PL/pgSQL functions. Reading a reservation aggregates its lines into a JSON array with `jsonb_agg … FILTER (WHERE l.sku IS NOT NULL)`, so a reservation comes back as one row with its lines, the shape the API returns.

The repository class

api/src/db/postgres.ts · lines 48–79

```
export class PostgresRepo implements InventoryRepo {
  readonly kind = "postgres";
  private readonly pool: pg.Pool;

  /** Connection settings come from the standard PG* environment variables, or `config`. */
  constructor(config: pg.PoolConfig = {}) {
    this.pool = new pg.Pool({ max: 10, ...config });
  }

  async init(): Promise<void> {
    await this.pool.query(readFileSync(new URL("./schema.sql", import.meta.url), "utf8"));
  }

  async seedIfEmpty(seed: readonly SeedPart[]): Promise<boolean> {
    const { rows } = await this.pool.query<{ n: number }>(Q.count);
```

```
    if (rows[0].n > 0) return false;
    const client = await this.pool.connect();
    try {
      await client.query("BEGIN");
      for (const s of seed) {
        await client.query(Q.insertPart, [s.sku, s.name, s.category, s.unit, s.onHand,
s.reorderLevel, s.location]);
      }
      await client.query("COMMIT");
      return true;
    } catch (err) {
      await client.query("ROLLBACK");
      throw err;
    } finally {
      client.release();
    }
  }
```

`pg.Pool` keeps up to ten connections open and lends one per query. With no arguments it reads the standard `PGHOST`, `PGUSER`, `PGPASSWORD` and `PGDATABASE` variables, the same ones `psql` uses, so Compose and Kubernetes configure it with environment variables alone.

Seeding is the one place with an explicit transaction: `BEGIN`, the inserts, `COMMIT`, or `ROLLBACK` on any error. It needs one dedicated connection from the pool (`connect()`), because a transaction lives on a connection, and `finally` returns it whatever happens.

Telling “not found” from “not enough”

api/src/db/postgres.ts · lines 112–142

```
async receive(sku: string, qty: number, note: string): Promise<Part> {
  const { rows } = await this.pool.query<PartRow>(Q.receive, [sku, qty, note]);
  if (!rows[0]) throw new NotFoundError(`No part ${sku}`);
  return toPart(rows[0]);
}

async consume(sku: string, qty: number, note: string): Promise<Part> {
  const { rows } = await this.pool.query<PartRow>(Q.consume, [sku, qty, note]);
  if (rows[0]) return toPart(rows[0]);
  // No row updated: either the SKU doesn't exist or there wasn't enough available.
  const [part] = await this.getParts([sku]);
  if (!part) throw new NotFoundError(`No part ${sku}`);
  throw new ConflictError(`Only ${part.available} of ${sku} available`);
```

```
}

async movements({ sku, limit }: { sku?: string; limit: number }): Promise<Movement[]> {
  const { rows } = await this.pool.query<MovementRow>(Q.movements, [sku ?? null, limit]);
  return rows.map((m) => ({
    id: String(m.id), sku: m.sku, kind: m.kind, qty: m.qty, note: m.note, reservationId:
m.reservation_id, at: m.at.toISOString(),
  }));
}

async reserve(quoteRef: string, lines: readonly Line[]): Promise<ReserveResult> {
  const { rows } = await this.pool.query<{ result: { ok: true; id: string } | { ok: false;
shortages: Shortage[] } }>(
    Q.reserve, [quoteRef, JSON.stringify(mergeLines(lines))],
  );
  const result = rows[0].result;
  if (!result.ok) return { ok: false, shortages: result.shortages };
  return { ok: true, reservation: await this.reservation(result.id) };
}
```

When the consume statement updates nothing, it doesn't say why. A follow-up read decides between 404 and 409. That read isn't atomic with the update, but it doesn't need to be: it only chooses the error message.

For reserve, the lines are merged and sent as one JSON parameter. The function's answer is already parsed by the driver (Postgres JSONB comes back as a JavaScript object), and on success the full reservation is read back.

Closing, and mapping rows

api/src/db/postgres.ts · lines 241–274

```
  private async closeReservation(id: string, action: "release" | "fulfil"):
Promise<Reservation> {
    if (!UUID.test(id)) throw new NotFoundError(`No reservation ${id}`);
    const { rows } = await this.pool.query<{ result: string }>(Q.close, [id, action]);
    const result = rows[0].result;
    if (result === "not_found") throw new NotFoundError(`No reservation ${id}`);
    if (result !== "ok") throw new ConflictError(`Reservation is already ${result}`);
    return this.reservation(id);
  }

  private async reservation(id: string): Promise<Reservation> {
```

```
    const { rows } = await this.pool.query<ReservationRow>(Q.reservations, [id, 1]);
    if (!rows[0]) throw new NotFoundError(`No reservation ${id}`);
    return toReservation(rows[0]);
  }
}

/** `%` and `_` in a search box are literal characters, not wildcards. */
function escapeLike(s: string): string {
  return s.replace(/[\\%_]/g, (c) => `\\${c}`);
}

function toPart(r: PartRow): Part {
  return withDerived({
    sku: r.sku, name: r.name, category: r.category, unit: r.unit, onHand: r.on_hand, reserved:
r.reserved,
    reorderLevel: r.reorder_level, location: r.location, updatedAt: r.updated_at.toISOString(),
  });
}

function toReservation(r: ReservationRow): Reservation {
  return {
    id: r.id, quoteRef: r.quote_ref, status: r.status, lines: r.lines,
    createdAt: r.created_at.toISOString(), closedAt: r.closed_at ? r.closed_at.toISOString() :
null,
  };
}
```

An ID that isn't a UUID can't exist, so it's “not found” before reaching the database, where it would otherwise be a type error. The mapping functions are where `snake_case` columns become `camelCase` fields and `Date` objects become ISO strings, so nothing outside this file knows Postgres's naming.

GOING DEEPER

Optimistic concurrency, the other approach

Row locks are **pessimistic**: assume conflict, make others wait. **Optimistic** concurrency assumes conflicts are rare: each row has a `version`, an update says `WHERE version = $seen`, and if no row matched, someone else got there first and the caller retries. It never blocks, which suits long user edits (two people editing the same part's details), but under heavy contention retries pile up. Stock reservations, where contention is the whole point, suit locks better.

PART 5.7 · `api/src/db/mongo.ts`

The MongoDB repository

The same interface on a document database. MongoDB guarantees atomicity per document, so every rule is written into an update's filter, and the multi-document reservation becomes a small saga.

The approach

api/src/db/mongo.ts · lines 1–19

```
/**
 * MongoDB implementation.
 *
 * Each part is one document, and MongoDB updates a single document atomically. So every
 * stock rule is written into the update's filter: "add 4 to reserved, but only if
 * onHand - reserved is at least 4". If another request got there first, the filter no
 * longer matches and nothing changes. That gives the same guarantees as the Postgres
 * row-level checks, with no multi-document transaction, so a standalone mongod works.
 *
 * A reservation touches several documents, so it's done as a small saga:
 *   1. insert the reservation as "pending";
 *   2. hold each line with a conditional $inc, in SKU order;
 *   3. if any line fails, undo the holds already made and delete the pending reservation;
 *   4. otherwise mark it "active" and write the ledger.
 * Concurrent readers can briefly see a hold that's about to be undone. Stock is never
 * over-reserved; at worst, a racing reservation is refused when it could just have fit.
 * A crash between steps leaves a "pending" reservation that says exactly which holds to
 * undo. With a replica set, the same steps could run inside a transaction instead.
 */
```

The comment is the design in brief, and it's honest about the trade-off: no transaction, so a racing request can briefly see a hold that's about to be undone. Stock is never over-promised; the worst case is a refusal that a transaction would have accepted.

Documents

api/src/db/mongo.ts · lines 32–51

```
interface PartDoc {
  _id: string; // the SKU
  name: string; category: Part["category"]; unit: string;
  onHand: number; reserved: number; reorderLevel: number; location: string; updatedAt: Date;
}
interface MovementDoc { _id: string; seq: number; sku: string; kind: MovementKind; qty: number;
note: string; reservationId: string | null; at: Date }
interface ReservationDoc {
  _id: string; quoteRef: string; status: Reservation["status"] | "pending"; lines: Line[];
createdAt: Date; closedAt: Date | null;
}
interface CounterDoc { _id: string; value: number }
/** A shape is stored almost exactly as the API returns it: documents fit nested data like size
and position. */
interface ShapeDoc extends Omit<Shape, "id" | "createdAt" | "updatedAt"> { _id: string;
createdAt: Date; updatedAt: Date }
/** Conveyors too: the nested position stays nested. */
interface ConveyorDoc extends Omit<Conveyor, "id" | "createdAt" | "updatedAt"> { _id: string;
createdAt: Date; updatedAt: Date }
/** A splitter's rules are nested lists of objects: a document holds them as they are. */
interface SplitterDoc extends Omit<Splitter, "id" | "createdAt" | "updatedAt"> { _id: string;
createdAt: Date; updatedAt: Date }

/** `onHand - reserved >= qty`, as a query filter. */
const atLeastAvailable = (qty: number) => ({ $expr: { $gte: [{ $subtract: ["$onHand",
"$reserved"] }, qty] } });
```

The SKU *is* the part's `_id`, so lookups by SKU use the primary index and two parts can't share a SKU. A reservation **embeds** its lines, the document model's answer to Postgres's `reservation_lines` table: they're always read together, so they're stored together.

`atLeastAvailable` builds the rule as a filter. `$expr` lets a query compare two fields of the same document, “onHand minus reserved is at least qty”, which plain query operators can't.

Connecting, indexes and seeding

api/src/db/mongo.ts · lines 53–100

```
readonly kind = "mongo";
private readonly client: MongoClient;
```

```
private readonly db: Db;
private readonly parts: Collection<PartDoc>;
private readonly moves: Collection<MovementDoc>;
private readonly res: Collection<ReservationDoc>;
private readonly counters: Collection<CounterDoc>;
private readonly shapes: Collection<ShapeDoc>;
private readonly conveyors: Collection<ConveyorDoc>;
private readonly splitters: Collection<SplitterDoc>;

constructor(url: string, dbName = "inventory") {
  this.client = new MongoClient(url, { serverSelectionTimeoutMS: 5000 });
  this.db = this.client.db(dbName);
  this.parts = this.db.collection<PartDoc>("parts");
  this.moves = this.db.collection<MovementDoc>("movements");
  this.res = this.db.collection<ReservationDoc>("reservations");
  this.counters = this.db.collection<CounterDoc>("counters");
  this.shapes = this.db.collection<ShapeDoc>("shapes");
  this.conveyors = this.db.collection<ConveyorDoc>("conveyors");
  this.splitters = this.db.collection<SplitterDoc>("splitters");
}

async init(): Promise<void> {
  await this.client.connect();
  await this.moves.createIndex({ sku: 1, seq: -1 });
  await this.moves.createIndex({ seq: -1 });
  await this.res.createIndex({ createdAt: -1 });
}

async seedIfEmpty(seed: readonly SeedPart[]): Promise<boolean> {
  if ((await this.parts.estimatedDocumentCount()) > 0) return false;
  const now = new Date();
  try {
    await this.parts.insertMany(
      seed.map((s) => ({
        _id: s.sku, name: s.name, category: s.category, unit: s.unit,
        onHand: s.onHand, reserved: 0, reorderLevel: s.reorderLevel, location: s.location,
updatedAt: now,
      })),
      { ordered: false },
    );
  } catch (err) {
```

```
    // Another API replica seeded at the same moment: duplicate keys are fine.
    if ((err as { code?: number }).code !== 11000) throw err;
  }
  return true;
}
```

Indexes are created on start; `createIndex` does nothing if one already exists. Seeding inserts with `ordered: false` and ignores duplicate-key errors (code 11000): with two API replicas starting together, both may try to seed, and the second's inserts simply collide with the first's.

Conditional updates

api/src/db/mongo.ts · lines 147–171

```
  const doc = await this.parts.findOneAndUpdate(
    { _id: sku },
    { $inc: { onHand: qty }, $set: { updatedAt: new Date() } },
    { returnDocument: "after" },
  );
  if (!doc) throw new NotFoundError(`No part ${sku}`);
  await this.log([{ sku, kind: "receive", qty, note, reservationId: null }]);
  return toPart(doc);
}

async consume(sku: string, qty: number, note: string): Promise<Part> {
  const doc = await this.parts.findOneAndUpdate(
    { _id: sku, ...atLeastAvailable(qty) },
    { $inc: { onHand: -qty }, $set: { updatedAt: new Date() } },
    { returnDocument: "after" },
  );
  if (!doc) {
    const [part] = await this.getParts([sku]);
    if (!part) throw new NotFoundError(`No part ${sku}`);
    throw new ConflictError(`Only ${part.available} of ${sku} available`);
  }
  await this.log([{ sku, kind: "consume", qty, note, reservationId: null }]);
  return toPart(doc);
}
```

`findOneAndUpdate` with the rule in the filter is the MongoDB equivalent of Postgres's `UPDATE … WHERE … RETURNING`. MongoDB applies a single-document update atomically: the filter is evaluated and the `$inc` applied as one step, so two racing consumes can't both pass the check. `returnDocument:`

`"after"` returns the updated document.

The reservation saga

api/src/db/mongo.ts · lines 180–204

```
  const want = mergeLines(lines);
  const id = randomUUID();
  await this.res.insertOne({ _id: id, quoteRef, status: "pending", lines: want, createdAt: new
Date(), closedAt: null });

  const held: Line[] = [];
  for (const line of want) {
    const r = await this.parts.updateOne(
      { _id: line.sku, ...atLeastAvailable(line.qty) },
      { $inc: { reserved: line.qty }, $set: { updatedAt: new Date() } },
    );
    if (r.modifiedCount !== 1) break;
    held.push(line);
  }

  if (held.length < want.length) {
    for (const line of held) await this.parts.updateOne({ _id: line.sku }, { $inc: { reserved:
-line.qty } });
    await this.res.deleteOne({ _id: id });
    return { ok: false, shortages: await this.shortages(want, want[held.length]) };
  }

  await this.res.updateOne({ _id: id }, { $set: { status: "active" } });
  await this.log(want.map((l) => ({ sku: l.sku, kind: "reserve" as const, qty: l.qty, note:
`Reserved for ${quoteRef}`, reservationId: id })));
  return { ok: true, reservation: toReservation((await this.res.findOne({ _id: id }))!) };
}
```

Step by step: insert the reservation as `pending`; hold each line with a conditional `$inc`, in SKU order; at the first line that doesn't fit, stop. If anything failed, undo the holds already made, delete the pending reservation, and report the shortages. Otherwise mark it `active` and write the ledger.

Why `pending` first? If the process crashed halfway, the holds would otherwise be orphaned: reserved stock with no reservation explaining it. A pending reservation records exactly which holds might exist, so a cleanup job can undo them. `reservations()` hides pending ones, so users never see a half-made reservation.

Closing exactly once, and the ledger

api/src/db/mongo.ts · lines 297–346

```
   * Claim the reservation first by flipping its status with a conditional update, so two
   * simultaneous "release" clicks can't both give the stock back.
   */
  private async closeReservation(id: string, action: "release" | "fulfil"):
Promise<Reservation> {
    const doc = await this.res.findOneAndUpdate(
      { _id: id, status: "active" },
      { $set: { status: action === "fulfil" ? "fulfilled" : "released", closedAt: new Date() }
},
      { returnDocument: "after" },
    );
    if (!doc) {
      const existing = await this.res.findOne({ _id: id });
      if (!existing || existing.status === "pending") throw new NotFoundError(`No reservation
${id}`);
      throw new ConflictError(`Reservation is already ${existing.status}`);
    }
    for (const { sku, qty } of doc.lines) {
      const inc = action === "fulfil" ? { reserved: -qty, onHand: -qty } : { reserved: -qty };
      await this.parts.updateOne({ _id: sku }, { $inc: inc, $set: { updatedAt: new Date() } });
    }
    await this.log(doc.lines.map((l) => ({
      sku: l.sku, kind: action, qty: l.qty, note: `${action === "fulfil" ? "Shipped" :
"Released"} for ${doc.quoteRef}`, reservationId: id,
    })));
    return toReservation(doc);
  }

  /** Current shortages for `want`. If a race already freed the stock, still report the line
that failed. */
  private async shortages(want: Line[], failed: Line): Promise<Shortage[]> {
    const current = await this.getParts(want.map((l) => l.sku));
    const bySku = new Map(current.map((p) => [p.sku, p]));
    const out: Shortage[] = [];
    for (const { sku, qty } of want) {
      const p = bySku.get(sku);
      const available = p?.available ?? 0;
      if (available < qty || sku === failed.sku) out.push({ sku, need: qty, available, known: p
```

```
!== undefined });
    }
    return out;
  }

  /** Ledger entries get an increasing sequence number from a counter document, so "newest
first" is exact. */
  private async log(entries: Omit<MovementDoc, "_id" | "seq" | "at">[]): Promise<void> {
    if (entries.length === 0) return;
    const counter = await this.counters.findOneAndUpdate(
      { _id: "movements" },
      { $inc: { value: entries.length } },
      { upsert: true, returnDocument: "after" },
    );
    const last = counter!.value;
    const at = new Date();
    await this.moves.insertMany(entries.map((e, i) => ({ ...e, _id: randomUUID(), seq: last -
entries.length + i + 1, at })));
  }
}
```

Closing starts by **claiming** the reservation: a conditional update from `active` to the new status. Exactly one of several simultaneous requests can match `status: "active"`; the others find nothing and get a conflict. Only the winner then adjusts the parts.

The ledger needs “newest first” to be exact, but timestamps can tie and random IDs don't sort. A counter document, incremented atomically with `$inc` and `upsert`, hands out a block of sequence numbers in one round trip: the classic MongoDB auto-increment pattern.

GOING DEEPER

Doing it with a MongoDB transaction

On a replica set (even a one-node one, `--replSet rs0`), the saga can be replaced by `session.withTransaction(async () => { … })` around the same conditional updates. The driver retries the whole callback on transient conflicts. The reservation then becomes invisible until committed, like Postgres's. The saga version was chosen so the demo runs on a plain standalone `mongo` container; switching is a contained change inside one method.

PART 5.8 · `api/src/db/index.ts`

Choosing a database, and the service layer

One environment variable picks the implementation. A thin service layer sits between the routes and the repository.

The factory

api/src/db/index.ts · lines 1–31

```
import type { DbKind, InventoryRepo } from "../domain/types.ts";
import { MemoryRepo } from "./memory.ts";

/**
 * Picks the database from the DB environment variable: "postgres", "mongo" or "memory".
 * The drivers are imported only when chosen, so the API runs with neither installed in memory
mode.
 */
export async function createRepo(env: NodeJS.ProcessEnv = process.env): Promise<InventoryRepo>
{
  const kind = (env.DB ?? "memory") as DbKind;
  switch (kind) {
    case "postgres": {
      const { PostgresRepo } = await import("./postgres.ts");
      return new PostgresRepo(); // reads PGHOST, PGUSER, PGPASSWORD, PGDATABASE, PGPORT
    }
    case "mongo": {
      const { MongoRepo } = await import("./mongo.ts");
      return new MongoRepo(mongoUrl(env), env.MONGO_DB ?? "inventory");
    }
    case "memory":
      return new MemoryRepo();
    default:
      throw new Error(`DB must be postgres, mongo or memory, not "${kind}"`);
  }
}

export function mongoUrl(env: NodeJS.ProcessEnv): string {
  if (env.MONGO_URL) return env.MONGO_URL;
  const host = env.MONGO_HOST ?? "localhost";
  const auth = env.MONGO_USER ?
```

```
  `${encodeURIComponent(env.MONGO_USER)}:${encodeURIComponent(env.MONGO_PASSWORD ?? "")}@` : "";
  return `mongodb://${auth}${host}:27017/?authSource=admin`;
}
```

`await import()` loads a driver only when it's chosen. The API in memory mode runs with neither `pg` nor `mongodb` installed, which is also how the unit tests run anywhere.

`mongoUrl` builds a connection string from parts, URL-encoding the user and password: a password containing `@` or `:` would otherwise break the URL. `authSource=admin` is where the official image creates its root user.

Checking and reserving designs

api/src/service.ts · lines 1–42

```
/**
 * What the HTTP routes call. Turns designs into stock lines and answers "can we build
 * this?" without touching the database's own rules, which live in the repositories.
 */
import { parseDesign, plan, type Requirement } from "./domain/design.ts";
import { ValidationError } from "./domain/errors.ts";
import type { InventoryRepo, Reservation, Shortage } from "./domain/types.ts";

export interface CheckLine extends Requirement {
  name: string | null;
  unit: string | null;
  available: number;
  short: number;
}

export interface CheckResult {
  ok: boolean;
  lines: CheckLine[];
  warnings: string[];
}

export async function checkDesign(repo: InventoryRepo, input: unknown): Promise<CheckResult> {
  const { requirements, warnings } = plan(parseDesign(input));
  const parts = new Map((await repo.getParts(requirements.map((r) => r.sku))).map((p) =>
[p.sku, p]));
  const lines = requirements.map((r) => {
    const p = parts.get(r.sku);
    const available = p?.available ?? 0;
```

```
    return { ...r, name: p?.name ?? null, unit: p?.unit ?? null, available, short: Math.max(0,
r.qty - available) };
  });
  return { ok: lines.every((l) => l.short === 0), lines, warnings };
}

export type ReserveOutcome =
  | { ok: true; reservation: Reservation; warnings: string[] }
  | { ok: false; shortages: Shortage[]; warnings: string[] };

export async function reserveDesign(repo: InventoryRepo, quoteRef: string, input: unknown):
Promise<ReserveOutcome> {
  const { requirements, warnings } = plan(parseDesign(input));
  if (requirements.length === 0) throw new ValidationError(["nothing in this design maps to
stock", ...warnings]);
  const result = await repo.reserve(quoteRef, requirements.map(({ sku, qty }) => ({ sku, qty
})));
  return { ...result, warnings };
}
```

The service turns a design into requirements, then asks the repository. `checkDesign` is read-only and answers “can we build it?”; `reserveDesign` makes the promise. They're separate on purpose: a check is a snapshot that can be stale a second later, so reserving always re-checks inside the database.

A design with nothing reservable (every item warned about) is rejected up front, rather than creating an empty reservation.

PART 5.9 · `api/src/app.ts`

The HTTP API

About a dozen routes on Node's built-in `http` module, with a twenty-line router.

A tiny router

`api/src/app.ts · lines 1–28`

```
/**
 * The HTTP API on Node's built-in http module: about a dozen routes don't need a framework.
 * createApp() returns a plain request handler, so tests can mount it on a random port.
 */
```

```
import type { IncomingMessage, ServerResponse } from "node:http";
import { hostname } from "node:os";
import { isQty, isSku } from "./domain/design.ts";
import { NotFoundError, ValidationError } from "./domain/errors.ts";
import { parseNewPart, parsePartPatch } from "./domain/newPart.ts";
import { parseNewShape, parseShapePatch } from "./domain/shapes.ts";
import { parseConveyorPatch, parseNewConveyor } from "./domain/conveyors.ts";
import { parseNewSplitter, parseSplitterPatch } from "./domain/splitters.ts";
import type { InventoryRepo } from "./domain/types.ts";
import { checkDesign, reserveDesign } from "./service.ts";

type Params = Record<string, string>;
type Handler = (ctx: { params: Params; query: URLSearchParams; body: unknown }) =>
Promise<[number, unknown]>;

const MAX_BODY = 100_000; // bytes

export function createApp(repo: InventoryRepo, version = "1.0.0") {
  const routes: { method: string; pattern: RegExp; keys: string[]; handler: Handler }[] = [];
  const route = (method: string, path: string, handler: Handler) => {
    const keys: string[] = [];
    const pattern = new RegExp(`^${path.replace(/:(\w+)/g, (_, k: string) => (keys.push(k), "
([^/]+)"))}$`);
    routes.push({ method, pattern, keys, handler });
  };
```

Each route's path, like `/api/parts/:sku/receive`, is turned into a regular expression where `:sku` becomes a capture group, `([^/]+)`, and its name is remembered in `keys`. The comma expression `(keys.push(k), "([^/]+)")` records the name and returns the replacement in one go.

A handler receives path parameters, the query string and the parsed body, and returns a `[status, payload]` pair. Handlers never touch the response object, which makes them easy to read and test.

Routes

api/src/app.ts · lines 29–73

```
// ---- Health and info ----
route("GET", "/api/health/live", async () => [200, { ok: true }]);
route("GET", "/api/health/ready", async () => {
  await repo.ping();
  return [200, { ok: true, db: repo.kind }];
});
```

```
route("GET", "/api/info", async () => [200, { db: repo.kind, host: hostname(), version }]);

// ---- Parts ----
route("GET", "/api/parts", async ({ query }) => [200, await repo.listParts({
  search: query.get("search") ?? undefined,
  lowOnly: query.get("low") === "true",
})]);
route("POST", "/api/parts", async ({ body }) => [201, await
repo.createPart(parseNewPart(body))]);
route("GET", "/api/parts/:sku", async ({ params }) => {
  const [part] = await repo.getParts([sku(params.sku)]);
  return part ? [200, part] : [404, { error: `No part ${params.sku}` }];
});
route("PATCH", "/api/parts/:sku", async ({ params, body }) => [200, await
repo.updatePart(sku(params.sku), parsePartPatch(body))]);
route("POST", "/api/parts/:sku/receive", async ({ params, body }) => {
  const { qty, note } = adjustment(body);
  return [200, await repo.receive(sku(params.sku), qty, note)];
});
route("POST", "/api/parts/:sku/consume", async ({ params, body }) => {
  const { qty, note } = adjustment(body);
  return [200, await repo.consume(sku(params.sku), qty, note)];
});
route("GET", "/api/movements", async ({ query }) => {
  const s = query.get("sku");
  return [200, await repo.movements({ sku: s ? sku(s) : undefined, limit: limit(query, 50, 500)
})];
});

// ---- Designs and reservations ----
route("POST", "/api/check", async ({ body }) => [200, await checkDesign(repo, field(body,
"design"))]);
route("POST", "/api/reservations", async ({ body }) => {
  const quoteRef = field(body, "quoteRef");
  if (typeof quoteRef !== "string" || !/^[\w.-]{1,64}$/.test(quoteRef)) {
    throw new ValidationError(["quoteRef must be 1-64 letters, digits, dots, dashes or
underscores"]);
  }
  const out = await reserveDesign(repo, quoteRef, field(body, "design"));
  return out.ok ? [201, out] : [409, { error: "Not enough stock", ...out }];
});
```

```
route("GET", "/api/reservations", async ({ query }) => [200, await
repo.reservations(limit(query, 50, 200))]);
route("POST", "/api/reservations/:id/release", async ({ params }) => [200, await
repo.release(params.id)]);
route("POST", "/api/reservations/:id/fulfil", async ({ params }) => [200, await
repo.fulfil(params.id)]);
```

Health checks come in two kinds, matching what Kubernetes asks: **live** means the process is up; **ready** means it can reach the database. A database outage then takes pods out of the load balancer without restarting them.

A refused reservation is a **409 Conflict** carrying the shortages, not a 500: it's a normal outcome the UI shows the user.

Handling a request

`api/src/app.ts · lines 125–164`

```
  return async function handle(req: IncomingMessage, res: ServerResponse): Promise<void> {
    const started = performance.now();
    const url = new URL(req.url ?? "/", "http://localhost");

    // The robot arm app runs on its own dev server (another origin), so the browser asks
    // first (a CORS "preflight") and only sends the request if these headers allow it.
    const cors = corsHeaders(req.headers.origin);
    if (req.method === "OPTIONS") {
      res.writeHead(cors ? 204 : 403, cors ?? {});
      res.end();
      return;
    }

    let status = 500;
    let payload: unknown;
    try {
      const matching = routes.filter((r) => r.pattern.test(url.pathname));
      const r = matching.find((m) => m.method === req.method);
      if (!r) {
        [status, payload] = matching.length ? [405, { error: "Method not allowed" }] : [404, {
error: "Not found" }];
      } else {
        const m = r.pattern.exec(url.pathname)!;
        const params = Object.fromEntries(r.keys.map((k, i) => [k, decode(m[i + 1])]));
        const body = req.method === "POST" || req.method === "PATCH" ? await readJson(req) :
```

```
undefined;
        [status, payload] = await r.handler({ params, query: url.searchParams, body });
      }
    } catch (err) {
      const e = err as Error & { status?: number; problems?: string[] };
      status = e.status ?? 500;
      payload = status === 500 ? { error: "Internal error" } : { error: e.message, ...
(e.problems ? { problems: e.problems } : {}) };
      if (status === 500) console.error(err);
    }
    const text = JSON.stringify(payload);
    res.writeHead(status, { "content-type": "application/json; charset=utf-8", "cache-control":
"no-store", ...cors });
    res.end(text);
    if (process.env.LOG_REQUESTS !== "0" && !url.pathname.startsWith("/api/health")) {
      console.log(`${req.method} ${url.pathname} ${status} ${(performance.now() -
started).toFixed(1)}ms`);
    }
  };
}
```

Find the routes whose path matches; if none matches the method, it's 405 (path exists) or 404 (doesn't). Parse the body only for POST. Any thrown error with a `status` becomes that status with its message; anything else is a 500 whose details go to the log, not to the client.

Reading bodies safely

api/src/app.ts · lines 166–213

```
async function readJson(req: IncomingMessage): Promise<unknown> {
  const chunks: Buffer[] = [];
  let size = 0;
  for await (const chunk of req) {
    size += (chunk as Buffer).length;
    if (size > MAX_BODY) throw new ValidationError([`body is larger than ${MAX_BODY} bytes`]);
    chunks.push(chunk as Buffer);
  }
  if (size === 0) return {};
  try {
    return JSON.parse(Buffer.concat(chunks).toString("utf8"));
  } catch {
    throw new ValidationError(["body is not valid JSON"]);
```

```
  }
}

function field(body: unknown, key: string): unknown {
  return body && typeof body === "object" ? (body as Record<string, unknown>)[key] : undefined;
}

function sku(s: string): string {
  if (!isSku(s)) throw new ValidationError([`"${s}" is not a valid SKU`]);
  return s;
}

function adjustment(body: unknown): { qty: number; note: string } {
  const qty = field(body, "qty");
  const note = field(body, "note") ?? "";
  const problems: string[] = [];
  if (!isQty(qty)) problems.push("qty must be a whole number from 1 to 100000");
  if (typeof note !== "string" || note.length > 200) problems.push("note must be text up to 200
characters");
  if (problems.length) throw new ValidationError(problems);
  return { qty: qty as number, note: note as string };
}

function limit(query: URLSearchParams, fallback: number, max: number): number {
  const n = Number(query.get("limit") ?? fallback);
  return Number.isInteger(n) && n > 0 ? Math.min(n, max) : fallback;
}

/** A malformed escape like "%E0" in a path is the client's mistake, not a server error. */
function decode(s: string): string {
  try {
    return decodeURIComponent(s);
  } catch {
    throw new ValidationError([`"${s}" is not a valid URL segment`]);
  }
}
```

The body is read in chunks with a running size check, so a client can't make the server buffer a gigabyte. Invalid JSON, a bad quantity or a malformed URL escape are all 400s with a reason. Quantities must be whole numbers from 1 to 100,000; `limit` clamps page sizes so no one can ask for a million rows.

Starting up

api/src/server.ts · lines 1–38

```
/**
 * Entry point: connect to the database (retrying while it boots), create the schema,
 * seed the starting stock if the database is empty, then listen on PORT (8080).
 */
import { createServer } from "node:http";
import { createApp } from "./app.ts";
import { createRepo } from "./db/index.ts";
import { SEED_PARTS } from "./domain/seed.ts";

const port = Number(process.env.PORT ?? 8080);
const repo = await createRepo();

for (let attempt = 1; ; attempt++) {
  try {
    await repo.init();
    await repo.ping();
    break;
  } catch (err) {
    if (attempt >= 30) throw err;
    console.log(`Waiting for ${repo.kind} (attempt ${attempt}): ${(err as Error).message}`);
    await new Promise((r) => setTimeout(r, 2000));
  }
}
if (await repo.seedIfEmpty(SEED_PARTS)) console.log(`Seeded ${SEED_PARTS.length} parts`);

const server = createServer(createApp(repo));
server.listen(port, () => console.log(`Inventory API on :${port}, database: ${repo.kind}`));

// Kubernetes sends SIGTERM before stopping a pod: finish in-flight requests, then close the
pool.
for (const signal of ["SIGTERM", "SIGINT"] as const) {
  process.once(signal, () => {
    server.close(async () => {
      await repo.close();
      process.exit(0);
    });
    setTimeout(() => process.exit(1), 10_000).unref();
```

```
  });
}
```

Top-level `await` (allowed in ES modules) keeps startup linear. The retry loop matters in containers: Compose and Kubernetes may start the API before the database accepts connections, so it tries for up to a minute instead of crashing.

On `SIGTERM`, which Kubernetes sends before stopping a pod, the server stops accepting connections, finishes in-flight requests, closes the database pool, and exits. A 10 s backstop timer forces the exit; `unref()` keeps that timer from holding the process open by itself.

GOING DEEPER

Idempotency keys

If the network drops after the server reserved stock but before the browser got the answer, the user clicks again and reserves twice. Payment APIs solve this with an **idempotency key**: the client sends a unique key per intended action, the server stores the response under it, and a retry with the same key gets the stored response instead of a second reservation. Here, a unique constraint on `quote_ref` would be the simplest version.

PART 5.10 · `api/tests/contract.ts`

Contract tests: one suite, every database

The same nineteen tests run against each implementation. If MongoDB and Postgres disagree about anything the API relies on, one of them fails.

A test suite as a function

api/tests/contract.ts · lines 1–33

```
/**
 * The behaviour every InventoryRepo must have. Run against each database by
 * memory.test.ts, postgres.test.ts and mongo.test.ts, so the three can't drift apart.
 */
import { after, before, beforeEach, describe, test } from "node:test";
import assert from "node:assert/strict";
import { ConflictError, NotFoundError } from "../src/domain/errors.ts";
import type { InventoryRepo, SeedPart } from "../src/domain/types.ts";

export const FIXTURE: SeedPart[] = [
```

```
  { sku: "BAR-4040-CLR", name: "40x40 extrusion", category: "extrusion", unit: "bar (3 m)",
onHand: 10, reorderLevel: 4, location: "A-1" },
  { sku: "LA-300", name: "Linear actuator", category: "actuator", unit: "piece", onHand: 3,
reorderLevel: 2, location: "D-1" },
  { sku: "MC-1", name: "Motion controller", category: "controller", unit: "piece", onHand: 2,
reorderLevel: 2, location: "D-2" },
  { sku: "PL-200", name: "Mounting plate 200_mm", category: "plate", unit: "plate", onHand: 20,
reorderLevel: 5, location: "B-1" },
];

export function contractTests(name: string, makeRepo: () => Promise<InventoryRepo>) {
  describe(`${name} repository`, () => {
    let repo: InventoryRepo;

    before(async () => {
      repo = await makeRepo();
      await repo.init();
    });
    beforeEach(async () => {
      await repo.reset();
      assert.equal(await repo.seedIfEmpty(FIXTURE), true);
    });
    after(async () => {
      await repo.reset();
      await repo.close();
    });
```

`contractTests(name, makeRepo)` defines the suite for whatever repository the factory makes. `beforeEach` resets and reseeds, so tests can't leak state into each other, and the seed fixture is small enough to reason about: three actuators, two controllers.

Behaviour, including the edge cases

api/tests/contract.ts · lines 102–145

```
test("consume can't take stock that's reserved for a quote", async () => {
  const r = await repo.reserve("Q-1", [{ sku: "LA-300", qty: 2 }]);
  assert.equal(r.ok, true);
  await assert.rejects(repo.consume("LA-300", 2, "line-side"), ConflictError);
  const p = await repo.consume("LA-300", 1, "line-side");
  assert.equal(p.onHand, 2);
  assert.equal(p.available, 0);
```

```
});

test("reserve holds every line and records who for", async () => {
  const r = await repo.reserve("Q-1", [{ sku: "LA-300", qty: 2 }, { sku: "BAR-4040-CLR", qty: 3
}]);
  assert.ok(r.ok);
  assert.equal(r.reservation.status, "active");
  assert.equal(r.reservation.quoteRef, "Q-1");
  assert.deepEqual(r.reservation.lines, [{ sku: "BAR-4040-CLR", qty: 3 }, { sku: "LA-300", qty:
2 }]);
  const [bar, la] = await repo.getParts(["BAR-4040-CLR", "LA-300"]);
  assert.equal(bar.reserved, 3);
  assert.equal(la.available, 1);
  const moves = await repo.movements({ limit: 10 });
  assert.deepEqual(moves.map((m) => m.kind), ["reserve", "reserve"]);
  assert.ok(moves.every((m) => m.reservationId === r.reservation.id));
});

test("reserve adds up lines for the same SKU", async () => {
  const r = await repo.reserve("Q-1", [{ sku: "LA-300", qty: 1 }, { sku: "LA-300", qty: 2 }]);
  assert.ok(r.ok);
  assert.deepEqual(r.reservation.lines, [{ sku: "LA-300", qty: 3 }]);
  assert.ok(!(await repo.reserve("Q-2", [{ sku: "LA-300", qty: 1 }])).ok);
});

test("reserve is all or nothing, and reports every shortage", async () => {
  const r = await repo.reserve("Q-1", [
    { sku: "BAR-4040-CLR", qty: 2 }, { sku: "LA-300", qty: 5 }, { sku: "GHOST-1", qty: 1 }, {
sku: "PL-200", qty: 1 },
  ]);
  assert.equal(r.ok, false);
  assert.ok(!r.ok);
  assert.deepEqual(
    r.shortages.map((s) => ({ ...s })).sort((a, b) => a.sku.localeCompare(b.sku)),
    [{ sku: "GHOST-1", need: 1, available: 0, known: false }, { sku: "LA-300", need: 5,
available: 3, known: true }],
  );
  for (const p of await repo.listParts()) assert.equal(p.reserved, 0, `${p.sku} was left
reserved`);
  assert.deepEqual(await repo.reservations(10), []);
```

```
  assert.deepEqual(await repo.movements({ limit: 10 }), []);
});
```

These tests describe rules, not implementations. “Consume can't take reserved stock” and “reserve is all or nothing” check the state after the call too, not just the return value: after a refused reservation, every part must have zero reserved, and there must be no reservation and no ledger line.

Concurrency, tested

api/tests/contract.ts · lines 147–185

```
test("concurrent reservations never over-reserve", async () => {
  const results = await Promise.all(Array.from({ length: 8 }, (_, i) => repo.reserve(`Q-${i}`,
[{ sku: "LA-300", qty: 1 }])));
  assert.equal(results.filter((r) => r.ok).length, 3);
  const [la] = await repo.getParts(["LA-300"]);
  assert.equal(la.reserved, 3);
  assert.equal(la.available, 0);
});

test("release gives the stock back", async () => {
  const r = await repo.reserve("Q-1", [{ sku: "MC-1", qty: 2 }]);
  assert.ok(r.ok);
  const released = await repo.release(r.reservation.id);
  assert.equal(released.status, "released");
  assert.ok(released.closedAt);
  const [mc] = await repo.getParts(["MC-1"]);
  assert.equal(mc.reserved, 0);
  assert.equal(mc.onHand, 2);
});

test("fulfil ships the reserved stock", async () => {
  const r = await repo.reserve("Q-1", [{ sku: "MC-1", qty: 2 }, { sku: "PL-200", qty: 4 }]);
  assert.ok(r.ok);
  const done = await repo.fulfil(r.reservation.id);
  assert.equal(done.status, "fulfilled");
  const [mc, pl] = await repo.getParts(["MC-1", "PL-200"]);
  assert.deepEqual([mc.onHand, mc.reserved, pl.onHand, pl.reserved], [0, 0, 16, 0]);
  const kinds = (await repo.movements({ limit: 10 })).map((m) => m.kind);
  assert.deepEqual(kinds.slice(0, 2), ["fulfil", "fulfil"]);
});
```

```
test("a reservation can only be closed once, even by simultaneous requests", async () => {
  const r = await repo.reserve("Q-1", [{ sku: "LA-300", qty: 2 }]);
  assert.ok(r.ok);
  const outcomes = await Promise.allSettled([repo.release(r.reservation.id),
repo.release(r.reservation.id), repo.fulfil(r.reservation.id)]);
  assert.equal(outcomes.filter((o) => o.status === "fulfilled").length, 1);
  assert.ok(outcomes.filter((o) => o.status === "rejected").every((o) => (o as
PromiseRejectedResult).reason instanceof ConflictError));
  const [la] = await repo.getParts(["LA-300"]);
  assert.equal(la.reserved, 0);
});
```

`Promise.all` fires eight reservations for one actuator each, at once, when three exist. Exactly three must succeed. Against Postgres or MongoDB these are real concurrent requests on separate connections; against the in-memory store they interleave on the event loop. Either way, a broken implementation over-reserves and fails.

The close test fires two releases and a fulfil at the same reservation. Exactly one may win; the losers must be conflicts, and the stock must be released once.

Running the suite per database

api/tests/postgres.test.ts · lines 1–14

```
/**
 * Runs the contract against a real PostgreSQL. Skipped unless PGHOST is set, e.g.:
 *   docker compose --profile test run --rm api-test
 * Uses its own database name (PGDATABASE), and wipes it between tests.
 */
import { test } from "node:test";
import { contractTests } from "./contract.ts";

if (process.env.PGHOST) {
  const { PostgresRepo } = await import("../src/db/postgres.ts");
  contractTests("postgres", async () => new PostgresRepo());
} else {
  test("postgres repository", { skip: "set PGHOST (and PGUSER, PGPASSWORD, PGDATABASE) to run"
}, () => {});
}
```

Database runs skip themselves unless connection settings are present, so `npm test` works on any machine, and `docker compose --profile test run api-test` runs everything against throwaway Postgres and MongoDB containers.

GOING DEEPER

What concurrency tests can and can't prove

A test that fires eight requests at once and sees three succeed shows the race is handled **in that run**. It can't prove there's no interleaving that fails, because the scheduler decides the timing. That's why the design doesn't rely on the tests alone: constraints in the database make the bad state impossible to store, and the tests check the error handling and the happy path around them.

PART 5.11 · `api/src/domain/newPart.ts`

Adding a part, end to end

The newest feature, followed through every layer: a form in the browser, validation, one route, and the same operation on three storage back ends, each safe against two people adding the same SKU at once.

Validating a new part

api/src/domain/newPart.ts · lines 1–43

```
/** Validates a new part sent from the "Add part" form. */
import { isSku } from "./design.ts";
import { ValidationError } from "./errors.ts";
import type { Category, SeedPart } from "./types.ts";

export const CATEGORIES: readonly Category[] = ["extrusion", "plate", "fastener", "motor",
"actuator", "controller", "gripper"];

const MAX_QTY = 100_000;

export function parseNewPart(input: unknown): SeedPart {
  const b = (input && typeof input === "object" ? input : {}) as Record<string, unknown>;
  const problems: string[] = [];
  const text = (key: string, max: number, required = true): string => {
    const v = b[key];
```

```
    if (v === undefined && !required) return "";
    if (typeof v !== "string" || (required && v.trim() === "") || v.trim().length > max) {
      problems.push(`${key} must be text${required ? "" : " (optional)"} up to ${max}
characters`);
      return "";
    }
    return v.trim();
  };
  const whole = (key: string): number => {
    const v = b[key] ?? 0;
    if (typeof v !== "number" || !Number.isInteger(v) || v < 0 || v > MAX_QTY) {
      problems.push(`${key} must be a whole number from 0 to ${MAX_QTY}`);
      return 0;
    }
    return v;
  };

  const sku = typeof b.sku === "string" ? b.sku.trim().toUpperCase() : "";
  if (!isSku(sku)) problems.push('sku must be 2-32 capital letters, digits or dashes, like "PL-
400"');
  const name = text("name", 80);
  const category = b.category as Category;
  if (!CATEGORIES.includes(category)) problems.push(`category must be one of
${CATEGORIES.join(", ")}`);
  const unit = text("unit", 30);
  const location = text("location", 20, false);
  const onHand = whole("onHand");
  const reorderLevel = whole("reorderLevel");

  if (problems.length) throw new ValidationError(problems);
  return { sku, name, category, unit, onHand, reorderLevel, location };
}
```

The same approach as `parseDesign`: check every field, collect every problem, and build a clean object from the checked values. Two small kindnesses happen here rather than in the form: the SKU is trimmed and upper-cased, so `pl-400` becomes `PL-400`, and text fields are trimmed so a stray space can't create a second “Mounting plate”.

`text` and `whole` are local helpers, closures over `b` and `problems`, so each field is one line. Quantities are whole numbers from 0 (a part can be added before any stock arrives) up to 100,000.

One route

api/src/app.ts · line 42

```
route("POST", "/api/parts", async ({ body }) => [201, await
repo.createPart(parseNewPart(body))]);
```

A `POST` to the collection creates a member: the REST convention. It returns **201 Created** with the new part. A taken SKU is a `ConflictError` from the repository, so it becomes **409**; invalid fields are a `ValidationError`, **400**, listing every problem. The route itself is one line because every decision lives in a layer that can be tested on its own.

The contract: create, and record the starting stock

api/src/domain/types.ts · lines 113–117

```
/**
 * Adds a new part. Initial stock is written to the ledger as a receipt, so the
 * ledger still explains every unit. Throws ConflictError if the SKU already exists.
 */
createPart(part: SeedPart): Promise<Part>;
```

The interface says what all three implementations must do, including the part that's easy to forget: starting stock is written to the ledger as a receipt. Otherwise the ledger would explain every change except the first, and “why do we have 12 of these?” would have no answer.

In memory

api/src/db/memory.ts · lines 54–60

```
async createPart(s: SeedPart): Promise<Part> {
  if (this.parts.has(s.sku)) throw new ConflictError(`Part ${s.sku} already exists`);
  const p: PartRow = { ...s, reserved: 0, updatedAt: new Date().toISOString() };
  this.parts.set(s.sku, p);
  if (s.onHand > 0) this.log(s.sku, "receive", s.onHand, "Initial stock", null);
  return out(p);
}
```

Check, insert, log. Atomic because nothing awaits in between, as with every method in this class.

In PostgreSQL: one statement

api/src/db/postgres-sql.ts · lines 15–24

```
createPart: `
  WITH p AS (
    INSERT INTO parts (sku, name, category, unit, on_hand, reorder_level, location)
    VALUES ($1, $2, $3, $4, $5, $6, $7)
    ON CONFLICT (sku) DO NOTHING
    RETURNING sku, name, category, unit, on_hand, reserved, reorder_level, location, updated_at
  ), m AS (
    INSERT INTO movements (sku, kind, qty, note) SELECT sku, 'receive', on_hand, 'Initial
stock' FROM p WHERE on_hand > 0
  )
  SELECT * FROM p`,
```

api/src/db/postgres.ts · lines 99–104

```
async createPart(s: SeedPart): Promise<Part> {
  const { rows } = await this.pool.query<PartRow>(Q.createPart, [s.sku, s.name, s.category,
s.unit, s.onHand, s.reorderLevel, s.location]);
  // ON CONFLICT DO NOTHING returns no row when the SKU is taken, even if two requests race.
  if (!rows[0]) throw new ConflictError(`Part ${s.sku} already exists`);
  return toPart(rows[0]);
}
```

`ON CONFLICT (sku) DO NOTHING` relies on the primary key. If the SKU exists, or another request inserts it a microsecond earlier, Postgres inserts nothing and `RETURNING` returns no row, which the repository turns into a conflict. There's no check-then-insert gap to race through: the unique index is the check.

The ledger insert is chained in a CTE and only runs for rows the insert returned with stock above zero (the movements table requires `qty > 0`). The foreign key from the movement to the new part is satisfied within the same statement, because Postgres checks foreign keys at the end of the statement, not in the middle.

In MongoDB: let the unique index decide

api/src/db/mongo.ts · lines 124–138

```
async createPart(s: SeedPart): Promise<Part> {
  const doc: PartDoc = {
    _id: s.sku, name: s.name, category: s.category, unit: s.unit,
    onHand: s.onHand, reserved: 0, reorderLevel: s.reorderLevel, location: s.location,
updatedAt: new Date(),
  };
  try {
    await this.parts.insertOne(doc);
  } catch (err) {
    // The SKU is the _id, so the unique index rejects a duplicate, even if two requests race.
    if ((err as { code?: number }).code === 11000) throw new ConflictError(`Part ${s.sku}
already exists`);
    throw err;
  }
  if (s.onHand > 0) await this.log([{ sku: s.sku, kind: "receive", qty: s.onHand, note:
"Initial stock", reservationId: null }]);
  return toPart(doc);
}
```

The SKU is the document's `_id`, and `_id` always has a unique index. So `insertOne` either inserts or fails with error code 11000 (duplicate key), which becomes a conflict. As in Postgres, the database's uniqueness guarantee replaces any “does it exist?” query, which would leave a gap for a racing request.

Tested on every database

api/tests/contract.ts · lines 66–85

```
test("createPart adds a part and logs its initial stock as a receipt", async () => {
  const p = await repo.createPart({ sku: "PL-400", name: "Mounting plate, 400 mm", category:
"plate", unit: "plate", onHand: 12, reorderLevel: 5, location: "B-01-4" });
  assert.deepEqual([p.sku, p.onHand, p.reserved, p.available, p.low], ["PL-400", 12, 0, 12,
false]);
  assert.equal((await repo.getParts(["PL-400"]))[0].name, "Mounting plate, 400 mm");
  const [m] = await repo.movements({ sku: "PL-400", limit: 5 });
  assert.deepEqual([m.kind, m.qty, m.note], ["receive", 12, "Initial stock"]);
  // A part can start empty: no ledger line, since nothing was received.
  await repo.createPart({ sku: "SRV-750", name: "Servo motor, 750 W", category: "motor", unit:
"piece", onHand: 0, reorderLevel: 2, location: "" });
  assert.equal((await repo.movements({ sku: "SRV-750", limit: 5 })).length, 0);
  assert.equal((await repo.getParts(["SRV-750"]))[0].low, true);
});

test("createPart refuses a SKU that exists, even from simultaneous requests", async () => {
  await assert.rejects(repo.createPart({ ...FIXTURE[1], onHand: 99 }), ConflictError);
  assert.equal((await repo.getParts(["LA-300"]))[0].onHand, 3);
  const part = { sku: "GR-2", name: "Vacuum gripper", category: "gripper" as const, unit:
"piece", onHand: 1, reorderLevel: 0, location: "" };
  const results = await Promise.allSettled([repo.createPart(part), repo.createPart(part),
repo.createPart(part)]);
  assert.equal(results.filter((r) => r.status === "fulfilled").length, 1);
  assert.equal((await repo.movements({ sku: "GR-2", limit: 5 })).length, 1);
});
```

The first test checks the part, its derived fields, and the “Initial stock” receipt, then that a part added with no stock writes no ledger line and starts as low stock. The second fires three simultaneous creates of the same SKU: exactly one may succeed, and there must be exactly one ledger line. Against real PostgreSQL, the same statement was also fired from ten connections at once: one part, one receipt.

The form

web/src/AddPartPanel.tsx · lines 4–44

```
const CATEGORIES: { id: Category; unit: string; prefix: string; example: string }[] = [
  { id: "extrusion", unit: "bar (3 m)", prefix: "BAR-", example: "BAR-2040-CLR" },
  { id: "plate", unit: "plate", prefix: "PL-", example: "PL-400" },
```

```
  { id: "fastener", unit: "pack of 50", prefix: "", example: "M8-BOLT" },
  { id: "motor", unit: "piece", prefix: "SRV-", example: "SRV-750" },
  { id: "actuator", unit: "piece", prefix: "LA-", example: "LA-500" },
  { id: "controller", unit: "piece", prefix: "MC-", example: "MC-2" },
  { id: "gripper", unit: "piece", prefix: "GR-", example: "GR-2" },
];
const UNITS = ["piece", "plate", "bar (3 m)", "pack of 50", "pack of 100", "metre"];
const SKU = /^[A-Z0-9][A-Z0-9-]{1,31}$/;

interface Props {
  existing: Part[];
  onCreated: (part: Part) => void;
  onCancel: () => void;
}

/** Form for a new part. Checks what it can before sending; the API checks everything again. */
export function AddPartPanel({ existing, onCreated, onCancel }: Props) {
  const [category, setCategory] = useState<Category>("plate");
  const [sku, setSku] = useState("");
  const [name, setName] = useState("");
  const [unit, setUnit] = useState("plate");
  const [unitTouched, setUnitTouched] = useState(false);
  const [onHand, setOnHand] = useState("0");
  const [reorderLevel, setReorderLevel] = useState("0");
  const [location, setLocation] = useState("");
  const [busy, setBusy] = useState(false);
  const [problems, setProblems] = useState<string[]>([]);

  const meta = CATEGORIES.find((c) => c.id === category)!;
  const skuTaken = existing.some((p) => p.sku === sku);
  const skuBad = sku !== "" && !SKU.test(sku);

  function pickCategory(c: Category) {
    setCategory(c);
    // Suggest the usual unit for the category, unless the user already chose one.
    if (!unitTouched) setUnit(CATEGORIES.find((x) => x.id === c)!.unit);
  }
```

Each category carries its usual unit, SKU prefix and an example. Picking a category fills in the unit, **unless the user has already typed one** (`unitTouched`): helpful defaults shouldn't overwrite deliberate choices.

`skuTaken` and `skuBad` are computed on every render from state, not stored, so they can never be out of date. They give instant feedback while typing; the server still checks everything, because another user may add the same SKU between the check and the click.

web/src/AddPartPanel.tsx · lines 45–61

```
async function submit(e: FormEvent) {
  e.preventDefault();
  setBusy(true);
  setProblems([]);
  try {
    const part = await api.createPart({
      sku, name, category, unit, location,
      onHand: Number(onHand), reorderLevel: Number(reorderLevel),
    });
    onCreated(part);
  } catch (err) {
    const body = (err as { body?: { problems?: string[] } }).body;
    setProblems(body?.problems ?? [(err as Error).message]);
  } finally {
    setBusy(false);
  }
}
```

On failure, the API's list of problems is shown as it is. On success, the parent decides what happens next.

web/src/AddPartPanel.tsx · lines 84–92

```
<label className="full">
  SKU
  <input value={sku} required maxLength={32} placeholder={`e.g. ${meta.example}`} autoFocus
    onChange={(e) => setSku(e.target.value.toUpperCase().replace(/\s+/g, "-"))}
    aria-invalid={skuTaken || skuBad} className="mono" />
  {skuTaken && <span className="field-error">{sku} already exists</span>}
  {skuBad && <span className="field-error">Capital letters, digits and dashes only, 2–32
characters</span>}
  {!sku && meta.prefix && <span className="hint">Parts like this usually start with
{meta.prefix}</span>}
</label>
```

The SKU input upper-cases as you type and turns spaces into dashes. `aria-invalid` marks the field for screen readers and, through CSS, outlines it in red: one attribute for both, as with the panel toggles in Snack 1.

Showing the new part

web/src/StockView.tsx · lines 73–87

```
{adding && (
  <AddPartPanel
    existing={parts}
    onCancel={() => setAdding(false)}
    onCreated={(p) => {
      setAdding(false);
      setSearch("");
      setCategory("all");
      setLowOnly(false);
      setSelected(p.sku);
      setJustAdded(p.sku);
      onChanged();
    }}
  />
)}
```

After creating, the stock view clears every filter (a new part hidden by an old search would look like a failure), selects the part so its panel opens with the “Initial stock” receipt, marks the row so CSS can flash it green, and calls `onChanged`, which refetches the parts and bumps the version so every other view updates too.

PART 5.12 · `api/src/domain/shapes.ts`

Shapes for the robot arm's scene

Chapter 1's arm app places shapes on its floor, and some of them stand for inventory parts. They're stored here, beside the stock, so every replica and every browser sees the same scene. This part follows them through the API.

The model

api/src/domain/shapes.ts · lines 1–52

```
/**
 * Simple 3D shapes placed in the robot arm's scene (Snack 1). They're stored with the
 * inventory, so any API replica, and any browser, sees the same scene.
 *
 * Units are meters and degrees. The floor is y = 0; a shape always rests on it, so only
 * x and z are stored and its height decides where its center is.
 */
import { isSku } from "./design.ts";
import { ValidationError } from "./errors.ts";

export type ShapeKind = "box" | "cylinder" | "sphere" | "cone";
export const SHAPE_KINDS: readonly ShapeKind[] = ["box", "cylinder", "sphere", "cone"];

/**
 * size.x, size.y, size.z are the bounding box: width, height, depth.
 * A cylinder or cone is round, so its depth equals its width (the diameter);
 * a sphere's three sizes are all its diameter. parseNewShape enforces both.
 */
export interface Size {
  x: number;
  y: number;
  z: number;
}

export interface Shape {
  id: string;
  name: string;
  kind: ShapeKind;
  size: Size;
  /** "#rrggbb" */
  color: string;
  /** Floor position of the shape's center. */
  position: { x: number; z: number };
  /** Turn about the vertical axis, in degrees. */
  rotationY: number;
  /** The inventory part this shape stands for, or null for a plain shape. */
  sku: string | null;
  createdAt: string;
```

```
  updatedAt: string;
}

export type NewShape = Omit<Shape, "id" | "createdAt" | "updatedAt">;
/** What can change after a shape is placed: move it, turn it, rename or recolor it. */
export type ShapePatch = Partial<Pick<Shape, "name" | "color" | "position" | "rotationY">>;

export const LIMITS = {
  minSize: 0.01, // 1 cm
  maxSize: 2, // 2 m
  floor: 3, // positions within ±3 m of the arm's base
};

const COLOR = /^#[0-9a-f]{6}$/;
```

Meters and degrees, as in the arm app. `size` is always the bounding box; for round shapes the depth follows the diameter. `sku` links a shape to an inventory part, or is null for a plain shape. `ShapePatch` lists what may change after placing: position, turn, name and color. Kind and size can't, because the arm app treats a different size as a different object: delete and re-create instead.

Validation, including what can't change

`api/src/domain/shapes.ts · lines 102–157`

```
export function parseNewShape(input: unknown): NewShape {
  const b = asBody(input);
  const problems: string[] = [];

  const kind = b.kind as ShapeKind;
  if (!SHAPE_KINDS.includes(kind)) problems.push(`kind must be one of ${SHAPE_KINDS.join(",
")}`);

  const s = asBody(b.size);
  const dim = (v: unknown) => num(v, LIMITS.minSize, LIMITS.maxSize);
  let size: Size | undefined;
  const range = `from ${LIMITS.minSize} to ${LIMITS.maxSize} m`;
  if (kind === "sphere") {
    const d = dim(s.x);
    if (d === null) problems.push(`size.x (diameter) must be ${range}`);
    else size = { x: d, y: d, z: d };
  } else if (kind === "cylinder" || kind === "cone") {
    const d = dim(s.x);
```

```
    const h = dim(s.y);
    if (d === null) problems.push(`size.x (diameter) must be ${range}`);
    if (h === null) problems.push(`size.y (height) must be ${range}`);
    if (d !== null && h !== null) size = { x: d, y: h, z: d };
  } else if (kind === "box") {
    const [x, y, z] = [dim(s.x), dim(s.y), dim(s.z)];
    if (x === null || y === null || z === null) problems.push(`size.x, size.y and size.z
(width, height, depth) must each be ${range}`);
    else size = { x, y, z };
  }

  const name = readName(b.name, problems);
  const color = readColor(b.color, problems);
  const position = readPosition(b.position, problems);
  const rotationY = b.rotationY === undefined ? 0 : readRotation(b.rotationY, problems);
  let sku: string | null = null;
  if (b.sku !== undefined && b.sku !== null) {
    if (isSku(b.sku)) sku = b.sku;
    else problems.push('sku must be an inventory SKU like "PL-200", or left out for a plain
shape');
  }

  if (problems.length) throw new ValidationError(problems);
  return { name: name!, kind, size: size!, color: color!, position: position!, rotationY:
rotationY!, sku };
}

/** Validates a change to a placed shape. At least one field must be given. */
export function parseShapePatch(input: unknown): ShapePatch {
  const b = asBody(input);
  const problems: string[] = [];
  const patch: ShapePatch = {};
  if (b.name !== undefined) patch.name = readName(b.name, problems);
  if (b.color !== undefined) patch.color = readColor(b.color, problems);
  if (b.position !== undefined) patch.position = readPosition(b.position, problems);
  if (b.rotationY !== undefined) patch.rotationY = readRotation(b.rotationY, problems);
  const unknown = Object.keys(b).filter((k) => !["name", "color", "position",
"rotationY"].includes(k));
  if (unknown.length) problems.push(`can't change ${unknown.join(", ")}; delete and re-create
the shape instead`);
  if (!problems.length && Object.keys(patch).length === 0) problems.push("nothing to change:
```

```
send name, color, position or rotationY");
  if (problems.length) throw new ValidationError(problems);
  return patch;
}
```

Round shapes are normalized here: send a cylinder with any depth and it's stored with depth = diameter. So the rule lives in one place, and the database never holds a cylinder that's wider one way than the other.

`parseShapePatch` rejects unknown fields by name (“can't change kind”) instead of silently ignoring them, and an empty patch is an error. Silent ignoring is how clients end up believing a change was saved.

Postgres: a table with its rules

`api/src/db/schema.sql · lines 48–65`

```
-- Shapes placed in the robot arm's scene (Snack 1). Sizes and positions in meters.
CREATE TABLE IF NOT EXISTS shapes (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL CHECK (length(name) BETWEEN 1 AND 40),
  kind        text NOT NULL CHECK (kind IN ('box', 'cylinder', 'sphere', 'cone')),
  size_x      double precision NOT NULL CHECK (size_x BETWEEN 0.01 AND 2),
  size_y      double precision NOT NULL CHECK (size_y BETWEEN 0.01 AND 2),
  size_z      double precision NOT NULL CHECK (size_z BETWEEN 0.01 AND 2),
  color       text NOT NULL CHECK (color ~ '^#[0-9a-f]{6}$'),
  x           double precision NOT NULL CHECK (x BETWEEN -3 AND 3),
  z           double precision NOT NULL CHECK (z BETWEEN -3 AND 3),
  rotation_y  double precision NOT NULL DEFAULT 0,
  created_at  timestamptz NOT NULL DEFAULT clock_timestamp(),
  updated_at  timestamptz NOT NULL DEFAULT clock_timestamp()
);
-- A shape can stand for an inventory part. Added separately so databases created before it
gain the column.
ALTER TABLE shapes ADD COLUMN IF NOT EXISTS sku text REFERENCES parts (sku);
CREATE INDEX IF NOT EXISTS shapes_by_sku ON shapes (sku);
```

The table repeats the limits as `CHECK` constraints, including a regular expression for the color, so even a direct SQL insert can't store a 5 m box or the color “red”. The `sku` column is added with `ALTER TABLE … ADD COLUMN IF NOT EXISTS`, after the table: a database created before parts could be placed gains the column on the next start. That's the smallest possible migration, and why real projects move to a migrations tool once the schema starts changing.

The foreign key means a shape can only stand for a part that exists. The API checks first to give a friendly 400; the key is the guarantee.

api/src/db/postgres-sql.ts · lines 63–77

```
listShapes: `
  SELECT id, name, kind, size_x, size_y, size_z, color, x, z, rotation_y, sku, created_at,
updated_at
    FROM shapes ORDER BY created_at, id`,
createShape: `
  INSERT INTO shapes (name, kind, size_x, size_y, size_z, color, x, z, rotation_y, sku)
  VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10)
  RETURNING id, name, kind, size_x, size_y, size_z, color, x, z, rotation_y, sku, created_at,
updated_at`,
updateShape: `
  UPDATE shapes
     SET name = coalesce($2, name), color = coalesce($3, color),
         x = coalesce($4, x), z = coalesce($5, z), rotation_y = coalesce($6, rotation_y),
         updated_at = clock_timestamp()
   WHERE id = $1
  RETURNING id, name, kind, size_x, size_y, size_z, color, x, z, rotation_y, sku, created_at,
updated_at`,
deleteShape: `DELETE FROM shapes WHERE id = $1 RETURNING id`,
```

The update uses `coalesce($n, column)`: a NULL parameter means “keep what's there”. One statement handles every combination of fields a patch might carry, instead of building SQL text for each, which is where injection bugs come from.

api/src/db/postgres.ts · lines 156–182

```
async listShapes(): Promise<Shape[]> {
  const { rows } = await this.pool.query<ShapeRow>(Q.listShapes);
  return rows.map(toShape);
}

async createShape(s: NewShape): Promise<Shape> {
  const { rows } = await this.pool.query<ShapeRow>(Q.createShape, [
    s.name, s.kind, s.size.x, s.size.y, s.size.z, s.color, s.position.x, s.position.z,
s.rotationY, s.sku,
  ]);
  return toShape(rows[0]);
}
```

```
async updateShape(id: string, p: ShapePatch): Promise<Shape> {
  if (!UUID.test(id)) throw new NotFoundError(`No shape ${id}`);
  // NULL means "keep the current value" (coalesce in the SQL).
  const { rows } = await this.pool.query<ShapeRow>(Q.updateShape, [
    id, p.name ?? null, p.color ?? null, p.position?.x ?? null, p.position?.z ?? null,
p.rotationY ?? null,
  ]);
  if (!rows[0]) throw new NotFoundError(`No shape ${id}`);
  return toShape(rows[0]);
}

async deleteShape(id: string): Promise<void> {
  if (!UUID.test(id)) throw new NotFoundError(`No shape ${id}`);
  const { rowCount } = await this.pool.query(Q.deleteShape, [id]);
  if (!rowCount) throw new NotFoundError(`No shape ${id}`);
}
```

MongoDB: nested data as it is

api/src/db/mongo.ts · lines 42–43

```
/** A shape is stored almost exactly as the API returns it: documents fit nested data like size
and position. */
interface ShapeDoc extends Omit<Shape, "id" | "createdAt" | "updatedAt"> { _id: string;
createdAt: Date; updatedAt: Date }
```

api/src/db/mongo.ts · lines 219–243

```
  return (await this.shapes.find({}).sort({ createdAt: 1, _id: 1 }).toArray()).map(toShape);
}

async createShape(s: NewShape): Promise<Shape> {
  const now = new Date();
  const doc: ShapeDoc = { ...s, _id: randomUUID(), createdAt: now, updatedAt: now };
  await this.shapes.insertOne(doc);
  return toShape(doc);
}

async updateShape(id: string, patch: ShapePatch): Promise<Shape> {
  const doc = await this.shapes.findOneAndUpdate(
    { _id: id },
    { $set: { ...patch, updatedAt: new Date() } },
```

```
    { returnDocument: "after" },
  );
  if (!doc) throw new NotFoundError(`No shape ${id}`);
  return toShape(doc);
}

async deleteShape(id: string): Promise<void> {
  const r = await this.shapes.deleteOne({ _id: id });
  if (r.deletedCount !== 1) throw new NotFoundError(`No shape ${id}`);
}
```

Here the document model shines. A shape has nested `size` and `position` objects, and MongoDB stores them as they are: the document is the API response with `_id` for `id`. Postgres needs five columns and a mapping function for the same data. `$set` with the patch updates exactly the fields it carries.

Shapes saved before the `sku` field existed simply don't have it; `toShape` reads a missing `sku` as null. That's MongoDB's version of the `ALTER TABLE` above: old documents keep their old shape, and the reading code copes.

Routes, and letting another page call the API

`api/src/app.ts · lines 75–88`

```
// ---- Shapes in the robot arm's scene (Snack 1) ----
route("GET", "/api/shapes", async () => [200, await repo.listShapes()]);
route("POST", "/api/shapes", async ({ body }) => {
  const shape = parseNewShape(body);
  // A shape that stands for a part must name one that exists.
  if (shape.sku && (await repo.getParts([shape.sku])).length === 0) throw new
ValidationError([`No part ${shape.sku} in the inventory`]);
  return [201, await repo.createShape(shape)];
});
route("PATCH", "/api/shapes/:id", async ({ params, body }) => [200, await
repo.updateShape(params.id, parseShapePatch(body))]);
route("DELETE", "/api/shapes/:id", async ({ params }) => {
  await repo.deleteShape(params.id);
  return [200, { deleted: params.id }];
});
```

Four routes: list, create, patch, delete. Creating a shape for a part checks that the part exists first.

The arm app runs on its own dev server, `http://localhost:5173`, a different **origin** from the API. Browsers block a page from reading responses from another origin unless the server says it's allowed: that's **CORS**. For anything but a simple GET, the browser first sends an `OPTIONS` **preflight** asking “may I send a PATCH with a JSON body?”. The handler answers it before routing: 204 with the allowed methods and headers, or 403.

api/src/app.ts · lines 215–233

```
/**
 * Which other web pages may call this API from a browser. By default, any page served from
 * this machine (http://localhost:<any port>), which covers the robot arm's Vite dev server.
 * CORS_ORIGINS="https://a.example,https://b.example" replaces that list.
 */
function corsHeaders(origin: string | undefined): Record<string, string> | null {
  if (!origin) return null;
  const allowed = process.env.CORS_ORIGINS
    ? process.env.CORS_ORIGINS.split(",").map((o) => o.trim()).includes(origin)
    : /^http:\/\/(localhost|127\.0\.0\.1)(:\d+)?$/.test(origin);
  if (!allowed) return null;
  return {
    "access-control-allow-origin": origin,
    "access-control-allow-methods": "GET, POST, PATCH, DELETE",
    "access-control-allow-headers": "content-type",
    "access-control-max-age": "600",
    vary: "origin",
  };
}
```

By default only pages served from this machine are allowed, matched with an anchored regular expression so `http://localhost.evil.example` doesn't sneak through. `Vary: Origin` tells caches the answer depends on who's asking. In production, `CORS_ORIGINS` lists the exact sites instead. CORS protects users' browsers, not the API: a script outside a browser ignores it, which is why it's no substitute for authentication.

Tested on every database

api/tests/contract.ts · lines 192–212

```
test("shapes: create, list oldest first, move, rename, delete", async () => {
  const box = await repo.createShape({ name: "Crate", kind: "box", size: { x: 0.3, y: 0.2, z:
0.25 }, color: "#d97706", position: { x: 0.5, z: -0.2 }, rotationY: 15, sku: null });
  const can = await repo.createShape({ name: "Actuator", kind: "box", size: { x: 0.4, y: 0.06,
z: 0.06 }, color: "#2563eb", position: { x: -0.4, z: 0.3 }, rotationY: 0, sku: "LA-300" });
  assert.equal(box.sku, null);
  assert.equal(can.sku, "LA-300");
  assert.equal((await repo.listShapes())[1].sku, "LA-300");
  assert.match(box.id, /^[0-9a-f-]{36}$/);
  assert.deepEqual(box.size, { x: 0.3, y: 0.2, z: 0.25 });
  assert.deepEqual((await repo.listShapes()).map((s) => s.name), ["Crate", "Actuator"]);

  const moved = await repo.updateShape(box.id, { position: { x: 0.6, z: 0.1 }, rotationY: 90
});
  assert.deepEqual([moved.position, moved.rotationY, moved.name, moved.color], [{ x: 0.6, z:
0.1 }, 90, "Crate", "#d97706"]);
  const renamed = await repo.updateShape(box.id, { name: "Tote", color: "#15803d" });
  assert.deepEqual([renamed.name, renamed.color, renamed.position], ["Tote", "#15803d", { x:
0.6, z: 0.1 }]);
  assert.ok(Date.parse(renamed.updatedAt) >= Date.parse(renamed.createdAt));

  await repo.deleteShape(can.id);
  assert.deepEqual((await repo.listShapes()).map((s) => s.id), [box.id]);
  await assert.rejects(repo.deleteShape(can.id), NotFoundError);
  await assert.rejects(repo.updateShape("00000000-0000-4000-8000-000000000000", { rotationY: 1
}), NotFoundError);
});
```

One more contract test, so the three back ends agree on shapes too: creation with and without a SKU, oldest-first order, partial updates that leave other fields alone, and not-found on a second delete. The API tests add validation, the unknown-SKU check, and CORS: a localhost page gets the headers, `https://evil.example` gets a 403.

PART 5.13 · `web/src/api.ts`

The React page

Four views over the same data: stock, design check, reservations and the ledger.

A typed client

web/src/api.ts · lines 23–60

```
export class ApiError extends Error {
  readonly status: number;
  readonly body: Record<string, unknown>;
  constructor(status: number, message: string, body: Record<string, unknown>) {
    super(message);
    this.status = status;
    this.body = body;
  }
}

async function call<T>(method: string, path: string, body?: unknown): Promise<T> {
  const res = await fetch(`/api${path}`, {
    method,
    headers: body === undefined ? undefined : { "content-type": "application/json" },
    body: body === undefined ? undefined : JSON.stringify(body),
  });
  const data = (await res.json().catch(() => ({}))) as Record<string, unknown>;
  if (!res.ok) {
    const problems = Array.isArray(data.problems) ? `: ${(data.problems as string[]).join(";
")}` : "";
    throw new ApiError(res.status, `${String(data.error ?? res.statusText)}${problems}`, data);
  }
  return data as T;
}

export const api = {
  info: () => call<Info>("GET", "/info"),
  parts: () => call<Part[]>("GET", "/parts"),
  movements: (sku?: string, limit = 50) => call<Movement[]>("GET", `/movements?
limit=${limit}${sku ? `&sku=${encodeURIComponent(sku)}` : ""}`),
  createPart: (part: Omit<Part, "reserved" | "available" | "low" | "updatedAt">) => call<Part>
("POST", "/parts", part),
  receive: (sku: string, qty: number, note: string) => call<Part>("POST",
`/parts/${encodeURIComponent(sku)}/receive`, { qty, note }),
  consume: (sku: string, qty: number, note: string) => call<Part>("POST",
`/parts/${encodeURIComponent(sku)}/consume`, { qty, note }),
  check: (design: unknown) => call<CheckResult>("POST", "/check", { design }),
  reserve: (quoteRef: string, design: unknown) =>
```

```
    call<{ ok: true; reservation: Reservation; warnings: string[] }>("POST", "/reservations", {
quoteRef, design }),
  reservations: () => call<Reservation[]>("GET", "/reservations"),
  release: (id: string) => call<Reservation>("POST", `/reservations/${id}/release`),
  fulfil: (id: string) => call<Reservation>("POST", `/reservations/${id}/fulfil`),
};
```

One `call` function handles every request: JSON in, JSON out, and any non-2xx response becomes an `ApiError` that keeps the status and body. The design view needs the body of a 409 to show the shortages, which is why it isn't thrown away.

App state and refreshing

web/src/App.tsx · lines 18–63

```
export function App() {
  const [tab, setTab] = useState<Tab>("stock");
  const [info, setInfo] = useState<Info | null>(null);
  const [parts, setParts] = useState<Part[]>([]);
  const [reservations, setReservations] = useState<Reservation[]>([]);
  const [version, setVersion] = useState(0); // bumps after every change, so views refetch
  const [error, setError] = useState<string | null>(null);

  const refresh = useCallback(async () => {
    try {
      const [p, r, i] = await Promise.all([api.parts(), api.reservations(), api.info()]);
      setParts(p);
      setReservations(r);
      setInfo(i);
      setError(null);
    } catch (e) {
      setError(`Can't reach the API: ${(e as Error).message}`);
    }
  }, []);

  const changed = useCallback(() => {
    setVersion((v) => v + 1);
    void refresh();
  }, [refresh]);

  // Load now, then every 10 s and whenever the tab gets focus, so changes made elsewhere show
up.
```

```
  useEffect(() => {
    void refresh();
    const timer = setInterval(() => void refresh(), 10_000);
    const onFocus = () => void refresh();
    window.addEventListener("focus", onFocus);
    return () => {
      clearInterval(timer);
      window.removeEventListener("focus", onFocus);
    };
  }, [refresh]);

  const kpis = useMemo(() => {
    const active = reservations.filter((r) => r.status === "active");
    return [
      { label: "SKUs", value: parts.length },
      { label: "Low stock", value: parts.filter((p) => p.low).length, tone: parts.some((p) =>
p.low) ? "warn" : undefined },
      { label: "Units reserved", value: parts.reduce((n, p) => n + p.reserved, 0) },
      { label: "Active reservations", value: active.length },
    ];
  }, [parts, reservations]);
```

The app holds parts and reservations, and a `version` number that increases after every change. Views that load their own data (a part's history, the ledger) list `version` in their effect dependencies, so any change anywhere refetches them.

It also refreshes every 10 s and when the window regains focus. With two API replicas and possibly two users, the page can't assume it made the last change; this is the simplest way to stay current. WebSockets or server-sent events would push changes instead.

The part panel

web/src/StockView.tsx · lines 99–131

```
function PartPanel({ part, version, onChanged, onClose }: { part: Part; version: number;
onChanged: () => void; onClose: () => void }) {
  const [history, setHistory] = useState<Movement[]>([]);
  const [qty, setQty] = useState("1");
  const [note, setNote] = useState("");
  const [busy, setBusy] = useState(false);
  const [message, setMessage] = useState<{ ok: boolean; text: string } | null>(null);

  useEffect(() => {
```

```
    let live = true;
    api.movements(part.sku, 12).then((m) => live && setHistory(m)).catch(() => {});
    return () => {
      live = false;
    };
  }, [part.sku, part.updatedAt, version]);

  async function submit(kind: "receive" | "consume", e?: FormEvent) {
    e?.preventDefault();
    setBusy(true);
    setMessage(null);
    try {
      const n = Number(qty);
      const p = kind === "receive" ? await api.receive(part.sku, n, note) : await
api.consume(part.sku, n, note);
      setMessage({ ok: true, text: `${kind === "receive" ? "Received" : "Consumed"} ${n}. On
hand is now ${p.onHand}.` });
      setNote("");
      onChanged();
    } catch (err) {
      setMessage({ ok: false, text: (err as Error).message });
    } finally {
      setBusy(false);
    }
  }

  const short = Math.max(0, part.reorderLevel + 1 - part.available);
```

The effect's `live` flag ignores a response that arrives after the panel closed or switched part: without it, a slow request for one part could overwrite another's history. One `submit` serves both buttons, and errors from the API (like “Only 4 of LA-300 available”) are shown as they are.

Checking and reserving a design

web/src/DesignView.tsx · lines 23–53

```
async function check() {
  setError(null);
  try {
    setResult(await api.check(parsed()));
  } catch (e) {
    setResult(null);
```

```
    setError((e as Error).message);
  }
}

// Re-check whenever stock changes (including from other tabs or users).
useEffect(() => {
  if (result) void check();
}, [version]);

async function reserve() {
  setBusy(true);
  setOutcome(null);
  try {
    const r = await api.reserve(quoteRef, parsed());
    setOutcome({ ok: true, text: `Reserved ${r.reservation.lines.length} lines for
${r.reservation.quoteRef}.` });
    setQuoteRef(`Q-${Date.now()}`);
    onChanged();
  } catch (e) {
    const shortages = e instanceof ApiError ? (e.body.shortages as Shortage[] | undefined) :
undefined;
    setOutcome({ ok: false, text: shortages ? "Nothing was reserved: the database found these
short." : (e as Error).message, shortages });
    onChanged();
  } finally {
    setBusy(false);
  }
}
```

Reserving catches the `ApiError` and pulls `shortages` out of its body, so a refusal reads as “the database found these short”, with each line. The view calls `onChanged` even after a refusal: the refusal proves the page's numbers were stale.

The stock bar

web/src/ui.tsx · lines 3–19

```
/**
 * On hand as a bar: the reserved share, the available share, and a tick at the reorder
 * level. Scaled to whichever is bigger, stock or twice the reorder level, so a part
 * near its reorder level always looks it.
 */
export function StockBar({ part, large = false }: { part: Part; large?: boolean }) {
  const scale = Math.max(part.onHand, part.reorderLevel * 2, 1);
  const pct = (n: number) => `${Math.min(100, (n / scale) * 100)}%`;
  return (
    <div className={`bar ${large ? "large" : ""} ${part.low ? "is-low" : ""}`} role="img"
      aria-label={`${part.available} available, ${part.reserved} reserved, reorder at
${part.reorderLevel}`}>
      <span className="reserved" style={{ width: pct(part.reserved) }} />
      <span className="available" style={{ width: pct(part.available) }} />
      <span className="reorder" style={{ left: pct(part.reorderLevel) }} />
    </div>
  );
}
```

Reserved and available stack in one bar, with a tick at the reorder level. Scaling to at least twice the reorder level means a part close to reordering always looks close, even if it has a big absolute number. The `aria-label` gives screen readers the same three numbers the bar shows.

PART 5.14 · `docker-compose.yml`

Containers: Compose profiles and Kubernetes overlays

One set of images, two databases. Compose chooses with profiles; Kubernetes with kustomize overlays.

One API definition, reused

docker-compose.yml · lines 1–19

```
# Runs the inventory app with one database or the other: http://localhost:8081
#
#   docker compose --profile postgres up --build     # PostgreSQL 17
#   docker compose --profile mongo up --build        # MongoDB 8
#
# The API and web images are the same either way; only the API's DB setting changes.
# Run the repository contract tests against BOTH databases (throwaway containers, no volumes):
#
#   docker compose --profile test run --rm --build api-test
#   docker compose --profile test down
name: inventory

x-api: &api
  build: ./api
  image: inventory-api:1.0
  networks:
    default:
      aliases: [api] # nginx forwards /api to "api", whichever database is behind it
```

`x-api` is a YAML **anchor** (`&api`), merged into both API services with `<<: *api`. The network alias `api` is the trick that makes switching invisible to the front end: nginx forwards to `api`, and whichever API service is running answers to that name.

Profiles

docker-compose.yml · lines 50–82

```
api-postgres:
  <<: *api
  profiles: [postgres]
  environment:
    DB: postgres
    PGHOST: postgres
    PGUSER: inventory
    PGPASSWORD: inventory-dev-password
    PGDATABASE: inventory
  depends_on:
    postgres:
      condition: service_healthy
```

```
api-mongo:
  <<: *api
  profiles: [mongo]
  environment:
    DB: mongo
    MONGO_HOST: mongo
    MONGO_USER: inventory
    MONGO_PASSWORD: inventory-dev-password
    MONGO_DB: inventory
  depends_on:
    mongo:
      condition: service_healthy

web:
  profiles: [postgres, mongo]
  build: ./web
  image: inventory-web:1.0
  ports:
    - "8081:80"
```

A service with `profiles` only starts when that profile is named: `--profile postgres` starts Postgres, its API and the web server; `--profile mongo` the Mongo pair and the same web server. `depends_on` with `service_healthy` waits for the database's healthcheck, not just for its container to exist.

Throwaway test databases

`docker-compose.yml · lines 83–122`

```
# ---- tests: fresh databases in memory (tmpfs), so your app data is never touched ----
postgres-test:
  profiles: [test]
  image: postgres:17-alpine
  environment:
    POSTGRES_USER: inventory
    POSTGRES_PASSWORD: test
    POSTGRES_DB: inventory_test
  tmpfs: [/var/lib/postgresql/data]
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U inventory -d inventory_test"]
    interval: 2s
```

```
    retries: 30

mongo-test:
  profiles: [test]
  image: mongo:8
  tmpfs: [/data/db]
  healthcheck:
    test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping').ok"]
    interval: 2s
    retries: 30

api-test:
  profiles: [test]
  image: inventory-api:1.0
  build: ./api
  command: ["node", "--test", "tests/**/*.test.ts"]
  environment:
    LOG_REQUESTS: "0"
    PGHOST: postgres-test
    PGUSER: inventory
    PGPASSWORD: test
    PGDATABASE: inventory_test
    MONGO_URL: mongodb://mongo-test:27017
  depends_on:
    postgres-test:
      condition: service_healthy
    mongo-test:
      condition: service_healthy
```

The test profile starts separate databases on **tmpfs**, memory-backed storage that vanishes with the container. The contract tests wipe their database between tests; giving them their own instances means they can never wipe your demo data.

The API image

api/Dockerfile · lines 1–13

```
# Node 22.18+ runs TypeScript directly (type stripping), so there's no build stage:
# the image holds the production dependencies and the .ts sources.
FROM node:22-alpine
ENV NODE_ENV=production
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev && npm cache clean --force
COPY src ./src
COPY tests ./tests
# Don't run as root.
USER node
EXPOSE 8080
CMD ["node", "src/server.ts"]
```

Node 22.18+ strips TypeScript types itself, so there's no compile step and no build stage: install production dependencies, copy the sources, and run `node src/server.ts` as the unprivileged `node` user.

Kubernetes: base plus overlay

k8s/postgres/kustomization.yaml · lines 1–24

```
# kubectl apply -k k8s/postgres
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: inventory
resources:
  - ../base
  - postgres.yaml
configMapGenerator:
  - name: api-config
    literals:
      - DB=postgres
      - PGHOST=postgres
      - PGDATABASE=inventory
secretGenerator:
  # Demo credentials. In a real cluster these come from a secret manager, not the repo.
  - name: api-db-credentials
    literals:
      - PGUSER=inventory
      - PGPASSWORD=inventory-dev-password
  - name: postgres-credentials
    literals:
      - POSTGRES_USER=inventory
      - POSTGRES_PASSWORD=inventory-dev-password
      - POSTGRES_DB=inventory
```

The base (namespace, API, web) is the same for both databases. Each overlay adds its database's StatefulSet and generates the API's ConfigMap and Secret. The API Deployment reads them with `envFrom`, so the overlay alone decides which database the same image talks to.

Generated ConfigMaps get a hash suffix in their name, and kustomize rewrites the Deployment to match. Changing a setting therefore changes the Deployment, which triggers a rolling restart automatically.

PART 5.15 · `api/src/domain/conveyors.ts`

Conveyor belts in the database

The arm app's conveyor belts (Chapter 1) are stored here too, beside its shapes, so a line layout is saved with the parts it uses. The pattern is the shapes' pattern; this part shows what's the same and what's new.

The model and its limits

api/src/domain/conveyors.ts · lines 1–45

```
/**
 * Conveyor belts placed in the robot arm's scene (Snack 1), stored beside its shapes.
 *
 * A belt is a box on the floor: its top surface is at `height`, it runs along its own
 * length, and it carries what rests on it at `speed` meters per second. Positive speed
 * moves things toward the belt's +x end (before rotation); negative runs it backwards.
 */
import { ValidationError } from "./errors.ts";

export interface Conveyor {
  id: string;
  name: string;
  /** Meters along the direction of travel. */
  length: number;
  /** Meters across. */
  width: number;
  /** Height of the belt surface above the floor, meters. */
  height: number;
  /** Meters per second; negative runs it backwards. */
  speed: number;
  /** Floor position of the belt's center. */
  position: { x: number; z: number };
  /** Degrees about the vertical axis; 0 means travel along +x. */
  rotationY: number;
  createdAt: string;
  updatedAt: string;
}

export type NewConveyor = Omit<Conveyor, "id" | "createdAt" | "updatedAt">;
/** A placed belt can be moved, turned, renamed and have its speed changed. Its size is fixed.
```

```
*/
export type ConveyorPatch = Partial<Pick<Conveyor, "name" | "position" | "rotationY" |
"speed">>;

export const CONVEYOR_LIMITS = {
  length: [0.3, 4],
  width: [0.1, 1.2],
  height: [0.05, 1.2],
  speed: [-1, 1],
  floor: 3,
} as const;

type Body = Record<string, unknown>;
const asBody = (v: unknown): Body => (v && typeof v === "object" && !Array.isArray(v) ? (v as
Body) : {});
const num = (v: unknown, [min, max]: readonly [number, number]) =>
  typeof v === "number" && Number.isFinite(v) && v >= min && v <= max ? v : null;
```

Meters, meters per second and degrees, like the shapes. A positive `speed` runs toward the belt's own +x end, so turning a belt never needs its speed changed. The limits are the physical sanity checks: belts from 30 cm to 4 m, tops between 5 cm and 1.2 m, at most 1 m/s either way, and positions inside the ±3 m floor the arm app draws.

Validation with defaults

`api/src/domain/conveyors.ts · lines 46–105`

```
function field<T>(problems: string[], value: T | null, message: string): T | undefined {
  if (value === null) {
    problems.push(message);
    return undefined;
  }
  return value;
}

function name(v: unknown) {
  return typeof v === "string" && v.trim() !== "" && v.trim().length <= 40 ? v.trim() : null;
}

function position(v: unknown) {
  const p = asBody(v);
  const range = [-CONVEYOR_LIMITS.floor, CONVEYOR_LIMITS.floor] as const;
```

```
  const x = num(p.x, range);
  const z = num(p.z, range);
  return x === null || z === null ? null : { x, z };
}

const MESSAGES = {
  name: "name must be text up to 40 characters",
  length: `length must be ${CONVEYOR_LIMITS.length[0]} to ${CONVEYOR_LIMITS.length[1]} m`,
  width: `width must be ${CONVEYOR_LIMITS.width[0]} to ${CONVEYOR_LIMITS.width[1]} m`,
  height: `height must be ${CONVEYOR_LIMITS.height[0]} to ${CONVEYOR_LIMITS.height[1]} m`,
  speed: `speed must be ${CONVEYOR_LIMITS.speed[0]} to ${CONVEYOR_LIMITS.speed[1]} m/s`,
  position: `position must be { x, z } in meters, each from -${CONVEYOR_LIMITS.floor} to
${CONVEYOR_LIMITS.floor}`,
  rotationY: "rotationY must be degrees from -360 to 360",
};

export function parseNewConveyor(input: unknown): NewConveyor {
  const b = asBody(input);
  const problems: string[] = [];
  const out = {
    name: field(problems, name(b.name), MESSAGES.name),
    length: field(problems, num(b.length, CONVEYOR_LIMITS.length), MESSAGES.length),
    width: field(problems, num(b.width, CONVEYOR_LIMITS.width), MESSAGES.width),
    height: field(problems, num(b.height, CONVEYOR_LIMITS.height), MESSAGES.height),
    speed: field(problems, num(b.speed ?? 0.2, CONVEYOR_LIMITS.speed), MESSAGES.speed),
    position: field(problems, position(b.position), MESSAGES.position),
    rotationY: field(problems, num(b.rotationY ?? 0, [-360, 360]), MESSAGES.rotationY),
  };
  if (problems.length) throw new ValidationError(problems);
  return out as NewConveyor;
}

export function parseConveyorPatch(input: unknown): ConveyorPatch {
  const b = asBody(input);
  const problems: string[] = [];
  const patch: ConveyorPatch = {};
  if (b.name !== undefined) patch.name = field(problems, name(b.name), MESSAGES.name);
  if (b.position !== undefined) patch.position = field(problems, position(b.position),
MESSAGES.position);
  if (b.rotationY !== undefined) patch.rotationY = field(problems, num(b.rotationY, [-360,
360]), MESSAGES.rotationY);
```

```
  if (b.speed !== undefined) patch.speed = field(problems, num(b.speed, CONVEYOR_LIMITS.speed),
MESSAGES.speed);
  const unknown = Object.keys(b).filter((k) => !["name", "position", "rotationY",
"speed"].includes(k));
  if (unknown.length) problems.push(`can't change ${unknown.join(", ")}; delete and re-create
the belt instead`);
  if (!problems.length && Object.keys(patch).length === 0) problems.push("nothing to change:
send name, position, rotationY or speed");
  if (problems.length) throw new ValidationError(problems);
  return patch;
}
```

`parseNewConveyor` collects every problem rather than stopping at the first, so a form can show them all together; the API test sends four mistakes and gets four messages. `speed` and `rotationY` are optional with defaults, 0.2 m/s and 0°: the smallest useful request is a name, a size and a position.

`parseConveyorPatch` allows name, position, angle and speed, and refuses size by name, for the same reason shapes refuse it: the arm app treats a belt of another size as another belt.

Postgres: one more table

`api/src/db/schema.sql · lines 66–80`

```
-- Conveyor belts in the robot arm's scene. Meters, meters per second, degrees.
CREATE TABLE IF NOT EXISTS conveyors (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL CHECK (length(name) BETWEEN 1 AND 40),
  length      double precision NOT NULL CHECK (length BETWEEN 0.3 AND 4),
  width       double precision NOT NULL CHECK (width BETWEEN 0.1 AND 1.2),
  height      double precision NOT NULL CHECK (height BETWEEN 0.05 AND 1.2),
  speed       double precision NOT NULL DEFAULT 0.2 CHECK (speed BETWEEN -1 AND 1),
  x           double precision NOT NULL CHECK (x BETWEEN -3 AND 3),
  z           double precision NOT NULL CHECK (z BETWEEN -3 AND 3),
  rotation_y  double precision NOT NULL DEFAULT 0,
  created_at  timestamptz NOT NULL DEFAULT clock_timestamp(),
  updated_at  timestamptz NOT NULL DEFAULT clock_timestamp()
);
```

The limits again as `CHECK` constraints. `DEFAULT 0.2` on `speed` mirrors the API default, so a row inserted by hand behaves the same. `created_at` uses `clock_timestamp()` rather than `now()`: `now()` is the start of the transaction, so two belts inserted in one transaction would tie and their order would depend on their random UUIDs.

api/src/db/postgres-sql.ts · lines 78–92

```
listConveyors: `
  SELECT id, name, length, width, height, speed, x, z, rotation_y, created_at, updated_at
    FROM conveyors ORDER BY created_at, id`,
createConveyor: `
  INSERT INTO conveyors (name, length, width, height, speed, x, z, rotation_y)
  VALUES ($1, $2, $3, $4, $5, $6, $7, $8)
  RETURNING id, name, length, width, height, speed, x, z, rotation_y, created_at, updated_at`,
updateConveyor: `
  UPDATE conveyors
     SET name = coalesce($2, name), x = coalesce($3, x), z = coalesce($4, z),
         rotation_y = coalesce($5, rotation_y), speed = coalesce($6, speed),
         updated_at = clock_timestamp()
   WHERE id = $1
  RETURNING id, name, length, width, height, speed, x, z, rotation_y, created_at, updated_at`,
deleteConveyor: `DELETE FROM conveyors WHERE id = $1 RETURNING id`,
```

The same `coalesce($n, column)` partial update as shapes. Because the API layer has already refused unknown fields, the SQL never needs to be assembled from user input: one fixed statement with six parameters covers every patch.

MongoDB: the document is the belt

api/src/db/mongo.ts · lines 44–45

```
/** Conveyors too: the nested position stays nested. */
interface ConveyorDoc extends Omit<Conveyor, "id" | "createdAt" | "updatedAt"> { _id: string;
createdAt: Date; updatedAt: Date }
```

api/src/db/mongo.ts · lines 244–268

```
async listConveyors(): Promise<Conveyor[]> {
  return (await this.conveyors.find({}).sort({ createdAt: 1, _id: 1
}).toArray()).map(toConveyor);
}

async createConveyor(c: NewConveyor): Promise<Conveyor> {
  const now = new Date();
  const doc: ConveyorDoc = { ...c, _id: randomUUID(), createdAt: now, updatedAt: now };
  await this.conveyors.insertOne(doc);
  return toConveyor(doc);
}
```

```
async updateConveyor(id: string, patch: ConveyorPatch): Promise<Conveyor> {
  const doc = await this.conveyors.findOneAndUpdate({ _id: id }, { $set: { ...patch, updatedAt:
new Date() } }, { returnDocument: "after" });
  if (!doc) throw new NotFoundError(`No conveyor ${id}`);
  return toConveyor(doc);
}

async deleteConveyor(id: string): Promise<void> {
  const r = await this.conveyors.deleteOne({ _id: id });
  if (r.deletedCount !== 1) throw new NotFoundError(`No conveyor ${id}`);
  // No foreign keys here: delete the belt's splitters ourselves. A splitter created for this
  // belt in the instant between the two deletes would be left pointing at nothing; the arm
  // app ignores splitters whose belt is gone, and Postgres's foreign key rules it out.
  await this.splitters.deleteMany({ conveyorId: id });
}
```

As with shapes, the document is the API object with `_id` for `id`, nested `position` and all. `findOneAndUpdate` with `returnDocument: "after"` changes and returns the belt in one round trip. In Postgres, `RETURNING` does the same job.

Routes and tests

`api/src/app.ts · lines 89–96`

```
// ---- Conveyor belts in the robot arm's scene ----
route("GET", "/api/conveyors", async () => [200, await repo.listConveyors()]);
route("POST", "/api/conveyors", async ({ body }) => [201, await
repo.createConveyor(parseNewConveyor(body))]);
route("PATCH", "/api/conveyors/:id", async ({ params, body }) => [200, await
repo.updateConveyor(params.id, parseConveyorPatch(body))]);
route("DELETE", "/api/conveyors/:id", async ({ params }) => {
  await repo.deleteConveyor(params.id);
  return [200, { deleted: params.id }];
});
```

api/tests/contract.ts · lines 214–225

```
test("conveyors: create, list, move, change speed, delete", async () => {
  const a = await repo.createConveyor({ name: "Infeed", length: 1.2, width: 0.3, height: 0.25,
speed: 0.2, position: { x: 0.6, z: -0.5 }, rotationY: 0 });
  const b = await repo.createConveyor({ name: "Outfeed", length: 0.8, width: 0.25, height:
0.15, speed: -0.1, position: { x: -0.4, z: 0.6 }, rotationY: 90 });
  assert.match(a.id, /^[0-9a-f-]{36}$/);
  assert.deepEqual((await repo.listConveyors()).map((c) => [c.name, c.speed]), [["Infeed",
0.2], ["Outfeed", -0.1]]);
  const moved = await repo.updateConveyor(a.id, { position: { x: 0.5, z: -0.6 }, speed: 0.35
});
  assert.deepEqual([moved.position, moved.speed, moved.length, moved.rotationY], [{ x: 0.5, z:
-0.6 }, 0.35, 1.2, 0]);
  await repo.deleteConveyor(b.id);
  assert.deepEqual((await repo.listConveyors()).map((c) => c.id), [a.id]);
  await assert.rejects(repo.deleteConveyor(b.id), NotFoundError);
  await assert.rejects(repo.updateConveyor("00000000-0000-4000-8000-000000000000", { speed: 0
}), NotFoundError);
});
```

The contract test runs on all three back ends: two belts in creation order, a partial update that changes position and speed and leaves length and angle alone, and not-found for a deleted belt and for a well-formed id that never existed. The Postgres SQL was also run against a real PostgreSQL 16, including the `CHECK` on length.

PART 5.16 · `api/src/domain/splitters.ts`

Splitters: sorting rules as data, and editable parts

The arm app's splitters (Chapter 1) send items left, right or straight on by rules. The rules are stored here, beside the belts, and the parts they test can now be edited, so the sorting can be changed by changing the database.

The model

api/src/domain/splitters.ts · lines 1–76

```
/**
 * Splitters: diverter gates on a conveyor belt that send each item left, right or straight on,
 * by rules you write. The rules are data, stored here, so they can be changed from the arm
app,
```

```
 * the API or the database directly, even while the line is running.
 *
 * A splitter has an ordered list of rules. Each rule is one or more conditions (all must hold)
 * and an exit. The first rule that matches an item decides its exit; if none matches, the
 * splitter's `otherwise` exit does.
 *
 * Conditions can test the shape itself (kind, color, size, name, SKU) or the inventory part it
 * stands for (category, location, stock). Part fields are read when the item reaches the gate,
 * so changing a part in the inventory changes where its items go.
 */
import { ValidationError } from "./errors.ts";

export const EXITS = ["left", "straight", "right"] as const;
export type Exit = (typeof EXITS)[number];

/** Every field a condition can test, and its type. Sizes are meters, like the rest of the API.
*/
export const FIELDS = {
  kind: "text",
  color: "text",
  name: "text",
  sku: "text",
  width: "number",
  height: "number",
  depth: "number",
  isPart: "boolean",
  "part.category": "text",
  "part.location": "text",
  "part.name": "text",
  "part.onHand": "number",
  "part.available": "number",
  "part.low": "boolean",
} as const;
export type Field = keyof typeof FIELDS;
type FieldType = (typeof FIELDS)[Field];

export const OPS: Record<FieldType, readonly Op[]> = {
  text: ["eq", "ne", "in", "contains"],
  number: ["eq", "ne", "lt", "lte", "gt", "gte"],
  boolean: ["eq"],
};
```

```
export type Op = "eq" | "ne" | "in" | "contains" | "lt" | "lte" | "gt" | "gte";

export interface Condition {
  field: Field;
  op: Op;
  /** Text, a number, true/false, or a list of texts for "in". */
  value: string | number | boolean | string[];
}

export interface Rule {
  conditions: Condition[];
  exit: Exit;
}

export interface Splitter {
  id: string;
  name: string;
  conveyorId: string;
  /** Meters from the start of the belt (its -x end before turning) to the gate. */
  along: number;
  rules: Rule[];
  /** Where an item goes when no rule matches. */
  otherwise: Exit;
  createdAt: string;
  updatedAt: string;
}

export type NewSplitter = Omit<Splitter, "id" | "createdAt" | "updatedAt">;
/** Everything but the belt can change: move a gate to another belt by re-creating it. */
export type SplitterPatch = Partial<Pick<Splitter, "name" | "along" | "rules" | "otherwise">>;

export const MAX_RULES = 20;
export const MAX_CONDITIONS = 5;
```

A splitter belongs to a belt (`conveyorId`) and sits `along` meters from the belt's start. Its rules are an ordered list; each rule is up to five conditions and an exit; `otherwise` catches the rest. The field and operator tables are the same as the arm app's, so a rule the API accepts is a rule the arm app can evaluate.

Validating a rule list

api/src/domain/splitters.ts · lines 78–180

```
type Body = Record<string, unknown>;
const asBody = (v: unknown): Body => (v && typeof v === "object" && !Array.isArray(v) ? (v as
Body) : {});

/** Checks one condition, returning a normalized copy or a problem. Texts are compared ignoring
case. */
function parseCondition(c: unknown, where: string, problems: string[]): Condition | null {
  const b = asBody(c);
  const field = b.field as Field;
  if (!(typeof field === "string" && field in FIELDS)) {
    problems.push(`${where}: field must be one of ${Object.keys(FIELDS).join(", ")}`);
    return null;
  }
  const type = FIELDS[field];
  const op = b.op as Op;
  if (!OPS[type].includes(op)) {
    problems.push(`${where}: ${field} can be compared with ${OPS[type].join(", ")}`);
    return null;
  }
  const v = b.value;
  if (type === "boolean") {
    if (typeof v !== "boolean") return problems.push(`${where}: value must be true or false`),
null;
    return { field, op, value: v };
  }
  if (type === "number") {
    if (typeof v !== "number" || !Number.isFinite(v)) return problems.push(`${where}: value
must be a number`), null;
    return { field, op, value: v };
  }
  if (op === "in") {
    const list = Array.isArray(v) ? v : null;
    if (!list || list.length === 0 || list.length > 20 || !list.every((s) => typeof s ===
"string" && s.trim() !== "" && s.length <= 80)) {
      return problems.push(`${where}: value must be a list of 1 to 20 texts`), null;
    }
    return { field, op, value: (list as string[]).map((s) => s.trim().toLowerCase()) };
```

```
  }
  if (typeof v !== "string" || v.trim() === "" || v.length > 80) return
problems.push(`${where}: value must be text up to 80 characters`), null;
  return { field, op, value: v.trim().toLowerCase() };
}

function parseRules(v: unknown, problems: string[]): Rule[] | undefined {
  if (!Array.isArray(v) || v.length > MAX_RULES) {
    problems.push(`rules must be a list of at most ${MAX_RULES} rules`);
    return undefined;
  }
  const rules: Rule[] = [];
  v.forEach((r, i) => {
    const b = asBody(r);
    const where = `rule ${i + 1}`;
    if (!EXITS.includes(b.exit as Exit)) problems.push(`${where}: exit must be left, straight
or right`);
    const cs = b.conditions;
    if (!Array.isArray(cs) || cs.length === 0 || cs.length > MAX_CONDITIONS) {
      problems.push(`${where}: conditions must be a list of 1 to ${MAX_CONDITIONS}`);
      return;
    }
    const conditions = cs.map((c, j) => parseCondition(c, `${where}, condition ${j + 1}`,
problems));
    if (conditions.every((c) => c !== null) && EXITS.includes(b.exit as Exit)) rules.push({
conditions: conditions as Condition[], exit: b.exit as Exit });
  });
  return rules;
}

const name = (v: unknown) => (typeof v === "string" && v.trim() !== "" && v.trim().length <= 40
? v.trim() : null);
const along = (v: unknown) => (typeof v === "number" && Number.isFinite(v) && v >= 0 && v <= 4
? v : null);

/** Shape checks only; that `along` fits on the belt is checked against the belt in app.ts. */
export function parseNewSplitter(input: unknown): NewSplitter {
  const b = asBody(input);
  const problems: string[] = [];
  const n = name(b.name);
  if (n === null) problems.push("name must be text up to 40 characters");
```

```
  if (typeof b.conveyorId !== "string" || !UUID.test(b.conveyorId)) problems.push("conveyorId
must be a conveyor's id");
  const a = along(b.along);
  if (a === null) problems.push("along must be meters from the start of the belt, 0 to 4");
  const otherwise = (b.otherwise ?? "straight") as Exit;
  if (!EXITS.includes(otherwise)) problems.push("otherwise must be left, straight or right");
  const rules = parseRules(b.rules ?? [], problems);
  if (problems.length) throw new ValidationError(problems);
  return { name: n!, conveyorId: b.conveyorId as string, along: a!, rules: rules!, otherwise };
}

export function parseSplitterPatch(input: unknown): SplitterPatch {
  const b = asBody(input);
  const problems: string[] = [];
  const patch: SplitterPatch = {};
  if (b.name !== undefined) {
    const n = name(b.name);
    if (n === null) problems.push("name must be text up to 40 characters");
    else patch.name = n;
  }
  if (b.along !== undefined) {
    const a = along(b.along);
    if (a === null) problems.push("along must be meters from the start of the belt, 0 to 4");
    else patch.along = a;
  }
  if (b.otherwise !== undefined) {
    if (!EXITS.includes(b.otherwise as Exit)) problems.push("otherwise must be left, straight
or right");
    else patch.otherwise = b.otherwise as Exit;
  }
  if (b.rules !== undefined) patch.rules = parseRules(b.rules, problems);
  const unknown = Object.keys(b).filter((k) => !["name", "along", "rules",
"otherwise"].includes(k));
  if (unknown.length) problems.push(`can't change ${unknown.join(", ")}; delete and re-create
the splitter instead`);
  if (!problems.length && Object.keys(patch).length === 0) problems.push("nothing to change:
send name, along, rules or otherwise");
  if (problems.length) throw new ValidationError(problems);
  return patch;
}
```

Validation walks every rule and condition and reports each problem with its position, “rule 2, condition 1: kind can be compared with eq, ne, in, contains”, so whoever wrote the rule (a person in the panel, or a script against the API) can find it. The operator must suit the field's type, and the value must suit the operator: a number, true/false, text, or a list of texts for `in`. Text values are trimmed and lowercased once here, so the stored rules are already in the form they're compared in.

A patch replaces the **whole** rule list. Reordering, inserting and deleting rules are then all one operation, and two people editing the same splitter can't interleave half of each other's changes: the last save wins, whole.

Postgres: rules as jsonb

`api/src/db/schema.sql · lines 82–96`

```
-- Splitters: diverter gates on a belt, with their sorting rules. The rules are a list of
-- objects that the API validates and only the arm app interprets, so they're stored as one
-- jsonb value rather than spread over rule and condition tables. Deleting a belt deletes its
-- splitters (ON DELETE CASCADE).
CREATE TABLE IF NOT EXISTS splitters (
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name         text NOT NULL CHECK (length(name) BETWEEN 1 AND 40),
  conveyor_id  uuid NOT NULL REFERENCES conveyors (id) ON DELETE CASCADE,
  along        double precision NOT NULL CHECK (along BETWEEN 0 AND 4),
  rules        jsonb NOT NULL DEFAULT '[]' CHECK (jsonb_typeof(rules) = 'array' AND
jsonb_array_length(rules) <= 20),
  otherwise    text NOT NULL DEFAULT 'straight' CHECK (otherwise IN ('left', 'straight',
'right')),
  created_at   timestamptz NOT NULL DEFAULT clock_timestamp(),
  updated_at   timestamptz NOT NULL DEFAULT clock_timestamp()
);
CREATE INDEX IF NOT EXISTS splitters_conveyor ON splitters (conveyor_id);
```

The rules go in one `jsonb` column rather than `rules` and `conditions` tables. The database never queries inside a rule; only the arm app interprets them, and they're always read and written as a whole list. Splitting them into tables would add two joins and an ordering column to rebuild a value that's naturally a document. The `CHECK` still guards the outline: it must be an array of at most 20.

`REFERENCES conveyors (id) ON DELETE CASCADE` ties a splitter to its belt: delete the belt and its splitters go with it, in the same statement, and a splitter for a belt that doesn't exist can't be inserted.

api/src/db/postgres-sql.ts · lines 93–107

```
listSplitters: `
  SELECT id, name, conveyor_id, along, rules, otherwise, created_at, updated_at
    FROM splitters ORDER BY created_at, id`,
createSplitter: `
  INSERT INTO splitters (name, conveyor_id, along, rules, otherwise)
  VALUES ($1, $2, $3, $4::jsonb, $5)
  RETURNING id, name, conveyor_id, along, rules, otherwise, created_at, updated_at`,
updateSplitter: `
  UPDATE splitters
     SET name = coalesce($2, name), along = coalesce($3, along),
         rules = coalesce($4::jsonb, rules), otherwise = coalesce($5, otherwise),
         updated_at = clock_timestamp()
   WHERE id = $1
  RETURNING id, name, conveyor_id, along, rules, otherwise, created_at, updated_at`,
deleteSplitter: `DELETE FROM splitters WHERE id = $1 RETURNING id`,
```

api/src/db/postgres.ts · lines 211–235

```
async listSplitters(): Promise<Splitter[]> {
  const { rows } = await this.pool.query<SplitterRow>(Q.listSplitters);
  return rows.map(toSplitter);
}

async createSplitter(s: NewSplitter): Promise<Splitter> {
  // The driver turns jsonb into objects on the way out; on the way in it needs a JSON string.
  const { rows } = await this.pool.query<SplitterRow>(Q.createSplitter, [s.name, s.conveyorId,
s.along, JSON.stringify(s.rules), s.otherwise]);
  return toSplitter(rows[0]);
}

async updateSplitter(id: string, p: SplitterPatch): Promise<Splitter> {
  if (!UUID.test(id)) throw new NotFoundError(`No splitter ${id}`);
  const { rows } = await this.pool.query<SplitterRow>(Q.updateSplitter, [
    id, p.name ?? null, p.along ?? null, p.rules ? JSON.stringify(p.rules) : null, p.otherwise
?? null,
  ]);
  if (!rows[0]) throw new NotFoundError(`No splitter ${id}`);
  return toSplitter(rows[0]);
}
```

```
async deleteSplitter(id: string): Promise<void> {
  if (!UUID.test(id)) throw new NotFoundError(`No splitter ${id}`);
  const { rowCount } = await this.pool.query(Q.deleteSplitter, [id]);
  if (!rowCount) throw new NotFoundError(`No splitter ${id}`);
}
```

The `pg` driver parses `jsonb` into objects when reading, but needs a JSON string when writing, hence `JSON.stringify(s.rules)` and the `$4::jsonb` cast. `coalesce($4::jsonb, rules)` keeps the old rules when the patch doesn't include any.

MongoDB: no foreign keys

api/src/db/mongo.ts · lines 261–290

```
async deleteConveyor(id: string): Promise<void> {
  const r = await this.conveyors.deleteOne({ _id: id });
  if (r.deletedCount !== 1) throw new NotFoundError(`No conveyor ${id}`);
  // No foreign keys here: delete the belt's splitters ourselves. A splitter created for this
  // belt in the instant between the two deletes would be left pointing at nothing; the arm
  // app ignores splitters whose belt is gone, and Postgres's foreign key rules it out.
  await this.splitters.deleteMany({ conveyorId: id });
}

async listSplitters(): Promise<Splitter[]> {
  return (await this.splitters.find({}).sort({ createdAt: 1, _id: 1
}).toArray()).map(toSplitter);
}

async createSplitter(s: NewSplitter): Promise<Splitter> {
  const now = new Date();
  const doc: SplitterDoc = { ...s, _id: randomUUID(), createdAt: now, updatedAt: now };
  await this.splitters.insertOne(doc);
  return toSplitter(doc);
}

async updateSplitter(id: string, patch: SplitterPatch): Promise<Splitter> {
  const doc = await this.splitters.findOneAndUpdate({ _id: id }, { $set: { ...patch, updatedAt:
new Date() } }, { returnDocument: "after" });
  if (!doc) throw new NotFoundError(`No splitter ${id}`);
  return toSplitter(doc);
}
```

```
async deleteSplitter(id: string): Promise<void> {
  const r = await this.splitters.deleteOne({ _id: id });
  if (r.deletedCount !== 1) throw new NotFoundError(`No splitter ${id}`);
}
```

The document holds the rules as they are, nested lists and all. MongoDB has no foreign keys, so `deleteConveyor` deletes the belt's splitters itself, in a second step. The comment says what that costs: a splitter created in the instant between the two deletes would point at a belt that's gone. The arm app skips splitters whose belt is missing, so it's harmless here; in a system where it mattered, a transaction on a replica set would close the gap.

Routes: does it fit on the belt?

api/src/app.ts · lines 98–123

```
// ---- Splitters: diverter gates on belts, with their sorting rules ----
/** The gate has to be on its belt: between the belt's start (0) and its length. */
async function checkFits(conveyorId: string, along: number) {
  const belt = (await repo.listConveyors()).find((c) => c.id === conveyorId);
  if (!belt) throw new ValidationError([`No conveyor ${conveyorId}`]);
  if (along > belt.length) throw new ValidationError([`along must be at most ${belt.length} m,
the length of ${belt.name}`]);
}
route("GET", "/api/splitters", async () => [200, await repo.listSplitters()]);
route("POST", "/api/splitters", async ({ body }) => {
  const s = parseNewSplitter(body);
  await checkFits(s.conveyorId, s.along);
  return [201, await repo.createSplitter(s)];
});
route("PATCH", "/api/splitters/:id", async ({ params, body }) => {
  const patch = parseSplitterPatch(body);
  if (patch.along !== undefined) {
    const current = (await repo.listSplitters()).find((s) => s.id === params.id);
    if (!current) throw new NotFoundError(`No splitter ${params.id}`);
    await checkFits(current.conveyorId, patch.along);
  }
  return [200, await repo.updateSplitter(params.id, patch)];
});
route("DELETE", "/api/splitters/:id", async ({ params }) => {
  await repo.deleteSplitter(params.id);
  return [200, { deleted: params.id }];
});
```

“The gate is on its belt” involves two records, so it's checked in the route: find the belt, compare `along` with its length. A belt's length can never change (its patch refuses size), so the check can't be invalidated later. The error names the belt and its length: “along must be at most 1.6 m, the length of Sort belt”.

Editing parts, so the database can steer the sorting

api/src/domain/newPart.ts · lines 45–77

```
/** What can change about an existing part. Stock changes go through receive and consume, so
the ledger explains them. */
export type PartPatch = Partial<Pick<SeedPart, "name" | "category" | "location" |
"reorderLevel">>;

export function parsePartPatch(input: unknown): PartPatch {
  const b = (input && typeof input === "object" && !Array.isArray(input) ? input : {}) as
Record<string, unknown>;
  const problems: string[] = [];
  const patch: PartPatch = {};
  if (b.name !== undefined) {
    if (typeof b.name !== "string" || b.name.trim() === "" || b.name.trim().length > 80)
problems.push("name must be text up to 80 characters");
    else patch.name = b.name.trim();
  }
  if (b.category !== undefined) {
    if (!CATEGORIES.includes(b.category as Category)) problems.push(`category must be one of
${CATEGORIES.join(", ")}`);
    else patch.category = b.category as Category;
  }
  if (b.location !== undefined) {
    if (typeof b.location !== "string" || b.location.trim().length > 20)
problems.push("location must be text up to 20 characters");
    else patch.location = b.location.trim();
  }
  if (b.reorderLevel !== undefined) {
    const v = b.reorderLevel;
    if (typeof v !== "number" || !Number.isInteger(v) || v < 0 || v > MAX_QTY)
problems.push(`reorderLevel must be a whole number from 0 to ${MAX_QTY}`);
    else patch.reorderLevel = v;
  }
  const unknown = Object.keys(b).filter((k) => !["name", "category", "location",
"reorderLevel"].includes(k));
```

```
  if (unknown.length) {
    const stock = unknown.filter((k) => ["onHand", "reserved", "available"].includes(k));
    problems.push(stock.length ? "stock changes go through receive and consume, so the ledger
records them" : `can't change ${unknown.join(", ")}`);
  }
  if (!problems.length && Object.keys(patch).length === 0) problems.push("nothing to change:
send name, category, location or reorderLevel");
  if (problems.length) throw new ValidationError(problems);
  return patch;
}
```

Rules can test a part's category, location and stock, so changing a part changes where its items go. Stock already changes through receive and consume. This adds `PATCH /api/parts/{sku}` for the rest: name, category, location and reorder level. Stock fields are refused with a reason: a direct edit to `onHand` would bypass the ledger, and every change to stock must be explained by a movement.

api/src/db/postgres-sql.ts · lines 25–30

```
updatePart: `
  UPDATE parts
     SET name = coalesce($2, name), category = coalesce($3, category),
         location = coalesce($4, location), reorder_level = coalesce($5, reorder_level),
updated_at = now()
   WHERE sku = $1
  RETURNING sku, name, category, unit, on_hand, reserved, reorder_level, location, updated_at`,
```

One `coalesce` statement again. The category column has no `CHECK`, so with SQL you can give a part a category of your own, and write a splitter rule for it; the API's list of categories is for the forms.

Tested on every database

api/tests/contract.ts · lines 227–251

```
test("splitters: rules stored as written, updated whole, deleted with their belt", async () =>
{
  const belt = await repo.createConveyor({ name: "Sorter", length: 2, width: 0.4, height: 0.3,
speed: 0.3, position: { x: 0.8, z: 0 }, rotationY: 0 });
  const rules = [
    { conditions: [{ field: "part.category" as const, op: "eq" as const, value: "plate" }],
exit: "left" as const },
    { conditions: [{ field: "kind" as const, op: "in" as const, value: ["sphere", "cylinder"]
}, { field: "height" as const, op: "gt" as const, value: 0.1 }], exit: "right" as const },
```

```
  ];
  const s = await repo.createSplitter({ name: "Gate 1", conveyorId: belt.id, along: 1.2, rules,
otherwise: "straight" });
  assert.deepEqual([s.conveyorId, s.along, s.rules, s.otherwise], [belt.id, 1.2, rules,
"straight"]);
  // Reordering is a whole-list update; other fields stay.
  const swapped = await repo.updateSplitter(s.id, { rules: [rules[1], rules[0]], otherwise:
"left" });
  assert.deepEqual([swapped.rules.map((r) => r.exit), swapped.otherwise, swapped.along,
swapped.name], [["right", "left"], "left", 1.2, "Gate 1"]);
  assert.deepEqual((await repo.listSplitters()).map((x) => x.id), [s.id]);
  await repo.deleteConveyor(belt.id);
  assert.deepEqual(await repo.listSplitters(), []);
  await assert.rejects(repo.updateSplitter(s.id, { name: "x" }), NotFoundError);
  await assert.rejects(repo.deleteSplitter(s.id), NotFoundError);
});

test("updatePart changes details but never stock", async () => {
  const before = (await repo.getParts(["PL-200"]))[0];
  const p = await repo.updatePart("PL-200", { category: "gripper", location: "Z-9" });
  assert.deepEqual([p.category, p.location, p.name, p.onHand, p.reorderLevel], ["gripper", "Z-
9", before.name, before.onHand, before.reorderLevel]);
  assert.equal((await repo.getParts(["PL-200"]))[0].category, "gripper");
  await assert.rejects(repo.updatePart("NOPE-1", { location: "A" }), NotFoundError);
});
```

The splitter contract test stores two rules (one with two conditions and an `in` list) and expects them back exactly, swaps their order with a whole-list update, then deletes the belt and expects the splitter gone: that's `ON DELETE CASCADE` in Postgres, and the explicit delete in MongoDB and memory. The SQL was also run on a real PostgreSQL 16, including the `CHECK`s on the rule list and the exit, and the foreign key.

api/tests/api.test.ts · lines 159–191

```
test("splitters: validate rules, fit on the belt, patch, delete", async () => {
  const belt = (await call("POST", "/api/conveyors", { name: "Sort belt", length: 1.6, width:
0.4, height: 0.3, position: { x: 0.8, z: 0 } })).body;
  const rule = { conditions: [{ field: "color", op: "eq", value: " #E11D48 " }], exit: "left"
};
  const made = await call("POST", "/api/splitters", { name: "Reds", conveyorId: belt.id, along:
1, rules: [rule] });
  assert.equal(made.status, 201);
```

```
  assert.deepEqual([made.body.rules[0].conditions[0].value, made.body.otherwise], ["#e11d48",
"straight"]); // trimmed, lowercased; default exit

  const bad = await call("POST", "/api/splitters", { name: "Bad", conveyorId: belt.id, along:
1, rules: [
    { conditions: [{ field: "weight", op: "eq", value: 1 }], exit: "left" },
    { conditions: [{ field: "kind", op: "gt", value: "box" }], exit: "up" },
    { conditions: [], exit: "right" },
  ] });
  assert.equal(bad.status, 400);
  assert.deepEqual(bad.body.problems.map((p: string) => p.split(":")[0]), ["rule 1, condition
1", "rule 2", "rule 2, condition 1", "rule 3"]);
  const tooFar = await call("POST", "/api/splitters", { name: "Off the end", conveyorId:
belt.id, along: 2, rules: [] });
  assert.match(tooFar.body.problems[0], /at most 1.6 m/);
  assert.equal((await call("PATCH", `/api/splitters/${made.body.id}`, { along: 1.7 })).status,
400);
  assert.equal((await call("PATCH", `/api/splitters/${made.body.id}`, { conveyorId: belt.id
})).status, 400);
  const moved = await call("PATCH", `/api/splitters/${made.body.id}`, { along: 0.5, otherwise:
"right", rules: [] });
  assert.deepEqual([moved.body.along, moved.body.otherwise, moved.body.rules], [0.5, "right",
[]]);
  assert.equal((await call("DELETE", `/api/splitters/${made.body.id}`)).status, 200);
  assert.equal((await call("DELETE", `/api/conveyors/${belt.id}`)).status, 200);
});

test("parts: change details, not stock", async () => {
  const p = await call("PATCH", "/api/parts/MC-1", { category: "motor", location: "C-01-1" });
  assert.deepEqual([p.status, p.body.category, p.body.location], [200, "motor", "C-01-1"]);
  const stock = await call("PATCH", "/api/parts/MC-1", { onHand: 99 });
  assert.equal(stock.status, 400);
  assert.match(stock.body.problems[0], /receive and consume/);
  assert.equal((await call("PATCH", "/api/parts/MC-1", { category: "widget" })).status, 400);
  assert.equal((await call("PATCH", "/api/parts/NOPE-1", { location: "A" })).status, 404);
});
```

CHAPTER 5

Interview questions

Q1 **Why both PostgreSQL and MongoDB? Which would you choose?**

To show the same business rules on two consistency models. For this domain I'd choose Postgres: stock, reservations and orders are relational, constraints catch bugs, and multi-row transactions make all-or-nothing trivial. MongoDB fits better when records have varied shapes (a catalog where every part type has different attributes) or need to scale horizontally, and its single-document atomicity is enough when each operation touches one document.

Q2 **Two people reserve the last actuator at the same moment. Walk me through what happens on each database.**

Postgres: both call `inv_reserve`; the first locks the part row with `FOR UPDATE`, the second waits. The first checks, reserves and commits; the second then gets the lock, sees zero available and returns the shortage. MongoDB: both send a conditional `$inc`; the server applies them one at a time, the first matches the filter, the second no longer does and modifies nothing, so the saga undoes any earlier holds and reports the shortage. Either way, exactly one succeeds.

Q3 **What's a lost update, and how do you prevent it?**

Two transactions read the same value, each computes a new one, and the second write overwrites the first. Prevent it with a lock (`SELECT … FOR UPDATE`), a conditional or atomic update (`SET x = x - 1 WHERE x >= 1`), optimistic version checks, or a stricter isolation level with retries. Back it with a constraint so the bad state can't be stored at all.

Q4 **Two people add a part with the same SKU at the same moment. What stops a duplicate?**

The database's unique index, not a check in the code. A “does it exist?” query followed by an insert leaves a gap a second request can slip through. In Postgres the insert says `ON CONFLICT (sku) DO NOTHING`, and an empty `RETURNING` means someone else won; in MongoDB the SKU is the `_id`, so the second `insertOne` fails with duplicate-key error 11000. Both become a 409. The form's own “already exists” message is only a convenience: it can be out of date by the time the button is clicked.

Q5 **How do you avoid deadlocks?**

Lock resources in a consistent global order (here, by SKU), keep transactions short, and don't wait on user input inside one. Postgres detects deadlocks and aborts one transaction, so code should also be ready to retry, but with ordered locking it shouldn't happen.

Q6 **MongoDB has transactions. Why didn't you use one?**

Multi-document transactions need a replica set; the demo runs a standalone container to keep setup simple. Single-document atomic updates already prevent over-reservation; the saga adds all-or-nothing with compensation. On a replica set I'd wrap the same steps in `withTransaction`, which removes the window where a partial hold is visible.

Q7 **What does the repository pattern buy you, and what does it cost?**

Storage can change without touching routes or UI, tests can use an in-memory version, and the domain operations get names. The cost is a lowest common denominator: features one database is great at (Postgres window functions, Mongo aggregation pipelines) have to be expressed through the interface, or the interface grows database-specific methods.

Q8 **How do you know the three implementations really behave the same?**

One contract suite, nineteen tests, run against each: creating parts (including duplicate SKUs), filtering, ledger entries, consuming around reservations, all-or-nothing with every shortage reported, concurrent reservations and concurrent closes. A database-specific quirk shows up as one run failing.

Q9 **How is the database set up? Is there a migration step?**

The API does it on start: wait for the database (Compose healthcheck, plus a retry loop for Kubernetes), run an idempotent schema script, and seed only if the parts table is empty. That's fine for a demo and safe with several replicas. A production system would use versioned migrations (node-pg-migrate, Flyway) so schema changes are reviewed, ordered and reversible, and would run them once per deploy instead of from every pod.

Q10 **How does the API behave in Kubernetes when the database goes down?**

Liveness only checks the process, so pods aren't restarted in a loop. Readiness checks the database, so pods are taken out of the service and nginx gets errors quickly instead of hanging. When the database returns, readiness passes and traffic resumes. On first start, the API retries the connection for up to a minute instead of crashing.

Q11 **How would you scale this to thousands of SKUs and many warehouses?**

Paginate parts with keyset pagination (by SKU) instead of returning everything; index search with a trigram index (`pg_trgm`) instead of a scan; add a `warehouse_id` to stock rows so a part has a row per location; and move refreshes from polling to server-sent events. Reservation contention is per SKU, so it parallelises well across parts.

Q12 **The arm app runs on another port. How can it call your API, and is that safe?**

Browsers block cross-origin reads unless the server allows them with CORS headers. The API answers the OPTIONS preflight and adds `Access-Control-Allow-Origin` for pages on localhost by default, or an explicit list in production. It's safe in the sense that matters: CORS only controls what browsers let other sites' scripts read. It isn't access control, so a real deployment still needs authentication on the API.

Q13 **Why store a splitter's rules as jsonb instead of rule and condition tables?**

The database never queries inside a rule; rules are always read and written as one ordered list, and only the client interprets them. jsonb keeps that value whole, with a CHECK on its outline, and avoids two joins and an ordering column. If we needed to ask “which splitters test part.category?” across thousands of them, a jsonb path query or a GIN index would do it, and that's when I'd reconsider tables.

CHAPTER 5

Exercises

5.1 Reservations that expire

Give reservations an `expires_at` and release expired ones automatically, on both databases.

Hint. A periodic job calling `release` for each expired, still-active reservation reuses all the existing locking and ledger logic.

5.2 Mongo with a transaction

Run MongoDB as a one-node replica set in Compose and implement `reserve` inside `session.withTransaction`. Keep the contract tests green.

Hint. Start mongod with `--replSet rs0` and run `rs.initiate()` once from a healthcheck or init container. Pass `{ session }` to every operation inside the callback.

5.3 Idempotent reservations

Make reserving the same `quoteRef` twice return the existing reservation instead of creating a second one.

Hint. A unique index on `quote_ref` (Postgres) or `quoteRef` (Mongo), and handling the duplicate-key error by reading the existing reservation.

5.4 Reorder suggestions

Add `GET /api/reorder` returning, for every low part, how many to order to reach twice its reorder level, and show it on the stock page.

Hint. Write it once in the service from `listParts({ lowOnly: true })`, not in each repository: it's arithmetic on data every repository already returns.

5.5 Optimistic edits

Add editing of a part's name, bin and reorder level with optimistic concurrency: a `version` field, and 409 if it changed since the form loaded.

Hint. `UPDATE … SET …, version = version + 1 WHERE sku = $1 AND version = $2`; in Mongo, the same condition in the filter and `$inc: { version: 1 }`.

APPENDIX A

Running the projects

Every project needs Node.js 20 or newer. From `Portfolio/vention/`:

Project	Commands
`snack1-robot-arm`	`npm install` · `npm run dev` (http://localhost:5173) · `npm test` · `npm run build:gripper` regenerates the .glb
`snack2-control-panel`	`npm install` · `npm run dev` · `npm test` (IK and program tests) · `npm run build`
`snack3-conveyor-physics`	`npm install` · `npm run dev` · `npm test` · `npm run bench` (headless physics benchmark)
`snack4-refactor-kata`	`npm install` · `npm test` (spec + golden master) · `npm run typecheck` · `npm run demo`
`snack5-inventory`	`docker compose --profile postgres up --build` (or `--profile mongo`), then http://localhost:8081 · `docker compose --profile test run --rm --build api-test` runs the contract tests on both databases · without Docker: `cd api; npm install; npm test`

Snack 4's full commit history ships as `history/refactor-steps.bundle`: `git clone history/refactor-steps.bundle steps`, then `git log --stat` in the new folder.

APPENDIX B

Glossary

AABB
Axis-aligned bounding box: a box whose faces line up with the world axes, described by its center and half sizes or its min and max corners.

Accessor (glTF)
Says how to read a range of a bufferView as numbers: component type, element type (VEC3, SCALAR…), count, bounds.

ACID
Atomicity, consistency, isolation, durability: the guarantees a database transaction gives.

Bin packing
Packing items into the fewest fixed-size bins; cut lists into stock bars are an instance.

Broad phase
The cheap first pass of collision detection that finds pairs which might touch.

Buffer view (glTF)
A byte range within a buffer, usable directly as a GPU buffer.

Characterization test
A test that records what code currently does, to detect changes, rather than what it should do.

CHECK constraint
A rule on a table that the database enforces on every write, rejecting rows that break it.

Conditional update
An update whose filter contains the rule (“only if enough is available”), so check and write are one atomic step.

Contract test
One test suite run against every implementation of an interface, so they behave the same.

CORS
Cross-origin resource sharing: headers by which a server lets pages from other origins read its responses in a browser.

Counting sort
A non-comparison sort that buckets items by a small integer key in O(n).

CTE
Common table expression (WITH … AS): names a sub-query; in Postgres it can also chain data-changing statements.

Deadlock
Two transactions each waiting for a lock the other holds; avoided by locking in a consistent order.

Denavit–Hartenberg
A standard way to describe each joint of a robot arm with four parameters, giving one transform matrix per joint.

Diverter
A device on a conveyor that steers items off the line to one side, here a hinged flap.

Document database
A database storing self-contained JSON-like documents, which can embed arrays and sub-objects (MongoDB).

Draw call
One request from the CPU to the GPU to draw geometry with a given state. Each has fixed overhead.

Emissive
Light a material gives off regardless of the scene's lights; used to make the selected housing glow.

Event delegation
One listener on a container handling events from its children, identified by attributes, so re-rendered children need no new listeners.

First-fit decreasing
A bin-packing heuristic: sort items largest first, put each in the first bin with room.

Fixed time step
Advancing a simulation in equal small steps, independent of the display's frame rate, so results don't depend on it.

Foreign key
A column that must match a key in another table, so references are always valid.

Forward kinematics
Computing the tool's position from joint angles. Always one answer.

Frustum culling
Skipping objects whose bounding sphere is outside the camera's view.

Golden master
Recorded outputs of existing code for many inputs, compared against new code.

Idempotency key
A client-chosen key that makes retrying a request safe: the server returns the stored result instead of acting twice.

Impulse
An instant change in momentum, the solver's unit of correction.

Instancing
Drawing one geometry many times in one draw call, with per-instance data such as a matrix and color.

Interpolation (α)
Drawing a moment between the last two physics states, by the fraction α of a step left over.

Inverse kinematics
Computing joint angles that put the tool at a target. Zero, several or infinitely many answers.

Island
A group of bodies connected by contacts; they sleep and wake together.

Isolation level
How much concurrent transactions can see of each other (READ COMMITTED, REPEATABLE READ, SERIALIZABLE).

Jacobian
The matrix of partial derivatives of tool position with respect to joint angles; maps joint speeds to tool speed.

jsonb
PostgreSQL's binary JSON type: stores a document in a column, with operators and indexes to query inside it.

Kerf
The width of material a saw blade removes with each cut.

Kinematic body
A body moved by code, with a velocity, that pushes others but is never pushed itself.

Kinematic simulation
Motion set directly (positions and velocities) rather than computed from forces.

Kustomize overlay
A directory that builds on a base set of Kubernetes manifests, adding or changing resources.

Little-endian
Byte order with the least significant byte first; required by the glTF container.

Local frame
A coordinate system attached to an object, in which its own geometry is simple; points are transformed into it.

Lost update
A race where two transactions read the same value and one's write silently overwrites the other's.

moveJ / moveL
Robot moves that interpolate joint angles (curved tool path) or the tool position (straight line).

Narrow phase
The exact collision test on candidate pairs, producing contact normal and depth.

Normal
A unit vector perpendicular to a surface; drives lighting and, in physics, the direction of contact forces.

Normalization
Storing each fact once in its own table, related by keys, instead of repeating it.

Object pool
Preallocated objects or slots reused instead of created and destroyed.

ON DELETE CASCADE
A foreign-key option that deletes the referencing rows when the referenced row is deleted.

Origin
The scheme, host and port of a page (http://localhost:5173); browsers isolate pages by origin.

PBR
Physically based rendering: materials described by base color, metalness and roughness, lit consistently in any environment.

Preflight
The OPTIONS request a browser sends before a cross-origin request that isn't simple, asking whether it's allowed.

Pure function
A function whose output depends only on its inputs and which has no side effects.

Quaternion
Four numbers representing a 3D rotation, free of gimbal lock and easy to interpolate.

Raycasting
Intersecting a ray (here from the camera through the mouse) with scene geometry, for picking.

Re-parenting
Moving an object to a new parent in the scene graph; `attach` keeps its world pose, `add` keeps its local one.

Readiness probe
A Kubernetes check that decides whether a pod receives traffic; separate from liveness, which decides restarts.

Reconciliation
React's comparison of a new render's output with the previous one, to find the DOM changes to make.

Refactoring
Changing the structure of code without changing its behaviour, in small steps.

Replica set
A group of MongoDB servers holding the same data; required for multi-document transactions.

Repository pattern
Hiding storage behind an interface of domain operations, so the rest of the code doesn't know the database.

Ring buffer
A fixed array used as a queue by wrapping an index, so nothing is shifted or allocated.

Row lock
A lock on specific rows (SELECT … FOR UPDATE) that makes other writers wait until the transaction ends.

Rule engine
A small interpreter that evaluates conditions stored as data, instead of decisions written as code.

Saga
A multi-step operation made of individually atomic steps, with compensating steps to undo earlier ones on failure.

Scene graph
A tree of objects whose transforms compose from parent to child.

Seam
A place where code's behaviour can be observed or substituted without editing it, which makes testing possible.

Semi-implicit Euler
Integration that updates velocity first, then position from the new velocity; stable and cheap.

Sequential impulses
Solving contact constraints one at a time and repeating, a form of projected Gauss–Seidel.

Shock propagation
Treating the lower body of a vertical contact as immovable on a final, bottom-up pass, so stacks settle.

Singularity
A pose where the arm loses the ability to move the tool in some direction; the Jacobian loses rank.

Slop
Overlap allowed between resting bodies before position correction acts, to prevent jitter.

Smoothstep
The easing curve $3t^2 - 2t^3$: from 0 to 1 with zero slope at both ends.

Spatial hash
A uniform grid of cells stored in a hash table, for finding nearby objects in O(n).

Spiral of death
A slow frame causing more physics steps, making the next frame slower; avoided by capping steps.

StatefulSet
A Kubernetes workload with stable pod names and a persistent volume per pod, used for databases.

Structure of arrays
Storing each property of many entities in its own array, instead of one object per entity.

TCP
Tool center point: the point on the tool that motion is planned for; here, between the fingertips.

Tearing (React)
Different components showing different versions of external data in the same commit.

Tessellation
Converting exact surfaces (from CAD) into triangles for rendering.

Winding order
The order a triangle's corners are listed; counter-clockwise as seen marks the front face.